MDN JavaScript Documentation

Volume 4 of 9 — Built-in objects: AbstractModuleSource – Date

Unofficial print compilation. This volume contains 195 MDN articles, reproduced without changes to their wording.

Source, attribution and licence

All articles in this volume are from **MDN Web Docs**, `https://developer.mozilla.org/en-US/docs/Web/JavaScript`, retrieved on **2026-11-10** from MDN's rendered pages. They are written **by Mozilla Contributors** and licensed under the Creative Commons Attribution-ShareAlike licence, **CC-BY-SA 2.5** or any later version (`https://creativecommons.org/licenses/by-sa/2.5/`). The title and the URL of the specific MDN page are printed at the head of every article; the list of authors of a page is available through that page on MDN. Code samples added to MDN on or after 20 August 2010 are in the public domain (**CC0**, `https://creativecommons.org/publicdomain/zero/1.0/`); older samples are under the MIT licence. Browser compatibility data is from `@mdn/browser-compat-data` 8.1.8 (CC0).

Modifications. This is not an official MDN or Mozilla publication and is not endorsed by Mozilla. The content was re-laid-out for A4 paper: website navigation, banners and interactive widgets were removed or replaced by short print notes; compatibility and specification tables, which the website generates from data, were re-rendered from the same data; in-volume links were turned into PDF links. Wording, code and section order were not edited. Like the original, this compilation is distributed under CC-BY-SA 2.5 or later. “MDN” and “Mozilla” are trademarks of the Mozilla Foundation; its logos and website design are not part of the licensed content and are not used here.

How to read this book

- **Text.** Article wording, headings, section order, notes, tables and code are MDN's, taken programmatically from MDN's rendered pages. Only the layout was changed for paper.
- **Dashed boxes in italics starting with ▷** and the small grey lines under each article title (breadcrumb, source URL, availability) are additions of this print edition, not MDN text.
- **Code** is printed character for character in a monospaced font. A line longer than about 92 characters wraps at the right edge and continues at the left edge of the code box; no characters are added. Labels above a box give the language; **✗ bad example** and **✓ good example** replace MDN's red/green colouring. “code continues” / “code continued” in the page margin mark a code box that runs over a page break.
- **Interactive examples and live samples** cannot run on paper. Their source code is printed; output is shown only where MDN's source states it (for example in `// Expected output` comments).
- **Links.** Dotted underlines mark links. In the PDF, links to articles in this volume jump to them; all other links open the MDN page. Web API, DOM, HTML/CSS, Glossary and Learn pages are outside the scope of this collection (only `/en-US/docs/Web/JavaScript/` is included) and remain links to MDN.
- **Badges** such as Deprecated Experimental Non-standard Optional reproduce MDN's status icons and labels as text.
- **Browser compatibility tables** are rebuilt from MDN's browser-compat-data (the data the website's tables are generated from). Each cell gives the first browser version that supports the feature. **No** (grey cell) = not supported; **?** = unknown; **a–b** = supported from version a, removed in version b (grey cell); **Prev.** = only in a preview release of that browser; ≤ = that version or earlier; small raised numbers refer to the numbered notes below the table (flags, prefixes, partial support and other remarks).
- **“Baseline” availability lines** are condensed from the Baseline status MDN shows at the top of a page.

Contents

Standard built-in objects 5

AbstractModuleSource 9

AggregateError 11
- AggregateError() constructor 13
- AggregateError: errors 14

Array 16
- Array() constructor 37
- Array.from() 40
- Array.fromAsync() 44
- Array.isArray() 49
- Array.of() 51
- Array[Symbol.species] 53
- Array.prototype.at() 56
- Array.prototype.concat() 59
- Array.prototype.copyWithin() 62
- Array.prototype.entries() 65
- Array.prototype.every() 68
- Array.prototype.fill() 71
- Array.prototype.filter() 74
- Array.prototype.find() 79
- Array.prototype.findIndex() 83
- Array.prototype.findLast() 86
- Array.prototype.findLastIndex() 91
- Array.prototype.flat() 95
- Array.prototype.flatMap() 97
- Array.prototype.forEach() 102
- Array.prototype.includes() 107
- Array.prototype.indexOf() 111
- Array.prototype.join() 114
- Array.prototype.keys() 117
- Array.prototype.lastIndexOf() 119
- Array.prototype.map() 122
- Array.prototype.pop() 128
- Array.prototype.push() 131
- Array.prototype.reduce() 134
- Array.prototype.reduceRight() 142
- Array.prototype.reverse() 148
- Array.prototype.shift() 151
- Array.prototype.slice() 153
- Array.prototype.some() 158
- Array.prototype.sort() 162
- Array.prototype.splice() 169
- Array.prototype.toLocaleString() 174
- Array.prototype.toReversed() 177
- Array.prototype.toSorted() 179
- Array.prototype.toSpliced() 181
- Array.prototype.toString() 184
- Array.prototype.unshift() 186
- Array.prototype.values() 189
- Array.prototype.with() 193
- Array.prototype[Symbol.iterator]() 196
- Array.prototype[Symbol.unscopables] 199
- Array: length 201

ArrayBuffer 204
- ArrayBuffer() constructor 207
- ArrayBuffer.isView() 209
- ArrayBuffer[Symbol.species] 210
- ArrayBuffer.prototype.resize() 212
- ArrayBuffer.prototype.slice() 214
- ArrayBuffer.prototype.transfer() 216
- ArrayBuffer.prototype.transferToFixedLength() 219
- ArrayBuffer.prototype.byteLength 221
- ArrayBuffer.prototype.detached 222
- ArrayBuffer.prototype.maxByteLength 223
- ArrayBuffer.prototype.resizable 224

AsyncDisposableStack 227
- AsyncDisposableStack() constructor 228
- AsyncDisposableStack.prototype.adopt() 230
- AsyncDisposableStack.prototype.defer() 231
- AsyncDisposableStack.prototype.disposeAsync() 233
- AsyncDisposableStack.prototype.move() 235
- AsyncDisposableStack.prototype.use() 238
- AsyncDisposableStack.prototype[Symbol.asyncDispose]() 239
- AsyncDisposableStack.prototype.disposed 241

AsyncFunction 242
- AsyncFunction() constructor 243

AsyncGenerator 246
- AsyncGenerator.prototype.next() 248
- AsyncGenerator.prototype.return() 250
- AsyncGenerator.prototype.throw() 253

AsyncGeneratorFunction 255
- AsyncGeneratorFunction() constructor 257
- AsyncGeneratorFunction.prototype.prototype 258

AsyncIterator 262
- AsyncIterator.prototype[Symbol.asyncDispose]() 263
- AsyncIterator.prototype[Symbol.asyncIterator]() 265

Atomics 267
- Atomics.add() 270
- Atomics.and() 272
- Atomics.compareExchange() 274
- Atomics.exchange() 276
- Atomics.isLockFree() 278
- Atomics.load() 279
- Atomics.notify() 281
- Atomics.or() 283
- Atomics.pause() 285
- Atomics.store() 287
- Atomics.sub() 288
- Atomics.wait() 290
- Atomics.waitAsync() 292
- Atomics.xor() 294

BigInt 297
- BigInt() constructor 304
- BigInt.asIntN() 306
- BigInt.asUintN() 308
- BigInt.prototype.toLocaleString() 310
- BigInt.prototype.toString() 313
- BigInt.prototype.valueOf() 316

BigInt64Array 318

BigInt64Array() constructor 320
BigUint64Array **323**
BigUint64Array() constructor 325
Boolean **327**
Boolean() constructor 331
Boolean.prototype.toString() 333
Boolean.prototype.valueOf() 335
DataView **337**
DataView() constructor 341
DataView.prototype.getBigInt64() 343
DataView.prototype.getBigUint64() 344
DataView.prototype.getFloat16() 346
DataView.prototype.getFloat32() 348
DataView.prototype.getFloat64() 349
DataView.prototype.getInt16() 351
DataView.prototype.getInt32() 353
DataView.prototype.getInt8() 354
DataView.prototype.getUint16() 356
DataView.prototype.getUint32() 358
DataView.prototype.getUint8() 359
DataView.prototype.setBigInt64() 361
DataView.prototype.setBigUint64() 363
DataView.prototype.setFloat16() 365
DataView.prototype.setFloat32() 366
DataView.prototype.setFloat64() 368
DataView.prototype.setInt16() 370
DataView.prototype.setInt32() 372
DataView.prototype.setInt8() 373
DataView.prototype.setUint16() 375
DataView.prototype.setUint32() 377
DataView.prototype.setUint8() 378
DataView.prototype.buffer 380
DataView.prototype.byteLength 381
DataView.prototype.byteOffset 383
Date **385**
Date() constructor 396
Date.now() 400
Date.parse() 402
Date.UTC() 406
Date.prototype.getDate() 409
Date.prototype.getDay() 410
Date.prototype.getFullYear() 412
Date.prototype.getHours() 413
Date.prototype.getMilliseconds() 415
Date.prototype.getMinutes() 416
Date.prototype.getMonth() 417
Date.prototype.getSeconds() 419
Date.prototype.getTime() 421
Date.prototype.getTimezoneOffset() 423
Date.prototype.getUTCDate() 425
Date.prototype.getUTCDay() 427
Date.prototype.getUTCFullYear() 428
Date.prototype.getUTCHours() 430
Date.prototype.getUTCMilliseconds() 431
Date.prototype.getUTCMinutes() 433
Date.prototype.getUTCMonth() 434
Date.prototype.getUTCSeconds() 435
Date.prototype.getYear() 437
Date.prototype.setDate() 439
Date.prototype.setFullYear() 441
Date.prototype.setHours() 443
Date.prototype.setMilliseconds() 445
Date.prototype.setMinutes() 447
Date.prototype.setMonth() 449
Date.prototype.setSeconds() 451
Date.prototype.setTime() 453
Date.prototype.setUTCDate() 454
Date.prototype.setUTCFullYear() 456
Date.prototype.setUTCHours() 458
Date.prototype.setUTCMilliseconds() 460
Date.prototype.setUTCMinutes() 461
Date.prototype.setUTCMonth() 463
Date.prototype.setUTCSeconds() 465
Date.prototype.setYear() 467
Date.prototype.toDateString() 468
Date.prototype.toISOString() 470
Date.prototype.toJSON() 472
Date.prototype.toLocaleDateString() 474
Date.prototype.toLocaleString() 478
Date.prototype.toLocaleTimeString() 482
Date.prototype.toString() 486
Date.prototype.toTemporalInstant() 488
Date.prototype.toTimeString() 489
Date.prototype.toUTCString() 491
Date.prototype.valueOf() 493
Date.prototype[Symbol.toPrimitive]() 495

JavaScript › Reference

Standard built-in objects

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects · MDN page last modified 2026-09-11

This chapter documents all of JavaScript's standard, built-in objects, including their methods and properties.

The term "global objects" (or standard built-in objects) here is not to be confused with **the global object**. Here, "global objects" refer to **objects in the global scope**.

The **global object** itself can be accessed using the `this` operator in the global scope. In fact, the global scope **consists of** the properties of the global object, including inherited properties, if any.

Other objects in the global scope are either created by the user script or provided by the host application. The host objects available in browser contexts are documented in the API reference.

For more information about the distinction between the DOM and core JavaScript, see JavaScript technologies overview.

Standard objects by category

Value properties

These global properties return a simple value. They have no properties or methods.

- `globalThis`
- `Infinity`
- `NaN`
- `undefined`

Function properties

These global functions—functions which are called globally, rather than on an object—directly return their results to the caller.

- `eval()`
- `isFinite()`
- `isNaN()`
- `parseFloat()`
- `parseInt()`
- `decodeURI()`
- `decodeURIComponent()`
- `encodeURI()`
- `encodeURIComponent()`
- `escape()` Deprecated
- `unescape()` Deprecated

Fundamental objects

These objects represent fundamental language constructs.

- `Object`
- `Function`

- `Boolean`
- `Symbol`

Error objects

Error objects are a special type of fundamental object. They include the basic `Error` type, as well as several specialized error types.

- `Error`
- `AggregateError`
- `EvalError`
- `RangeError`
- `ReferenceError`
- `SuppressedError`
- `SyntaxError`
- `TypeError`
- `URIError`
- `InternalError` Non-standard

Numbers and dates

These are the base objects representing numbers, dates, and mathematical calculations.

- `Number`
- `BigInt`
- `Math`
- `Date`
- `Temporal`

Text processing

These objects represent strings and support manipulating them.

- `String`
- `RegExp`

Indexed collections

These objects represent collections of data which are ordered by an index value. This includes (typed) arrays and array-like constructs.

- `Array`
- `TypedArray`
- `Int8Array`
- `Uint8Array`
- `Uint8ClampedArray`
- `Int16Array`
- `Uint16Array`
- `Int32Array`
- `Uint32Array`
- `BigInt64Array`

- `BigUint64Array`
- `Float16Array`
- `Float32Array`
- `Float64Array`

Keyed collections

These objects represent collections which use keys. The iterable collections (`Map` and `Set`) contain elements which are easily iterated in the order of insertion.

- `Map`
- `Set`
- `WeakMap`
- `WeakSet`

Structured data

These objects represent and interact with structured data buffers and data coded using JavaScript Object Notation (JSON).

- `ArrayBuffer`
- `SharedArrayBuffer`
- `DataView`
- `Atomics`
- `JSON`

Managing memory

These objects interact with the garbage collection mechanism.

- `WeakRef`
- `FinalizationRegistry`

Control abstraction objects

Control abstractions can help to structure code, especially async code (without using deeply nested callbacks, for example).

- `Iterator`
- `AsyncIterator`
- `Promise`
- `GeneratorFunction`
- `AsyncGeneratorFunction`
- `Generator`
- `AsyncGenerator`
- `AsyncFunction`
- `DisposableStack`
- `AsyncDisposableStack`

Reflection

- AbstractModuleSource
- Reflect
- Proxy

Internationalization

Additions to the ECMAScript core for language-sensitive functionalities.

- Intl
- Intl.Collator
- Intl.DateTimeFormat
- Intl.DisplayNames
- Intl.DurationFormat
- Intl.ListFormat
- Intl.Locale
- Intl.NumberFormat
- Intl.PluralRules
- Intl.RelativeTimeFormat
- Intl.Segmenter

AbstractModuleSource

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/AbstractModuleSource · MDN page last modified 2026-09-11

An ***AbstractModuleSource*** object represents the compiled source of a module obtained with `import source` or `import.source()`. There is no directly visible `AbstractModuleSource` constructor.

Description

The `AbstractModuleSource` constructor (often referred to as `%AbstractModuleSource%` to indicate its "intrinsicness", since it does not correspond to any global exposed to a JavaScript program) serves as the superclass of all module source subclasses, providing a common interface of utility methods. This constructor is not directly exposed: there is no global `AbstractModuleSource` property. It is only accessible through `Object.getPrototypeOf(moduleSourceObject.constructor)` and similar.

AbstractModuleSource objects

- `WebAssembly.Module`

JavaScript module source objects will be added by the ECMAScript Module Phase Imports proposal.

Constructor

This object cannot be instantiated directly — attempting to construct it with `new` throws a `TypeError`.

```js
import source modSource from "./module.wasm";

new (Object.getPrototypeOf(modSource.constructor))();
// TypeError: Abstract class AbstractModuleSource not directly constructable
```

Instead, obtain a module source object using `import source`, `import.source()`, or a concrete module type's API, such as `WebAssembly.Module()`. These mechanisms do not call the abstract constructor.

Instance properties

These properties are defined on `AbstractModuleSource.prototype` and shared by all `AbstractModuleSource` subclass instances.

`AbstractModuleSource.prototype.constructor`

The constructor function that created the instance object.

`AbstractModuleSource.prototype.constructor` is the hidden `AbstractModuleSource` constructor function, but each module source subclass also defines its own `constructor` property.

`AbstractModuleSource.prototype[Symbol.toStringTag]`

The initial value of the `AbstractModuleSource.prototype[Symbol.toStringTag]` property is a getter that returns a string based on the constructor's identity, such as `"WebAssembly.Module"` or `"ModuleSource"`. It returns `undefined` if the `this` value is not an instance of one of the module source subclasses. This property is used in `Object.prototype.toString()`.

Specifications

Specification
Source Phase Imports `https://tc39.es/proposal-source-phase-imports/#sec-module-source-objects`

Browser compatibility

▷ *No compatibility data found for `javascript.builtins.AbstractModuleSource`.*

See also

- `WebAssembly.Module`
- `import.source()`
- `import source`

JavaScript › Reference › Standard built-in objects

AggregateError

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/AggregateError · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since September 2020.

The **`AggregateError`** object represents an error when several errors need to be wrapped in a single error. It is thrown when multiple errors need to be reported by an operation, for example by `Promise.any()`, when all promises passed to it reject.

Compared to `SuppressedError`, `AggregateError` represents a list of unrelated errors, while `SuppressedError` represents an error that happened during the handling of another error.

`AggregateError` is a subclass of `Error`.

Constructor

`AggregateError()`
: Creates a new `AggregateError` object.

Instance properties

Also inherits instance properties from its parent `Error`.

These properties are defined on `AggregateError.prototype` and shared by all `AggregateError` instances.

`AggregateError.prototype.constructor`
: The constructor function that created the instance object. For `AggregateError` instances, the initial value is the `AggregateError` constructor.

`AggregateError.prototype.name`
: Represents the name for the type of error. For `AggregateError.prototype.name`, the initial value is `"AggregateError"`.

These properties are own properties of each `AggregateError` instance.

`errors`
: An array representing the errors that were aggregated.

Instance methods

Inherits instance methods from its parent `Error`.

Examples

Catching an AggregateError

```js
Promise.any([Promise.reject(new Error("some error"))]).catch((e) => {
  console.log(e instanceof AggregateError); // true
  console.log(e.message); // "All Promises rejected"
  console.log(e.name); // "AggregateError"
  console.log(e.errors); // [ Error: "some error" ]
});
```

Creating an AggregateError

```js
try {
  throw new AggregateError([new Error("some error")], "Hello");
} catch (e) {
  console.log(e instanceof AggregateError); // true
  console.log(e.message); // "Hello"
  console.log(e.name); // "AggregateError"
  console.log(e.errors); // [ Error: "some error" ]
}
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-aggregate-error-objects

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
AggregateError	85	85	79	71	14	85	79	60	14	14.0	85	14	1.0.0	1.2	15.0.0
AggregateError() constructor	85	85	79	71	14	85	79	60	14	14.0	85	14	1.0.0	1.2	15.0.0
errors	85	85	79	71	14	85	79	60	14	14.0	85	14	1.0.0	1.2	15.0.0
AggregateError is serializable	98[1]	98[1]	103[2]	84[1]	17	98[1]	103[2]	68[1]	17	18.0[1]	98[1]	17	?	**No**	**No**

1. AggregateError serializes to the Error type, without additional properties.
2. AggregateError serializes to the AggregateError type, with properties name, message, cause, and errors.

Compatibility data: MDN browser-compat-data 8.1.8, javascript.builtins.AggregateError. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Polyfill of AggregateError in core-js
- es-shims polyfill of AggregateError
- Error
- Promise.any

JavaScript › Reference › Standard built-in objects › AggregateError

AggregateError() constructor

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/AggregateError/AggregateError · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since September 2020.

The **`AggregateError()`** constructor creates `AggregateError` objects.

Syntax

```js
new AggregateError(errors)
new AggregateError(errors, message)
new AggregateError(errors, message, options)

AggregateError(errors)
AggregateError(errors, message)
AggregateError(errors, message, options)
```

> **Note:** `AggregateError()` can be called with or without `new`. Both create a new `AggregateError` instance.

Parameters

`errors`

An iterable of errors, may not actually be `Error` instances.

`message` Optional

An optional human-readable description of the aggregate error.

`options` Optional

An object that has the following properties:

`cause` Optional

A property indicating the specific cause of the error. When catching and re-throwing an error with a more-specific or useful error message, this property can be used to pass the original error.

Examples

Creating an AggregateError

```js
try {
  throw new AggregateError([new Error("some error")], "Hello");
} catch (e) {
  console.log(e instanceof AggregateError); // true
  console.log(e.message); // "Hello"
  console.log(e.name); // "AggregateError"
  console.log(e.errors); // [ Error: "some error" ]
}
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-aggregate-error-constructor`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`AggregateError()` constructor	85	85	79	71	14	85	79	60	14	14.0	85	14	1.0.0	1.2	15.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.AggregateError.AggregateError`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `AggregateError` in `core-js`
- es-shims polyfill of `AggregateError`
- `Promise.any`

JavaScript › Reference › Standard built-in objects › AggregateError

AggregateError: errors

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/AggregateError/errors · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since September 2020.

The **`errors`** data property of an `AggregateError` instance contains an array representing the errors that were aggregated.

Value

An `Array` containing values in the same order as the iterable passed as the first argument of the `AggregateError()` constructor.

Property attributes of `AggregateError: errors`	
Writable	yes
Enumerable	no
Configurable	yes

Examples

Using errors

```js
try {
  throw new AggregateError(
    // An iterable of errors
    new Set([new Error("some error"), new Error("another error")]),
    "Multiple errors thrown",
  );
} catch (err) {
  console.log(err.errors);
  // [
  //   Error: some error,
  //   Error: another error
  // ]
}
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-aggregate-error

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
errors	85	85	79	71	14	85	79	60	14	14.0	85	14	1.0.0	1.2	15.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.AggregateError.errors`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Control flow and error handling guide
- `AggregateError`
- `Error: cause`

JavaScript › Reference › Standard built-in objects

Array

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array · MDN page last modified 2026-07-28

Baseline: Widely available — available across major browsers since July 2015.

The **`Array`** object, as with arrays in other programming languages, enables storing a collection of multiple items under a single variable name, and has members for performing common array operations.

Description

In JavaScript, arrays aren't primitives but are instead `Array` objects with the following core characteristics:

- **JavaScript arrays are resizable** and **can contain a mix of different data types**. (When those characteristics are undesirable, use typed arrays instead.)
- **JavaScript arrays are not associative arrays** and so, array elements cannot be accessed using arbitrary strings as indexes, but must be accessed using nonnegative integers (or their respective string form) as indexes.
- **JavaScript arrays are zero-indexed**: the first element of an array is at index `0`, the second is at index `1`, and so on — and the last element is at the value of the array's `length` property minus `1`.
- **JavaScript array-copy operations create shallow copies**. (All standard built-in copy operations with *any* JavaScript objects create shallow copies, rather than deep copies).

Array indices

`Array` objects cannot use arbitrary strings as element indexes (as in an associative array) but must use nonnegative integers (or their respective string form). Setting or accessing via non-integers will not set or retrieve an element from the array list itself, but will set or access a variable associated with that array's object property collection. The array's object properties and list of array elements are separate, and the array's traversal and mutation operations cannot be applied to these named properties.

Array elements are object properties in the same way that `toString` is a property (to be specific, however, `toString()` is a method). Nevertheless, trying to access an element of an array as follows throws a syntax error because the property name is not valid:

js · ✗ bad example

```
arr.0; // a syntax error
```

JavaScript syntax requires properties beginning with a digit to be accessed using bracket notation instead of dot notation. It's also possible to quote the array indices (e.g., `years['2']` instead of `years[2]`), although usually not necessary.

The `2` in `years[2]` is coerced into a string by the JavaScript engine through an implicit `toString` conversion. As a result, `'2'` and `'02'` would refer to two different slots on the `years` object, and the following example could be `true`:

js

```
console.log(years["2"] !== years["02"]);
```

Only `years['2']` is an actual array index. `years['02']` is an arbitrary string property that will not be visited in array iteration.

Relationship between length and numerical properties

A JavaScript array's `length` property and numerical properties are connected.

Several of the built-in array methods (e.g., `join()`, `slice()`, `indexOf()`, etc.) take into account the value of an array's `length` property when they're called.

Other methods (e.g., `push()`, `splice()`, etc.) also result in updates to an array's `length` property.

```js
const fruits = [];
fruits.push("banana", "apple", "peach");
console.log(fruits.length); // 3
```

When setting a property on a JavaScript array when the property is a valid array index and that index is outside the current bounds of the array, the engine will update the array's `length` property accordingly:

```js
fruits[5] = "mango";
console.log(fruits[5]); // 'mango'
console.log(Object.keys(fruits)); // ['0', '1', '2', '5']
console.log(fruits.length); // 6
```

Increasing the `length` extends the array by adding empty slots without creating any new elements — not even `undefined`.

```js
fruits.length = 10;
console.log(fruits); // ['banana', 'apple', 'peach', empty x 2, 'mango', empty x 4]
console.log(Object.keys(fruits)); // ['0', '1', '2', '5']
console.log(fruits.length); // 10
console.log(fruits[8]); // undefined
```

Decreasing the `length` property does, however, delete elements.

```js
fruits.length = 2;
console.log(Object.keys(fruits)); // ['0', '1']
console.log(fruits.length); // 2
```

This is explained further on the `length` page.

Array methods and empty slots

Array methods have different behaviors when encountering empty slots in sparse arrays. In general, older methods (e.g., `forEach`) treat empty slots differently from indices that contain `undefined`.

Methods that have special treatment for empty slots include the following: `concat()`, `copyWithin()`, `every()`, `filter()`, `flat()`, `flatMap()`, `forEach()`, `indexOf()`, `lastIndexOf()`, `map()`, `reduce()`, `reduceRight()`, `reverse()`, `slice()`, `some()`, `sort()`, and `splice()`. Iteration methods such as `forEach` don't visit empty slots at all. Other methods, such as `concat`, `copyWithin`, etc., preserve empty slots when doing the copying, so in the end the array is still sparse.

```js
const colors = ["red", "yellow", "blue"];
colors[5] = "purple";
colors.forEach((item, index) => {
  console.log(`${index}: ${item}`);
});
// Output:
// 0: red
// 1: yellow
// 2: blue
// 5: purple

colors.reverse(); // ['purple', empty × 2, 'blue', 'yellow', 'red']
```

Newer methods (e.g., `keys`) do not treat empty slots specially and treat them as if they contain `undefined`. Methods that conflate empty slots with `undefined` elements include the following: `entries()`, `fill()`, `find()`, `findIndex()`, `findLast()`, `findLastIndex()`, `includes()`, `join()`, `keys()`, `toLocaleString()`, `toReversed()`, `toSorted()`, `toSpliced()`, `values()`, and `with()`.

```js
const colors = ["red", "yellow", "blue"];
colors[5] = "purple";
const iterator = colors.keys();
for (const key of iterator) {
  console.log(`${key}: ${colors[key]}`);
}
// Output
// 0: red
// 1: yellow
// 2: blue
// 3: undefined
// 4: undefined
// 5: purple

const newColors = colors.toReversed(); // ['purple', undefined, undefined, 'blue', 'yellow',
'red']
```

Copying methods and mutating methods

Some methods do not mutate the existing array that the method was called on, but instead return a new array. They do so by first constructing a new array and then populating it with elements. The copy always happens *shallowly* — the method never copies anything beyond the initially created array. Elements of the original array(s) are copied into the new array as follows:

- Objects: the object reference is copied into the new array. Both the original and new array refer to the same object. That is, if a referenced object is modified, the changes are visible to both the new and original arrays.
- Primitive types such as strings, numbers and booleans (not `String`, `Number`, and `Boolean` objects): their values are copied into the new array.

Other methods mutate the array that the method was called on, in which case their return value differs depending on the method: sometimes a reference to the same array, sometimes the length of the new array.

The following methods create new arrays by accessing `this.constructor[Symbol.species]` to determine the constructor to use: `concat()`, `filter()`, `flat()`, `flatMap()`, `map()`, `slice()`, and `splice()`

(to construct the array of removed elements that's returned).

The following methods always create new arrays with the `Array` base constructor: `toReversed()`, `toSorted()`, `toSpliced()`, and `with()`.

The following table lists the methods that mutate the original array, and the corresponding non-mutating alternative:

Mutating method	Non-mutating alternative
`copyWithin()`	No one-method alternative
`fill()`	No one-method alternative
`pop()`	`slice(0, -1)`
`push(v1, v2)`	`concat([v1, v2])`
`reverse()`	`toReversed()`
`shift()`	`slice(1)`
`sort()`	`toSorted()`
`splice()`	`toSpliced()`
`unshift(v1, v2)`	`toSpliced(0, 0, v1, v2)`

An easy way to change a mutating method into a non-mutating alternative is to use the spread syntax or `slice()` to create a copy first:

```js
arr.copyWithin(0, 1, 2); // mutates arr
const arr2 = arr.slice().copyWithin(0, 1, 2); // does not mutate arr
const arr3 = [...arr].copyWithin(0, 1, 2); // does not mutate arr
```

Iterative methods

Many array methods take a callback function as an argument. The callback function is called sequentially and at most once for each element in the array, and the return value of the callback function is used to determine the return value of the method. They all share the same signature:

```js
method(callbackFn, thisArg)
```

Where `callbackFn` takes three arguments:

`element`
: The current element being processed in the array.

`index`
: The index of the current element being processed in the array.

`array`
: The array that the method was called upon.

What `callbackFn` is expected to return depends on the array method that was called.

The `thisArg` argument (defaults to `undefined`) will be used as the `this` value when calling `callbackFn`. The `this` value ultimately observable by `callbackFn` is determined according to the usual rules: if `callbackFn` is non-strict, primitive `this` values are wrapped into objects, and `undefined`/`null` is

substituted with `globalThis`. The `thisArg` argument is irrelevant for any `callbackFn` defined with an arrow function, as arrow functions don't have their own `this` binding.

The `array` argument passed to `callbackFn` is most useful if you want to read another index during iteration, because you may not always have an existing variable that refers to the current array. You should generally not mutate the array during iteration (see mutating initial array in iterative methods), but you can also use this argument to do so. The `array` argument is *not* the array that is being built, in the case of methods like `map()`, `filter()`, and `flatMap()` — there is no way to access the array being built from the callback function.

All iterative methods are copying and generic, although they behave differently with empty slots.

The following methods are iterative: `every()`, `filter()`, `find()`, `findIndex()`, `findLast()`, `findLastIndex()`, `flatMap()`, `forEach()`, `map()`, and `some()`.

In particular, `every()`, `find()`, `findIndex()`, `findLast()`, `findLastIndex()`, and `some()` do not always invoke `callbackFn` on every element — they stop iteration as soon as the return value is determined.

The `reduce()` and `reduceRight()` methods also take a callback function and run it at most once for each element in the array, but they have slightly different signatures from typical iterative methods (for example, they don't accept `thisArg`).

The `sort()` method also takes a callback function, but it is not an iterative method. It mutates the array in-place, doesn't accept `thisArg`, and may invoke the callback multiple times on an index.

Iterative methods iterate the array like the following (with a lot of technical details omitted):

```js
function method(callbackFn, thisArg) {
  const length = this.length;
  for (let i = 0; i < length; i++) {
    if (i in this) {
      const result = callbackFn.call(thisArg, this[i], i, this);
      // Do something with result; maybe return early
    }
  }
}
```

Note the following:

1. Not all methods do the `i in this` test. The `find`, `findIndex`, `findLast`, and `findLastIndex` methods do not, but other methods do.
2. The `length` is memorized before the loop starts. This affects how insertions and deletions during iteration are handled (see mutating initial array in iterative methods).
3. The method doesn't memorize the array contents, so if any index is modified during iteration, the new value might be observed.
4. The code above iterates the array in ascending order of index. Some methods iterate in descending order of index (`for (let i = length - 1; i >= 0; i--)`): `reduceRight()`, `findLast()`, and `findLastIndex()`.
5. `reduce` and `reduceRight` have slightly different signatures and do not always start at the first/last element.

Generic array methods

Array methods are always generic — they don't access any internal data of the array object. They only access the array elements through the `length` property and the indexed elements. This means that they can be called on array-like objects as well.

```js
const arrayLike = {
  0: "a",
  1: "b",
  length: 2,
};
console.log(Array.prototype.join.call(arrayLike, "+")); // 'a+b'
```

Normalization of the length property

The `length` property is converted to an integer and then clamped to the range between 0 and 2^{53} - 1. `NaN` becomes `0`, so even when `length` is not present or is `undefined`, it behaves as if it has value `0`.

The language avoids setting `length` to an unsafe integer. All built-in methods will throw a `TypeError` if `length` will be set to a number greater than 2^{53} - 1. However, because the `length` property of arrays throws an error if it's set to greater than 2^{32} - 1, the safe integer threshold is usually not reached unless the method is called on a non-array object.

```js
Array.prototype.flat.call({}); // []
```

Some array methods set the `length` property of the array object. They always set the value after normalization, so `length` always ends as an integer.

```js
const a = { length: 0.7 };
Array.prototype.push.call(a);
console.log(a.length); // 0
```

Array-like objects

The term *array-like object* refers to any object that doesn't throw during the `length` conversion process described above. In practice, such object is expected to actually have a `length` property and to have indexed elements in the range `0` to `length - 1`. (If it doesn't have all indices, it will be functionally equivalent to a sparse array.) Any integer index less than zero or greater than `length - 1` is ignored when an array method operates on an array-like object.

Many DOM objects are array-like — for example, `NodeList` and `HTMLCollection`. The `arguments` object is also array-like. You can call array methods on them even if they don't have these methods themselves.

```js
function f() {
  console.log(Array.prototype.join.call(arguments, "+"));
}

f("a", "b"); // 'a+b'
```

Constructor

`Array()`
Creates a new `Array` object.

Static properties

`Array[Symbol.species]`
Returns the `Array` constructor.

Static methods

`Array.from()`
Creates a new `Array` instance from an iterable or array-like object.

`Array.fromAsync()`
Creates a new `Array` instance from an async iterable, iterable, or array-like object.

`Array.isArray()`
Returns `true` if the argument is an array, or `false` otherwise.

`Array.of()`
Creates a new `Array` instance with a variable number of arguments, regardless of number or type of the arguments.

Instance properties

These properties are defined on `Array.prototype` and shared by all `Array` instances.

`Array.prototype.constructor`
The constructor function that created the instance object. For `Array` instances, the initial value is the `Array` constructor.

`Array.prototype[Symbol.unscopables]`
Contains property names that were not included in the ECMAScript standard prior to the ES2015 version and that are ignored for `with` statement-binding purposes.

These properties are own properties of each `Array` instance.

`length`
Reflects the number of elements in an array.

Instance methods

`Array.prototype.at()`
Returns the array item at the given index. Accepts negative integers, which count back from the last item.

`Array.prototype.concat()`
Returns a new array that is the calling array joined with other array(s) and/or value(s).

`Array.prototype.copyWithin()`
Copies a sequence of array elements within an array.

`Array.prototype.entries()`
Returns a new *array iterator* object that contains the key/value pairs for each index in an array.

`Array.prototype.every()`
Returns `false` if it finds an element in the array that does not satisfy the provided testing function. Otherwise, it returns `true`.

`Array.prototype.fill()`
Fills all the elements of an array from a start index to an end index with a static value.

`Array.prototype.filter()`
Returns a new array containing all elements of the calling array for which the provided filtering function returns `true`.

`Array.prototype.find()`
Returns the value of the first element in the array that satisfies the provided testing function, or `undefined` if no appropriate element is found.

`Array.prototype.findIndex()`
Returns the index of the first element in the array that satisfies the provided testing function, or `-1` if no appropriate element was found.

`Array.prototype.findLast()`
Returns the value of the last element in the array that satisfies the provided testing function, or `undefined` if no appropriate element is found.

`Array.prototype.findLastIndex()`
Returns the index of the last element in the array that satisfies the provided testing function, or `-1` if no appropriate element was found.

`Array.prototype.flat()`
Returns a new array with all sub-array elements concatenated into it recursively up to the specified depth.

`Array.prototype.flatMap()`
Returns a new array formed by applying a given callback function to each element of the calling array, and then flattening the result by one level.

`Array.prototype.forEach()`
Calls a function for each element in the calling array.

`Array.prototype.includes()`
Determines whether the calling array contains a value, returning `true` or `false` as appropriate.

`Array.prototype.indexOf()`
Returns the first (least) index at which a given element can be found in the calling array.

`Array.prototype.join()`
Returns a new string that is the concatenation of all elements in this array, separated by commas or a specified separator string.

`Array.prototype.keys()`
Returns a new *array iterator* that contains the keys for each index in the calling array.

`Array.prototype.lastIndexOf()`
Returns the last (greatest) index at which a given element can be found in the calling array, or `-1` if none is found.

`Array.prototype.map()`
Returns a new array containing the results of invoking a function on every element in the calling array.

`Array.prototype.pop()`
Removes the last element from an array and returns that element.

`Array.prototype.push()`
Adds one or more elements to the end of an array, and returns the new `length` of the array.

`Array.prototype.reduce()`
Executes a user-supplied "reducer" callback function on each element of the array (from left to right), to reduce it to a single value.

`Array.prototype.reduceRight()`
Executes a user-supplied "reducer" callback function on each element of the array (from right to left), to reduce it to a single value.

`Array.prototype.reverse()`
Reverses the order of the elements of an array *in place*. (First becomes the last, last becomes first.)

`Array.prototype.shift()`
Removes the first element from an array and returns that element.

`Array.prototype.slice()`
Extracts a section of the calling array and returns a new array.

`Array.prototype.some()`
Returns `true` if it finds an element in the array that satisfies the provided testing function. Otherwise, it returns `false`.

`Array.prototype.sort()`
Sorts the elements of an array in place and returns the array.

`Array.prototype.splice()`
Adds and/or removes elements from an array.

`Array.prototype.toLocaleString()`
Returns a localized string representing the calling array and its elements. Overrides the `Object.prototype.toLocaleString()` method.

`Array.prototype.toReversed()`
Returns a new array with the elements in reversed order, without modifying the original array.

`Array.prototype.toSorted()`
Returns a new array with the elements sorted in ascending order, without modifying the original array.

`Array.prototype.toSpliced()`
Returns a new array with some elements removed and/or replaced at a given index, without modifying the original array.

`Array.prototype.toString()`
Returns a string representing the calling array and its elements. Overrides the `Object.prototype.toString()` method.

`Array.prototype.unshift()`
Adds one or more elements to the front of an array, and returns the new `length` of the array.

`Array.prototype.values()`

Returns a new *array iterator* object that contains the values for each index in the array.

`Array.prototype.with()`

Returns a new array with the element at the given index replaced with the given value, without modifying the original array.

`Array.prototype[Symbol.iterator]()`

An alias for the `values()` method by default.

Examples

This section provides some examples of common array operations in JavaScript.

> **Note:** If you're not yet familiar with array basics, consider first reading JavaScript First Steps: Arrays, which explains what arrays are, and includes other examples of common array operations.

Create an array

This example shows three ways to create new array: first using array literal notation, then using the `Array()` constructor, and finally using `String.prototype.split()` to build the array from a string.

```js
// 'fruits' array created using array literal notation.
const fruits = ["Apple", "Banana"];
console.log(fruits.length);
// 2

// 'fruits2' array created using the Array() constructor.
const fruits2 = new Array("Apple", "Banana");
console.log(fruits2.length);
// 2

// 'fruits3' array created using String.prototype.split().
const fruits3 = "Apple, Banana".split(", ");
console.log(fruits3.length);
// 2
```

Create a string from an array

This example uses the `join()` method to create a string from the `fruits` array.

```js
const fruits = ["Apple", "Banana"];
const fruitsString = fruits.join(", ");
console.log(fruitsString);
// "Apple, Banana"
```

Access an array item by its index

This example shows how to access items in the `fruits` array by specifying the index number of their position in the array.

```js
const fruits = ["Apple", "Banana"];

// The index of an array's first element is always 0.
fruits[0]; // Apple

// The index of an array's second element is always 1.
fruits[1]; // Banana

// The index of an array's last element is always one
// less than the length of the array.
fruits[fruits.length - 1]; // Banana

// Using an index number larger than the array's length
// returns 'undefined'.
fruits[99]; // undefined
```

Find the index of an item in an array

This example uses the `indexOf()` method to find the position (index) of the string `"Banana"` in the `fruits` array.

```js
const fruits = ["Apple", "Banana"];
console.log(fruits.indexOf("Banana"));
// 1
```

Check if an array contains a certain item

This example shows two ways to check if the `fruits` array contains `"Banana"` and `"Cherry"`: first with the `includes()` method, and then with the `indexOf()` method to test for an index value that's not `-1`.

```js
const fruits = ["Apple", "Banana"];

fruits.includes("Banana"); // true
fruits.includes("Cherry"); // false

// If indexOf() doesn't return -1, the array contains the given item.
fruits.indexOf("Banana") !== -1; // true
fruits.indexOf("Cherry") !== -1; // false
```

Append an item to an array

This example uses the `push()` method to append a new string to the `fruits` array.

```js
const fruits = ["Apple", "Banana"];
const newLength = fruits.push("Orange");
console.log(fruits);
// ["Apple", "Banana", "Orange"]
console.log(newLength);
// 3
```

Remove the last item from an array

This example uses the `pop()` method to remove the last item from the `fruits` array.

```js
const fruits = ["Apple", "Banana", "Orange"];
const removedItem = fruits.pop();
console.log(fruits);
// ["Apple", "Banana"]
console.log(removedItem);
// Orange
```

> **Note:** `pop()` can only be used to remove the last item from an array. To remove multiple items from the end of an array, see the next example.

Remove multiple items from the end of an array

This example uses the `splice()` method to remove the last 3 items from the `fruits` array.

```js
const fruits = ["Apple", "Banana", "Strawberry", "Mango", "Cherry"];
const start = -3;
const removedItems = fruits.splice(start);
console.log(fruits);
// ["Apple", "Banana"]
console.log(removedItems);
// ["Strawberry", "Mango", "Cherry"]
```

Truncate an array down to just its first N items

This example uses the `splice()` method to truncate the `fruits` array down to just its first 2 items.

```js
const fruits = ["Apple", "Banana", "Strawberry", "Mango", "Cherry"];
const start = 2;
const removedItems = fruits.splice(start);
console.log(fruits);
// ["Apple", "Banana"]
console.log(removedItems);
// ["Strawberry", "Mango", "Cherry"]
```

Remove the first item from an array

This example uses the `shift()` method to remove the first item from the `fruits` array.

```js
const fruits = ["Apple", "Banana"];
const removedItem = fruits.shift();
console.log(fruits);
// ["Banana"]
console.log(removedItem);
// Apple
```

> **Note:** `shift()` can only be used to remove the first item from an array. To remove multiple items from the beginning of an array, see the next example.

Remove multiple items from the beginning of an array

This example uses the `splice()` method to remove the first 3 items from the `fruits` array.

```js
const fruits = ["Apple", "Strawberry", "Cherry", "Banana", "Mango"];
const start = 0;
const deleteCount = 3;
const removedItems = fruits.splice(start, deleteCount);
console.log(fruits);
// ["Banana", "Mango"]
console.log(removedItems);
// ["Apple", "Strawberry", "Cherry"]
```

Add a new first item to an array

This example uses the `unshift()` method to add, at index `0`, a new item to the `fruits` array — making it the new first item in the array.

```js
const fruits = ["Banana", "Mango"];
const newLength = fruits.unshift("Strawberry");
console.log(fruits);
// ["Strawberry", "Banana", "Mango"]
console.log(newLength);
// 3
```

Remove a single item by index

This example uses the `splice()` method to remove the string `"Banana"` from the `fruits` array — by specifying the index position of `"Banana"`.

```js
const fruits = ["Strawberry", "Banana", "Mango"];
const start = fruits.indexOf("Banana");
const deleteCount = 1;
const removedItems = fruits.splice(start, deleteCount);
console.log(fruits);
// ["Strawberry", "Mango"]
console.log(removedItems);
// ["Banana"]
```

Remove multiple items by index

This example uses the `splice()` method to remove the strings `"Banana"` and `"Strawberry"` from the `fruits` array — by specifying the index position of `"Banana"`, along with a count of the number of total items to remove.

```js
const fruits = ["Apple", "Banana", "Strawberry", "Mango"];
const start = 1;
const deleteCount = 2;
const removedItems = fruits.splice(start, deleteCount);
console.log(fruits);
// ["Apple", "Mango"]
console.log(removedItems);
// ["Banana", "Strawberry"]
```

Replace multiple items in an array

This example uses the `splice()` method to replace the last 2 items in the `fruits` array with new items.

```js
const fruits = ["Apple", "Banana", "Strawberry"];
const start = -2;
const deleteCount = 2;
const removedItems = fruits.splice(start, deleteCount, "Mango", "Cherry");
console.log(fruits);
// ["Apple", "Mango", "Cherry"]
console.log(removedItems);
// ["Banana", "Strawberry"]
```

Iterate over an array

This example uses a `for...of` loop to iterate over the `fruits` array, logging each item to the console.

```js
const fruits = ["Apple", "Mango", "Cherry"];
for (const fruit of fruits) {
  console.log(fruit);
}
// Apple
// Mango
// Cherry
```

But `for...of` is just one of many ways to iterate over any array; for more ways, see Loops and iteration, and see the documentation for the `every()`, `filter()`, `flatMap()`, `map()`, `reduce()`, and `reduceRight()` methods — and see the next example, which uses the `forEach()` method.

Call a function on each element in an array

This example uses the `forEach()` method to call a function on each element in the `fruits` array; the function causes each item to be logged to the console, along with the item's index number.

```js
const fruits = ["Apple", "Mango", "Cherry"];
fruits.forEach((item, index, array) => {
  console.log(item, index);
});
// Apple 0
// Mango 1
// Cherry 2
```

Merge multiple arrays together

This example uses the `concat()` method to merge the `fruits` array with a `moreFruits` array, to produce a new `combinedFruits` array. Notice that `fruits` and `moreFruits` remain unchanged.

```js
const fruits = ["Apple", "Banana", "Strawberry"];
const moreFruits = ["Mango", "Cherry"];
const combinedFruits = fruits.concat(moreFruits);
console.log(combinedFruits);
// ["Apple", "Banana", "Strawberry", "Mango", "Cherry"]

// The 'fruits' array remains unchanged.
console.log(fruits);
// ["Apple", "Banana", "Strawberry"]

// The 'moreFruits' array also remains unchanged.
console.log(moreFruits);
// ["Mango", "Cherry"]
```

Copy an array

This example shows three ways to create a new array from the existing `fruits` array: first by using spread syntax, then by using the `from()` method, and then by using the `slice()` method.

```js
const fruits = ["Strawberry", "Mango"];

// Create a copy using spread syntax.
const fruitsCopy = [...fruits];
// ["Strawberry", "Mango"]

// Create a copy using the from() method.
const fruitsCopy2 = Array.from(fruits);
// ["Strawberry", "Mango"]

// Create a copy using the slice() method.
const fruitsCopy3 = fruits.slice();
// ["Strawberry", "Mango"]
```

All built-in array-copy operations (spread syntax, `Array.from()`, `Array.prototype.slice()`, and `Array.prototype.concat()`) create shallow copies. If you instead want a deep copy of an array, you can use `JSON.stringify()` to convert the array to a JSON string, and then `JSON.parse()` to convert the string back into a new array that's completely independent from the original array.

```js
const fruitsDeepCopy = JSON.parse(JSON.stringify(fruits));
```

You can also create deep copies using the `structuredClone()` method, which has the advantage of allowing transferable objects in the source to be *transferred* to the new copy, rather than just cloned.

Finally, it's important to understand that assigning an existing array to a new variable doesn't create a copy of either the array or its elements. Instead the new variable is just a reference, or alias, to the original array; that is, the original array's name and the new variable name are just two names for the exact same object (and so will always evaluate as strictly equivalent). Therefore, if you make any changes at all either to the value of the original array or to the value of the new variable, the other will change, too:

```js
const fruits = ["Strawberry", "Mango"];
const fruitsAlias = fruits;
// 'fruits' and 'fruitsAlias' are the same object, strictly equivalent.
fruits === fruitsAlias; // true
// Any changes to the 'fruits' array change 'fruitsAlias' too.
fruits.unshift("Apple", "Banana");
console.log(fruits);
// ['Apple', 'Banana', 'Strawberry', 'Mango']
console.log(fruitsAlias);
// ['Apple', 'Banana', 'Strawberry', 'Mango']
```

Creating a two-dimensional array

The following creates a chessboard as a two-dimensional array of strings. The first move is made by copying the `'p'` in `board[6][4]` to `board[4][4]`. The old position at `[6][4]` is made blank.

```js
const board = [
  ["R", "N", "B", "Q", "K", "B", "N", "R"],
  ["P", "P", "P", "P", "P", "P", "P", "P"],
  [" ", " ", " ", " ", " ", " ", " ", " "],
  [" ", " ", " ", " ", " ", " ", " ", " "],
  [" ", " ", " ", " ", " ", " ", " ", " "],
  [" ", " ", " ", " ", " ", " ", " ", " "],
  ["p", "p", "p", "p", "p", "p", "p", "p"],
  ["r", "n", "b", "q", "k", "b", "n", "r"],
];

console.log(`${board.join("\n")}\n\n`);

// Move King's Pawn forward 2
board[4][4] = board[6][4];
board[6][4] = " ";
console.log(board.join("\n"));
```

Here is the output:

```
R,N,B,Q,K,B,N,R
P,P,P,P,P,P,P,P
 , , , , , , ,
 , , , , , , ,
 , , , , , , ,
 , , , , , , ,
p,p,p,p,p,p,p,p
r,n,b,q,k,b,n,r

R,N,B,Q,K,B,N,R
P,P,P,P,P,P,P,P
 , , , , , , ,
 , , , , , , ,
 , , , ,p, , ,
 , , , , , , ,
p,p,p,p, ,p,p,p
r,n,b,q,k,b,n,r
```

Using an array to tabulate a set of values

```js
const values = [];
for (let x = 0; x < 10; x++) {
  values.push([2 ** x, 2 * x ** 2]);
}
console.table(values);
```

Results in

```
// The first column is the index
0  1    0
1  2    2
2  4    8
3  8    18
4  16   32
5  32   50
6  64   72
7  128  98
8  256  128
9  512  162
```

Creating an array using the result of a match

The result of a match between a `RegExp` and a string can create a JavaScript array that has properties and elements which provide information about the match. Such an array is returned by `RegExp.prototype.exec()` and `String.prototype.match()`.

For example:

```js
// Match one d followed by one or more b's followed by one d
// Remember matched b's and the following d
// Ignore case

const myRe = /d(b+)(d)/i;
const execResult = myRe.exec("cdbBdbsbz");

console.log(execResult.input); // 'cdbBdbsbz'
console.log(execResult.index); // 1
console.log(execResult); // [ "dbBd", "bB", "d" ]
```

For more information about the result of a match, see the `RegExp.prototype.exec()` and `String.prototype.match()` pages.

Mutating initial array in iterative methods

Iterative methods do not mutate the array on which it is called, but the function provided as `callbackFn` can. The key principle to remember is that only indexes between 0 and `arrayLength - 1` are visited, where `arrayLength` is the length of the array at the time the array method was first called, but the element passed to the callback is the value at the time the index is visited. Therefore:

- `callbackFn` will not visit any elements added beyond the array's initial length when the call to the iterative method began.
- Changes to already-visited indexes do not cause `callbackFn` to be invoked on them again.

- If an existing, yet-unvisited element of the array is changed by `callbackFn`, its value passed to the `callbackFn` will be the value at the time that element gets visited. Removed elements are not visited.

> **Warning:** Concurrent modifications of the kind described above frequently lead to hard-to-understand code and are generally to be avoided (except in special cases).

The following examples use the `forEach` method as an example, but other methods that visit indexes in ascending order work in the same way. We will first define a helper function:

```js
function testSideEffect(effect) {
  const arr = ["e1", "e2", "e3", "e4"];
  arr.forEach((elem, index, arr) => {
    console.log(`array: [${arr.join(", ")}], index: ${index}, elem: ${elem}`);
    effect(arr, index);
  });
  console.log(`Final array: [${arr.join(", ")}]`);
}
```

Modification to indexes not visited yet will be visible once the index is reached:

```js
testSideEffect((arr, index) => {
  if (index + 1 < arr.length) arr[index + 1] += "*";
});
// array: [e1, e2, e3, e4], index: 0, elem: e1
// array: [e1, e2*, e3, e4], index: 1, elem: e2*
// array: [e1, e2*, e3*, e4], index: 2, elem: e3*
// array: [e1, e2*, e3*, e4*], index: 3, elem: e4*
// Final array: [e1, e2*, e3*, e4*]
```

Modification to already visited indexes does not change iteration behavior, although the array will be different afterwards:

```js
testSideEffect((arr, index) => {
  if (index > 0) arr[index - 1] += "*";
});
// array: [e1, e2, e3, e4], index: 0, elem: e1
// array: [e1, e2, e3, e4], index: 1, elem: e2
// array: [e1*, e2, e3, e4], index: 2, elem: e3
// array: [e1*, e2*, e3, e4], index: 3, elem: e4
// Final array: [e1*, e2*, e3*, e4]
```

Inserting *n* elements at unvisited indexes that are less than the initial array length will make them be visited. The last *n* elements in the original array that now have index greater than the initial array length will not be visited:

```js
testSideEffect((arr, index) => {
  if (index === 1) arr.splice(2, 0, "new");
});
// array: [e1, e2, e3, e4], index: 0, elem: e1
// array: [e1, e2, e3, e4], index: 1, elem: e2
// array: [e1, e2, new, e3, e4], index: 2, elem: new
// array: [e1, e2, new, e3, e4], index: 3, elem: e3
// Final array: [e1, e2, new, e3, e4]
// e4 is not visited because it now has index 4
```

Inserting *n* elements with index greater than the initial array length will not make them be visited:

```js
testSideEffect((arr) => arr.push("new"));
// array: [e1, e2, e3, e4], index: 0, elem: e1
// array: [e1, e2, e3, e4, new], index: 1, elem: e2
// array: [e1, e2, e3, e4, new, new], index: 2, elem: e3
// array: [e1, e2, e3, e4, new, new, new], index: 3, elem: e4
// Final array: [e1, e2, e3, e4, new, new, new, new]
```

Inserting *n* elements at already visited indexes will not make them be visited, but it shifts remaining elements back by *n*, so the current index and the *n - 1* elements before it are visited again:

```js
testSideEffect((arr, index) => arr.splice(index, 0, "new"));
// array: [e1, e2, e3, e4], index: 0, elem: e1
// array: [new, e1, e2, e3, e4], index: 1, elem: e1
// array: [new, new, e1, e2, e3, e4], index: 2, elem: e1
// array: [new, new, new, e1, e2, e3, e4], index: 3, elem: e1
// Final array: [new, new, new, new, e1, e2, e3, e4]
// e1 keeps getting visited because it keeps getting shifted back
```

Deleting *n* elements at unvisited indexes will make them not be visited anymore. Because the array has shrunk, the last *n* iterations will visit out-of-bounds indexes. If the method ignores non-existent indexes (see array methods and empty slots), the last *n* iterations will be skipped; otherwise, they will receive `undefined`:

```js
testSideEffect((arr, index) => {
  if (index === 1) arr.splice(2, 1);
});
// array: [e1, e2, e3, e4], index: 0, elem: e1
// array: [e1, e2, e3, e4], index: 1, elem: e2
// array: [e1, e2, e4], index: 2, elem: e4
// Final array: [e1, e2, e4]
// Does not visit index 3 because it's out-of-bounds

// Compare this with find(), which treats nonexistent indexes as undefined:
const arr2 = ["e1", "e2", "e3", "e4"];
arr2.find((elem, index, arr) => {
  console.log(`array: [${arr.join(", ")}], index: ${index}, elem: ${elem}`);
  if (index === 1) arr.splice(2, 1);
  return false;
});
// array: [e1, e2, e3, e4], index: 0, elem: e1
// array: [e1, e2, e3, e4], index: 1, elem: e2
// array: [e1, e2, e4], index: 2, elem: e4
// array: [e1, e2, e4], index: 3, elem: undefined
```

Deleting *n* elements at already visited indexes does not change the fact that they were visited before they get deleted. Because the array has shrunk, the next *n* elements after the current index are skipped. If the method ignores non-existent indexes, the last *n* iterations will be skipped; otherwise, they will receive `undefined`:

```js
testSideEffect((arr, index) => arr.splice(index, 1));
// array: [e1, e2, e3, e4], index: 0, elem: e1
// Does not visit e2 because e2 now has index 0, which has already been visited
// array: [e2, e3, e4], index: 1, elem: e3
// Does not visit e4 because e4 now has index 1, which has already been visited
// Final array: [e2, e4]
// Index 2 is out-of-bounds, so it's not visited

// Compare this with find(), which treats nonexistent indexes as undefined:
const arr2 = ["e1", "e2", "e3", "e4"];
arr2.find((elem, index, arr) => {
  console.log(`array: [${arr.join(", ")}], index: ${index}, elem: ${elem}`);
  arr.splice(index, 1);
  return false;
});
// array: [e1, e2, e3, e4], index: 0, elem: e1
// array: [e2, e3, e4], index: 1, elem: e3
// array: [e2, e4], index: 2, elem: undefined
// array: [e2, e4], index: 3, elem: undefined
```

For methods that iterate in descending order of index, insertion causes elements to be skipped, and deletion causes elements to be visited multiple times. Adjust the code above yourself to see the effects.

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/indexed-collections.html#sec-array-objects

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`Array`	1	12	1	4	1	18	4	10.1	1	1.0	1	1	1.0.0	1.0	0.10.0
[Symbol.iterator]	38	12	36[1,2,3,4]	25	10	38	36[1,2,3,4]	25	10	3.0	38	10	1.0.0	1.0	0.12.0
[Symbol.species]	51	79	48	38	10	51	48	41	10	5.0	51	10	1.0.0	1.0	6.5.0
[Symbol.unscopables]	38	12	48	25	10	38	48	25	10	3.0	38	10	1.0.0	1.0	0.12.0
`Array()` constructor	1	12	1	4	1	18	4	10.1	1	1.0	1	1	1.0.0	1.0	0.10.0
`at`	92	92	90	78	15.4	92	90	65	15.4	16.0	92	15.4	1.0.0	1.12	16.6.0
`concat`	1	12	1	4	1	18	4	10.1	1	1.0	1	1	1.0.0	1.0	0.10.0
`copyWithin`	45	12	32	32	9	45	32	32	9	5.0	45	9	1.0.0	1.0	4.0.0
`entries`	38	12	28	25	8	38	28	25	8	3.0	38	8	1.0.0	1.0	0.12.0
`every`	1	12	1.5	9.5	3	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`fill`	45	12	31	32	8	45	31	32	8	5.0	45	8	1.0.0	1.0	4.0.0
`filter`	1	12	1.5	9.5	3	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`find`	45	12	25	32	8	45	4	32	8	5.0	45	8	1.0.0	1.0	4.0.0
`findIndex`	45	12	25	32	8	45	4	32	8	5.0	45	8	1.0.0	1.0	4.0.0
`findLast`	97	97	104	83	15.4	97	104	68	15.4	18.0	97	15.4	1.0.0	1.16	18.0.0
`findLastIndex`	97	97	104	83	15.4	97	104	68	15.4	18.0	97	15.4	1.0.0	1.16	18.0.0
`flat`	69	79	62	56	12	69	62	48	12	10.0	69	12	1.0.0	1.0	11.0.0
`flatMap`	69	79	62	56	12	69	62	48	12	10.0	69	12	1.0.0	1.0	11.0.0
`forEach`	1	12	1.5	9.5	3	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`from`	45	12	32	32	9	45	32	32	9	5.0	45	9	1.0.0	1.0	4.0.0
`fromAsync`	121	121	115	107	16.4	121	115	81	16.4	25.0	121	16.4	1.0.0	1.38	22.0.0
`includes`	47	14	43	34	9	47	43	34	9	5.0	47	9	1.0.0	1.0	6.0.0
`indexOf`	1	12	1.5	9.5	3	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`isArray`	4	12	4	10.5	5	18	4	14	5	1.0	4.4	5	1.0.0	1.0	0.10.0
`join`	1	12	1	4	1	18	4	10.1	1	1.0	1	1	1.0.0	1.0	0.10.0
`keys`	38	12	28	25	8	38	28	25	8	3.0	38	8	1.0.0	1.0	0.12.0
`lastIndexOf`	1	12	1.5	9.5	3	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`length`	1	12	1	4	1	18	4	10.1	1	1.0	1	1	1.0.0	1.0	0.10.0
`map`	1	12	1.5	9.5	3	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`of`	45	12	25	26	9	39	25	26	9	4.0	39	9	1.0.0	1.0	4.0.0
`pop`	1	12	1	4	1	18	4	10.1	1	1.0	1	1	1.0.0	1.0	0.10.0
`push`	1	12	1	4	1	18	4	10.1	1	1.0	1	1	1.0.0	1.0	0.10.0
`reduce`	3	12	3	10.5	4	18	4	14	3.2	1.0	4.4	3.2	1.0.0	1.0	0.10.0
`reduceRight`	3	12	3	10.5	4	18	4	14	3.2	1.0	4.4	3.2	1.0.0	1.0	0.10.0
`reverse`	1	12	1	4	1	18	4	10.1	1	1.0	1	1	1.0.0	1.0	0.10.0
`shift`	1	12	1	4	1	18	4	10.1	1	1.0	1	1	1.0.0	1.0	0.10.0
`slice`	1	12	1	4	1	18	4	10.1	1	1.0	1	1	1.0.0	1.0	0.10.0
`some`	1	12	1.5	9.5	3	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`sort`	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Stable sorting	70	79	3	57	10.1	70	4	49	10.3	10.0	70	10.3	1.0.0	1.0	12.0.0

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`splice`	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`toLocaleString`	1	12	1	4	1	18	4	10.1	1	1.0	1	1	1.0.0	1.0	0.10.0
`locales` parameter	24	79	52	15	7	25	56	14	7	2.0	4.4	7	1.0.0	1.8[5,6]	13.0.0[7,8]
`options` parameter	24	79	52	15	7	25	56	14	7	2.0	4.4	7	1.0.0	1.0	0.12.0
`toReversed`	110	110	115	96	16	110	115	74	16	21.0	110	16	1.0.0	1.31	20.0.0
`toSorted`	110	110	115	96	16	110	115	74	16	21.0	110	16	1.0.0	1.31	20.0.0
`toSpliced`	110	110	115	96	16	110	115	74	16	21.0	110	16	1.0.0	1.31	20.0.0
`toString`	1	12	1	4	1	18	4	10.1	1	1.0	1	1	1.0.0	1.0	0.10.0
`unshift`	1	12	1	4	1	18	4	10.1	1	1.0	1	1	1.0.0	1.0	0.10.0
`values`	66	14	60	53	9	66	60	47	9	9.0	66	9	1.0.0	1.0	10.9.0[9]
`with`	110	110	115	96	16	110	115	74	16	21.0	110	16	1.0.0	1.31	20.0.0

1. Also from version 27–36: uses the non-standard name `@@iterator`.
2. Also from version 27–36: A placeholder property named `@@iterator` is used.
3. Also from version 17–27: uses the non-standard name `iterator`.
4. Also from version 17–27: A placeholder property named `iterator` is used.
5. Also from version 1.0–1.8: partial support.
6. Also from version 1.0–1.8: Only the locale data for `en-US` is available.
7. Also from version 0.12.0–13.0.0: partial support.
8. Also from version 0.12.0–13.0.0: Before version 13.0.0, only the locale data for `en-US` is available by default. When other locales are specified, the function silently falls back to `en-US`. To make full ICU (locale) data available before version 13, see Node.js documentation on the `--with-intl` option and how to provide the data.
9. Also from version 0.12.0–4.0.0: supported.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Array`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Indexed collections guide
- `TypedArray`
- `ArrayBuffer`

JavaScript › Reference › Standard built-in objects › Array

Array() constructor

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/Array · MDN page last modified 2026-10-08

Baseline: Widely available — available across major browsers since July 2015.

The **`Array()`** constructor creates `Array` objects.

Syntax

```js
new Array()
new Array(element1)
new Array(element1, element2)
new Array(element1, element2, /* …, */ elementN)
new Array(arrayLength)

Array()
Array(element1)
Array(element1, element2)
Array(element1, element2, /* …, */ elementN)
Array(arrayLength)
```

> **Note:** `Array()` can be called with or without new. Both create a new `Array` instance.

Parameters

`element1, ..., elementN`

A JavaScript array is initialized with the given elements, except in the case where a single argument is passed to the `Array` constructor and that argument is a number (see the `arrayLength` parameter below). Note that this special case only applies to JavaScript arrays created with the `Array` constructor, not array literals created with the square bracket syntax.

`arrayLength`

If the only argument passed to the `Array` constructor is an integer between 0 and 2^{32} - 1 (inclusive), this returns a new JavaScript array with its `length` property set to that number.

> **Note:** This implies an array of `arrayLength` empty slots, not slots with actual `undefined` values — see sparse arrays.

Exceptions

`RangeError`

Thrown if there's only one argument (`arrayLength`) that is a number, but its value is not an integer or not between 0 and 2^{32} - 1 (inclusive).

Examples

Array literal notation

Arrays can be created using the literal notation:

```js
const fruits = ["Apple", "Banana"];

console.log(fruits.length); // 2
console.log(fruits[0]); // "Apple"
```

Array constructor with a single parameter

Arrays can be created using a constructor with a single number parameter. An array is created with its `length` property set to that number, and the array elements are empty slots.

```js
const arrayEmpty = new Array(2);

console.log(arrayEmpty.length); // 2
console.log(arrayEmpty[0]); // undefined; actually, it is an empty slot
console.log(0 in arrayEmpty); // false
console.log(1 in arrayEmpty); // false
```

```js
const arrayOfOne = new Array("2"); // Not the number 2 but the string "2"

console.log(arrayOfOne.length); // 1
console.log(arrayOfOne[0]); // "2"
```

Array constructor with multiple parameters

If more than one argument is passed to the constructor, a new `Array` with the given elements is created.

```js
const fruits = new Array("Apple", "Banana");

console.log(fruits.length); // 2
console.log(fruits[0]); // "Apple"
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/indexed-collections.html#sec-array-constructor`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`Array()` constructor	1	12	1	4	1	18	4	10.1	1	1.0	1	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Array.Array`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Indexed collections guide
- `Array`

JavaScript › Reference › Standard built-in objects › Array

Array.from()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/from · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since September 2015.

The **`Array.from()`** static method creates a new, shallow-copied `Array` instance from an iterable or array-like object.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Array.from()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
console.log(Array.from("foo"));
// Expected output: Array ["f", "o", "o"]

console.log(Array.from([1, 2, 3], (x) => x + x));
// Expected output: Array [2, 4, 6]
```

Syntax

js

```
Array.from(items)
Array.from(items, mapFn)
Array.from(items, mapFn, thisArg)
```

Parameters

`items`

An iterable or array-like object to convert to an array.

`mapFn` Optional

A function to call on every element of the array. If provided, every value to be added to the array is first passed through this function, and `mapFn`'s return value is added to the array instead. The function is called with the following arguments:

`element`

The current element being processed in the array.

`index`

The index of the current element being processed in the array.

`thisArg` Optional

Value to use as `this` when executing `mapFn`.

Return value

A new `Array` instance.

Description

`Array.from()` lets you create `Array`s from:

- iterable objects (objects such as `Map` and `Set`); or, if the object is not iterable,

- array-like objects (objects with a `length` property and indexed elements).

To convert an ordinary object that's not iterable or array-like to an array (by enumerating its property keys, values, or both), use `Object.keys()`, `Object.values()`, or `Object.entries()`. To convert an async iterable to an array, use `Array.fromAsync()`.

`Array.from()` never creates a sparse array. If the `items` object is missing some index properties, they become `undefined` in the new array.

`Array.from()` has an optional parameter `mapFn`, which allows you to execute a function on each element of the array being created, similar to `map()`. More clearly, `Array.from(obj, mapFn, thisArg)` has the same result as `Array.from(obj).map(mapFn, thisArg)`, except that it does not create an intermediate array, and `mapFn` only receives two arguments (`element`, `index`) without the whole array, because the array is still under construction.

> **Note:** This behavior is more important for typed arrays, since the intermediate array would necessarily have values truncated to fit into the appropriate type. `Array.from()` is implemented to have the same signature as `TypedArray.from()`.

The `Array.from()` method is a generic factory method. For example, if a subclass of `Array` inherits the `from()` method, the inherited `from()` method will return new instances of the subclass instead of `Array` instances. In fact, the `this` value can be any constructor function that accepts a single argument representing the length of the new array. When an iterable is passed as `items`, the constructor is called with no arguments; when an array-like object is passed, the constructor is called with the normalized length of the array-like object. The final `length` will be set again when iteration finishes. If the `this` value is not a constructor function, the plain `Array` constructor is used instead.

Examples

Array from a String

```js
Array.from("foo");
// [ "f", "o", "o" ]
```

Array from a Set

```js
const set = new Set(["foo", "bar", "baz", "foo"]);
Array.from(set);
// [ "foo", "bar", "baz" ]
```

Array from a Map

```js
const map = new Map([
  [1, 2],
  [2, 4],
  [4, 8],
]);
Array.from(map);
// [[1, 2], [2, 4], [4, 8]]
```

(code continues on the next page)

(code continued from the previous page)

```
const mapper = new Map([
  ["1", "a"],
  ["2", "b"],
]);
Array.from(mapper.values());
// ['a', 'b'];

Array.from(mapper.keys());
// ['1', '2'];
```

Array from a NodeList

js

```
// Create an array based on a property of DOM Elements
const images = document.querySelectorAll("img");
const sources = Array.from(images, (image) => image.src);
const insecureSources = sources.filter((link) => link.startsWith("http://"));
```

Array from an Array-like object (arguments)

js

```
function f() {
  return Array.from(arguments);
}

f(1, 2, 3);

// [ 1, 2, 3 ]
```

Using arrow functions and Array.from()

js

```
// Using an arrow function as the map function to
// manipulate the elements
Array.from([1, 2, 3], (x) => x + x);
// [2, 4, 6]

// Generate a sequence of numbers
// Since the array is initialized with `undefined` on each position,
// the value of `v` below will be `undefined`
Array.from({ length: 5 }, (v, i) => i);
// [0, 1, 2, 3, 4]
```

Sequence generator (range)

js

```
// Sequence generator function (commonly referred to as "range", cf. Python, Clojure, etc.)
const range = (start, stop, step) =>
  Array.from(
    { length: Math.ceil((stop - start) / step) },
    (_, i) => start + i * step,
  );
```

(code continues on the next page)

(code continued from the previous page)

```
// Generate a sequence of numbers from 0 (inclusive) to 5 (exclusive), incrementing by 1
range(0, 5, 1);
// [0, 1, 2, 3, 4]

// Generate a sequence of numbers from 1 (inclusive) to 10 (exclusive), incrementing by 2
range(1, 10, 2);
// [1, 3, 5, 7, 9]

// Generate the Latin alphabet making use of it being ordered as a sequence
range("A".charCodeAt(0), "Z".charCodeAt(0) + 1, 1).map((x) =>
  String.fromCharCode(x),
);
// ["A", "B", "C", "D", "E", "F", "G", "H", "I", "J", "K", "L", "M", "N", "O", "P", "Q",
"R", "S", "T", "U", "V", "W", "X", "Y", "Z"]
```

Calling from() on non-array constructors

The `from()` method can be called on any constructor function that accepts a single argument representing the length of the new array.

js

```
function NotArray(len) {
  console.log("NotArray called with length", len);
}

// Iterable
console.log(Array.from.call(NotArray, new Set(["foo", "bar", "baz"])));
// NotArray called with length undefined
// NotArray { '0': 'foo', '1': 'bar', '2': 'baz', length: 3 }

// Array-like
console.log(Array.from.call(NotArray, { length: 1, 0: "foo" }));
// NotArray called with length 1
// NotArray { '0': 'foo', length: 1 }
```

When the `this` value is not a constructor, a plain `Array` object is returned.

js

```
console.log(Array.from.call({}, { length: 1, 0: "foo" })); // [ 'foo' ]
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/indexed-collections.html#sec-array.from

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
from	45	12	32	32	9	45	32	32	9	5.0	45	9	1.0.0	1.0	4.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Array.from`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Polyfill of `Array.from` in `core-js`
- es-shims polyfill of `Array.from`
- Indexed collections guide
- `Array`
- `Array()`
- `Array.of()`
- `Array.fromAsync()`
- `Array.prototype.map()`
- `TypedArray.from()`

JavaScript › Reference › Standard built-in objects › Array

Array.fromAsync()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/fromAsync · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since January 2024.

The **`Array.fromAsync()`** static method creates a new, shallow-copied `Array` instance from an async iterable, iterable, or array-like object.

Syntax

```js
Array.fromAsync(items)
Array.fromAsync(items, mapFn)
Array.fromAsync(items, mapFn, thisArg)
```

Parameters

`items`

An async iterable, iterable, or array-like object to convert to an array.

`mapFn` Optional

A function to call on every element of the array. If provided, every value to be added to the array is first passed through this function, and `mapFn`'s return value is added to the array instead (after being awaited). The function is called with the following arguments:

`element`

The current element being processed in the array. If `items` is a sync iterable or array-like object, then all elements are first awaited, and `element` will never be a thenable. If `items` is an async iterable, then each yielded value is passed as-is.

`index`

The index of the current element being processed in the array.

`thisArg` Optional

Value to use as `this` when executing `mapFn`.

Return value

A new `Promise` whose fulfillment value is a new `Array` instance.

Description

`Array.fromAsync()` lets you create arrays from:

- async iterable objects (objects such as `ReadableStream` and `AsyncGenerator`); or, if the object is not async iterable,
- iterable objects (objects such as `Map` and `Set`); or, if the object is not iterable,
- array-like objects (objects with a `length` property and indexed elements).

`Array.fromAsync()` iterates the async iterable in a fashion very similar to `for await...of`. `Array.fromAsync(items)` is generally equivalent to the following code, if `items` is an async iterable or sync iterable:

```js
const result = [];
for await (const element of items) {
  result.push(element);
}
```

`Array.fromAsync()` is almost equivalent to `Array.from()` in terms of behavior, except the following:

- `Array.fromAsync()` handles async iterable objects.
- `Array.fromAsync()` returns a `Promise` that fulfills to the array instance.
- If `Array.fromAsync()` is called with a non-async iterable object, each element to be added to the array is first awaited.
- If a `mapFn` is provided, its output is also internally awaited.

`Array.fromAsync()` and `Promise.all()` can both turn an iterable of promises into a promise of an array. However, there are two key differences:

- `Array.fromAsync()` awaits each value yielded from the object sequentially. `Promise.all()` awaits all values concurrently.
- `Array.fromAsync()` iterates the iterable lazily, and doesn't retrieve the next value until the current one is settled. `Promise.all()` retrieves all values in advance and awaits them all.

Examples

Array from an async iterable

```js
const asyncIterable = (async function* () {
  for (let i = 0; i < 5; i++) {
    await new Promise((resolve) => setTimeout(resolve, 10 * i));
    yield i;
  }
})();

Array.fromAsync(asyncIterable).then((array) => console.log(array));
// [0, 1, 2, 3, 4]
```

When `items` is an async iterable where each result's `value` is also a promise, then those promises are added to the resulting array without being awaited. This is consistent with the behavior of `for await...of`.

```js
function createAsyncIter() {
  let i = 0;
  return {
    [Symbol.asyncIterator]() {
      return {
        async next() {
          if (i > 2) return { done: true };
          i++;
          return { value: Promise.resolve(i), done: false };
        },
      };
    },
  };
}

Array.fromAsync(createAsyncIter()).then((array) => console.log(array));
// (3) [Promise, Promise, Promise]
```

Note: In practice, you will rarely encounter an async iterable that yields promises, because if you implement it using an async generator function, then the `yield` expression automatically unwraps promises.

Array from a sync iterable

```js
Array.fromAsync(
  new Map([
    [1, 2],
    [3, 4],
  ]),
).then((array) => console.log(array));
// [[1, 2], [3, 4]]
```

Array from a sync iterable that yields promises

```js
Array.fromAsync(
  new Set([Promise.resolve(1), Promise.resolve(2), Promise.resolve(3)]),
).then((array) => console.log(array));
// [1, 2, 3]
```

Array from an array-like object of promises

```js
Array.fromAsync({
  length: 3,
  0: Promise.resolve(1),
  1: Promise.resolve(2),
  2: Promise.resolve(3),
}).then((array) => console.log(array));
// [1, 2, 3]
```

Using mapFn with a sync iterable

When `items` is a sync iterable or array-like object, both the input and output of `mapFn` are awaited internally by `Array.fromAsync()`.

```js
function delayedValue(v) {
  return new Promise((resolve) => setTimeout(() => resolve(v), 100));
}

Array.fromAsync(
  [delayedValue(1), delayedValue(2), delayedValue(3)],
  (element) => delayedValue(element * 2),
).then((array) => console.log(array));
// [2, 4, 6]
```

Using mapFn with an async iterable

When `items` is an async iterable, the input to `mapFn` is not awaited, but the output is. Using the same `createAsyncIter` function as above:

```js
Array.fromAsync(createAsyncIter(), async (element) => (await element) * 2).then(
  (array) => console.log(array),
);
// [2, 4, 6]
```

Curiously, this means that `Array.fromAsync(createAsyncIter())` is not equivalent to `Array.fromAsync(createAsyncIter(), (element) => element)`, because the latter awaits each yielded value, while the former does not.

```js
Array.fromAsync(createAsyncIter(), (element) => element).then((array) =>
  console.log(array),
);
// [1, 2, 3]
```

Comparison with Promise.all()

`Array.fromAsync()` awaits each value yielded from the object sequentially. `Promise.all()` awaits all values concurrently.

```js
function* makeIterableOfPromises() {
  for (let i = 0; i < 5; i++) {
    yield new Promise((resolve) => setTimeout(resolve, 100));
  }
}

(async () => {
  console.time("Array.fromAsync() time");
  await Array.fromAsync(makeIterableOfPromises());
  console.timeEnd("Array.fromAsync() time");
  // Array.fromAsync() time: 503.610ms
```

(code continues on the next page)

(code continued from the previous page)

```
  console.time("Promise.all() time");
  await Promise.all(makeIterableOfPromises());
  console.timeEnd("Promise.all() time");
  // Promise.all() time: 101.728ms
})();
```

Closing sync iterables on rejection

Similar to `for await...of`, if the object being iterated is a sync iterable, and a yielded promise rejects, the `return()` method of the underlying iterator will be called, if one exists, to allow the iterator to perform cleanup.

js

```
function* generatorWithRejectedPromises() {
  try {
    yield 0;
    yield Promise.reject(new Error("error"));
  } finally {
    console.log("called finally");
  }
}

(async () => {
  try {
    await Array.fromAsync(generatorWithRejectedPromises());
  } catch (e) {
    console.log("caught", e);
  }
})();
// called finally
// caught Error: error
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/indexed-collections.html#sec-array.fromasync`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`fromAsync`	121	121	115	107	16.4	121	115	81	16.4	25.0	121	16.4	1.0.0	1.38	22.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Array.fromAsync`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Polyfill of `Array.fromAsync` in `core-js`
- Indexed collections guide

- Array
- Array()
- Array.of()
- Array.from()
- for await...of

JavaScript › Reference › Standard built-in objects › Array

Array.isArray()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/isArray · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since July 2015.

The **Array.isArray()** static method determines whether the passed value is an Array.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Array.isArray()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
console.log(Array.isArray([1, 3, 5]));
// Expected output: true

console.log(Array.isArray("[]"));
// Expected output: false

console.log(Array.isArray(new Array(5)));
// Expected output: true

console.log(Array.isArray(new Int16Array([15, 33])));
// Expected output: false
```

Syntax

js

```
Array.isArray(value)
```

Parameters

value

The value to be checked.

Return value

`true` if `value` is an Array; otherwise, `false`. `false` is always returned if `value` is a TypedArray instance.

Description

`Array.isArray()` checks if the passed value is an Array. It performs a *branded check*, similar to the `in` operator, for a private field initialized by the `Array()` constructor.

It is a more robust alternative to `instanceof Array` because it avoids false positives and false negatives:

- `Array.isArray()` rejects values that aren't actual `Array` instances, even if they have `Array.prototype` in their prototype chain — `instanceof Array` would accept these as it does check

the prototype chain.

- `Array.isArray()` accepts `Array` objects constructed in another realm — `instanceof Array` returns `false` for these because the identity of the `Array` constructor is different across realms.

See the article "Determining with absolute accuracy whether or not a JavaScript object is an array" for more details.

Examples

Using Array.isArray()

```js
// all following calls return true
Array.isArray([]);
Array.isArray([1]);
Array.isArray(new Array());
Array.isArray(new Array("a", "b", "c", "d"));
Array.isArray(new Array(3));
// Little known fact: Array.prototype itself is an array:
Array.isArray(Array.prototype);

// all following calls return false
Array.isArray();
Array.isArray({});
Array.isArray(null);
Array.isArray(undefined);
Array.isArray(17);
Array.isArray("Array");
Array.isArray(true);
Array.isArray(false);
Array.isArray(new Uint8Array(32));
// This is not an array, because it was not created using the
// array literal syntax or the Array constructor
Array.isArray({ __proto__: Array.prototype });
```

instanceof vs. Array.isArray()

When checking for `Array` instance, `Array.isArray()` is preferred over `instanceof` because it works across realms.

```js
const iframe = document.createElement("iframe");
document.body.appendChild(iframe);
const xArray = window.frames[window.frames.length - 1].Array;
const arr = new xArray(1, 2, 3); // [1, 2, 3]

// Correctly checking for Array
Array.isArray(arr); // true
// The prototype of arr is xArray.prototype, which is a
// different object from Array.prototype
arr instanceof Array; // false
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/indexed-collections.html#sec-array.isarray`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`isArray`	4	12	4	10.5	5	18	4	14	5	1.0	4.4	5	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Array.isArray`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Array.isArray` in `core-js`
- es-shims polyfill of `Array.isArray`
- Indexed collections guide
- `Array`

JavaScript › Reference › Standard built-in objects › Array

Array.of()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/of · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since September 2015.

The **`Array.of()`** static method creates a new `Array` instance from a variable number of arguments, regardless of number or type of the arguments.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Array.of()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
console.log(Array.of("foo", 2, "bar", true));
// Expected output: Array ["foo", 2, "bar", true]

console.log(Array.of());
// Expected output: Array []
```

Syntax

js

```
Array.of()
Array.of(element1)
Array.of(element1, element2)
Array.of(element1, element2, /* …, */ elementN)
```

Parameters

`element1, ..., elementN`

Elements used to create the array.

Return value

A new `Array` instance.

Description

The difference between `Array.of()` and the `Array()` constructor is in the handling of single arguments: `Array.of(7)` creates an array with a single element, `7`, whereas `Array(7)` creates an empty array with a `length` property of `7`. (That implies an array of 7 empty slots, not slots with actual `undefined` values.)

```js
Array.of(7); // [7]
Array(7); // array of 7 empty slots

Array.of(1, 2, 3); // [1, 2, 3]
Array(1, 2, 3); // [1, 2, 3]
```

The `Array.of()` method is a generic factory method. For example, if a subclass of `Array` inherits the `of()` method, the inherited `of()` method will return new instances of the subclass instead of `Array` instances. In fact, the `this` value can be any constructor function that accepts a single argument representing the length of the new array, and the constructor will be called with the number of arguments passed to `of()`. The final `length` will be set again when all elements are assigned. If the `this` value is not a constructor function, the plain `Array` constructor is used instead.

Examples

Using Array.of()

```js
Array.of(1); // [1]
Array.of(1, 2, 3); // [1, 2, 3]
Array.of(undefined); // [undefined]
```

Calling of() on non-array constructors

The `of()` method can be called on any constructor function that accepts a single argument representing the length of the new array.

```js
function NotArray(len) {
  console.log("NotArray called with length", len);
}

console.log(Array.of.call(NotArray, 1, 2, 3));
// NotArray called with length 3
// NotArray { '0': 1, '1': 2, '2': 3, length: 3 }

console.log(Array.of.call(Object)); // [Number: 0] { length: 0 }
```

When the `this` value is not a constructor, a plain `Array` object is returned.

```js
console.log(Array.of.call({}, 1)); // [ 1 ]
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/indexed-collections.html#sec-array.of

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
of	45	12	25	26	9	39	25	26	9	4.0	39	9	1.0.0	1.0	4.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Array.of`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Array.of` in `core-js`
- es-shims polyfill of `Array.of`
- Indexed collections guide
- `Array`
- `Array()`
- `Array.from()`
- `TypedArray.of()`

JavaScript › Reference › Standard built-in objects › Array

Array[Symbol.species]

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/Symbol.species · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since January 2020.

The **`Array[Symbol.species]`** static accessor property returns the constructor used to construct return values from array methods.

> **Warning:** The existence of `[Symbol.species]` allows execution of arbitrary code and may create security vulnerabilities. It also makes certain optimizations much harder. Engine implementers are investigating whether to remove this feature. Avoid relying on it if possible. Modern array methods, such as `toReversed()`, do not use `[Symbol.species]` and always return a new `Array` base class instance.

Syntax

```js
Array[Symbol.species]
```

Return value

The value of the constructor (`this`) on which `get [Symbol.species]` was called. The return value is used to construct return values from array methods that create new arrays.

Description

The `[Symbol.species]` accessor property returns the default constructor for `Array` objects. Subclass constructors may override it to change the constructor assignment. The default implementation is basically:

```js
// Hypothetical underlying implementation for illustration
class Array {
  static get [Symbol.species]() {
    return this;
  }
}
```

Because of this polymorphic implementation, `[Symbol.species]` of derived subclasses would also return the constructor itself by default.

```js
class SubArray extends Array {}
SubArray[Symbol.species] === SubArray; // true
```

When calling array methods that do not mutate the existing array but return a new array instance (for example, `filter()` and `map()`), the array's `constructor[Symbol.species]` will be accessed. The returned constructor will be used to construct the return value of the array method. This makes it technically possible to make array methods return objects unrelated to arrays.

```js
class NotAnArray {
  constructor(length) {
    this.length = length;
  }
}

const arr = [0, 1, 2];
arr.constructor = { [Symbol.species]: NotAnArray };
arr.map((i) => i); // NotAnArray { '0': 0, '1': 1, '2': 2, length: 3 }
arr.filter((i) => i); // NotAnArray { '0': 1, '1': 2, length: 0 }
arr.concat([1, 2]); // NotAnArray { '0': 0, '1': 1, '2': 2, '3': 1, '4': 2, length: 5 }
```

Examples

Species in ordinary objects

The `[Symbol.species]` property returns the default constructor function, which is the `Array` constructor for `Array`.

```js
Array[Symbol.species]; // [Function: Array]
```

Species in derived objects

In an instance of a custom `Array` subclass, such as `MyArray`, the `MyArray` species is the `MyArray` constructor. However, you might want to overwrite this, in order to return parent `Array` objects in your derived class methods:

```js
class MyArray extends Array {
  // Overwrite MyArray species to the parent Array constructor
  static get [Symbol.species]() {
    return Array;
  }
}
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/indexed-collections.html#sec-get-array-%symbol.species%

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
[Symbol.species]	51	79	48	38	10	51	48	41	10	5.0	51	10	1.0.0	1.0	6.5.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Array.@@species`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Polyfill of `Array[Symbol.species]` and support of `[Symbol.species]` in all affected `Array` methods in `core-js`
- Indexed collections guide
- `Array`
- `Symbol.species`

JavaScript › Reference › Standard built-in objects › Array

Array.prototype.at()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/at · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since March 2022.

The **`at()`** method of `Array` instances takes an integer value and returns the item at that index, allowing for positive and negative integers. Negative integers count back from the last item in the array.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Array.prototype.at()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const array = [5, 12, 8, 130, 44];

let index = 2;

console.log(`An index of ${index} returns ${array.at(index)}`);
// Expected output: "An index of 2 returns 8"

index = -2;

console.log(`An index of ${index} returns ${array.at(index)}`);
// Expected output: "An index of -2 returns 130"
```

Syntax

js

```
at(index)
```

Parameters

`index`

Zero-based index of the array element to be returned, converted to an integer. Negative index counts back from the end of the array — if `index < 0`, `index + array.length` is accessed.

Return value

The element in the array matching the given index. Always returns `undefined` if `index < -array.length` or `index >= array.length` without attempting to access the corresponding property.

Description

The `at()` method is equivalent to the bracket notation when `index` is a non-negative integer. For example, `array[0]` and `array.at(0)` both return the first item. However, when counting elements from the end of the array, you cannot use `array[-1]` like you may in Python or R, because all values inside the square brackets are treated literally as string properties, so you will end up reading `array["-1"]`, which is just a normal string property instead of an array index.

The usual practice is to access `length` and calculate the index from that — for example, `array[array.length - 1]`. The `at()` method allows relative indexing, so this can be shortened to `array.at(-1)`.

By combining `at()` with `with()`, you can both read and write (respectively) an array using negative indices.

The `at()` method is generic. It only expects the `this` value to have a `length` property and integer-keyed properties.

Examples

Return the last value of an array

The following example provides a function which returns the last element found in a specified array.

```js
// Our array with items
const cart = ["apple", "banana", "pear"];

// A function which returns the last item of a given array
function returnLast(arr) {
  return arr.at(-1);
}

// Get the last item of our array 'cart'
const item1 = returnLast(cart);
console.log(item1); // 'pear'

// Add an item to our 'cart' array
cart.push("orange");
const item2 = returnLast(cart);
console.log(item2); // 'orange'
```

Comparing methods

This example compares different ways to select the penultimate (last but one) item of an `Array`. While all the methods shown below are valid, this example highlights the succinctness and readability of the `at()` method.

```js
// Our array with items
const colors = ["red", "green", "blue"];

// Using length property
const lengthWay = colors[colors.length - 2];
console.log(lengthWay); // 'green'

// Using slice() method. Note an array is returned
const sliceWay = colors.slice(-2, -1);
console.log(sliceWay[0]); // 'green'

// Using at() method
const atWay = colors.at(-2);
console.log(atWay); // 'green'
```

Calling at() on non-array objects

The `at()` method reads the `length` property of `this` and calculates the index to access.

```js
const arrayLike = {
  length: 2,
  0: "a",
  1: "b",
  2: "c", // ignored by at() since length is 2
};
console.log(Array.prototype.at.call(arrayLike, 0)); // "a"
console.log(Array.prototype.at.call(arrayLike, 2)); // undefined
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/indexed-collections.html#sec-array.prototype.at`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`at`	92	92	90	78	15.4	92	90	65	15.4	16.0	92	15.4	1.0.0	1.12	16.6.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Array.at`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Array.prototype.at` in `core-js`
- es-shims polyfill of `Array.prototype.at`
- Indexed collections guide
- `Array`
- `Array.prototype.findIndex()`
- `Array.prototype.indexOf()`
- `Array.prototype.with()`
- `TypedArray.prototype.at()`
- `String.prototype.at()`

JavaScript › Reference › Standard built-in objects › Array

Array.prototype.concat()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/concat · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since July 2015.

The **`concat()`** method of `Array` instances is used to merge two or more arrays. This method does not change the existing arrays, but instead returns a new array.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Array.prototype.concat()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const array1 = ["a", "b", "c"];
const array2 = ["d", "e", "f"];
const array3 = array1.concat(array2);

console.log(array3);
// Expected output: Array ["a", "b", "c", "d", "e", "f"]
```

Syntax

js

```
concat()
concat(value1)
concat(value1, value2)
concat(value1, value2, /* …, */ valueN)
```

Parameters

`value1, ..., valueN` Optional

Arrays and/or values to concatenate into a new array. If all `valueN` parameters are omitted, `concat` returns a shallow copy of the existing array on which it is called. See the description below for more details.

Return value

A new `Array` instance.

Description

The `concat` method creates a new array. The array will first be populated by the elements in the object on which it is called. Then, for each argument, its value will be concatenated into the array — for normal objects or primitives, the argument itself will become an element of the final array; for arrays or array-like objects with the property `Symbol.isConcatSpreadable` set to a truthy value, each element of the argument will be independently added to the final array. The `concat` method does not recurse into nested array arguments.

The `concat()` method is a copying method. It does not alter `this` or any of the arrays provided as arguments but instead returns a shallow copy that contains the same elements as the ones from the original arrays.

The `concat()` method preserves empty slots if any of the source arrays is sparse.

The `concat()` method is generic. The `this` value is treated in the same way as the other arguments (except it will be converted to an object first), which means plain objects will be directly prepended to the resulting array, while array-like objects with truthy `[Symbol.isConcatSpreadable]` will be spread into the resulting array.

Examples

Concatenating two arrays

The following code concatenates two arrays:

```js
const letters = ["a", "b", "c"];
const numbers = [1, 2, 3];

const alphaNumeric = letters.concat(numbers);
console.log(alphaNumeric);
// results in ['a', 'b', 'c', 1, 2, 3]
```

Concatenating three arrays

The following code concatenates three arrays:

```js
const num1 = [1, 2, 3];
const num2 = [4, 5, 6];
const num3 = [7, 8, 9];

const numbers = num1.concat(num2, num3);

console.log(numbers);
// results in [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Concatenating values to an array

The following code concatenates three values to an array:

```js
const letters = ["a", "b", "c"];

const alphaNumeric = letters.concat(1, [2, 3]);

console.log(alphaNumeric);
// results in ['a', 'b', 'c', 1, 2, 3]
```

Concatenating nested arrays

The following code concatenates nested arrays and demonstrates retention of references:

```js
const num1 = [[1]];
const num2 = [2, [3]];

const numbers = num1.concat(num2);

console.log(numbers);
// results in [[1], 2, [3]]

// modify the first element of num1
num1[0].push(4);

console.log(numbers);
// results in [[1, 4], 2, [3]]
```

Concatenating array-like objects with Symbol.isConcatSpreadable

`concat` does not treat all array-like objects as arrays by default — only if `Symbol.isConcatSpreadable` is set to a truthy value (e.g., `true`).

```js
const obj1 = { 0: 1, 1: 2, 2: 3, length: 3 };
const obj2 = { 0: 1, 1: 2, 2: 3, length: 3, [Symbol.isConcatSpreadable]: true };
console.log([0].concat(obj1, obj2));
// [ 0, { '0': 1, '1': 2, '2': 3, length: 3 }, 1, 2, 3 ]
```

Using concat() on sparse arrays

If any of the source arrays is sparse, the resulting array will also be sparse:

```js
console.log([1, , 3].concat([4, 5])); // [1, empty, 3, 4, 5]
console.log([1, 2].concat([3, , 5])); // [1, 2, 3, empty, 5]
```

Calling concat() on non-array objects

If the `this` value is not an array, it is converted to an object and then treated in the same way as the arguments for `concat()`. In this case the return value is always a plain new array.

```js
console.log(Array.prototype.concat.call({}, 1, 2, 3)); // [{}, 1, 2, 3]
console.log(Array.prototype.concat.call(1, 2, 3)); // [ [Number: 1], 2, 3 ]
const arrayLike = {
  [Symbol.isConcatSpreadable]: true,
  length: 2,
  0: 1,
  1: 2,
  2: 99, // ignored by concat() since length is 2
};
console.log(Array.prototype.concat.call(arrayLike, 3, 4)); // [1, 2, 3, 4]
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/indexed-collections.html#sec-array.prototype.concat

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`concat`	1	12	1	4	1	18	4	10.1	1	1.0	1	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Array.concat`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Array.prototype.concat` in `core-js` with fixes and implementation of modern behavior like `Symbol.isConcatSpreadable` support
- es-shims polyfill of `Array.prototype.concat`
- Indexed collections guide
- `Array`
- `Array.prototype.push()`
- `Array.prototype.unshift()`
- `Array.prototype.splice()`
- `String.prototype.concat()`
- `Symbol.isConcatSpreadable`

JavaScript › Reference › Standard built-in objects › Array

Array.prototype.copyWithin()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/copyWithin · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since September 2015.

The **`copyWithin()`** method of `Array` instances shallow copies part of this array to another location in the same array and returns this array without modifying its length.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Array.prototype.copyWithin()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const array = ["a", "b", "c", "d", "e"];

// Copy to index 0 the element at index 3
console.log(array.copyWithin(0, 3, 4));
// Expected output: Array ["d", "b", "c", "d", "e"]

// Copy to index 1 all elements from index 3 to the end
console.log(array.copyWithin(1, 3));
// Expected output: Array ["d", "d", "e", "d", "e"]
```

Syntax

```js
copyWithin(target, start)
copyWithin(target, start, end)
```

Parameters

`target`

Zero-based index at which to copy the sequence to, converted to an integer. This corresponds to where the element at `start` will be copied to, and all elements between `start` and `end` are copied to succeeding indices.

- Negative index counts back from the end of the array — if `-array.length <= target < 0`, `target + array.length` is used.
- If `target < -array.length`, `0` is used.
- If `target >= array.length`, nothing is copied.
- If `target` is positioned after `start` after normalization, copying only happens until the end of `array.length` (in other words, `copyWithin()` never extends the array).

`start`

Zero-based index at which to start copying elements from, converted to an integer.

- Negative index counts back from the end of the array — if `-array.length <= start < 0`, `start + array.length` is used.
- If `start < -array.length`, `0` is used.
- If `start >= array.length`, nothing is copied.

`end` Optional

Zero-based index at which to end copying elements from, converted to an integer. `copyWithin()` copies up to but not including `end`.

- Negative index counts back from the end of the array — if `-array.length <= end < 0`, `end + array.length` is used.
- If `end < -array.length`, `0` is used.
- If `end >= array.length` or `end` is omitted or `undefined`, `array.length` is used, causing all elements until the end to be copied.
- If `end` implies a position before or at the position that `start` implies, nothing is copied.

Return value

The modified array.

Description

The `copyWithin()` method works like C and C++'s `memmove`, and is a high-performance method to shift the data of an `Array`. This especially applies to the `TypedArray` method of the same name. The sequence is copied and pasted as one operation; the pasted sequence will have the copied values even when the copy and paste region overlap.

Because `undefined` becomes `0` when converted to an integer, omitting the `start` parameter has the same effect as passing `0`, which copies the entire array to the target position, equivalent to a right shift

where the right boundary is clipped off and the left boundary is duplicated. This behavior may confuse readers of your code, so you should explicitly pass `0` as `start` instead.

```js
console.log([1, 2, 3, 4, 5].copyWithin(2));
// [1, 2, 1, 2, 3]; move all elements to the right by 2 positions
```

The `copyWithin()` method is a mutating method. It does not alter the length of `this`, but it will change the content of `this` and create new properties or delete existing properties, if necessary.

The `copyWithin()` method preserves empty slots. If the region to be copied from is sparse, the empty slots' corresponding new indices are deleted and also become empty slots.

The `copyWithin()` method is generic. It only expects the `this` value to have a `length` property and integer-keyed properties. Although strings are also array-like, this method is not suitable to be applied on them, as strings are immutable.

Examples

Using copyWithin()

```js
console.log([1, 2, 3, 4, 5].copyWithin(0, 3));
// [4, 5, 3, 4, 5]

console.log([1, 2, 3, 4, 5].copyWithin(0, 3, 4));
// [4, 2, 3, 4, 5]

console.log([1, 2, 3, 4, 5].copyWithin(-2, -3, -1));
// [1, 2, 3, 3, 4]
```

Using copyWithin() on sparse arrays

`copyWithin()` will propagate empty slots.

```js
console.log([1, , 3].copyWithin(2, 1, 2)); // [1, empty, empty]
```

Calling copyWithin() on non-array objects

The `copyWithin()` method reads the `length` property of `this` and then manipulates the integer indices involved.

```js
const arrayLike = {
  length: 5,
  3: 1,
};
console.log(Array.prototype.copyWithin.call(arrayLike, 0, 3));
// { '0': 1, '3': 1, length: 5 }
console.log(Array.prototype.copyWithin.call(arrayLike, 3, 1));
// { '0': 1, length: 5 }
// The '3' property is deleted because the copied source is an empty slot
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/indexed-collections.html#sec-array.prototype.copywithin

Browser compatibility

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
copyWithin	45	12	32	32	9	45	32	32	9	5.0	45	9	1.0.0	1.0	4.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Array.copyWithin`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Array.prototype.copyWithin` in `core-js`
- es-shims polyfill of `Array.prototype.copyWithin`
- Indexed collections guide
- `Array`
- `TypedArray.prototype.copyWithin()`

JavaScript › Reference › Standard built-in objects › Array

Array.prototype.entries()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/entries · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since July 2015.

The **`entries()`** method of `Array` instances returns a new *array iterator* object that contains the key/value pairs for each index in the array.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Array.prototype.entries()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const array = ["a", "b", "c"];

const iterator = array.entries();

console.log(iterator.next().value);
// Expected output: Array [0, "a"]

console.log(iterator.next().value);
// Expected output: Array [1, "b"]
```

Syntax

```js
entries()
```

Parameters

None.

Return value

A new iterable iterator object.

Description

When used on sparse arrays, the `entries()` method iterates empty slots as if they have the value `undefined`.

The `entries()` method is generic. It only expects the `this` value to have a `length` property and integer-keyed properties.

Examples

Iterating with index and element

```js
const a = ["a", "b", "c"];

for (const [index, element] of a.entries()) {
  console.log(index, element);
}

// 0 'a'
// 1 'b'
// 2 'c'
```

Using a for...of loop

```js
const array = ["a", "b", "c"];
const arrayEntries = array.entries();

for (const element of arrayEntries) {
  console.log(element);
}

// [0, 'a']
// [1, 'b']
// [2, 'c']
```

Iterating sparse arrays

`entries()` will visit empty slots as if they are `undefined`.

```js
for (const element of [, "a"].entries()) {
  console.log(element);
}
// [0, undefined]
// [1, 'a']
```

Calling entries() on non-array objects

The `entries()` method reads the `length` property of `this` and then accesses each property whose key is a nonnegative integer less than `length`.

```js
const arrayLike = {
  length: 3,
  0: "a",
  1: "b",
  2: "c",
  3: "d", // ignored by entries() since length is 3
};
for (const entry of Array.prototype.entries.call(arrayLike)) {
  console.log(entry);
}
// [ 0, 'a' ]
// [ 1, 'b' ]
// [ 2, 'c' ]
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/indexed-collections.html#sec-array.prototype.entries

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`entries`	38	12	28	25	8	38	28	25	8	3.0	38	8	1.0.0	1.0	0.12.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Array.entries`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Polyfill of `Array.prototype.entries` in `core-js`
- es-shims polyfill of `Array.prototype.entries`
- Indexed collections guide
- `Array`
- `Array.prototype.keys()`
- `Array.prototype.values()`

- `Array.prototype[Symbol.iterator]()`
- `TypedArray.prototype.entries()`
- Iteration protocols

JavaScript › Reference › Standard built-in objects › Array

Array.prototype.every()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/every · MDN page last modified 2026-10-26
Baseline: Widely available — available across major browsers since July 2015.

The **`every()`** method of `Array` instances returns `false` if it finds an element in the array that does not satisfy the provided testing function. Otherwise, it returns `true`.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Array.prototype.every()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const isBelowThreshold = (currentValue) => currentValue < 40;

const array1 = [1, 30, 39, 29, 10, 13];

console.log(array1.every(isBelowThreshold));
// Expected output: true
```

Syntax

js

```
every(callbackFn)
every(callbackFn, thisArg)
```

Parameters

`callbackFn`

A function to execute for each element in the array. It should return a truthy value to indicate the element passes the test, and a falsy value otherwise. The function is called with the following arguments:

`element`

The current element being processed in the array.

`index`

The index of the current element being processed in the array.

`array`

The array `every()` was called upon.

`thisArg` Optional

A value to use as `this` when executing `callbackFn`. See iterative methods.

Return value

`true` unless `callbackFn` returns a falsy value for an array element, in which case `false` is immediately returned.

Description

The `every()` method is an iterative method. It calls a provided `callbackFn` function once for each element in an array, until the `callbackFn` returns a falsy value. If such an element is found, `every()` immediately returns `false` and stops iterating through the array. Otherwise, if `callbackFn` returns a truthy value for all elements, `every()` returns `true`. Read the iterative methods section for more information about how these methods work in general.

`every` acts like the "for all" quantifier in mathematics. In particular, for an empty array, it returns `true`. (It is vacuously true that all elements of the empty set satisfy any given condition.)

`callbackFn` is invoked only for array indexes which have assigned values. It is not invoked for empty slots in sparse arrays.

The `every()` method is generic. It only expects the `this` value to have a `length` property and integer-keyed properties.

Examples

Testing size of all array elements

The following example tests whether all elements in the array are 10 or bigger.

```js
function isBigEnough(element, index, array) {
  return element >= 10;
}
[12, 5, 8, 130, 44].every(isBigEnough); // false
[12, 54, 18, 130, 44].every(isBigEnough); // true
```

Check if one array is a subset of another array

The following example tests if all the elements of an array are present in another array.

```js
const isSubset = (array1, array2) =>
  array2.every((element) => array1.includes(element));

console.log(isSubset([1, 2, 3, 4, 5, 6, 7], [5, 7, 6])); // true
console.log(isSubset([1, 2, 3, 4, 5, 6, 7], [5, 8, 7])); // false
```

Using the third argument of callbackFn

The `array` argument is useful if you want to access another element in the array. The following example first uses `filter()` to extract the positive values and then uses `every()` to check whether the array is strictly increasing.

```js
const numbers = [-2, 4, -8, 16, -32];
const isIncreasing = numbers
  .filter((num) => num > 0)
  .every((num, idx, arr) => {
    // Without the arr argument, there's no way to easily access the
    // intermediate array without saving it to a variable.
    if (idx === 0) return true;
    return num > arr[idx - 1];
  });
console.log(isIncreasing); // true
```

Using every() on sparse arrays

`every()` will not run its predicate on empty slots.

```js
console.log([1, , 3].every((x) => x !== undefined)); // true
console.log([2, , 2].every((x) => x === 2)); // true
```

Calling every() on non-array objects

The `every()` method reads the `length` property of `this` and then accesses each property with a nonnegative integer key less than `length` until they all have been accessed or `callbackFn` returns `false`.

```js
const arrayLike = {
  length: 3,
  0: "a",
  1: "b",
  2: "c",
  3: 345, // ignored by every() since length is 3
};
console.log(
  Array.prototype.every.call(arrayLike, (x) => typeof x === "string"),
); // true
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/indexed-collections.html#sec-array.prototype.every

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
every	1	12	1.5	9.5	3	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Array.every`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Polyfill of `Array.prototype.every` in `core-js`
- es-shims polyfill of `Array.prototype.every`
- Indexed collections guide
- `Array`
- `Array.prototype.forEach()`
- `Array.prototype.some()`
- `Array.prototype.find()`
- `TypedArray.prototype.every()`

JavaScript › Reference › Standard built-in objects › Array

Array.prototype.fill()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/fill · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since September 2015.

The **`fill()`** method of `Array` instances changes all elements within a range of indices in an array to a static value. It returns the modified array.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Array.prototype.fill()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const array = [1, 2, 3, 4];

// Fill with 0 from position 2 until position 4
console.log(array.fill(0, 2, 4));
// Expected output: Array [1, 2, 0, 0]

// Fill with 5 from position 1
console.log(array.fill(5, 1));
// Expected output: Array [1, 5, 5, 5]

console.log(array.fill(6));
// Expected output: Array [6, 6, 6, 6]
```

Syntax

js

```
fill(value)
fill(value, start)
fill(value, start, end)
```

Parameters

`value`

Value to fill the array with. Note all elements in the array will be this exact value: if `value` is an object, each slot in the array will reference that object.

`start` Optional

Zero-based index at which to start filling, converted to an integer.

- Negative index counts back from the end of the array — if `-array.length <= start < 0`, `start + array.length` is used.
- If `start < -array.length` or `start` is omitted, `0` is used.
- If `start >= array.length`, no index is filled.

`end` Optional

Zero-based index at which to end filling, converted to an integer. `fill()` fills up to but not including `end`.

- Negative index counts back from the end of the array — if `-array.length <= end < 0`, `end + array.length` is used.
- If `end < -array.length`, `0` is used.
- If `end >= array.length` or `end` is omitted or `undefined`, `array.length` is used, causing all indices until the end to be filled.
- If `end` implies a position before or at the position that `start` implies, nothing is filled.

Return value

The modified array, filled with `value`.

Description

The `fill()` method is a mutating method. It does not alter the length of `this`, but it will change the content of `this`.

The `fill()` method fills empty slots in sparse arrays with `value` as well.

The `fill()` method is generic. It only expects the `this` value to have a `length` property. Although strings are also array-like, this method is not suitable to be applied on them, as strings are immutable.

> **Note:** Using `Array.prototype.fill()` on an empty array (`length = 0`) would not modify it as the array has nothing to be modified. To use `Array.prototype.fill()` when declaring an array, make sure the array has non-zero `length`. See example.

Examples

Using fill()

```js
console.log([1, 2, 3].fill(4)); // [4, 4, 4]
console.log([1, 2, 3].fill(4, 1)); // [1, 4, 4]
console.log([1, 2, 3].fill(4, 1, 2)); // [1, 4, 3]
console.log([1, 2, 3].fill(4, 1, 1)); // [1, 2, 3]
console.log([1, 2, 3].fill(4, 3, 3)); // [1, 2, 3]
console.log([1, 2, 3].fill(4, -3, -2)); // [4, 2, 3]
console.log([1, 2, 3].fill(4, NaN, NaN)); // [1, 2, 3]
console.log([1, 2, 3].fill(4, 3, 5)); // [1, 2, 3]
console.log(Array(3).fill(4)); // [4, 4, 4]

// A single object, referenced by each slot of the array:
const arr = Array(3).fill({}); // [{}, {}, {}]
arr[0].hi = "hi"; // [{ hi: "hi" }, { hi: "hi" }, { hi: "hi" }]
```

Using fill() to create a matrix of all 1

This example shows how to create a matrix of all 1, like the `ones()` function of Octave or MATLAB.

```js
const arr = new Array(3);
for (let i = 0; i < arr.length; i++) {
  arr[i] = new Array(4).fill(1); // Creating an array of size 4 and filled of 1
}
arr[0][0] = 10;
console.log(arr[0][0]); // 10
console.log(arr[1][0]); // 1
console.log(arr[2][0]); // 1
```

Using fill() to populate an empty array

This example shows how to populate an array, setting all elements to a specific value. The `end` parameter does not have to be specified.

```js
const tempGirls = Array(5).fill("girl", 0);
```

Note that the array was initially a sparse array with no assigned indices. `fill()` is still able to fill this array.

Calling fill() on non-array objects

The `fill()` method reads the `length` property of `this` and sets the value of each integer-keyed property from `start` to `end`.

```js
const arrayLike = { length: 2 };
console.log(Array.prototype.fill.call(arrayLike, 1));
// { '0': 1, '1': 1, length: 2 }
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/indexed-collections.html#sec-array.prototype.fill

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`fill`	45	12	31	32	8	45	31	32	8	5.0	45	8	1.0.0	1.0	4.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Array.fill`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Polyfill of `Array.prototype.fill` in `core-js`
- Indexed collections guide
- `Array`
- `TypedArray.prototype.fill()`

JavaScript › Reference › Standard built-in objects › Array

Array.prototype.filter()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/filter · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since July 2015.

The **`filter()`** method of `Array` instances creates a shallow copy of a portion of a given array, filtered down to just the elements from the given array that pass the test implemented by the provided function.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Array.prototype.filter()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const words = ["spray", "elite", "exuberant", "destruction", "present"];

const result = words.filter((word) => word.length > 6);

console.log(result);
// Expected output: Array ["exuberant", "destruction", "present"]
```

Syntax

js

```
filter(callbackFn)
filter(callbackFn, thisArg)
```

Parameters

`callbackFn`

A function to execute for each element in the array. It should return a truthy value to keep the element in the resulting array, and a falsy value otherwise. The function is called with the following

arguments:

`element`
The current element being processed in the array.

`index`
The index of the current element being processed in the array.

`array`
The array `filter()` was called upon.

`thisArg` Optional
A value to use as `this` when executing `callbackFn`. See iterative methods.

Return value

A shallow copy of the given array containing just the elements that pass the test. If no elements pass the test, an empty array is returned.

Description

The `filter()` method is an iterative method. It calls a provided `callbackFn` function once for each element in an array, and constructs a new array of all the values for which `callbackFn` returns a truthy value. Array elements which do not pass the `callbackFn` test are not included in the new array. Read the iterative methods section for more information about how these methods work in general.

`callbackFn` is invoked only for array indexes which have assigned values. It is not invoked for empty slots in sparse arrays.

The `filter()` method is generic. It only expects the `this` value to have a `length` property and integer-keyed properties.

Examples

Filtering out all small values

The following example uses `filter()` to create a filtered array that has all elements with values less than 10 removed.

```js
function isBigEnough(value) {
  return value >= 10;
}

const filtered = [12, 5, 8, 130, 44].filter(isBigEnough);
// filtered is [12, 130, 44]
```

Find all prime numbers in an array

The following example returns all prime numbers in the array:

```js
const array = [-3, -2, -1, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13];

function isPrime(n) {
  if (n < 2) {
    return false;
```

(code continues on the next page)

(code continued from the previous page)

```
  }
  if (n % 2 === 0) {
    return n === 2;
  }
  for (let factor = 3; factor * factor <= n; factor += 2) {
    if (n % factor === 0) {
      return false;
    }
  }
  return true;
}

console.log(array.filter(isPrime)); // [2, 3, 5, 7, 11, 13]
```

> **Note:** The `isPrime()` implementation is for demonstration only. For a real-world application, you would want to use a heavily memoized algorithm such as the Sieve of Eratosthenes to avoid repeated calculations.

Filtering invalid objects from an array of objects

The following example uses `filter()` to create a filtered array of all objects with non-zero, numeric `id`.

js

```
const arr = [
  { id: 15 },
  { id: -1 },
  { id: 0 },
  { id: 3 },
  { id: 12.2 },
  {},
  { id: null },
  { id: NaN },
  { id: "undefined" },
];

let invalidEntries = 0;

function filterByID(item) {
  if (Number.isFinite(item.id) && item.id !== 0) {
    return true;
  }
  invalidEntries++;
  return false;
}

const arrByID = arr.filter(filterByID);

console.log("Filtered Array\n", arrByID);
// Filtered Array
// [{ id: 15 }, { id: -1 }, { id: 3 }, { id: 12.2 }]

console.log("Number of Invalid Entries =", invalidEntries);
// Number of Invalid Entries = 5
```

Searching in array

The following example uses `filter()` to filter array content based on search criteria.

```js
const fruits = ["apple", "banana", "grapes", "mango", "orange"];

/**
 * Filter array items based on search criteria (query)
 */
function filterItems(arr, query) {
  return arr.filter((el) => el.toLowerCase().includes(query.toLowerCase()));
}

console.log(filterItems(fruits, "ap")); // ['apple', 'grapes']
console.log(filterItems(fruits, "an")); // ['banana', 'mango', 'orange']
```

Using the third argument of callbackFn

The `array` argument is useful if you want to access another element in the array, especially when you don't have an existing variable that refers to the array. The following example first uses `map()` to extract the numerical ID from each name and then uses `filter()` to select the ones that are greater than its neighbors.

```js
const names = ["JC63", "Bob132", "Ursula89", "Ben96"];
const greatIDs = names
  .map((name) => parseInt(name.match(/\d+/)[0], 10))
  .filter((id, idx, arr) => {
    // Without the arr argument, there's no way to easily access the
    // intermediate array without saving it to a variable.
    if (idx > 0 && id <= arr[idx - 1]) return false;
    if (idx < arr.length - 1 && id <= arr[idx + 1]) return false;
    return true;
  });
console.log(greatIDs); // [132, 96]
```

The `array` argument is *not* the array that is being built — there is no way to access the array being built from the callback function.

Using filter() on sparse arrays

`filter()` will skip empty slots.

```js
console.log([1, , undefined].filter((x) => x === undefined)); // [undefined]
console.log([1, , undefined].filter((x) => x !== 2)); // [1, undefined]
```

Calling filter() on non-array objects

The `filter()` method reads the `length` property of `this` and then accesses each property whose key is a nonnegative integer less than `length`.

```js
const arrayLike = {
  length: 3,
  0: "a",
  1: "b",
  2: "c",
  3: "a", // ignored by filter() since length is 3
};
console.log(Array.prototype.filter.call(arrayLike, (x) => x <= "b"));
// [ 'a', 'b' ]
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/indexed-collections.html#sec-array.prototype.filter

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`filter`	1	12	1.5	9.5	3	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Array.filter`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Array.prototype.filter` in `core-js`
- es-shims polyfill of `Array.prototype.filter`
- Indexed collections guide
- `Array`
- `Array.prototype.forEach()`
- `Array.prototype.every()`
- `Array.prototype.map()`
- `Array.prototype.some()`
- `Array.prototype.reduce()`
- `TypedArray.prototype.filter()`

JavaScript › Reference › Standard built-in objects › Array

Array.prototype.find()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/find · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since September 2015.

The **`find()`** method of `Array` instances returns the first element in the provided array that satisfies the provided testing function. If no values satisfy the testing function, `undefined` is returned.

- If you need the **index** of the found element in the array, use `findIndex()`.
- If you need to find the **index of a value**, use `indexOf()`. (It's similar to `findIndex()`, but checks each element for equality with the value instead of using a testing function.)
- If you need to find if a value **exists** in an array, use `includes()`. Again, it checks each element for equality with the value instead of using a testing function.
- If you need to find if any element satisfies the provided testing function, use `some()`.
- If you need to find all elements that satisfy the provided testing function, use `filter()`.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Array.prototype.find()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const array = [5, 12, 8, 130, 44];

const found = array.find((element) => element > 10);

console.log(found);
// Expected output: 12
```

Syntax

js

```
find(callbackFn)
find(callbackFn, thisArg)
```

Parameters

`callbackFn`

A function to execute for each element in the array. It should return a truthy value to indicate a matching element has been found, and a falsy value otherwise. The function is called with the following arguments:

`element`

The current element being processed in the array.

`index`

The index of the current element being processed in the array.

`array`

The array `find()` was called upon.

`thisArg` Optional

A value to use as `this` when executing `callbackFn`. See iterative methods.

Return value

The first element in the array that satisfies the provided testing function. Otherwise, `undefined` is returned.

Description

The `find()` method is an iterative method. It calls a provided `callbackFn` function once for each element in an array in ascending-index order, until `callbackFn` returns a truthy value. `find()` then returns that element and stops iterating through the array. If `callbackFn` never returns a truthy value, `find()` returns `undefined`. Read the iterative methods section for more information about how these methods work in general.

`callbackFn` is invoked for *every* index of the array, not just those with assigned values. Empty slots in sparse arrays behave the same as `undefined`.

The `find()` method is generic. It only expects the `this` value to have a `length` property and integer-keyed properties.

Examples

Find an object in an array by one of its properties

js

```
const inventory = [
  { name: "apples", quantity: 2 },
  { name: "bananas", quantity: 0 },
  { name: "cherries", quantity: 5 },
];

function isCherries(fruit) {
  return fruit.name === "cherries";
}

console.log(inventory.find(isCherries));
// { name: 'cherries', quantity: 5 }
```

Using arrow function and destructuring

js

```
const inventory = [
  { name: "apples", quantity: 2 },
  { name: "bananas", quantity: 0 },
  { name: "cherries", quantity: 5 },
];

const result = inventory.find(({ name }) => name === "cherries");

console.log(result); // { name: 'cherries', quantity: 5 }
```

Find the first prime number in an array

The following example returns the first element in the array that is a prime number, or `undefined` if there is no prime number.

```js
function isPrime(n) {
  if (n < 2) {
    return false;
  }
  if (n % 2 === 0) {
    return n === 2;
  }
  for (let factor = 3; factor * factor <= n; factor += 2) {
    if (n % factor === 0) {
      return false;
    }
  }
  return true;
}

console.log([4, 6, 8, 12].find(isPrime)); // undefined, not found
console.log([4, 5, 8, 12].find(isPrime)); // 5
```

> **Note:** The `isPrime()` implementation is for demonstration only. For a real-world application, you would want to use a heavily memoized algorithm such as the Sieve of Eratosthenes to avoid repeated calculations.

Using the third argument of callbackFn

The `array` argument is useful if you want to access another element in the array, especially when you don't have an existing variable that refers to the array. The following example first uses `filter()` to extract the positive values and then uses `find()` to find the first element that is less than its neighbors.

```js
const numbers = [3, -1, 1, 4, 1, 5, 9, 2, 6];
const firstTrough = numbers
  .filter((num) => num > 0)
  .find((num, idx, arr) => {
    // Without the arr argument, there's no way to easily access the
    // intermediate array without saving it to a variable.
    if (idx > 0 && num >= arr[idx - 1]) return false;
    if (idx < arr.length - 1 && num >= arr[idx + 1]) return false;
    return true;
  });
console.log(firstTrough); // 1
```

Using find() on sparse arrays

Empty slots in sparse arrays *are* visited, and are treated the same as `undefined`.

```js
// Declare array with no elements at indexes 2, 3, and 4
const array = [0, 1, , , , 5, 6];

// Shows all indexes, not just those with assigned values
array.find((value, index) => {
  console.log("Visited index", index, "with value", value);
  return false;
});
// Visited index 0 with value 0
// Visited index 1 with value 1
// Visited index 2 with value undefined
```

(code continues on the next page)

(code continued from the previous page)

```
// Visited index 3 with value undefined
// Visited index 4 with value undefined
// Visited index 5 with value 5
// Visited index 6 with value 6

// Shows all indexes, including deleted
array.find((value, index) => {
  // Delete element 5 on first iteration
  if (index === 0) {
    console.log("Deleting array[5] with value", array[5]);
    delete array[5];
  }
  // Element 5 is still visited even though deleted
  console.log("Visited index", index, "with value", value);
  return false;
});
// Deleting array[5] with value 5
// Visited index 0 with value 0
// Visited index 1 with value 1
// Visited index 2 with value undefined
// Visited index 3 with value undefined
// Visited index 4 with value undefined
// Visited index 5 with value undefined
// Visited index 6 with value 6
```

Calling find() on non-array objects

The `find()` method reads the `length` property of `this` and then accesses each property whose key is a nonnegative integer less than `length`.

js

```
const arrayLike = {
  length: 3,
  "-1": 0.1, // ignored by find() since -1 < 0
  0: 2,
  1: 7.3,
  2: 4,
};
console.log(Array.prototype.find.call(arrayLike, (x) => !Number.isInteger(x)));
// 7.3
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/indexed-collections.html#sec-array.prototype.find

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`find`	45	12	25	32	8	45	4	32	8	5.0	45	8	1.0.0	1.0	4.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Array.find`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Array.prototype.find` in `core-js`
- es-shims polyfill of `Array.prototype.find`
- Indexed collections guide
- `Array`
- `Array.prototype.findIndex()`
- `Array.prototype.findLast()`
- `Array.prototype.findLastIndex()`
- `Array.prototype.includes()`
- `Array.prototype.filter()`
- `Array.prototype.every()`
- `Array.prototype.some()`
- `TypedArray.prototype.find()`

JavaScript › Reference › Standard built-in objects › Array

Array.prototype.findIndex()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/findIndex · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since September 2015.

The **`findIndex()`** method of `Array` instances returns the index of the first element in an array that satisfies the provided testing function. If no elements satisfy the testing function, -1 is returned.

See also the `find()` method, which returns the first element that satisfies the testing function (rather than its index).

Try it

▷ Interactive example on MDN: "JavaScript Demo: Array.prototype.findIndex()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const array = [5, 12, 8, 130, 44];

const isLargeNumber = (element) => element > 13;

console.log(array.findIndex(isLargeNumber));
// Expected output: 3
```

Syntax

js

```
findIndex(callbackFn)
findIndex(callbackFn, thisArg)
```

Parameters

`callbackFn`

A function to execute for each element in the array. It should return a truthy value to indicate a matching element has been found, and a falsy value otherwise. The function is called with the following arguments:

`element`

The current element being processed in the array.

`index`

The index of the current element being processed in the array.

`array`

The array `findIndex()` was called upon.

`thisArg` Optional

A value to use as `this` when executing `callbackFn`. See iterative methods.

Return value

The index of the first element in the array that passes the test. Otherwise, `-1`.

Description

The `findIndex()` method is an iterative method. It calls a provided `callbackFn` function once for each element in an array in ascending-index order, until `callbackFn` returns a truthy value. `findIndex()` then returns the index of that element and stops iterating through the array. If `callbackFn` never returns a truthy value, `findIndex()` returns `-1`. Read the iterative methods section for more information about how these methods work in general.

`callbackFn` is invoked for *every* index of the array, not just those with assigned values. Empty slots in sparse arrays behave the same as `undefined`.

The `findIndex()` method is generic. It only expects the `this` value to have a `length` property and integer-keyed properties.

Examples

Find the index of the first prime number in an array

The following example returns the index of the first element in the array that is a prime number, or `-1` if there is no prime number.

```js
function isPrime(n) {
  if (n < 2) {
    return false;
  }
  if (n % 2 === 0) {
    return n === 2;
  }
  for (let factor = 3; factor * factor <= n; factor += 2) {
    if (n % factor === 0) {
      return false;
    }
  }
```

(code continues on the next page)

(code continued from the previous page)

```
  return true;
}

console.log([4, 6, 8, 9, 12].findIndex(isPrime)); // -1, not found
console.log([4, 6, 7, 9, 12].findIndex(isPrime)); // 2 (array[2] is 7)
```

> **Note:** The `isPrime()` implementation is for demonstration only. For a real-world application, you would want to use a heavily memoized algorithm such as the Sieve of Eratosthenes to avoid repeated calculations.

Using the third argument of callbackFn

The `array` argument is useful if you want to access another element in the array, especially when you don't have an existing variable that refers to the array. The following example first uses `filter()` to extract the positive values and then uses `findIndex()` to find the first element that is less than its neighbors.

js

```
const numbers = [3, -1, 1, 4, 1, 5, 9, 2, 6];
const firstTrough = numbers
  .filter((num) => num > 0)
  .findIndex((num, idx, arr) => {
    // Without the arr argument, there's no way to easily access the
    // intermediate array without saving it to a variable.
    if (idx > 0 && num >= arr[idx - 1]) return false;
    if (idx < arr.length - 1 && num >= arr[idx + 1]) return false;
    return true;
  });
console.log(firstTrough); // 1
```

Using findIndex() on sparse arrays

You can search for `undefined` in a sparse array and get the index of an empty slot.

js

```
console.log([1, , 3].findIndex((x) => x === undefined)); // 1
```

Calling findIndex() on non-array objects

The `findIndex()` method reads the `length` property of `this` and then accesses each property whose key is a nonnegative integer less than `length`.

js

```
const arrayLike = {
  length: 3,
  "-1": 0.1, // ignored by findIndex() since -1 < 0
  0: 2,
  1: 7.3,
  2: 4,
};
console.log(
  Array.prototype.findIndex.call(arrayLike, (x) => !Number.isInteger(x)),
); // 1
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/indexed-collections.html#sec-array.prototype.findindex`

Browser compatibility

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Nodejs
`findIndex`	45	12	25	32	8	45	4	32	8	5.0	45	8	1.0.0	1.0	4.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Array.findIndex`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Array.prototype.findIndex` in `core-js`
- es-shims polyfill of `Array.prototype.findIndex`
- Indexed collections guide
- `Array`
- `Array.prototype.find()`
- `Array.prototype.findLast()`
- `Array.prototype.findLastIndex()`
- `Array.prototype.indexOf()`
- `Array.prototype.lastIndexOf()`
- `TypedArray.prototype.findIndex()`

JavaScript › Reference › Standard built-in objects › Array

Array.prototype.findLast()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/findLast · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since August 2022.

The **`findLast()`** method of `Array` instances iterates the array in reverse order and returns the value of the first element that satisfies the provided testing function. If no elements satisfy the testing function, `undefined` is returned.

If you need to find:

- the *first* element that matches, use `find()`.
- the *index* of the last matching element in the array, use `findLastIndex()`.
- the *index of a value*, use `indexOf()`. (It's similar to `findIndex()`, but checks each element for equality with the value instead of using a testing function.)
- whether a value *exists* in an array, use `includes()`. Again, it checks each element for equality with the value instead of using a testing function.

- if any element satisfies the provided testing function, use `some()`.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Array.prototype.findLast()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const array = [5, 12, 50, 130, 44];

const found = array.findLast((element) => element > 45);

console.log(found);
// Expected output: 130
```

Syntax

js

```
findLast(callbackFn)
findLast(callbackFn, thisArg)
```

Parameters

`callbackFn`

A function to execute for each element in the array. It should return a truthy value to indicate a matching element has been found, and a falsy value otherwise. The function is called with the following arguments:

`element`

The current element being processed in the array.

`index`

The index of the current element being processed in the array.

`array`

The array `findLast()` was called upon.

`thisArg` Optional

A value to use as `this` when executing `callbackFn`. See iterative methods.

Return value

The last (highest-index) element in the array that satisfies the provided testing function; `undefined` if no matching element is found.

Description

The `findLast()` method is an iterative method. It calls a provided `callbackFn` function once for each element in an array in descending-index order, until `callbackFn` returns a truthy value. `findLast()` then returns that element and stops iterating through the array. If `callbackFn` never returns a truthy value, `findLast()` returns `undefined`. Read the iterative methods section for more information about how these methods work in general.

`callbackFn` is invoked for *every* index of the array, not just those with assigned values. Empty slots in sparse arrays behave the same as `undefined`.

The `findLast()` method is generic. It only expects the `this` value to have a `length` property and integer-keyed properties.

Examples

Find last object in an array matching on element properties

This example shows how you might create a test based on the properties of array elements.

```js
const inventory = [
  { name: "apples", quantity: 2 },
  { name: "bananas", quantity: 0 },
  { name: "fish", quantity: 1 },
  { name: "cherries", quantity: 5 },
];

// return true inventory stock is low
function isNotEnough(item) {
  return item.quantity < 2;
}

console.log(inventory.findLast(isNotEnough));
// { name: "fish", quantity: 1 }
```

Using arrow function and destructuring

The previous example might be written using an arrow function and object destructuring:

```js
const inventory = [
  { name: "apples", quantity: 2 },
  { name: "bananas", quantity: 0 },
  { name: "fish", quantity: 1 },
  { name: "cherries", quantity: 5 },
];

const result = inventory.findLast(({ quantity }) => quantity < 2);

console.log(result);
// { name: "fish", quantity: 1 }
```

Find the last prime number in an array

The following example returns the last element in the array that is a prime number, or `undefined` if there is no prime number.

```js
function isPrime(n) {
  if (n < 2) {
    return false;
  }
  if (n % 2 === 0) {
    return n === 2;
  }
  for (let factor = 3; factor * factor <= n; factor += 2) {
    if (n % factor === 0) {
      return false;
    }
```

(code continues on the next page)

(code continued from the previous page)

```
  }
  return true;
}

console.log([4, 6, 8, 12].findLast(isPrime)); // undefined, not found
console.log([4, 5, 7, 8, 9, 11, 12].findLast(isPrime)); // 11
```

> **Note:** The `isPrime()` implementation is for demonstration only. For a real-world application, you would want to use a heavily memoized algorithm such as the Sieve of Eratosthenes to avoid repeated calculations.

Find the most recent completed task

This example demonstrates a practical use case for `findLast()` when working with time-ordered data. It finds the most recent completed task from a list already sorted by `timestamp`.

js

```
const tasks = [
  { name: "Setup project", completed: true, timestamp: 1609459200000 },
  { name: "Write tests", completed: false, timestamp: 1609545600000 },
  { name: "Fix bug #42", completed: true, timestamp: 1609632000000 },
  { name: "Deploy to staging", completed: true, timestamp: 1609718400000 },
  { name: "Review PR", completed: false, timestamp: 1609804800000 },
];

const lastCompletedTask = tasks.findLast((task) => task.completed);

console.log(lastCompletedTask.name); // Deploy to staging
```

This is more efficient than using `find()` with a reversed array, because it avoids creating a new array.

Using the third argument of callbackFn

The `array` argument is useful if you want to access another element in the array, especially when you don't have an existing variable that refers to the array. The following example first uses `filter()` to extract the positive values and then uses `findLast()` to find the last element that is less than its neighbors.

js

```
const numbers = [3, -1, 1, 4, 1, 5, 9, 2, 6];
const lastTrough = numbers
  .filter((num) => num > 0)
  .findLast((num, idx, arr) => {
    // Without the arr argument, there's no way to easily access the
    // intermediate array without saving it to a variable.
    if (idx > 0 && num >= arr[idx - 1]) return false;
    if (idx < arr.length - 1 && num >= arr[idx + 1]) return false;
    return true;
  });
console.log(lastTrough); // 2
```

Using findLast() on sparse arrays

Empty slots in sparse arrays *are* visited, and are treated the same as `undefined`.

```js
// Declare array with no elements at indexes 2, 3, and 4
const array = [0, 1, , , , 5, 6];

// Shows all indexes, not just those with assigned values
array.findLast((value, index) => {
  console.log(`Visited index ${index} with value ${value}`);
  return false;
});
// Visited index 6 with value 6
// Visited index 5 with value 5
// Visited index 4 with value undefined
// Visited index 3 with value undefined
// Visited index 2 with value undefined
// Visited index 1 with value 1
// Visited index 0 with value 0

// Shows all indexes, including deleted
array.findLast((value, index) => {
  // Delete element 5 on first iteration
  if (index === 6) {
    console.log(`Deleting array[5] with value ${array[5]}`);
    delete array[5];
  }
  // Element 5 is still visited even though deleted
  console.log(`Visited index ${index} with value ${value}`);
  return false;
});
// Deleting array[5] with value 5
// Visited index 6 with value 6
// Visited index 5 with value undefined
// Visited index 4 with value undefined
// Visited index 3 with value undefined
// Visited index 2 with value undefined
// Visited index 1 with value 1
// Visited index 0 with value 0
```

Calling findLast() on non-array objects

The `findLast()` method reads the `length` property of `this` and then accesses each property whose key is a nonnegative integer less than `length`.

```js
const arrayLike = {
  length: 3,
  0: 2,
  1: 7.3,
  2: 4,
  3: 3, // ignored by findLast() since length is 3
};
console.log(
  Array.prototype.findLast.call(arrayLike, (x) => Number.isInteger(x)),
); // 4
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/indexed-collections.html#sec-array.prototype.findlast

Browser compatibility

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
findLast	97	97	104	83	15.4	97	104	68	15.4	18.0	97	15.4	1.0.0	1.16	18.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Array.findLast`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Array.prototype.findLast` in `core-js`
- es-shims polyfill of `Array.prototype.findLast`
- Indexed collections guide
- `Array`
- `Array.prototype.find()`
- `Array.prototype.findIndex()`
- `Array.prototype.findLastIndex()`
- `Array.prototype.includes()`
- `Array.prototype.filter()`
- `Array.prototype.every()`
- `Array.prototype.some()`
- `TypedArray.prototype.findLast()`

JavaScript › Reference › Standard built-in objects › Array

Array.prototype.findLastIndex()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/findLastIndex · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since August 2022.

The **`findLastIndex()`** method of `Array` instances iterates the array in reverse order and returns the index of the first element that satisfies the provided testing function. If no elements satisfy the testing function, -1 is returned.

See also the `findLast()` method, which returns the value of last element that satisfies the testing function (rather than its index).

Try it

▷ Interactive example on MDN: "JavaScript Demo: Array.prototype.findLastIndex()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const array = [5, 12, 50, 130, 44];

const isLargeNumber = (element) => element > 45;

console.log(array.findLastIndex(isLargeNumber));
// Expected output: 3
// Index of element with value: 130
```

Syntax

js

```
findLastIndex(callbackFn)
findLastIndex(callbackFn, thisArg)
```

Parameters

`callbackFn`

A function to execute for each element in the array. It should return a truthy value to indicate a matching element has been found, and a falsy value otherwise. The function is called with the following arguments:

`element`

The current element being processed in the array.

`index`

The index of the current element being processed in the array.

`array`

The array `findLastIndex()` was called upon.

`thisArg` Optional

A value to use as `this` when executing `callbackFn`. See iterative methods.

Return value

The index of the last (highest-index) element in the array that passes the test. Otherwise `-1` if no matching element is found.

Description

The `findLastIndex()` method is an iterative method. It calls a provided `callbackFn` function once for each element in an array in descending-index order, until `callbackFn` returns a truthy value. `findLastIndex()` then returns the index of that element and stops iterating through the array. If `callbackFn` never returns a truthy value, `findLastIndex()` returns `-1`. Read the iterative methods section for more information about how these methods work in general.

`callbackFn` is invoked for *every* index of the array, not just those with assigned values. Empty slots in sparse arrays behave the same as `undefined`.

The `findLastIndex()` method is generic. It only expects the `this` value to have a `length` property and integer-keyed properties.

Examples

Find the index of the last prime number in an array

The following example returns the index of the last element in the array that is a prime number, or `-1` if there is no prime number.

```js
function isPrime(n) {
  if (n < 2) {
    return false;
  }
  if (n % 2 === 0) {
    return n === 2;
  }
  for (let factor = 3; factor * factor <= n; factor += 2) {
    if (n % factor === 0) {
      return false;
    }
  }
  return true;
}

console.log([4, 6, 8, 12].findLastIndex(isPrime)); // -1, not found
console.log([4, 5, 7, 8, 9, 11, 12].findLastIndex(isPrime)); // 5
```

> **Note:** The `isPrime()` implementation is for demonstration only. For a real-world application, you would want to use a heavily memoized algorithm such as the Sieve of Eratosthenes to avoid repeated calculations.

Using the third argument of callbackFn

The `array` argument is useful if you want to access another element in the array, especially when you don't have an existing variable that refers to the array. The following example first uses `filter()` to extract the positive values and then uses `findLastIndex()` to find the last element that is less than its neighbors.

```js
const numbers = [3, -1, 1, 4, 1, 5, 9, 2, 6];
const lastTrough = numbers
  .filter((num) => num > 0)
  .findLastIndex((num, idx, arr) => {
    // Without the arr argument, there's no way to easily access the
    // intermediate array without saving it to a variable.
    if (idx > 0 && num >= arr[idx - 1]) return false;
    if (idx < arr.length - 1 && num >= arr[idx + 1]) return false;
    return true;
  });
console.log(lastTrough); // 6
```

Using findLastIndex() on sparse arrays

You can search for `undefined` in a sparse array and get the index of an empty slot.

```js
console.log([1, , 3].findLastIndex((x) => x === undefined)); // 1
```

Calling findLastIndex() on non-array objects

The `findLastIndex()` method reads the `length` property of `this` and then accesses each property whose key is a nonnegative integer less than `length`.

```js
const arrayLike = {
  length: 3,
  0: 2,
  1: 7.3,
  2: 4,
  3: 3, // ignored by findLastIndex() since length is 3
};
console.log(
  Array.prototype.findLastIndex.call(arrayLike, (x) => Number.isInteger(x)),
); // 2
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/indexed-collections.html#sec-array.prototype.findlastindex

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
findLastIndex	97	97	104	83	15.4	97	104	68	15.4	18.0	97	15.4	1.0.0	1.16	18.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Array.findLastIndex`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Polyfill of `Array.prototype.findLastIndex` in `core-js`
- es-shims polyfill of `Array.prototype.findLastIndex`
- Indexed collections guide
- `Array`
- `Array.prototype.find()`
- `Array.prototype.findIndex()`
- `Array.prototype.findLast()`
- `Array.prototype.indexOf()`

- `Array.prototype.lastIndexOf()`
- `TypedArray.prototype.findLastIndex()`

JavaScript › Reference › Standard built-in objects › Array

Array.prototype.flat()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/flat · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since January 2020.

The **`flat()`** method of `Array` instances creates a new array with all sub-array elements concatenated into it recursively up to the specified depth.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Array.prototype.flat()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const arr1 = [0, 1, 2, [3, 4]];

console.log(arr1.flat());
// expected output: Array [0, 1, 2, 3, 4]

const arr2 = [0, 1, [2, [3, [4, 5]]]];

console.log(arr2.flat());
// expected output: Array [0, 1, 2, Array [3, Array [4, 5]]]

console.log(arr2.flat(2));
// expected output: Array [0, 1, 2, 3, Array [4, 5]]

console.log(arr2.flat(Infinity));
// expected output: Array [0, 1, 2, 3, 4, 5]
```

Syntax

js

```
flat()
flat(depth)
```

Parameters

`depth` Optional

The depth level specifying how deep a nested array structure should be flattened. Defaults to 1.

Return value

A new array with the sub-array elements concatenated into it.

Description

The `flat()` method is a copying method. It does not alter `this` but instead returns a shallow copy that contains the same elements as the ones from the original array.

The `flat()` method removes empty slots if the array being flattened is sparse. For example, if `depth` is 1, both empty slots in the root array and in the first level of nested arrays are ignored, but empty slots in

further nested arrays are preserved with the arrays themselves.

The `flat()` method is generic. It only expects the `this` value to have a `length` property and integer-keyed properties. However, its elements must be arrays if they are to be flattened.

Examples

Flattening nested arrays

```js
const arr1 = [1, 2, [3, 4]];
arr1.flat();
// [1, 2, 3, 4]

const arr2 = [1, 2, [3, 4, [5, 6]]];
arr2.flat();
// [1, 2, 3, 4, [5, 6]]

const arr3 = [1, 2, [3, 4, [5, 6]]];
arr3.flat(2);
// [1, 2, 3, 4, 5, 6]

const arr4 = [1, 2, [3, 4, [5, 6, [7, 8, [9, 10]]]]];
arr4.flat(Infinity);
// [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

Using flat() on sparse arrays

The `flat()` method removes empty slots in arrays:

```js
const arr5 = [1, 2, , 4, 5];
console.log(arr5.flat()); // [1, 2, 4, 5]

const array = [1, , 3, ["a", , "c"]];
console.log(array.flat()); // [ 1, 3, "a", "c" ]

const array2 = [1, , 3, undefined, ["a", , ["d", , "e"]], null];
console.log(array2.flat()); // [ 1, 3, undefined, "a", ["d", empty, "e"], null ]
console.log(array2.flat(2)); // [ 1, 3, undefined, "a", "d", "e", null ]
```

Calling flat() on non-array objects

The `flat()` method reads the `length` property of `this` and then accesses each property whose key is a nonnegative integer less than `length`. If the element is not an array, it's directly appended to the result. If the element is an array, it's flattened according to the `depth` parameter.

```js
const arrayLike = {
  length: 3,
  0: [1, 2],
  // Array-like objects aren't flattened
  1: { length: 2, 0: 3, 1: 4 },
  2: 5,
  3: 3, // ignored by flat() since length is 3
};
console.log(Array.prototype.flat.call(arrayLike));
// [ 1, 2, { '0': 3, '1': 4, length: 2 }, 5 ]
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/indexed-collections.html#sec-array.prototype.flat`

Browser compatibility

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`flat`	69	79	62	56	12	69	62	48	12	10.0	69	12	1.0.0	1.0	11.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Array.flat`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Array.prototype.flat` in `core-js`
- es-shims polyfill of `Array.prototype.flat`
- Indexed collections guide
- `Array`
- `Array.prototype.concat()`
- `Array.prototype.flatMap()`
- `Array.prototype.map()`
- `Array.prototype.reduce()`

JavaScript › Reference › Standard built-in objects › Array

Array.prototype.flatMap()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/flatMap · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since January 2020.

The **`flatMap()`** method of `Array` instances returns a new array formed by applying a given callback function to each element of the array, and then flattening the result by one level. It is identical to a `map()` followed by a `flat()` of depth 1 (`arr.map(...args).flat()`), but slightly more efficient than calling those two methods separately.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Array.prototype.flatMap()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const arr = [1, 2, 1];

const result = arr.flatMap((num) => (num === 2 ? [2, 2] : 1));

console.log(result);
// Expected output: Array [1, 2, 2, 1]
```

Syntax

js

```
flatMap(callbackFn)
flatMap(callbackFn, thisArg)
```

Parameters

`callbackFn`

A function to execute for each element in the array. It should return an array containing new elements of the new array, or a single non-array value to be added to the new array. The function is called with the following arguments:

`element`

The current element being processed in the array.

`index`

The index of the current element being processed in the array.

`array`

The array `flatMap()` was called upon.

`thisArg` Optional

A value to use as `this` when executing `callbackFn`. See iterative methods.

Return value

A new array with each element being the result of the callback function and flattened by a depth of 1.

Description

The `flatMap()` method is an iterative method. See `Array.prototype.map()` for a detailed description of the callback function. The `flatMap()` method is identical to `map(callbackFn, thisArg)` followed by `flat(1)` — for each element, it produces an array of new elements, and concatenates the resulting arrays together to form a new array. Read the iterative methods section for more information about how these methods work in general.

The `flatMap()` method is generic. It only expects the `this` value to have a `length` property and integer-keyed properties. However, the value returned from `callbackFn` must be an array if it is to be flattened.

Alternative

Pre-allocate and explicitly iterate

```js
const arr = [1, 2, 3, 4];

arr.flatMap((x) => [x, x * 2]);
// is equivalent to
const n = arr.length;
const acc = new Array(n * 2);
for (let i = 0; i < n; i++) {
  const x = arr[i];
  acc[i * 2] = x;
  acc[i * 2 + 1] = x * 2;
}
// [1, 2, 2, 4, 3, 6, 4, 8]
```

Note that in this particular case the `flatMap` approach is slower than the for-loop approach — due to the creation of temporary arrays that must be garbage collected, as well as the return array not needing to be frequently resized. However, `flatMap` may still be the correct solution in cases where its flexibility and readability are desired.

Examples

map() and flatMap()

```js
const arr = [1, 2, 3, 4];

arr.map((x) => [x * 2]);
// [[2], [4], [6], [8]]

arr.flatMap((x) => [x * 2]);
// [2, 4, 6, 8]

// only one level is flattened
arr.flatMap((x) => [[x * 2]]);
// [[2], [4], [6], [8]]
```

While the above could have been achieved by using map itself, here is an example that better showcases the use of `flatMap()`.

Let's generate a list of words from a list of sentences.

```js
const arr = ["it's Sunny in", "", "California"];

arr.map((x) => x.split(" "));
// [["it's","Sunny","in"],[""],["California"]]

arr.flatMap((x) => x.split(" "));
// ["it's","Sunny","in", "", "California"]
```

Notice, the output list length can be different from the input list length.

For adding and removing items during a map()

`flatMap` can be used as a way to add and remove items (modify the number of items) during a `map`. In other words, it allows you to map *many items to many items* (by handling each input item separately), rather than always *one-to-one*. In this sense, it works like the opposite of filter. Return a 1-element array to keep the item, a multiple-element array to add items, or a 0-element array to remove the item.

```js
// Let's say we want to remove all the negative numbers
// and split the odd numbers into an even number and a 1
const a = [5, 4, -3, 20, 17, -33, -4, 18];
//         |\  \  x   |  | \   x   x   |
//        [4,1, 4,   20, 16, 1,       18]

const result = a.flatMap((n) => {
  if (n < 0) {
    return [];
  }
  return n % 2 === 0 ? [n] : [n - 1, 1];
});
console.log(result); // [4, 1, 4, 20, 16, 1, 18]
```

Using the third argument of callbackFn

The `array` argument is useful if you want to access another element in the array, especially when you don't have an existing variable that refers to the array. The following example first uses `filter()` to extract operational stations and then uses `flatMap()` to create a new array where each element contains a station and its next station. On the last station, it returns an empty array to exclude it from the final array.

```js
const stations = ["New Haven", "West Haven", "Milford (closed)", "Stratford"];
const line = stations
  .filter((name) => !name.endsWith("(closed)"))
  .flatMap((name, idx, arr) => {
    // Without the arr argument, there's no way to easily access the
    // intermediate array without saving it to a variable.
    if (idx === arr.length - 1) return []; // last station has no next station
    return [`${name} - ${arr[idx + 1]}`];
  });
console.log(line); // ['New Haven - West Haven', 'West Haven - Stratford']
```

The `array` argument is *not* the array that is being built — there is no way to access the array being built from the callback function.

Using flatMap() on sparse arrays

The `callbackFn` won't be called for empty slots in the source array because `map()` doesn't, while `flat()` ignores empty slots in the returned arrays.

```js
console.log([1, 2, , 4, 5].flatMap((x) => [x, x * 2])); // [1, 2, 2, 4, 4, 8, 5, 10]
console.log([1, 2, 3, 4].flatMap((x) => [, x * 2])); // [2, 4, 6, 8]
```

Calling flatMap() on non-array objects

The `flatMap()` method reads the `length` property of `this` and then accesses each property whose key is a nonnegative integer less than `length`. If the return value of the callback function is not an array, it is always directly appended to the result array.

js

```
const arrayLike = {
  length: 3,
  0: 1,
  1: 2,
  2: 3,
  3: 4, // ignored by flatMap() since length is 3
};
console.log(Array.prototype.flatMap.call(arrayLike, (x) => [x, x * 2]));
// [1, 2, 2, 4, 3, 6]

// Array-like objects returned from the callback won't be flattened
console.log(
  Array.prototype.flatMap.call(arrayLike, (x) => ({
    length: 1,
    0: x,
  })),
);
// [ { '0': 1, length: 1 }, { '0': 2, length: 1 }, { '0': 3, length: 1 } ]
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/indexed-collections.html#sec-array.prototype.flatmap

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`flatMap`	69	79	62	56	12	69	62	48	12	10.0	69	12	1.0.0	1.0	11.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Array.flatMap`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Array.prototype.flatMap` in `core-js`
- es-shims polyfill of `Array.prototype.flatMap`
- Indexed collections guide
- `Array`
- `Array.prototype.concat()`
- `Array.prototype.flat()`

- `Array.prototype.map()`
- `Array.prototype.reduce()`

JavaScript › Reference › Standard built-in objects › Array

Array.prototype.forEach()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/forEach · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since July 2015.

The **`forEach()`** method of `Array` instances executes a provided function once for each array element.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Array.prototype.forEach()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const array = ["a", "b", "c"];

array.forEach((element) => console.log(element));

// Expected output: "a"
// Expected output: "b"
// Expected output: "c"
```

Syntax

js

```
forEach(callbackFn)
forEach(callbackFn, thisArg)
```

Parameters

`callbackFn`

A function to execute for each element in the array. Its return value is discarded. The function is called with the following arguments:

`element`

The current element being processed in the array.

`index`

The index of the current element being processed in the array.

`array`

The array `forEach()` was called upon.

`thisArg` Optional

A value to use as `this` when executing `callbackFn`. See iterative methods.

Return value

None (`undefined`).

Description

The `forEach()` method is an iterative method. It calls a provided `callbackFn` function once for each element in an array in ascending-index order. Unlike `map()`, `forEach()` always returns `undefined` and is not chainable. The typical use case is to execute side effects at the end of a chain. Read the iterative methods section for more information about how these methods work in general.

`callbackFn` is invoked only for array indexes which have assigned values. It is not invoked for empty slots in sparse arrays.

The `forEach()` method is generic. It only expects the `this` value to have a `length` property and integer-keyed properties.

There is no way to stop or break a `forEach()` loop other than by throwing an exception. If you need such behavior, the `forEach()` method is the wrong tool.

Early termination may be accomplished with looping statements like `for`, `for...of`, and `for...in`. Array methods like `every()`, `some()`, `find()`, and `findIndex()` also stops iteration immediately when further iteration is not necessary.

`forEach()` expects a synchronous function — it does not wait for promises. Make sure you are aware of the implications while using promises (or async functions) as `forEach` callbacks.

```js
const ratings = [5, 4, 5];
let sum = 0;

const sumFunction = async (a, b) => a + b;

ratings.forEach(async (rating) => {
  sum = await sumFunction(sum, rating);
});

console.log(sum);
// Naively expected output: 14
// Actual output: 0
```

To run a series of asynchronous operations sequentially or concurrently, see promise composition.

Examples

Converting a for loop to forEach

```js
const items = ["item1", "item2", "item3"];
const copyItems = [];

// before
for (let i = 0; i < items.length; i++) {
  copyItems.push(items[i]);
}

// after
items.forEach((item) => {
  copyItems.push(item);
});
```

Printing the contents of an array

> **Note:** In order to display the content of an array in the console, you can use `console.table()`, which prints a formatted version of the array.
>
> The following example illustrates an alternative approach, using `forEach()`.

The following code logs a line for each element in an array:

```js
const logArrayElements = (element, index /*, array */) => {
  console.log(`a[${index}] = ${element}`);
};

// Notice that index 2 is skipped, since there is no item at
// that position in the array.
[2, 5, , 9].forEach(logArrayElements);
// Logs:
// a[0] = 2
// a[1] = 5
// a[3] = 9
```

Using thisArg

The following (contrived) example updates an object's properties from each entry in the array:

```js
class Counter {
  constructor() {
    this.sum = 0;
    this.count = 0;
  }
  add(array) {
    // Only function expressions have their own this bindings.
    array.forEach(function countEntry(entry) {
      this.sum += entry;
      ++this.count;
    }, this);
  }
}

const obj = new Counter();
obj.add([2, 5, 9]);
console.log(obj.count); // 3
console.log(obj.sum); // 16
```

Since the `thisArg` parameter (`this`) is provided to `forEach()`, it is passed to `callback` each time it's invoked. The callback uses it as its `this` value.

> **Note:** If passing the callback function used an arrow function expression, the `thisArg` parameter could be omitted, since all arrow functions lexically bind the `this` value.

An object copy function

The following code creates a copy of a given object.

There are different ways to create a copy of an object. The following is just one way and is presented to explain how `Array.prototype.forEach()` works by using `Object.*` utility functions.

```js
const copy = (obj) => {
  const copy = Object.create(Object.getPrototypeOf(obj));
  const propNames = Object.getOwnPropertyNames(obj);
  propNames.forEach((name) => {
    const desc = Object.getOwnPropertyDescriptor(obj, name);
    Object.defineProperty(copy, name, desc);
  });
  return copy;
};

const obj1 = { a: 1, b: 2 };
const obj2 = copy(obj1); // obj2 looks like obj1 now
```

Flatten an array

The following example is only here for learning purpose. If you want to flatten an array using built-in methods, you can use `Array.prototype.flat()`.

```js
const flatten = (arr) => {
  const result = [];
  arr.forEach((item) => {
    if (Array.isArray(item)) {
      result.push(...flatten(item));
    } else {
      result.push(item);
    }
  });
  return result;
};

// Usage
const nested = [1, 2, 3, [4, 5, [6, 7], 8, 9]];
console.log(flatten(nested)); // [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Using the third argument of callbackFn

The `array` argument is useful if you want to access another element in the array, especially when you don't have an existing variable that refers to the array. The following example first uses `filter()` to extract the positive values and then uses `forEach()` to log its neighbors.

```js
const numbers = [3, -1, 1, 4, 1, 5];
numbers
  .filter((num) => num > 0)
  .forEach((num, idx, arr) => {
    // Without the arr argument, there's no way to easily access the
    // intermediate array without saving it to a variable.
    console.log(arr[idx - 1], num, arr[idx + 1]);
  });
// undefined 3 1
// 3 1 4
// 1 4 1
// 4 1 5
// 1 5 undefined
```

Using forEach() on sparse arrays

```js
const arraySparse = [1, 3, /* empty */, 7];
let numCallbackRuns = 0;

arraySparse.forEach((element) => {
  console.log({ element });
  numCallbackRuns++;
});

console.log({ numCallbackRuns });

// { element: 1 }
// { element: 3 }
// { element: 7 }
// { numCallbackRuns: 3 }
```

The callback function is not invoked for the missing value at index 2.

Calling forEach() on non-array objects

The `forEach()` method reads the `length` property of `this` and then accesses each property whose key is a nonnegative integer less than `length`.

```js
const arrayLike = {
  length: 3,
  0: 2,
  1: 3,
  2: 4,
  3: 5, // ignored by forEach() since length is 3
};
Array.prototype.forEach.call(arrayLike, (x) => console.log(x));
// 2
// 3
// 4
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/indexed-collections.html#sec-array.prototype.foreach`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`forEach`	1	12	1.5	9.5	3	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Array.forEach`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Array.prototype.forEach` in `core-js`
- es-shims polyfill of `Array.prototype.forEach`
- Indexed collections guide
- `Array`
- `Array.prototype.find()`
- `Array.prototype.map()`
- `Array.prototype.filter()`
- `Array.prototype.every()`
- `Array.prototype.some()`
- `TypedArray.prototype.forEach()`
- `Map.prototype.forEach()`
- `Set.prototype.forEach()`

JavaScript › Reference › Standard built-in objects › Array

Array.prototype.includes()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/includes · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since August 2016.

The **`includes()`** method of `Array` instances determines whether an array includes a certain value among its entries, returning `true` or `false` as appropriate.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Array.prototype.includes()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const array = [1, 2, 3];

console.log(array.includes(2));
// Expected output: true

const pets = ["cat", "dog", "bat"];

console.log(pets.includes("cat"));
// Expected output: true

console.log(pets.includes("at"));
// Expected output: false
```

Syntax

js

```
includes(searchElement)
includes(searchElement, fromIndex)
```

Parameters

`searchElement`

The value to search for.

`fromIndex` Optional

Zero-based index at which to start searching, converted to an integer.

- Negative index counts back from the end of the array — if `-array.length <= fromIndex < 0`, `fromIndex + array.length` is used. However, the array is still searched from front to back in this case.
- If `fromIndex < -array.length` or `fromIndex` is omitted, `0` is used, causing the entire array to be searched.
- If `fromIndex >= array.length`, the array is not searched and `false` is returned.

Return value

A boolean value which is `true` if the value `searchElement` is found within the array (or the part of the array indicated by the index `fromIndex`, if specified).

Description

The `includes()` method compares `searchElement` to elements of the array using the SameValueZero algorithm. This algorithm works like strict equality `===` (where `-0` and `+0` are considered equal), with the exception that `NaN` is considered equal to itself.

When used on sparse arrays, the `includes()` method iterates empty slots as if they have the value `undefined`.

The `includes()` method is generic. It only expects the `this` value to have a `length` property and integer-keyed properties.

Examples

Using includes()

```js
[1, 2, 3].includes(2); // true
[1, 2, 3].includes(4); // false
[1, 2, 3].includes(3, 3); // false
[1, 2, 3].includes(3, -1); // true
[1, 2, NaN].includes(NaN); // true
["1", "2", "3"].includes(3); // false
```

fromIndex is greater than or equal to the array length

If `fromIndex` is greater than or equal to the length of the array, `false` is returned. The array will not be searched.

```js
const arr = ["a", "b", "c"];

arr.includes("c", 3); // false
arr.includes("c", 100); // false
```

Computed index is less than 0

If `fromIndex` is negative, the computed index is calculated to be used as a position in the array at which to begin searching for `searchElement`. If the computed index is less than or equal to `0`, the entire array will be searched.

```js
// array length is 3
// fromIndex is -100
// computed index is 3 + (-100) = -97

const arr = ["a", "b", "c"];

arr.includes("a", -100); // true
arr.includes("b", -100); // true
arr.includes("c", -100); // true
arr.includes("a", -2); // false
```

Using includes() on sparse arrays

You can search for `undefined` in a sparse array and get `true`.

```js
console.log([1, , 3].includes(undefined)); // true
```

Calling includes() on non-array objects

The `includes()` method reads the `length` property of `this` and then accesses each property whose key is a nonnegative integer less than `length`.

```js
const arrayLike = {
  length: 3,
  0: 2,
  1: 3,
  2: 4,
  3: 1, // ignored by includes() since length is 3
};
console.log(Array.prototype.includes.call(arrayLike, 2));
// true
console.log(Array.prototype.includes.call(arrayLike, 1));
// false
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/indexed-collections.html#sec-array.prototype.includes

Browser compatibility

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
includes	47	14	43	34	9	47	43	34	9	5.0	47	9	1.0.0	1.0	6.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Array.includes`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Array.prototype.includes` in `core-js`
- es-shims polyfill of `Array.prototype.includes`
- Indexed collections guide
- `Array`
- `Array.prototype.indexOf()`
- `Array.prototype.find()`
- `Array.prototype.findIndex()`
- `TypedArray.prototype.includes()`
- `String.prototype.includes()`

JavaScript › Reference › Standard built-in objects › Array

Array.prototype.indexOf()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/indexOf · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since July 2015.

The **`indexOf()`** method of `Array` instances returns the first index at which a given element can be found in the array, or -1 if it is not present.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Array.prototype.indexOf()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const beasts = ["ant", "bison", "camel", "duck", "bison"];

console.log(beasts.indexOf("bison"));
// Expected output: 1

// Start from index 2
console.log(beasts.indexOf("bison", 2));
// Expected output: 4

console.log(beasts.indexOf("giraffe"));
// Expected output: -1
```

Syntax

js

```
indexOf(searchElement)
indexOf(searchElement, fromIndex)
```

Parameters

`searchElement`

Element to locate in the array.

`fromIndex` Optional

Zero-based index at which to start searching, converted to an integer.

- Negative index counts back from the end of the array — if `-array.length <= fromIndex < 0`, `fromIndex + array.length` is used. Note, the array is still searched from front to back in this case.
- If `fromIndex < -array.length` or `fromIndex` is omitted, `0` is used, causing the entire array to be searched.
- If `fromIndex >= array.length`, the array is not searched and `-1` is returned.

Return value

The first index of `searchElement` in the array; `-1` if not found.

Description

The `indexOf()` method compares `searchElement` to elements of the array using strict equality (the same algorithm used by the `===` operator). `NaN` values are never compared as equal, so `indexOf()` always returns `-1` when `searchElement` is `NaN`.

The `indexOf()` method skips empty slots in sparse arrays.

The `indexOf()` method is generic. It only expects the `this` value to have a `length` property and integer-keyed properties.

Examples

Using indexOf()

The following example uses `indexOf()` to locate values in an array.

```js
const array = [2, 9, 9];
array.indexOf(2); // 0
array.indexOf(7); // -1
array.indexOf(9, 2); // 2
array.indexOf(2, -1); // -1
array.indexOf(2, -3); // 0
```

You cannot use `indexOf()` to search for `NaN`.

```js
const array = [NaN];
array.indexOf(NaN); // -1
```

Finding all the occurrences of an element

```js
const indices = [];
const array = ["a", "b", "a", "c", "a", "d"];
const element = "a";
let idx = array.indexOf(element);
while (idx !== -1) {
  indices.push(idx);
  idx = array.indexOf(element, idx + 1);
}
console.log(indices);
// [0, 2, 4]
```

Finding if an element exists in the array or not and updating the array

```js
function updateVegetablesCollection(veggies, veggie) {
  if (veggies.indexOf(veggie) === -1) {
    veggies.push(veggie);
    console.log(`New veggies collection is: ${veggies}`);
  } else {
    console.log(`${veggie} already exists in the veggies collection.`);
  }
}
```

(code continues on the next page)

(code continued from the previous page)

```
const veggies = ["potato", "tomato", "chillies", "green-pepper"];

updateVegetablesCollection(veggies, "spinach");
// New veggies collection is: potato,tomato,chillies,green-pepper,spinach
updateVegetablesCollection(veggies, "spinach");
// spinach already exists in the veggies collection.
```

Using indexOf() on sparse arrays

You cannot use `indexOf()` to search for empty slots in sparse arrays.

js

```
console.log([1, , 3].indexOf(undefined)); // -1
```

Calling indexOf() on non-array objects

The `indexOf()` method reads the `length` property of `this` and then accesses each property whose key is a nonnegative integer less than `length`.

js

```
const arrayLike = {
  length: 3,
  0: 2,
  1: 3,
  2: 4,
  3: 5, // ignored by indexOf() since length is 3
};
console.log(Array.prototype.indexOf.call(arrayLike, 2));
// 0
console.log(Array.prototype.indexOf.call(arrayLike, 5));
// -1
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/indexed-collections.html#sec-array.prototype.indexof

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`indexOf`	1	12	1.5	9.5	3	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Array.indexOf`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Polyfill of `Array.prototype.indexOf` in `core-js`
- es-shims polyfill of `Array.prototype.indexOf`

- Indexed collections guide
- `Array`
- `Array.prototype.findIndex()`
- `Array.prototype.findLastIndex()`
- `Array.prototype.lastIndexOf()`
- `TypedArray.prototype.indexOf()`
- `String.prototype.indexOf()`

JavaScript › Reference › Standard built-in objects › Array

Array.prototype.join()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/join · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since July 2015.

The **`join()`** method of `Array` instances returns a new string that is the concatenation of all elements in this array, separated by commas or a specified separator string. If the array has only one item, that item's stringification is returned without using the separator.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Array.prototype.join()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const elements = ["Fire", "Air", "Water"];

console.log(elements.join());
// Expected output: "Fire,Air,Water"

console.log(elements.join(""));
// Expected output: "FireAirWater"

console.log(elements.join("-"));
// Expected output: "Fire-Air-Water"
```

Syntax

js

```
join()
join(separator)
```

Parameters

`separator` Optional

A string to separate each pair of adjacent elements of the array. If omitted, the array elements are separated with a comma (",").

Return value

A string with all array elements joined. If `array.length` is `0`, the empty string is returned.

Description

The string conversions of all array elements are joined into one string. If an element is `undefined` or `null`, it is converted to an empty string instead of the string `"null"` or `"undefined"`.

The `join` method is accessed internally by `Array.prototype.toString()` with no arguments. Overriding `join` of an array instance will override its `toString` behavior as well.

`Array.prototype.join` recursively converts each element, including other arrays, to strings. Because the string returned by `Array.prototype.toString` (which is the same as calling `join()`) does not have delimiters, nested arrays look like they are flattened. You can only control the separator of the first level, while deeper levels always use the default comma.

```js
const matrix = [
  [1, 2, 3],
  [4, 5, 6],
  [7, 8, 9],
];

console.log(matrix.join()); // 1,2,3,4,5,6,7,8,9
console.log(matrix.join(";")); // 1,2,3;4,5,6;7,8,9
```

When an array is cyclic (it contains an element that is itself), browsers avoid infinite recursion by ignoring the cyclic reference.

```js
const arr = [];
arr.push(1, [3, arr, 4], 2);
console.log(arr.join(";")); // 1;3,,4;2
```

When used on sparse arrays, the `join()` method iterates empty slots as if they have the value `undefined`.

The `join()` method is generic. It only expects the `this` value to have a `length` property and integer-keyed properties.

Examples

Joining an array four different ways

The following example creates an array, `a`, with three elements, then joins the array four times: using the default separator, then a comma and a space, then a plus and an empty string.

```js
const a = ["Wind", "Water", "Fire"];
a.join(); // 'Wind,Water,Fire'
a.join(", "); // 'Wind, Water, Fire'
a.join(" + "); // 'Wind + Water + Fire'
a.join(""); // 'WindWaterFire'
```

Using join() on sparse arrays

`join()` treats empty slots the same as `undefined` and produces an extra separator:

```js
console.log([1, , 3].join()); // '1,,3'
console.log([1, undefined, 3].join()); // '1,,3'
```

Calling join() on non-array objects

The `join()` method reads the `length` property of `this` and then accesses each property whose key is a nonnegative integer less than `length`.

```js
const arrayLike = {
  length: 3,
  0: 2,
  1: 3,
  2: 4,
  3: 5, // ignored by join() since length is 3
};
console.log(Array.prototype.join.call(arrayLike));
// 2,3,4
console.log(Array.prototype.join.call(arrayLike, "."));
// 2.3.4
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/indexed-collections.html#sec-array.prototype.join`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`join`	1	12	1	4	1	18	4	10.1	1	1.0	1	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Array.join`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Polyfill of `Array.prototype.join` in `core-js`
- es-shims polyfill of `Array.prototype.join`
- Indexed collections guide
- `Array`
- `Array.prototype.toString()`
- `TypedArray.prototype.join()`
- `String.prototype.split()`

JavaScript › Reference › Standard built-in objects › Array

Array.prototype.keys()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/keys · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since July 2015.

The **`keys()`** method of `Array` instances returns a new *array iterator* object that contains the keys for each index in the array.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Array.prototype.keys()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const array = ["a", "b", "c"];
const iterator = array.keys();

for (const key of iterator) {
  console.log(key);
}

// Expected output: 0
// Expected output: 1
// Expected output: 2
```

Syntax

js

```
keys()
```

Parameters

None.

Return value

A new iterable iterator object.

Description

When used on sparse arrays, the `keys()` method iterates empty slots as if they have the value `undefined`.

The `keys()` method is generic. It only expects the `this` value to have a `length` property and integer-keyed properties.

Examples

Using keys() on sparse arrays

Unlike `Object.keys()`, which only includes keys that actually exist in the array, the `keys()` iterator doesn't ignore holes representing missing properties.

```js
const arr = ["a", , "c"];
const sparseKeys = Object.keys(arr);
const denseKeys = [...arr.keys()];
console.log(sparseKeys); // ['0', '2']
console.log(denseKeys); // [0, 1, 2]
```

Calling keys() on non-array objects

The `keys()` method reads the `length` property of `this` and then yields all integer indices between 0 and `length - 1`. No index access actually happens.

```js
const arrayLike = {
  length: 3,
};
for (const entry of Array.prototype.keys.call(arrayLike)) {
  console.log(entry);
}
// 0
// 1
// 2
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/indexed-collections.html#sec-array.prototype.keys`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`keys`	38	12	28	25	8	38	28	25	8	3.0	38	8	1.0.0	1.0	0.12.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Array.keys`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Polyfill of `Array.prototype.keys` in `core-js`
- es-shims polyfill of `Array.prototype.keys`
- Indexed collections guide
- `Array`
- `Array.prototype.entries()`
- `Array.prototype.values()`
- `Array.prototype[Symbol.iterator]()`
- `TypedArray.prototype.keys()`

- Iteration protocols

JavaScript › Reference › Standard built-in objects › Array

Array.prototype.lastIndexOf()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/lastIndexOf · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since July 2015.

The **`lastIndexOf()`** method of `Array` instances returns the last index at which a given element can be found in the array, or -1 if it is not present. The array is searched backwards, starting at `fromIndex`.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Array.prototype.lastIndexOf()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const animals = ["Dodo", "Tiger", "Penguin", "Dodo"];

console.log(animals.lastIndexOf("Dodo"));
// Expected output: 3

console.log(animals.lastIndexOf("Tiger"));
// Expected output: 1
```

Syntax

js

```
lastIndexOf(searchElement)
lastIndexOf(searchElement, fromIndex)
```

Parameters

`searchElement`

Element to locate in the array.

`fromIndex` Optional

Zero-based index at which to start searching backwards, converted to an integer.

- Negative index counts back from the end of the array — if `-array.length <= fromIndex < 0`, `fromIndex + array.length` is used.
- If `fromIndex < -array.length`, the array is not searched and `-1` is returned. You can think of it conceptually as starting at a nonexistent position before the beginning of the array and going backwards from there. There are no array elements on the way, so `searchElement` is never found.
- If `fromIndex >= array.length` or `fromIndex` is omitted or `undefined`, `array.length - 1` is used, causing the entire array to be searched. You can think of it conceptually as starting at a nonexistent position beyond the end of the array and going backwards from there. It eventually reaches the real end position of the array, at which point it starts searching backwards through the actual array elements.

Return value

The last index of `searchElement` in the array; `-1` if not found.

Description

The `lastIndexOf()` method compares `searchElement` to elements of the array using strict equality (the same algorithm used by the `===` operator). `NaN` values are never compared as equal, so `lastIndexOf()` always returns `-1` when `searchElement` is `NaN`.

The `lastIndexOf()` method skips empty slots in sparse arrays.

The `lastIndexOf()` method is generic. It only expects the `this` value to have a `length` property and integer-keyed properties.

Examples

Using lastIndexOf()

The following example uses `lastIndexOf()` to locate values in an array.

```js
const numbers = [2, 5, 9, 2];
numbers.lastIndexOf(2); // 3
numbers.lastIndexOf(7); // -1
numbers.lastIndexOf(2, 3); // 3
numbers.lastIndexOf(2, 2); // 0
numbers.lastIndexOf(2, -2); // 0
numbers.lastIndexOf(2, -1); // 3
```

You cannot use `lastIndexOf()` to search for `NaN`.

```js
const array = [NaN];
array.lastIndexOf(NaN); // -1
```

Finding all the occurrences of an element

The following example uses `lastIndexOf` to find all the indices of an element in a given array, using `push()` to add them to another array as they are found.

```js
const indices = [];
const array = ["a", "b", "a", "c", "a", "d"];
const element = "a";
let idx = array.lastIndexOf(element);
while (idx !== -1) {
  indices.push(idx);
  idx = idx > 0 ? array.lastIndexOf(element, idx - 1) : -1;
}

console.log(indices);
// [4, 2, 0]
```

Note that we have to handle the case `idx === 0` separately here because the element will always be found regardless of the `fromIndex` parameter if it is the first element of the array. This is different from the `indexOf()` method.

Using lastIndexOf() on sparse arrays

You cannot use `lastIndexOf()` to search for empty slots in sparse arrays.

```js
console.log([1, , 3].lastIndexOf(undefined)); // -1
```

Calling lastIndexOf() on non-array objects

The `lastIndexOf()` method reads the `length` property of `this` and then accesses each property whose key is a nonnegative integer less than `length`.

```js
const arrayLike = {
  length: 3,
  0: 2,
  1: 3,
  2: 2,
  3: 5, // ignored by lastIndexOf() since length is 3
};
console.log(Array.prototype.lastIndexOf.call(arrayLike, 2));
// 2
console.log(Array.prototype.lastIndexOf.call(arrayLike, 5));
// -1
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/indexed-collections.html#sec-array.prototype.lastindexof

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`lastIndexOf`	1	12	1.5	9.5	3	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Array.lastIndexOf`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Array.prototype.lastIndexOf` in `core-js`
- es-shims polyfill of `Array.prototype.lastIndexOf`
- Indexed collections guide
- `Array`
- `Array.prototype.findIndex()`
- `Array.prototype.findLastIndex()`
- `Array.prototype.indexOf()`

- `TypedArray.prototype.lastIndexOf()`
- `String.prototype.lastIndexOf()`

JavaScript › Reference › Standard built-in objects › Array

Array.prototype.map()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/map · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since July 2015.

The **`map()`** method of `Array` instances creates a new array populated with the results of calling a provided function on every element in the calling array.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Array.prototype.map()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const array = [1, 4, 9, 16];

// Pass a function to map
const mapped = array.map((x) => x * 2);

console.log(mapped);
// Expected output: Array [2, 8, 18, 32]
```

Syntax

js

```
map(callbackFn)
map(callbackFn, thisArg)
```

Parameters

`callbackFn`

A function to execute for each element in the array. Its return value is added as a single element in the new array. The function is called with the following arguments:

`element`

The current element being processed in the array.

`index`

The index of the current element being processed in the array.

`array`

The array `map()` was called upon.

`thisArg` Optional

A value to use as `this` when executing `callbackFn`. See iterative methods.

Return value

A new array with each element being the result of the callback function.

Description

The `map()` method is an iterative method. It calls a provided `callbackFn` function once for each element in an array and constructs a new array from the results. Read the iterative methods section for more information about how these methods work in general.

`callbackFn` is invoked only for array indexes which have assigned values. It is not invoked for empty slots in sparse arrays.

The `map()` method is generic. It only expects the `this` value to have a `length` property and integer-keyed properties.

Since `map` builds a new array, calling it without using the returned array is an anti-pattern; use `forEach` or `for...of` instead.

Examples

Mapping an array of numbers to an array of square roots

The following code takes an array of numbers and creates a new array containing the square roots of the numbers in the first array.

js

```
const numbers = [1, 4, 9];
const roots = numbers.map((num) => Math.sqrt(num));

// roots is now     [1, 2, 3]
// numbers is still [1, 4, 9]
```

Using map to reformat objects in an array

The following code takes an array of objects and creates a new array containing the newly reformatted objects.

js

```
const kvArray = [
  { key: 1, value: 10 },
  { key: 2, value: 20 },
  { key: 3, value: 30 },
];

const reformattedArray = kvArray.map(({ key, value }) => ({ [key]: value }));

console.log(reformattedArray); // [{ 1: 10 }, { 2: 20 }, { 3: 30 }]
console.log(kvArray);
// [
//   { key: 1, value: 10 },
//   { key: 2, value: 20 },
//   { key: 3, value: 30 }
// ]
```

Using parseInt() with map()

It is common to use the callback with one argument (the element being traversed). Certain functions are also commonly used with one argument, even though they take additional optional arguments. These habits may lead to confusing behaviors. Consider:

```js
["1", "2", "3"].map(parseInt);
```

While one might expect `[1, 2, 3]`, the actual result is `[1, NaN, NaN]`.

`parseInt` is often used with one argument, but takes two. The first is an expression and the second is the radix to the callback function, `Array.prototype.map` passes 3 arguments: the element, the index, and the array. The third argument is ignored by `parseInt` — but *not* the second one! This is the source of possible confusion.

Here is a concise example of the iteration steps:

```js
/* first iteration  (index is 0): */ parseInt("1", 0); // 1
/* second iteration (index is 1): */ parseInt("2", 1); // NaN
/* third iteration  (index is 2): */ parseInt("3", 2); // NaN
```

To solve this, define another function that only takes one argument:

```js
["1", "2", "3"].map((str) => parseInt(str, 10)); // [1, 2, 3]
```

You can also use the `Number` function, which only takes one argument:

```js
["1", "2", "3"].map(Number); // [1, 2, 3]

// But unlike parseInt(), Number() will also return a float or (resolved) exponential
notation:
["1.1", "2.2e2", "3e300"].map(Number); // [1.1, 220, 3e+300]

// For comparison, if we use parseInt() on the array above:
["1.1", "2.2e2", "3e300"].map((str) => parseInt(str, 10)); // [1, 2, 3]
```

See A JavaScript optional argument hazard by Allen Wirfs-Brock for more discussions.

Mapped array contains undefined

When `undefined` or nothing is returned, the resulting array contains `undefined`. If you want to delete the element instead, chain a `filter()` method, or use the `flatMap()` method and return an empty array to signify deletion.

```js
const numbers = [1, 2, 3, 4];
const filteredNumbers = numbers.map((num, index) => {
  if (index < 3) {
    return num;
  }
});

// index goes from 0, so the filteredNumbers are 1,2,3 and undefined.
// filteredNumbers is [1, 2, 3, undefined]
// numbers is still [1, 2, 3, 4]
```

Side-effectful mapping

The callback can have side effects.

```js
const cart = [5, 15, 25];
let total = 0;
const withTax = cart.map((cost) => {
  total += cost;
  return cost * 1.2;
});
console.log(withTax); // [6, 18, 30]
console.log(total); // 45
```

This is not recommended, because copying methods are best used with pure functions. In this case, we can choose to iterate the array twice.

```js
const cart = [5, 15, 25];
const total = cart.reduce((acc, cost) => acc + cost, 0);
const withTax = cart.map((cost) => cost * 1.2);
```

Sometimes this pattern goes to its extreme and the *only* useful thing that `map()` does is causing side effects.

```js
const products = [
  { name: "sports car" },
  { name: "laptop" },
  { name: "phone" },
];

products.map((product) => {
  product.price = 100;
});
```

As mentioned previously, this is an anti-pattern. If you don't use the return value of `map()`, use `forEach()` or a `for...of` loop instead.

```js
products.forEach((product) => {
  product.price = 100;
});
```

Or, if you want to create a new array instead:

```js
const productsWithPrice = products.map((product) => ({
  ...product,
  price: 100,
}));
```

Using the third argument of callbackFn

The `array` argument is useful if you want to access another element in the array, especially when you don't have an existing variable that refers to the array. The following example first uses `filter()` to extract the positive values and then uses `map()` to create a new array where each element is the average of its neighbors and itself.

```js
const numbers = [3, -1, 1, 4, 1, 5, 9, 2, 6];
const averaged = numbers
  .filter((num) => num > 0)
  .map((num, idx, arr) => {
    // Without the arr argument, there's no way to easily access the
    // intermediate array without saving it to a variable.
    const prev = arr[idx - 1];
    const next = arr[idx + 1];
    let count = 1;
    let total = num;
    if (prev !== undefined) {
      count++;
      total += prev;
    }
    if (next !== undefined) {
      count++;
      total += next;
    }
    const average = total / count;
    // Keep two decimal places
    return Math.round(average * 100) / 100;
  });
console.log(averaged); // [2, 2.67, 2, 3.33, 5, 5.33, 5.67, 4]
```

The `array` argument is *not* the array that is being built — there is no way to access the array being built from the callback function.

Using map() on sparse arrays

A sparse array remains sparse after `map()`. The indices of empty slots are still empty in the returned array, and the callback function won't be called on them.

```js
console.log(
  [1, , 3].map((x, index) => {
    console.log(`Visit ${index}`);
    return x * 2;
  }),
);
// Visit 0
// Visit 2
// [2, empty, 6]
```

Calling map() on non-array objects

The `map()` method reads the `length` property of `this` and then accesses each property whose key is a nonnegative integer less than `length`.

```js
const arrayLike = {
  length: 3,
  0: 2,
  1: 3,
  2: 4,
  3: 5, // ignored by map() since length is 3
};
console.log(Array.prototype.map.call(arrayLike, (x) => x ** 2));
// [ 4, 9, 16 ]
```

This example shows how to iterate through a collection of objects collected by `querySelectorAll()`, returning an array containing the values of all the selected `option` elements. Because `querySelectorAll()` returns a `NodeList`, which is an array-like object without the `map()` method, we use `Array.prototype.map.call()`, passing `elems` as the `this` value and the callback as the second argument.

```js
const elems = document.querySelectorAll("select option:checked");
const values = Array.prototype.map.call(elems, ({ value }) => value);
```

You can also use `Array.from()` to transform `elems` to an array, and then access the `map()` method.

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/indexed-collections.html#sec-array.prototype.map

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`map`	1	12	1.5	9.5	3	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Array.map`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Polyfill of `Array.prototype.map` in `core-js`
- es-shims polyfill of `Array.prototype.map`
- Indexed collections guide
- `Array`
- `Array.prototype.forEach()`
- `Array.from()`
- `TypedArray.prototype.map()`
- `Map`

JavaScript › Reference › Standard built-in objects › Array

Array.prototype.pop()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/pop · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **`pop()`** method of `Array` instances removes the **last** element from an array and returns that element. This method changes the length of the array.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Array.prototype.pop()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const plants = ["broccoli", "cauliflower", "cabbage", "kale", "tomato"];

console.log(plants.pop());
// Expected output: "tomato"

console.log(plants);
// Expected output: Array ["broccoli", "cauliflower", "cabbage", "kale"]

plants.pop();

console.log(plants);
// Expected output: Array ["broccoli", "cauliflower", "cabbage"]
```

Syntax

js

```
pop()
```

Parameters

None.

Return value

The removed element from the array; `undefined` if the array is empty.

Description

The `pop()` method removes the last element from an array and returns that value to the caller. If you call `pop()` on an empty array, it returns `undefined`.

`Array.prototype.shift()` has similar behavior to `pop()`, but applied to the first element in an array.

The `pop()` method is a mutating method. It changes the length and the content of `this`. In case you want the value of `this` to be the same, but return a new array with the last element removed, you can use `arr.slice(0, -1)` instead.

The `pop()` method is generic. It only expects the `this` value to have a `length` property and integer-keyed properties. Although strings are also array-like, this method is not suitable to be applied on them, as strings are immutable.

Examples

Removing the last element of an array

The following code creates the `myFish` array containing four elements, then removes its last element.

```js
const myFish = ["angel", "clown", "mandarin", "sturgeon"];

const popped = myFish.pop();

console.log(myFish); // ['angel', 'clown', 'mandarin' ]

console.log(popped); // 'sturgeon'
```

Calling pop() on non-array objects

The `pop()` method reads the `length` property of `this`. If the normalized length is 0, `length` is set to `0` again (whereas it may be negative or `undefined` before). Otherwise, the property at `length - 1` is returned and deleted.

```js
const arrayLike = {
  length: 3,
  unrelated: "foo",
  2: 4,
};
console.log(Array.prototype.pop.call(arrayLike));
// 4
console.log(arrayLike);
// { length: 2, unrelated: 'foo' }

const plainObj = {};
// There's no length property, so the length is 0
Array.prototype.pop.call(plainObj);
console.log(plainObj);
// { length: 0 }
```

Using an object in an array-like fashion

`push` and `pop` are intentionally generic, and we can use that to our advantage — as the following example shows.

Note that in this example, we don't create an array to store a collection of objects. Instead, we store the collection on the object itself and use `call` on `Array.prototype.push` and `Array.prototype.pop` to trick those methods into thinking we're dealing with an array.

```js
const collection = {
  length: 0,
  addElements(...elements) {
    // obj.length will be incremented automatically
    // every time an element is added.
```

(code continues on the next page)

(code continued from the previous page)

```
    // Returning what push returns; that is
    // the new value of length property.
    return [].push.call(this, ...elements);
  },
  removeElement() {
    // obj.length will be decremented automatically
    // every time an element is removed.

    // Returning what pop returns; that is
    // the removed element.
    return [].pop.call(this);
  },
};

collection.addElements(10, 20, 30);
console.log(collection.length); // 3
collection.removeElement();
console.log(collection.length); // 2
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/indexed-collections.html#sec-array.prototype.pop`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
pop	1	12	1	4	1	18	4	10.1	1	1.0	1	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Array.pop`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Indexed collections guide
- `Array`
- `Array.prototype.push()`
- `Array.prototype.shift()`
- `Array.prototype.unshift()`
- `Array.prototype.concat()`
- `Array.prototype.splice()`

JavaScript › Reference › Standard built-in objects › Array

Array.prototype.push()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/push · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since July 2015.

The **`push()`** method of `Array` instances adds the specified elements to the end of an array and returns the new length of the array.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Array.prototype.push()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const animals = ["pigs", "goats", "sheep"];

const count = animals.push("cows");
console.log(count);
// Expected output: 4
console.log(animals);
// Expected output: Array ["pigs", "goats", "sheep", "cows"]

animals.push("chickens", "cats", "dogs");
console.log(animals);
// Expected output: Array ["pigs", "goats", "sheep", "cows", "chickens", "cats", "dogs"]
```

Syntax

js

```
push()
push(element1)
push(element1, element2)
push(element1, element2, /* …, */ elementN)
```

Parameters

`element1, ..., elementN`

The element(s) to add to the end of the array.

Return value

The new `length` property of the object upon which the method was called.

Description

The `push()` method appends values to an array.

`Array.prototype.unshift()` has similar behavior to `push()`, but applied to the start of an array.

The `push()` method is a mutating method. It changes the length and the content of `this`. In case you want the value of `this` to be the same, but return a new array with elements appended to the end, you can use `arr.concat([element0, element1, /* ... ,*/ elementN])` instead. Notice that the elements are wrapped in an extra array — otherwise, if the element is an array itself, it would be spread instead of pushed as a single element due to the behavior of `concat()`.

The `push()` method is generic. It only expects the `this` value to have a `length` property and integer-keyed properties. Although strings are also array-like, this method is not suitable to be applied on them, as strings are immutable.

Examples

Adding elements to an array

The following code creates the `sports` array containing two elements, then appends two elements to it. The `total` variable contains the new length of the array.

```js
const sports = ["soccer", "baseball"];
const total = sports.push("football", "swimming");

console.log(sports); // ['soccer', 'baseball', 'football', 'swimming']
console.log(total); // 4
```

Merging two arrays

This example uses spread syntax to push all elements from a second array into the first one.

```js
const vegetables = ["parsnip", "potato"];
const moreVegs = ["celery", "beetroot"];

// Merge the second array into the first one
vegetables.push(...moreVegs);

console.log(vegetables); // ['parsnip', 'potato', 'celery', 'beetroot']
```

Merging two arrays can also be done with the `concat()` method, which will make a new combined array instead of adding to the original. The spread syntax only works if the number of elements in the array is less than the maximum number of function arguments allowed by the engine. For longer arrays, use `concat()` or call `push()` multiple times in a loop.

Calling push() on non-array objects

The `push()` method reads the `length` property of `this`. It then sets each index of `this` starting at `length` with the arguments passed to `push()`. Finally, it sets the `length` to the previous length plus the number of pushed elements.

```js
const arrayLike = {
  length: 3,
  unrelated: "foo",
  2: 4,
};
Array.prototype.push.call(arrayLike, 1, 2);
console.log(arrayLike);
// { '2': 4, '3': 1, '4': 2, length: 5, unrelated: 'foo' }

const plainObj = {};
// There's no length property, so the length is 0
Array.prototype.push.call(plainObj, 1, 2);
console.log(plainObj);
// { '0': 1, '1': 2, length: 2 }
```

Using an object in an array-like fashion

As mentioned above, `push` is intentionally generic, and we can use that to our advantage. `Array.prototype.push` can work on an object just fine, as this example shows.

Note that we don't create an array to store a collection of objects. Instead, we store the collection on the object itself and use `call` on `Array.prototype.push` to trick the method into thinking we are dealing with an array—and it just works, thanks to the way JavaScript allows us to establish the execution context in any way we want.

```js
const obj = {
  length: 0,

  addElem(elem) {
    // obj.length is automatically incremented
    // every time an element is added.
    [].push.call(this, elem);
  },
};

// Let's add some empty objects just to illustrate.
obj.addElem({});
obj.addElem({});
console.log(obj.length); // 2
```

Note that although `obj` is not an array, the method `push` successfully incremented `obj`'s `length` property just like if we were dealing with an actual array.

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/indexed-collections.html#sec-array.prototype.push`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`push`	1	12	1	4	1	18	4	10.1	1	1.0	1	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Array.push`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Polyfill of `Array.prototype.push` in `core-js` with fixes of this method
- es-shims polyfill of `Array.prototype.push`
- Indexed collections guide
- `Array`

- `Array.prototype.pop()`
- `Array.prototype.shift()`
- `Array.prototype.unshift()`
- `Array.prototype.concat()`
- `Array.prototype.splice()`

JavaScript › Reference › Standard built-in objects › Array

Array.prototype.reduce()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/reduce · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since July 2015.

The **`reduce()`** method of `Array` instances executes a user-supplied "reducer" callback function on each element of the array, in order, passing in the return value from the calculation on the preceding element. The final result of running the reducer across all elements of the array is a single value.

The first time that the callback is run there is no "return value of the previous calculation". If supplied, an initial value may be used in its place. Otherwise the array element at index 0 is used as the initial value and iteration starts from the next element (index 1 instead of index 0).

Try it

▷ Interactive example on MDN: "JavaScript Demo: Array.prototype.reduce()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const array = [1, 2, 3, 4];

// 0 + 1 + 2 + 3 + 4
const initialValue = 0;
const sumWithInitial = array.reduce(
  (accumulator, currentValue) => accumulator + currentValue,
  initialValue,
);

console.log(sumWithInitial);
// Expected output: 10
```

Syntax

js

```
reduce(callbackFn)
reduce(callbackFn, initialValue)
```

Parameters

`callbackFn`

A function to execute for each element in the array. Its return value becomes the value of the `accumulator` parameter on the next invocation of `callbackFn`. For the last invocation, the return value becomes the return value of `reduce()`. The function is called with the following arguments:

`accumulator`

The value resulting from the previous call to `callbackFn`. On the first call, its value is `initialValue` if the latter is specified; otherwise its value is `array[0]`.

`currentValue`

The value of the current element. On the first call, its value is `array[0]` if `initialValue` is specified; otherwise its value is `array[1]`.

`currentIndex`

The index position of `currentValue` in the array. On the first call, its value is `0` if `initialValue` is specified, otherwise `1`.

`array`

The array `reduce()` was called upon.

`initialValue` Optional

A value to which `accumulator` is initialized the first time the callback is called. If `initialValue` is specified, `callbackFn` starts executing with the first value in the array as `currentValue`. If `initialValue` is *not* specified, `accumulator` is initialized to the first value in the array, and `callbackFn` starts executing with the second value in the array as `currentValue`. In this case, if the array is empty (so that there's no first value to return as `accumulator`), an error is thrown.

Return value

The value that results from running the "reducer" callback function to completion over the entire array.

Exceptions

`TypeError`

Thrown if the array contains no elements and `initialValue` is not provided.

Description

The `reduce()` method is an iterative method. It runs a "reducer" callback function over all elements in the array, in ascending-index order, and accumulates them into a single value. Every time, the return value of `callbackFn` is passed into `callbackFn` again on next invocation as `accumulator`. The final value of `accumulator` (which is the value returned from `callbackFn` on the final iteration of the array) becomes the return value of `reduce()`. Read the iterative methods section for more information about how these methods work in general.

`callbackFn` is invoked only for array indexes which have assigned values. It is not invoked for empty slots in sparse arrays.

Unlike other iterative methods, `reduce()` does not accept a `thisArg` argument. `callbackFn` is always called with `undefined` as `this`, which gets substituted with `globalThis` if `callbackFn` is non-strict.

`reduce()` is a central concept in functional programming, where it's not possible to mutate any value, so in order to accumulate all values in an array, one must return a new accumulator value on every iteration. This convention propagates to JavaScript's `reduce()`: you should use spreading or other copying methods where possible to create new arrays and objects as the accumulator, rather than mutating the existing one. If you decided to mutate the accumulator instead of copying it, remember to still return the modified object in the callback, or the next iteration will receive undefined. However, note that copying the accumulator may in turn lead to increased memory usage and degraded

performance — see When to not use reduce() for more details. In such cases, to avoid bad performance and unreadable code, it's better to use a `for` loop instead.

The `reduce()` method is generic. It only expects the `this` value to have a `length` property and integer-keyed properties.

Edge cases

If the array only has one element (regardless of position) and no `initialValue` is provided, or if `initialValue` is provided but the array is empty, the solo value will be returned *without* calling `callbackFn`.

If `initialValue` is provided and the array is not empty, then the reduce method will always invoke the callback function starting at index 0.

If `initialValue` is not provided then the reduce method will act differently for arrays with length larger than 1, equal to 1 and 0, as shown in the following example:

```js
const getMax = (a, b) => Math.max(a, b);

// callback is invoked for each element in the array starting at index 0
[1, 100].reduce(getMax, 50); // 100
[50].reduce(getMax, 10); // 50

// callback is invoked once for element at index 1
[1, 100].reduce(getMax); // 100

// callback is not invoked
[50].reduce(getMax); // 50
[].reduce(getMax, 1); // 1

[].reduce(getMax); // TypeError
```

Examples

How reduce() works without an initial value

The code below shows what happens if we call `reduce()` with an array and no initial value.

```js
const array = [15, 16, 17, 18, 19];

function reducer(accumulator, currentValue, index) {
  const returns = accumulator + currentValue;
  console.log(
    `accumulator: ${accumulator}, currentValue: ${currentValue}, index: ${index}, returns:
${returns}`,
  );
  return returns;
}

array.reduce(reducer);
```

The callback would be invoked four times, with the arguments and return values in each call being as follows:

	accumulator	currentValue	index	Return value
First call	15	16	1	31
Second call	31	17	2	48
Third call	48	18	3	66
Fourth call	66	19	4	85

The `array` parameter never changes through the process — it's always `[15, 16, 17, 18, 19]`. The value returned by `reduce()` would be that of the last callback invocation (`85`).

How reduce() works with an initial value

Here we reduce the same array using the same algorithm, but with an `initialValue` of `10` passed as the second argument to `reduce()`:

```js
[15, 16, 17, 18, 19].reduce(
  (accumulator, currentValue) => accumulator + currentValue,
  10,
);
```

The callback would be invoked five times, with the arguments and return values in each call being as follows:

	accumulator	currentValue	index	Return value
First call	10	15	0	25
Second call	25	16	1	41
Third call	41	17	2	58
Fourth call	58	18	3	76
Fifth call	76	19	4	95

The value returned by `reduce()` in this case would be `95`.

Sum of values in an object array

To sum up the values contained in an array of objects, you **must** supply an `initialValue`, so that each item passes through your function.

```js
const objects = [{ x: 1 }, { x: 2 }, { x: 3 }];
const sum = objects.reduce(
  (accumulator, currentValue) => accumulator + currentValue.x,
  0,
);

console.log(sum); // 6
```

Function sequential piping

The `pipe` function takes a sequence of functions and returns a new function. When the new function is called with an argument, the sequence of functions are called in order, which each one receiving the return value of the previous function.

```js
const pipe =
  (...functions) =>
  (initialValue) =>
    functions.reduce((acc, fn) => fn(acc), initialValue);

// Building blocks to use for composition
const double = (x) => 2 * x;
const triple = (x) => 3 * x;
const quadruple = (x) => 4 * x;

// Composed functions for multiplication of specific values
const multiply6 = pipe(double, triple);
const multiply9 = pipe(triple, triple);
const multiply16 = pipe(quadruple, quadruple);
const multiply24 = pipe(double, triple, quadruple);

// Usage
multiply6(6); // 36
multiply9(9); // 81
multiply16(16); // 256
multiply24(10); // 240
```

Running promises in sequence

Promise sequencing is essentially function piping demonstrated in the previous section, except done asynchronously.

```js
// Compare this with pipe: fn(acc) is changed to acc.then(fn),
// and initialValue is ensured to be a promise
const asyncPipe =
  (...functions) =>
  (initialValue) =>
    functions.reduce((acc, fn) => acc.then(fn), Promise.resolve(initialValue));

// Building blocks to use for composition
const p1 = async (a) => a * 5;
const p2 = async (a) => a * 2;
// The composed functions can also return non-promises, because the values are
// all eventually wrapped in promises
const f3 = (a) => a * 3;
const p4 = async (a) => a * 4;

asyncPipe(p1, p2, f3, p4)(10).then(console.log); // 1200
```

`asyncPipe` can also be implemented using `async/await`, which better demonstrates its similarity with `pipe`:

```js
const asyncPipe =
  (...functions) =>
  (initialValue) =>
    functions.reduce(async (acc, fn) => fn(await acc), initialValue);
```

Using reduce() with sparse arrays

`reduce()` skips missing elements in sparse arrays, but it does not skip `undefined` values.

```js
console.log([1, 2, , 4].reduce((a, b) => a + b)); // 7
console.log([1, 2, undefined, 4].reduce((a, b) => a + b)); // NaN
```

Calling reduce() on non-array objects

The `reduce()` method reads the `length` property of `this` and then accesses each property whose key is a nonnegative integer less than `length`.

```js
const arrayLike = {
  length: 3,
  0: 2,
  1: 3,
  2: 4,
  3: 99, // ignored by reduce() since length is 3
};
console.log(Array.prototype.reduce.call(arrayLike, (x, y) => x + y));
// 9
```

When to not use reduce()

Multipurpose higher-order functions like `reduce()` can be powerful but sometimes difficult to understand, especially for less-experienced JavaScript developers. If code becomes clearer when using other array methods, developers must weigh the readability tradeoff against the other benefits of using `reduce()`.

Note that `reduce()` is always equivalent to a `for...of` loop, except that instead of mutating a variable in the upper scope, we now return the new value for each iteration:

```js
const val = array.reduce((acc, cur) => update(acc, cur), initialValue);

// Is equivalent to:
let val = initialValue;
for (const cur of array) {
  val = update(val, cur);
}
```

As previously stated, the reason why people may want to use `reduce()` is to mimic functional programming practices of immutable data. Therefore, developers who uphold the immutability of the accumulator often copy the entire accumulator for each iteration, like this:

js · ✗ bad example

```js
const names = ["Alice", "Bob", "Tiff", "Bruce", "Alice"];
const countedNames = names.reduce((allNames, name) => {
  const currCount = Object.hasOwn(allNames, name) ? allNames[name] : 0;
  return {
    ...allNames,
    [name]: currCount + 1,
  };
}, {});
```

This code is ill-performing, because each iteration has to copy the entire `allNames` object, which could be big, depending how many unique names there are. This code has worst-case `O(N^2)` performance, where `N` is the length of `names`.

A better alternative is to *mutate* the `allNames` object on each iteration. However, if `allNames` gets mutated anyway, you may want to convert the `reduce()` to a `for` loop instead, which is much clearer:

js · ✗ bad example

```
const names = ["Alice", "Bob", "Tiff", "Bruce", "Alice"];
const countedNames = names.reduce((allNames, name) => {
  const currCount = allNames[name] ?? 0;
  allNames[name] = currCount + 1;
  // return allNames, otherwise the next iteration receives undefined
  return allNames;
}, Object.create(null));
```

js · ✓ good example

```
const names = ["Alice", "Bob", "Tiff", "Bruce", "Alice"];
const countedNames = Object.create(null);
for (const name of names) {
  const currCount = countedNames[name] ?? 0;
  countedNames[name] = currCount + 1;
}
```

Therefore, if your accumulator is an array or an object and you are copying the array or object on each iteration, you may accidentally introduce quadratic complexity into your code, causing performance to quickly degrade on large data. This has happened in real-world code — see for example Making Tanstack Table 1000x faster with a 1 line change.

Some of the acceptable use cases of `reduce()` are given above (most notably, summing an array, promise sequencing, and function piping). There are other cases where better alternatives than `reduce()` exist.

- Flattening an array of arrays. Use `flat()` instead.

  js · ✗ bad example

  ```
  const flattened = array.reduce((acc, cur) => acc.concat(cur), []);
  ```

  js · ✓ good example

  ```
  const flattened = array.flat();
  ```

- Grouping objects by a property. Use `Object.groupBy()` instead.

  js · ✗ bad example

  ```
  const groups = array.reduce((acc, obj) => {
    const key = obj.name;
    const curGroup = acc[key] ?? [];
    return { ...acc, [key]: [...curGroup, obj] };
  }, {});
  ```

  js · ✓ good example

  ```
  const groups = Object.groupBy(array, (obj) => obj.name);
  ```

- Concatenating arrays contained in an array of objects. Use `flatMap()` instead.

js · ✗ bad example

```
const friends = [
  { name: "Anna", books: ["Bible", "Harry Potter"] },
  { name: "Bob", books: ["War and peace", "Romeo and Juliet"] },
  { name: "Alice", books: ["The Lord of the Rings", "The Shining"] },
];
const allBooks = friends.reduce((acc, cur) => [...acc, ...cur.books], []);
```

js · ✓ good example

```
const allBooks = friends.flatMap((person) => person.books);
```

- Removing duplicate items in an array. Use `Set` and `Array.from()` instead.

js · ✗ bad example

```
const uniqArray = array.reduce(
  (acc, cur) => (acc.includes(cur) ? acc : [...acc, cur]),
  [],
);
```

js · ✓ good example

```
const uniqArray = Array.from(new Set(array));
```

- Eliminating or adding elements in an array. Use `flatMap()` instead.

js · ✗ bad example

```
// Takes an array of numbers and splits perfect squares into its square roots
const roots = array.reduce((acc, cur) => {
  if (cur < 0) return acc;
  const root = Math.sqrt(cur);
  if (Number.isInteger(root)) return [...acc, root, root];
  return [...acc, cur];
}, []);
```

js · ✓ good example

```
const roots = array.flatMap((val) => {
  if (val < 0) return [];
  const root = Math.sqrt(val);
  if (Number.isInteger(root)) return [root, root];
  return [val];
});
```

  If you are only eliminating elements from an array, you also can use `filter()`.

- Searching for elements or testing if elements satisfy a condition. Use `find()` and `findIndex()`, or `some()` and `every()` instead. These methods have the additional benefit that they return as soon as the result is certain, without iterating the entire array.

js · ✗ bad example

```
const allEven = array.reduce((acc, cur) => acc && cur % 2 === 0, true);
```

js · ✓ good example

```
const allEven = array.every((val) => val % 2 === 0);
```

In cases where `reduce()` is the best choice, documentation and semantic variable naming can help mitigate readability drawbacks.

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/indexed-collections.html#sec-array.prototype.reduce`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`reduce`	3	12	3	10.5	4	18	4	14	3.2	1.0	4.4	3.2	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Array.reduce`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Array.prototype.reduce` in `core-js`
- es-shims polyfill of `Array.prototype.reduce`
- Indexed collections guide
- `Array`
- `Array.prototype.map()`
- `Array.prototype.flat()`
- `Array.prototype.flatMap()`
- `Array.prototype.reduceRight()`
- `TypedArray.prototype.reduce()`
- `Object.groupBy()`
- `Map.groupBy()`

JavaScript › Reference › Standard built-in objects › Array

Array.prototype.reduceRight()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/reduceRight · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since July 2015.

The **`reduceRight()`** method of `Array` instances applies a function against an accumulator and each value of the array (from right-to-left) to reduce it to a single value.

See also `Array.prototype.reduce()` for left-to-right.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Array.prototype.reduceRight()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const array = [
  [0, 1],
  [2, 3],
  [4, 5],
];

const result = array.reduceRight((accumulator, currentValue) =>
  accumulator.concat(currentValue),
);

console.log(result);
// Expected output: Array [4, 5, 2, 3, 0, 1]
```

Syntax

js

```
reduceRight(callbackFn)
reduceRight(callbackFn, initialValue)
```

Parameters

`callbackFn`

A function to execute for each element in the array. Its return value becomes the value of the `accumulator` parameter on the next invocation of `callbackFn`. For the last invocation, the return value becomes the return value of `reduceRight()`. The function is called with the following arguments:

`accumulator`

The value resulting from the previous call to `callbackFn`. On the first call, its value is `initialValue` if the latter is specified; otherwise its value is the last element of the array.

`currentValue`

The value of the current element. On the first call, its value is the last element if `initialValue` is specified; otherwise its value is the second-to-last element.

`currentIndex`

The index position of `currentValue` in the array. On the first call, its value is `array.length - 1` if `initialValue` is specified, otherwise `array.length - 2`.

`array`

The array `reduceRight()` was called upon.

`initialValue` Optional

Value to use as accumulator to the first call of the `callbackFn`. If no initial value is supplied, the last element in the array will be used and skipped. Calling `reduceRight()` on an empty array without an initial value creates a `TypeError`.

Return value

The value that results from the reduction.

Description

The `reduceRight()` method is an iterative method. It runs a "reducer" callback function over all elements in the array, in descending-index order, and accumulates them into a single value. Read the iterative methods section for more information about how these methods work in general.

`callbackFn` is invoked only for array indexes which have assigned values. It is not invoked for empty slots in sparse arrays.

Unlike other iterative methods, `reduceRight()` does not accept a `thisArg` argument. `callbackFn` is always called with `undefined` as `this`, which gets substituted with `globalThis` if `callbackFn` is non-strict.

The `reduceRight()` method is generic. It only expects the `this` value to have a `length` property and integer-keyed properties.

All caveats about `reduce` discussed in when to not use reduce() apply to `reduceRight` as well. Because JavaScript has no lazy evaluation semantics, there is no performance difference between `reduce` and `reduceRight`.

Examples

How reduceRight() works without an initial value

The call to the reduceRight `callbackFn` would look something like this:

```js
arr.reduceRight((accumulator, currentValue, index, array) => {
  // …
});
```

The first time the function is called, the `accumulator` and `currentValue` can be one of two values. If an `initialValue` was provided in the call to `reduceRight`, then `accumulator` will be equal to `initialValue` and `currentValue` will be equal to the last value in the array. If no `initialValue` was provided, then `accumulator` will be equal to the last value in the array and `currentValue` will be equal to the second-to-last value.

If the array is empty and no `initialValue` was provided, `TypeError` would be thrown. If the array has only one element (regardless of position) and no `initialValue` was provided, or if `initialValue` is provided but the array is empty, the solo value would be returned without calling `callbackFn`.

Some example run-throughs of the function would look like this:

```js
[0, 1, 2, 3, 4].reduceRight(
  (accumulator, currentValue, index, array) => accumulator + currentValue,
);
```

The callback would be invoked four times, with the arguments and return values in each call being as follows:

	accumulator	currentValue	index	Return value
First call	4	3	3	7
Second call	7	2	2	9
Third call	9	1	1	10
Fourth call	10	0	0	10

The `array` parameter never changes through the process — it's always `[0, 1, 2, 3, 4]`. The value returned by `reduceRight` would be that of the last callback invocation (`10`).

How reduceRight() works with an initial value

Here we reduce the same array using the same algorithm, but with an `initialValue` of `10` passed as the second argument to `reduceRight()`:

```js
[0, 1, 2, 3, 4].reduceRight(
  (accumulator, currentValue, index, array) => accumulator + currentValue,
  10,
);
```

	accumulator	currentValue	index	Return value
First call	10	4	4	14
Second call	14	3	3	17
Third call	17	2	2	19
Fourth call	19	1	1	20
Fifth call	20	0	0	20

The value returned by `reduceRight` this time would be, of course, `20`.

Sum up all values within an array

```js
const sum = [0, 1, 2, 3].reduceRight((a, b) => a + b);
// sum is 6
```

Run a list of asynchronous functions with callbacks in series each passing their results to the next

```js
const waterfall =
  (...functions) =>
  (callback, ...args) =>
    functions.reduceRight(
      (composition, fn) =>
        (...results) =>
          fn(composition, ...results),
      callback,
    )(...args);

const randInt = (max) => Math.floor(Math.random() * max);

const add5 = (callback, x) => {
  setTimeout(callback, randInt(1000), x + 5);
};
```

(code continues on the next page)

(code continued from the previous page)

```
const mul3 = (callback, x) => {
  setTimeout(callback, randInt(1000), x * 3);
};
const sub2 = (callback, x) => {
  setTimeout(callback, randInt(1000), x - 2);
};
const split = (callback, x) => {
  setTimeout(callback, randInt(1000), x, x);
};
const add = (callback, x, y) => {
  setTimeout(callback, randInt(1000), x + y);
};
const div4 = (callback, x) => {
  setTimeout(callback, randInt(1000), x / 4);
};

const computation = waterfall(add5, mul3, sub2, split, add, div4);
computation(console.log, 5); // Logs 14

// same as:

const computation2 = (input, callback) => {
  const f6 = (x) => div4(callback, x);
  const f5 = (x, y) => add(f6, x, y);
  const f4 = (x) => split(f5, x);
  const f3 = (x) => sub2(f4, x);
  const f2 = (x) => mul3(f3, x);
  add5(f2, input);
};
```

Difference between reduce and reduceRight

js

```
const a = ["1", "2", "3", "4", "5"];
const left = a.reduce((prev, cur) => prev + cur);
const right = a.reduceRight((prev, cur) => prev + cur);

console.log(left); // "12345"
console.log(right); // "54321"
```

Defining composable functions

Function composition is a mechanism for combining functions, in which the output of each function is passed into the next one, and the output of the last function is the final result. In this example we use `reduceRight()` to implement function composition.

See also Function composition on Wikipedia.

js

```
const compose =
  (...args) =>
  (value) =>
    args.reduceRight((acc, fn) => fn(acc), value);

// Increment passed number
const inc = (n) => n + 1;
```

(code continues on the next page)

(code continued from the previous page)

```
// Doubles the passed value
const double = (n) => n * 2;

// using composition function
console.log(compose(double, inc)(2)); // 6

// using composition function
console.log(compose(inc, double)(2)); // 5
```

Using reduceRight() with sparse arrays

`reduceRight()` skips missing elements in sparse arrays, but it does not skip `undefined` values.

js

```
console.log([1, 2, , 4].reduceRight((a, b) => a + b)); // 7
console.log([1, 2, undefined, 4].reduceRight((a, b) => a + b)); // NaN
```

Calling reduceRight() on non-array objects

The `reduceRight()` method reads the `length` property of `this` and then accesses each property whose key is a nonnegative integer less than `length`.

js

```
const arrayLike = {
  length: 3,
  0: 2,
  1: 3,
  2: 4,
  3: 99, // ignored by reduceRight() since length is 3
};
console.log(Array.prototype.reduceRight.call(arrayLike, (x, y) => x - y));
// -1, which is 4 - 3 - 2
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/indexed-collections.html#sec-array.prototype.reduceright

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`reduceRight`	3	12	3	10.5	4	18	4	14	3.2	1.0	4.4	3.2	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Array.reduceRight`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Array.prototype.reduceRight` in `core-js`
- es-shims polyfill of `Array.prototype.reduceRight`
- Indexed collections guide
- `Array`
- `Array.prototype.map()`
- `Array.prototype.flat()`
- `Array.prototype.flatMap()`
- `Array.prototype.reduce()`
- `TypedArray.prototype.reduceRight()`
- `Object.groupBy()`
- `Map.groupBy()`

JavaScript › Reference › Standard built-in objects › Array

Array.prototype.reverse()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/reverse · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since July 2015.

The **`reverse()`** method of `Array` instances reverses an array *in place* and returns the reference to the same array, the first array element now becoming the last, and the last array element becoming the first. In other words, elements order in the array will be turned towards the direction opposite to that previously stated.

To reverse the elements in an array without mutating the original array, use `toReversed()`.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Array.prototype.reverse()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const array = ["one", "two", "three"];
console.log("array:", array);
// Expected output: "array:" Array ["one", "two", "three"]

const reversed = array.reverse();
console.log("reversed:", reversed);
// Expected output: "reversed:" Array ["three", "two", "one"]

// Careful: reverse is destructive -- it changes the original array.
console.log("array:", array);
// Expected output: "array:" Array ["three", "two", "one"]
```

Syntax

js

```
reverse()
```

Parameters

None.

Return value

The reference to the original array, now reversed. Note that the array is reversed *in place*, and no copy is made.

Description

The `reverse()` method transposes the elements of the calling array object in place, mutating the array, and returning a reference to the array.

The `reverse()` method preserves empty slots. If the source array is sparse, the empty slots' corresponding new indices are deleted and also become empty slots.

The `reverse()` method is generic. It only expects the `this` value to have a `length` property and integer-keyed properties. Although strings are also array-like, this method is not suitable to be applied on them, as strings are immutable.

Examples

Reversing the elements in an array

The following example creates an array `items`, containing three elements, then reverses the array. The call to `reverse()` returns a reference to the reversed array `items`.

```js
const items = [1, 2, 3];
console.log(items); // [1, 2, 3]

items.reverse();
console.log(items); // [3, 2, 1]
```

The reverse() method returns the reference to the same array

The `reverse()` method returns reference to the original array, so mutating the returned array will mutate the original array as well.

```js
const numbers = [3, 2, 4, 1, 5];
const reversed = numbers.reverse();
// numbers and reversed are both in reversed order [5, 1, 4, 2, 3]
reversed[0] = 5;
console.log(numbers[0]); // 5
```

In case you want `reverse()` to not mutate the original array, but return a shallow-copied array like other array methods (e.g., `map()`) do, use the `toReversed()` method. Alternatively, you can do a shallow copy before calling `reverse()`, using the spread syntax or `Array.from()`.

```js
const numbers = [3, 2, 4, 1, 5];
// [...numbers] creates a shallow copy, so reverse() does not mutate the original
const reverted = [...numbers].reverse();
reverted[0] = 5;
console.log(numbers[0]); // 3
```

Using reverse() on sparse arrays

Sparse arrays remain sparse after calling `reverse()`. Empty slots are copied over to their respective new indices as empty slots.

```js
console.log([1, , 3].reverse()); // [3, empty, 1]
console.log([1, , 3, 4].reverse()); // [4, 3, empty, 1]
```

Calling reverse() on non-array objects

The `reverse()` method reads the `length` property of `this`. It then visits each property having an integer key between `0` and `length / 2`, and swaps the two corresponding indices on both ends, deleting any destination property for which the source property did not exist.

```js
const arrayLike = {
  length: 3,
  unrelated: "foo",
  2: 4,
  3: 33, // ignored by reverse() since length is 3
};
console.log(Array.prototype.reverse.call(arrayLike));
// { 0: 4, 3: 33, length: 3, unrelated: 'foo' }
// The index 2 is deleted because there was no index 0 present originally
// The index 3 is unchanged since the length is 3
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/indexed-collections.html#sec-array.prototype.reverse`

Browser compatibility

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`reverse`	1	12	1	4	1	18	4	10.1	1	1.0	1	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Array.reverse`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Polyfill of `Array.prototype.reverse` in `core-js`
- es-shims polyfill of `Array.prototype.reverse`
- Indexed collections guide
- `Array`
- `Array.prototype.join()`

- Array.prototype.sort()
- Array.prototype.toReversed()
- TypedArray.prototype.reverse()

JavaScript › Reference › Standard built-in objects › Array

Array.prototype.shift()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/shift · MDN page last modified 2025-07-20

Baseline: Widely available — available across major browsers since July 2015.

The **`shift()`** method of Array instances removes the **first** element from an array and returns that removed element. This method changes the length of the array.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Array.prototype.shift()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const array = [1, 2, 3];

const firstElement = array.shift();

console.log(array);
// Expected output: Array [2, 3]

console.log(firstElement);
// Expected output: 1
```

Syntax

js

```
shift()
```

Parameters

None.

Return value

The removed element from the array; undefined if the array is empty.

Description

The `shift()` method shifts all values to the left by 1 and decrements the length by 1, resulting in the first element being removed. If the `length` property is 0, undefined is returned.

The pop() method has similar behavior to `shift()`, but applied to the last element in an array.

The `shift()` method is a mutating method. It changes the length and the content of `this`. In case you want the value of `this` to be the same, but return a new array with the first element removed, you can use arr.slice(1) instead.

The `shift()` method is generic. It only expects the `this` value to have a `length` property and integer-keyed properties. Although strings are also array-like, this method is not suitable to be applied on them, as strings are immutable.

Examples

Removing an element from an array

The following code displays the `myFish` array before and after removing its first element. It also displays the removed element:

```js
const myFish = ["angel", "clown", "mandarin", "surgeon"];

console.log("myFish before:", myFish);
// myFish before: ['angel', 'clown', 'mandarin', 'surgeon']

const shifted = myFish.shift();

console.log("myFish after:", myFish);
// myFish after: ['clown', 'mandarin', 'surgeon']

console.log("Removed this element:", shifted);
// Removed this element: angel
```

Using shift() method in while loop

The shift() method is often used in condition inside while loop. In the following example every iteration will remove the next element from an array, until it is empty:

```js
const names = ["Andrew", "Tyrone", "Paul", "Maria", "Gayatri"];

while (typeof (i = names.shift()) !== "undefined") {
  console.log(i);
}
// Andrew, Tyrone, Paul, Maria, Gayatri
```

Calling shift() on non-array objects

The `shift()` method reads the `length` property of `this`. If the normalized length is 0, `length` is set to `0` again (whereas it may be negative or `undefined` before). Otherwise, the property at `0` is returned, and the rest of the properties are shifted left by one. The property at `length - 1` is deleted, and the `length` property is decremented by one.

```js
const arrayLike = {
  length: 3,
  unrelated: "foo",
  2: 4,
};
console.log(Array.prototype.shift.call(arrayLike));
// undefined, because it is an empty slot
console.log(arrayLike);
// { '1': 4, length: 2, unrelated: 'foo' }

const plainObj = {};
// There's no length property, so the length is 0
Array.prototype.shift.call(plainObj);
console.log(plainObj);
// { length: 0 }
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/indexed-collections.html#sec-array.prototype.shift

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
shift	1	12	1	4	1	18	4	10.1	1	1.0	1	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Array.shift`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Indexed collections guide
- `Array`
- `Array.prototype.push()`
- `Array.prototype.pop()`
- `Array.prototype.unshift()`
- `Array.prototype.concat()`
- `Array.prototype.splice()`

JavaScript › Reference › Standard built-in objects › Array

Array.prototype.slice()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/slice · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since July 2015.

The **`slice()`** method of `Array` instances returns a shallow copy of a portion of an array into a new array object selected from `start` to `end` (`end` not included) where `start` and `end` represent the index of items in that array. The original array will not be modified.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Array.prototype.slice()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const animals = ["ant", "bison", "camel", "duck", "elephant"];

console.log(animals.slice(2));
// Expected output: Array ["camel", "duck", "elephant"]

console.log(animals.slice(2, 4));
// Expected output: Array ["camel", "duck"]
```

(code continues on the next page)

(code continued from the previous page)

```
console.log(animals.slice(1, 5));
// Expected output: Array ["bison", "camel", "duck", "elephant"]

console.log(animals.slice(-2));
// Expected output: Array ["duck", "elephant"]

console.log(animals.slice(2, -1));
// Expected output: Array ["camel", "duck"]

console.log(animals.slice());
// Expected output: Array ["ant", "bison", "camel", "duck", "elephant"]
```

Syntax

js

```
slice()
slice(start)
slice(start, end)
```

Parameters

`start` Optional

Zero-based index at which to start extraction, converted to an integer.

- Negative index counts back from the end of the array — if `-array.length <= start < 0`, `start + array.length` is used.
- If `start < -array.length` or `start` is omitted, `0` is used.
- If `start >= array.length`, an empty array is returned.

`end` Optional

Zero-based index at which to end extraction, converted to an integer. `slice()` extracts up to but not including `end`.

- Negative index counts back from the end of the array — if `-array.length <= end < 0`, `end + array.length` is used.
- If `end < -array.length`, `0` is used.
- If `end >= array.length` or `end` is omitted or `undefined`, `array.length` is used, causing all elements until the end to be extracted.
- If `end` implies a position before or at the position that `start` implies, an empty array is returned.

Return value

A new array containing the extracted elements.

Description

The `slice()` method is a copying method. It does not alter `this` but instead returns a shallow copy that contains some of the same elements as the ones from the original array.

The `slice()` method preserves empty slots. If the sliced portion is sparse, the returned array is sparse as well.

The `slice()` method is generic. It only expects the `this` value to have a `length` property and integer-keyed properties.

Examples

Return a portion of an existing array

```js
const fruits = ["Banana", "Orange", "Lemon", "Apple", "Mango"];
const citrus = fruits.slice(1, 3);

// fruits contains ['Banana', 'Orange', 'Lemon', 'Apple', 'Mango']
// citrus contains ['Orange','Lemon']
```

In this example, `slice(1, 3)` extracts elements from index `1` up to, but not including, index `3`, resulting in a new array `['Orange', 'Lemon']`.

Omitting the end parameter

```js
const fruits = ["Apple", "Banana", "Orange", "Mango", "Pineapple"];

const tropical = fruits.slice(2);
console.log(tropical); // ['Orange', 'Mango', 'Pineapple']
```

In this example, `slice(2)` extracts elements from index `2` to the end of the array.

Using negative indices

```js
const fruits = ["Apple", "Banana", "Orange", "Mango", "Pineapple"];

const lastTwo = fruits.slice(-2);
console.log(lastTwo); // ['Mango', 'Pineapple']
```

In this example, `slice(-2)` extracts the last two elements of the array. When using a negative index with the `slice` method, negative indices are counted from the end of the array, starting at `-1` for the last element, `-2` for the second-to-last element, and so on. The negative index `-2` itself is included because it is the starting point of the extraction.

```
|     |     |     |     |     |
|  S  |  L  |  I  |  C  |  E  |
|     |     |     |     |     |
  -5    -4    -3    -2    -1

<--- read from reverse
```

Using a positive start index and a negative end index

```js
const fruits = ["Apple", "Banana", "Orange", "Mango", "Pineapple"];

// Using positive start index and negative end index
const sliceExample = fruits.slice(1, -1);
console.log(sliceExample); // ['Banana', 'Orange', 'Mango']
```

In this example, `slice(1, -1)` starts extracting from index `1` and goes up to, but does not include, the element at index `-1` (which is the last element). This results in a new array with `['Banana', 'Orange',`

'Mango']. The slice method always excludes the element at the final index specified, regardless of whether it is positive or negative.

```
read from start --->

   0     1     2     3     4
|     |     |     |     |     |
|  S  |  L  |  I  |  C  |  E  |
|     |     |     |     |     |
  -5    -4    -3    -2    -1

<--- read from reverse
```

Using slice with arrays of objects

In the following example, slice creates a new array, newCar, from myCar. Both include a reference to the object myHonda. When the color of myHonda is changed to purple, both arrays reflect the change.

js

```
// Using slice, create newCar from myCar.
const myHonda = {
  color: "red",
  wheels: 4,
  engine: { cylinders: 4, size: 2.2 },
};
const myCar = [myHonda, 2, "cherry condition", "purchased 1997"];
const newCar = myCar.slice(0, 2);

console.log("myCar =", myCar);
console.log("newCar =", newCar);
console.log("myCar[0].color =", myCar[0].color);
console.log("newCar[0].color =", newCar[0].color);

// Change the color of myHonda.
myHonda.color = "purple";
console.log("The new color of my Honda is", myHonda.color);

console.log("myCar[0].color =", myCar[0].color);
console.log("newCar[0].color =", newCar[0].color);
```

This script writes:

```
myCar = [
  { color: 'red', wheels: 4, engine: { cylinders: 4, size: 2.2 } },
  2,
  'cherry condition',
  'purchased 1997'
]
newCar = [ { color: 'red', wheels: 4, engine: { cylinders: 4, size: 2.2 } }, 2 ]
myCar[0].color = red
newCar[0].color = red
The new color of my Honda is purple
myCar[0].color = purple
newCar[0].color = purple
```

Calling slice() on non-array objects

The `slice()` method reads the `length` property of `this`. It then reads the integer-keyed properties from `start` to `end` and defines them on a newly created array.

```js
const arrayLike = {
  length: 3,
  0: 2,
  1: 3,
  2: 4,
  3: 33, // ignored by slice() since length is 3
};
console.log(Array.prototype.slice.call(arrayLike, 1, 3));
// [ 3, 4 ]
```

Using slice() to convert array-like objects to arrays

The `slice()` method is often used with `bind()` and `call()` to create a utility method that converts an array-like object into an array.

```js
// slice() is called with `this` passed as the first argument
const slice = Function.prototype.call.bind(Array.prototype.slice);

function list() {
  return slice(arguments);
}

const listResult = list(1, 2, 3); // [1, 2, 3]
```

Using slice() on sparse arrays

The array returned from `slice()` may be sparse if the source is sparse.

```js
console.log([1, 2, , 4, 5].slice(1, 4)); // [2, empty, 4]
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/indexed-collections.html#sec-array.prototype.slice

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`slice`	1	12	1	4	1	18	4	10.1	1	1.0	1	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Array.slice`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Polyfill of `Array.prototype.slice` in `core-js`
- es-shims polyfill of `Array.prototype.slice`
- Indexed collections guide
- `Array`
- `Array.prototype.pop()`
- `Array.prototype.shift()`
- `Array.prototype.concat()`
- `Array.prototype.splice()`
- `TypedArray.prototype.slice()`
- `String.prototype.slice()`

JavaScript › Reference › Standard built-in objects › Array

Array.prototype.some()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/some · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since July 2015.

The **`some()`** method of `Array` instances returns `true` if it finds an element in the array that satisfies the provided testing function. Otherwise, it returns `false`.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Array.prototype.some()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const array = [1, 2, 3, 4, 5];

// Checks whether an element is even
const even = (element) => element % 2 === 0;

console.log(array.some(even));
// Expected output: true
```

Syntax

js

```
some(callbackFn)
some(callbackFn, thisArg)
```

Parameters

`callbackFn`

A function to execute for each element in the array. It should return a truthy value to indicate the element passes the test, and a falsy value otherwise. The function is called with the following arguments:

`element`

The current element being processed in the array.

`index`

The index of the current element being processed in the array.

`array`

The array `some()` was called upon.

`thisArg` Optional

A value to use as `this` when executing `callbackFn`. See iterative methods.

Return value

`false` unless `callbackFn` returns a truthy value for an array element, in which case `true` is immediately returned.

Description

The `some()` method is an iterative method. It calls a provided `callbackFn` function once for each element in an array, until the `callbackFn` returns a truthy value. If such an element is found, `some()` immediately returns `true` and stops iterating through the array. Otherwise, if `callbackFn` returns a falsy value for all elements, `some()` returns `false`. Read the iterative methods section for more information about how these methods work in general.

`some()` acts like the "there exists" quantifier in mathematics. In particular, for an empty array, it returns `false` for any condition.

`callbackFn` is invoked only for array indexes which have assigned values. It is not invoked for empty slots in sparse arrays.

`some()` does not mutate the array on which it is called, but the function provided as `callbackFn` can. Note, however, that the length of the array is saved *before* the first invocation of `callbackFn`. Therefore:

- `callbackFn` will not visit any elements added beyond the array's initial length when the call to `some()` began.
- Changes to already-visited indexes do not cause `callbackFn` to be invoked on them again.
- If an existing, yet-unvisited element of the array is changed by `callbackFn`, its value passed to the `callbackFn` will be the value at the time that element gets visited. Deleted elements are not visited.

> **Warning:** Concurrent modifications of the kind described above frequently lead to hard-to-understand code and are generally to be avoided (except in special cases).

The `some()` method is generic. It only expects the `this` value to have a `length` property and integer-keyed properties.

Examples

Testing value of array elements

The following example tests whether any element in the array is bigger than 10.

```js
function isBiggerThan10(element, index, array) {
  return element > 10;
}

[2, 5, 8, 1, 4].some(isBiggerThan10); // false
[12, 5, 8, 1, 4].some(isBiggerThan10); // true
```

Testing array elements using arrow functions

Arrow functions provide a shorter syntax for the same test.

```js
[2, 5, 8, 1, 4].some((x) => x > 10); // false
[12, 5, 8, 1, 4].some((x) => x > 10); // true
```

Checking whether a value exists in an array

To mimic the function of the `includes()` method, this custom function returns `true` if the element exists in the array:

```js
const fruits = ["apple", "banana", "mango", "guava"];

function checkAvailability(arr, val) {
  return arr.some((arrVal) => val === arrVal);
}

checkAvailability(fruits, "grapefruit"); // false
checkAvailability(fruits, "banana"); // true
```

Converting any value to Boolean

```js
const TRUTHY_VALUES = [true, "true", 1];

function getBoolean(value) {
  if (typeof value === "string") {
    value = value.toLowerCase().trim();
  }

  return TRUTHY_VALUES.some((t) => t === value);
}

getBoolean(false); // false
getBoolean("false"); // false
getBoolean(1); // true
getBoolean("true"); // true
```

Using the third argument of callbackFn

The `array` argument is useful if you want to access another element in the array, especially when you don't have an existing variable that refers to the array. The following example first uses `filter()` to extract the positive values and then uses `some()` to check whether the array is strictly increasing.

```js
const numbers = [3, -1, 1, 4, 1, 5];
const isIncreasing = !numbers
  .filter((num) => num > 0)
  .some((num, idx, arr) => {
    // Without the arr argument, there's no way to easily access the
    // intermediate array without saving it to a variable.
    if (idx === 0) return false;
    return num <= arr[idx - 1];
  });
console.log(isIncreasing); // false
```

Using some() on sparse arrays

`some()` will not run its predicate on empty slots.

```js
console.log([1, , 3].some((x) => x === undefined)); // false
console.log([1, , 1].some((x) => x !== 1)); // false
console.log([1, undefined, 1].some((x) => x !== 1)); // true
```

Calling some() on non-array objects

The `some()` method reads the `length` property of `this` and then accesses each property whose key is a nonnegative integer less than `length` until they all have been accessed or `callbackFn` returns `true`.

```js
const arrayLike = {
  length: 3,
  0: "a",
  1: "b",
  2: "c",
  3: 3, // ignored by some() since length is 3
};
console.log(Array.prototype.some.call(arrayLike, (x) => typeof x === "number"));
// false
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/indexed-collections.html#sec-array.prototype.some

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
some	1	12	1.5	9.5	3	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Array.some`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Polyfill of `Array.prototype.some` in `core-js`
- es-shims polyfill of `Array.prototype.some`
- Indexed collections guide
- `Array`
- `Array.prototype.every()`
- `Array.prototype.forEach()`
- `Array.prototype.find()`
- `Array.prototype.includes()`
- `TypedArray.prototype.some()`

JavaScript › Reference › Standard built-in objects › Array

Array.prototype.sort()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/sort · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since July 2015.

The **`sort()`** method of `Array` instances sorts the elements of an array *in place* and returns the reference to the same array, now sorted. The default sort order is ascending, built upon converting the elements into strings, then comparing their sequences of UTF-16 code unit values.

The time and space complexity of the sort cannot be guaranteed as it depends on the implementation.

To sort the elements in an array without mutating the original array, use `toSorted()`.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Array.prototype.sort()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const months = ["March", "Jan", "Feb", "Dec"];
months.sort();
console.log(months);
// Expected output: Array ["Dec", "Feb", "Jan", "March"]

const array = [1, 30, 4, 21, 100000];
array.sort();
console.log(array);
// Expected output: Array [1, 100000, 21, 30, 4]
```

Syntax

js

```
sort()
sort(compareFn)
```

Parameters

`compareFn` Optional

A function that determines the order of the elements. The function is called with the following arguments:

a

The first element for comparison. Will never be `undefined`.

b

The second element for comparison. Will never be `undefined`.

It should return a number where:

- A negative value indicates that `a` should come before `b`.
- A positive value indicates that `a` should come after `b`.
- Zero or `NaN` indicates that `a` and `b` are considered equal.

To memorize this, remember that `(a, b) => a - b` sorts numbers in ascending order.

If omitted, the array elements are converted to strings, then sorted according to each character's Unicode code point value.

Return value

The reference to the original array, now sorted. Note that the array is sorted *in place*, and no copy is made.

Description

If `compareFn` is not supplied, all non-`undefined` array elements are sorted by converting them to strings and comparing strings in UTF-16 code units order. For example, "banana" comes before "cherry". In a numeric sort, 9 comes before 80, but because numbers are converted to strings, "80" comes before "9" in the Unicode order. All `undefined` elements are sorted to the end of the array.

The `sort()` method preserves empty slots. If the source array is sparse, the empty slots are moved to the end of the array, and always come after all the `undefined`.

> **Note:** In UTF-16, Unicode characters above `\uFFFF` are encoded as two surrogate code units, of the range `\uD800` - `\uDFFF`. The value of each code unit is taken separately into account for the comparison. Thus the character formed by the surrogate pair `\uD855\uDE51` will be sorted before the character `\uFF3A`.

If `compareFn` is supplied, all non-`undefined` array elements are sorted according to the return value of the compare function (all `undefined` elements are sorted to the end of the array, with no call to `compareFn`).

`compareFn(a, b)` return value	sort order
> 0	sort `a` after `b`, e.g., `[b, a]`
< 0	sort `a` before `b`, e.g., `[a, b]`
=== 0	keep original order of `a` and `b`

So, the compare function has the following form:

```js
function compareFn(a, b) {
  if (a is less than b by some ordering criterion) {
    return -1;
  } else if (a is greater than b by the ordering criterion) {
    return 1;
  }
  // a must be equal to b
  return 0;
}
```

More formally, the comparator is expected to have the following properties, in order to ensure proper sort behavior:

- *Pure*: The comparator does not mutate the objects being compared or any external state. (This is important because there's no guarantee *when* and *how* the comparator will be called, so any particular call should not produce visible effects to the outside.)
- *Stable*: The comparator returns the same result with the same pair of input.
- *Reflexive*: `compareFn(a, a) === 0`.
- *Anti-symmetric*: `compareFn(a, b)` and `compareFn(b, a)` must both be `0` or have opposite signs.
- *Transitive*: If `compareFn(a, b)` and `compareFn(b, c)` are both positive, zero, or negative, then `compareFn(a, c)` has the same positivity as the previous two.

A comparator conforming to the constraints above will always be able to return all of `1`, `0`, and `-1`, or consistently return `0`. For example, if a comparator only returns `1` and `0`, or only returns `0` and `-1`, it will not be able to sort reliably because *anti-symmetry* is broken. A comparator that always returns `0` will cause the array to not be changed at all, but is reliable nonetheless.

The default lexicographic comparator satisfies all constraints above.

To compare numbers instead of strings, the compare function can subtract `b` from `a`. The following function will sort the array in ascending order (if it doesn't contain `NaN`):

```js
function compareNumbers(a, b) {
  return a - b;
}
```

The `sort()` method is generic. It only expects the `this` value to have a `length` property and integer-keyed properties. Although strings are also array-like, this method is not suitable to be applied on them, as strings are immutable.

Examples

Creating, displaying, and sorting an array

The following example creates four arrays and displays the original array, then the sorted arrays. The numeric arrays are sorted without a compare function, then sorted using one.

```js
const stringArray = ["Blue", "Humpback", "Beluga"];
const numberArray = [40, 1, 5, 200];
const numericStringArray = ["80", "9", "700"];
const mixedNumericArray = ["80", "9", "700", 40, 1, 5, 200];
```

(code continues on the next page)

(code continued from the previous page)

```
function compareNumbers(a, b) {
  return a - b;
}

stringArray.join(); // 'Blue,Humpback,Beluga'
stringArray.sort(); // ['Beluga', 'Blue', 'Humpback']

numberArray.join(); // '40,1,5,200'
numberArray.sort(); // [1, 200, 40, 5]
numberArray.sort(compareNumbers); // [1, 5, 40, 200]

numericStringArray.join(); // '80,9,700'
numericStringArray.sort(); // ['700', '80', '9']
numericStringArray.sort(compareNumbers); // ['9', '80', '700']

mixedNumericArray.join(); // '80,9,700,40,1,5,200'
mixedNumericArray.sort(); // [1, 200, 40, 5, '700', '80', '9']
mixedNumericArray.sort(compareNumbers); // [1, 5, '9', 40, '80', 200, '700']
```

Sorting array of objects

Arrays of objects can be sorted by comparing the value of one of their properties.

js

```
const items = [
  { name: "Edward", value: 21 },
  { name: "Sharpe", value: 37 },
  { name: "And", value: 45 },
  { name: "The", value: -12 },
  { name: "Magnetic", value: 13 },
  { name: "Zeros", value: 37 },
];

// sort by value
items.sort((a, b) => a.value - b.value);

// sort by name
items.sort((a, b) => {
  const nameA = a.name.toUpperCase(); // ignore upper and lowercase
  const nameB = b.name.toUpperCase(); // ignore upper and lowercase
  if (nameA < nameB) {
    return -1;
  }
  if (nameA > nameB) {
    return 1;
  }

  // names must be equal
  return 0;
});
```

Sorting non-ASCII characters

For sorting strings with non-ASCII characters, i.e., strings with accented characters (e, é, è, a, ä, etc.), strings from languages other than English, use `String.prototype.localeCompare()`. This function can compare those characters so they appear in the right order.

```js
const items = ["réservé", "premier", "communiqué", "café", "adieu", "éclair"];
items.sort((a, b) => a.localeCompare(b));

// items is ['adieu', 'café', 'communiqué', 'éclair', 'premier', 'réservé']
```

Sorting with map

The `compareFn` can be invoked multiple times per element within the array. Depending on the `compareFn`'s nature, this may yield a high overhead. The more work a `compareFn` does and the more elements there are to sort, it may be more efficient to use `map()` for sorting. The idea is to traverse the array once to extract the actual values used for sorting into a temporary array, sort the temporary array, and then traverse the temporary array to achieve the right order.

```js
// the array to be sorted
const data = ["delta", "alpha", "charlie", "bravo"];

// temporary array holds objects with position and sort-value
const mapped = data.map((v, i) => ({ i, value: someSlowOperation(v) }));

// sorting the mapped array containing the reduced values
mapped.sort((a, b) => {
  if (a.value > b.value) {
    return 1;
  }
  if (a.value < b.value) {
    return -1;
  }
  return 0;
});

const result = mapped.map((v) => data[v.i]);
```

There is an open source library available called mapsort which applies this approach.

sort() returns the reference to the same array

The `sort()` method returns a reference to the original array, so mutating the returned array will mutate the original array as well.

```js
const numbers = [3, 1, 4, 1, 5];
const sorted = numbers.sort((a, b) => a - b);
// numbers and sorted are both [1, 1, 3, 4, 5]
sorted[0] = 10;
console.log(numbers[0]); // 10
```

In case you want `sort()` to not mutate the original array, but return a shallow-copied array like other array methods (e.g., `map()`) do, use the `toSorted()` method. Alternatively, you can do a shallow copy before calling `sort()`, using the spread syntax or `Array.from()`.

```js
const numbers = [3, 1, 4, 1, 5];
// [...numbers] creates a shallow copy, so sort() does not mutate the original
const sorted = [...numbers].sort((a, b) => a - b);
sorted[0] = 10;
console.log(numbers[0]); // 3
```

Sort stability

Since version 10 (or ECMAScript 2019), the specification dictates that `Array.prototype.sort` is stable.

For example, say you had a list of students alongside their grades. Note that the list of students is already pre-sorted by name in alphabetical order:

```js
const students = [
  { name: "Alex", grade: 15 },
  { name: "Devlin", grade: 15 },
  { name: "Eagle", grade: 13 },
  { name: "Sam", grade: 14 },
];
```

After sorting this array by `grade` in ascending order:

```js
students.sort((firstItem, secondItem) => firstItem.grade - secondItem.grade);
```

The `students` variable will then have the following value:

```js
[
  { name: "Eagle", grade: 13 },
  { name: "Sam", grade: 14 },
  { name: "Alex", grade: 15 }, // original maintained for similar grade (stable sorting)
  { name: "Devlin", grade: 15 }, // original maintained for similar grade (stable sorting)
];
```

It's important to note that students that have the same grade (for example, Alex and Devlin), will remain in the same order as before calling the sort. This is what a stable sorting algorithm guarantees.

Before version 10 (or ECMAScript 2019), sort stability was not guaranteed, meaning that you could end up with the following:

```js
[
  { name: "Eagle", grade: 13 },
  { name: "Sam", grade: 14 },
  { name: "Devlin", grade: 15 }, // original order not maintained
  { name: "Alex", grade: 15 }, // original order not maintained
];
```

Sorting with non-well-formed comparator

If a comparing function does not satisfy all of purity, stability, reflexivity, anti-symmetry, and transitivity rules, as explained in the description, the program's behavior is not well-defined.

For example, consider this code:

```js
const arr = [3, 1, 4, 1, 5, 9];
const compareFn = (a, b) => (a > b ? 1 : 0);
arr.sort(compareFn);
```

The `compareFn` function here is not well-formed, because it does not satisfy anti-symmetry: if `a > b`, it returns `1`; but by swapping `a` and `b`, it returns `0` instead of a negative value. Therefore, the resulting

array will be different across engines. For example, V8 (used by Chrome, Node.js, etc.) and JavaScriptCore (used by Safari) would not sort the array at all and return `[3, 1, 4, 1, 5, 9]`, while SpiderMonkey (used by Firefox) will return the array sorted ascendingly, as `[1, 1, 3, 4, 5, 9]`.

However, if the `compareFn` function is changed slightly so that it returns `-1` or `0`:

```js
const arr = [3, 1, 4, 1, 5, 9];
const compareFn = (a, b) => (a > b ? -1 : 0);
arr.sort(compareFn);
```

Then V8 and JavaScriptCore sorts it descendingly, as `[9, 5, 4, 3, 1, 1]`, while SpiderMonkey returns it as-is: `[3, 1, 4, 1, 5, 9]`.

Due to this implementation inconsistency, you are always advised to make your comparator well-formed by following the five constraints.

Using sort() on sparse arrays

Empty slots are moved to the end of the array.

```js
console.log(["a", "c", , "b"].sort()); // ['a', 'b', 'c', empty]
console.log([, undefined, "a", "b"].sort()); // ["a", "b", undefined, empty]
```

Calling sort() on non-array objects

The `sort()` method reads the `length` property of `this`. It then collects all existing integer-keyed properties in the range of `0` to `length - 1`, sorts them, and writes them back. If there are missing properties in the range, the corresponding trailing properties are deleted, as if the non-existent properties are sorted towards the end.

```js
const arrayLike = {
  length: 3,
  unrelated: "foo",
  0: 5,
  2: 4,
};
console.log(Array.prototype.sort.call(arrayLike));
// { '0': 4, '1': 5, length: 3, unrelated: 'foo' }
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/indexed-collections.html#sec-array.prototype.sort

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`sort`	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Stable sorting	70	79	3	57	10.1	70	4	49	10.3	10.0	70	10.3	1.0.0	1.0	12.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Array.sort`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Array.prototype.sort` with modern behavior like stable sort in `core-js`
- Indexed collections guide
- `Array`
- `Array.prototype.reverse()`
- `Array.prototype.toSorted()`
- `String.prototype.localeCompare()`
- `TypedArray.prototype.sort()`
- Getting things sorted in V8 on v8.dev (2018)
- Stable `Array.prototype.sort` on v8.dev (2019)
- `Array.prototype.sort` stability by Mathias Bynens

JavaScript › Reference › Standard built-in objects › Array

Array.prototype.splice()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/splice · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **`splice()`** method of `Array` instances changes the contents of an array by removing or replacing existing elements and/or adding new elements in place.

To create a new array with a segment removed and/or replaced without mutating the original array, use `toSpliced()`. To access part of an array without modifying it, see `slice()`.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Array.prototype.splice()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const months = ["Jan", "March", "April", "June"];
months.splice(1, 0, "Feb");
// Inserts at index 1
console.log(months);
// Expected output: Array ["Jan", "Feb", "March", "April", "June"]

months.splice(4, 1, "May");
// Replaces 1 element at index 4
console.log(months);
// Expected output: Array ["Jan", "Feb", "March", "April", "May"]
```

Syntax

js

```
splice(start)
splice(start, deleteCount)
splice(start, deleteCount, item1)
splice(start, deleteCount, item1, item2)
splice(start, deleteCount, item1, item2, /* …, */ itemN)
```

Parameters

`start`

Zero-based index at which to start changing the array, converted to an integer.

- Negative index counts back from the end of the array — if `-array.length <= start < 0`, `start + array.length` is used.
- If `start < -array.length`, `0` is used.
- If `start >= array.length`, no element will be deleted, but the method will behave as an adding function, adding as many elements as provided.
- If `start` is omitted (and `splice()` is called with no arguments), nothing is deleted. This is different from passing `undefined`, which is converted to `0`.

`deleteCount` Optional

An integer indicating the number of elements in the array to remove from `start`.

If `deleteCount` is omitted, or if its value is greater than or equal to the number of elements after the position specified by `start`, then all the elements from `start` to the end of the array will be deleted. However, if you wish to pass any `itemN` parameter, you should pass `Infinity` as `deleteCount` to delete all elements after `start`, because an explicit `undefined` gets converted to `0`.

If `deleteCount` is `0` or negative, no elements are removed. In this case, you should specify at least one new element (see below).

`item1, ..., itemN` Optional

The elements to add to the array, beginning from `start`.

If you do not specify any elements, `splice()` will only remove elements from the array.

Return value

An array containing the deleted elements.

If only one element is removed, an array of one element is returned.

If no elements are removed, an empty array is returned.

Description

The `splice()` method is a mutating method. It may change the content of `this`. If the specified number of elements to insert differs from the number of elements being removed, the array's `length` will be changed as well. At the same time, it uses `[Symbol.species]` to create a new array instance to be returned.

If the deleted portion is sparse, the array returned by `splice()` is sparse as well, with those corresponding indices being empty slots.

The `splice()` method is generic. It only expects the `this` value to have a `length` property and integer-keyed properties. Although strings are also array-like, this method is not suitable to be applied on them, as strings are immutable.

Examples

Remove 0 (zero) elements before index 2, and insert "drum"

```js
const myFish = ["angel", "clown", "mandarin", "sturgeon"];
const removed = myFish.splice(2, 0, "drum");

// myFish is ["angel", "clown", "drum", "mandarin", "sturgeon"]
// removed is [], no elements removed
```

Remove 0 (zero) elements before index 2, and insert "drum" and "guitar"

```js
const myFish = ["angel", "clown", "mandarin", "sturgeon"];
const removed = myFish.splice(2, 0, "drum", "guitar");

// myFish is ["angel", "clown", "drum", "guitar", "mandarin", "sturgeon"]
// removed is [], no elements removed
```

Remove 0 (zero) elements at index 0, and insert "angel"

`splice(0, 0, ...elements)` inserts elements at the start of the array like `unshift()`.

```js
const myFish = ["clown", "mandarin", "sturgeon"];
const removed = myFish.splice(0, 0, "angel");

// myFish is ["angel", "clown", "mandarin", "sturgeon"]
// no items removed
```

Remove 0 (zero) elements at last index, and insert "sturgeon"

`splice(array.length, 0, ...elements)` inserts elements at the end of the array like `push()`.

```js
const myFish = ["angel", "clown", "mandarin"];
const removed = myFish.splice(myFish.length, 0, "sturgeon");

// myFish is ["angel", "clown", "mandarin", "sturgeon"]
// no items removed
```

Remove 1 element at index 3

```js
const myFish = ["angel", "clown", "drum", "mandarin", "sturgeon"];
const removed = myFish.splice(3, 1);

// myFish is ["angel", "clown", "drum", "sturgeon"]
// removed is ["mandarin"]
```

Remove 1 element at index 2, and insert "trumpet"

```js
const myFish = ["angel", "clown", "drum", "sturgeon"];
const removed = myFish.splice(2, 1, "trumpet");

// myFish is ["angel", "clown", "trumpet", "sturgeon"]
// removed is ["drum"]
```

Remove 2 elements from index 0, and insert "parrot", "anemone" and "blue"

```js
const myFish = ["angel", "clown", "trumpet", "sturgeon"];
const removed = myFish.splice(0, 2, "parrot", "anemone", "blue");

// myFish is ["parrot", "anemone", "blue", "trumpet", "sturgeon"]
// removed is ["angel", "clown"]
```

Remove 2 elements, starting from index 2

```js
const myFish = ["parrot", "anemone", "blue", "trumpet", "sturgeon"];
const removed = myFish.splice(2, 2);

// myFish is ["parrot", "anemone", "sturgeon"]
// removed is ["blue", "trumpet"]
```

Remove 1 element from index -2

```js
const myFish = ["angel", "clown", "mandarin", "sturgeon"];
const removed = myFish.splice(-2, 1);

// myFish is ["angel", "clown", "sturgeon"]
// removed is ["mandarin"]
```

Remove all elements, starting from index 2

```js
const myFish = ["angel", "clown", "mandarin", "sturgeon"];
const removed = myFish.splice(2);

// myFish is ["angel", "clown"]
// removed is ["mandarin", "sturgeon"]
```

Using splice() on sparse arrays

The `splice()` method preserves the array's sparseness.

```js
const arr = [1, , 3, 4, , 6];
console.log(arr.splice(1, 2)); // [empty, 3]
console.log(arr); // [1, 4, empty, 6]
```

Calling splice() on non-array objects

The `splice()` method reads the `length` property of `this`. It then updates the integer-keyed properties and the `length` property as needed.

```js
const arrayLike = {
  length: 3,
  unrelated: "foo",
  0: 5,
  2: 4,
};
console.log(Array.prototype.splice.call(arrayLike, 0, 1, 2, 3));
// [ 5 ]
console.log(arrayLike);
// { '0': 2, '1': 3, '3': 4, length: 4, unrelated: 'foo' }
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/indexed-collections.html#sec-array.prototype.splice

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
splice	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Array.splice`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Indexed collections guide
- `Array`
- `Array.prototype.concat()`
- `Array.prototype.push()`
- `Array.prototype.pop()`
- `Array.prototype.shift()`
- `Array.prototype.slice()`
- `Array.prototype.toSpliced()`
- `Array.prototype.unshift()`

JavaScript › Reference › Standard built-in objects › Array

Array.prototype.toLocaleString()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/toLocaleString · MDN page last modified 2025-09-24

Baseline: Widely available — available across major browsers since July 2015.

The **`toLocaleString()`** method of `Array` instances returns a string representing the elements of the array. The elements are converted to strings using their `toLocaleString` methods and these strings are separated by a locale-specific string (such as a comma ",").

Try it

▷ Interactive example on MDN: "JavaScript Demo: Array.prototype.toLocaleString()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const array = [1, "a", new Date("21 Dec 1997 14:12:00 UTC")];
const localeString = array.toLocaleString("en", { timeZone: "UTC" });

console.log(localeString);
// Expected output: "1,a,12/21/1997, 2:12:00 PM",
// This assumes "en" locale and UTC timezone - your results may vary
```

Syntax

js

```
toLocaleString()
toLocaleString(locales)
toLocaleString(locales, options)
```

Parameters

`locales` Optional

A string with a BCP 47 language tag, or an array of such strings. For the general form and interpretation of the `locales` argument, see the parameter description on the `Intl` main page.

`options` Optional

An object with configuration properties. What you can pass here depends on what elements are being converted. For example, for numbers, see `Number.prototype.toLocaleString()`.

Return value

A string representing the elements of the array.

Description

The `Array.prototype.toLocaleString` method traverses its content, calling the `toLocaleString` method of every element with the `locales` and `options` parameters provided, and concatenates them with an implementation-defined separator (such as a comma ",").

> **Note:** The `locales` or `options` arguments do not control the separator used between array elements; they are simply passed to the `toLocaleString()` method of each element. The actual separator (usually a comma) depends solely on the host's current locale. If you expect localized list formatting, consider using `Intl.ListFormat` instead.

If an element is `undefined`, `null`, it is converted to an empty string instead of the string `"null"` or `"undefined"`.

When used on sparse arrays, the `toLocaleString()` method iterates empty slots as if they have the value `undefined`.

The `toLocaleString()` method is generic. It only expects the `this` value to have a `length` property and integer-keyed properties.

Examples

Using locales and options

The elements of the array are converted to strings using their `toLocaleString` methods. For example, this snippet implicitly calls the `Number.prototype.toLocaleString()` method to display the currency for the strings and numbers in the `prices` array:

```js
const prices = ["￥7", 500, 8123, 12];
prices.toLocaleString("ja-JP", { style: "currency", currency: "JPY" });

// "￥7,￥500,￥8,123,￥12"
```

List separators

The list separator is not affected by the `locales` parameter. To configure it, use `Intl.ListFormat` instead.

```js
const nums = [8888, 9999];
console.log(nums.toLocaleString("zh")); // "8,888,9,999"

const formatter = new Intl.ListFormat("zh", {
  type: "conjunction",
  style: "narrow",
});
console.log(formatter.format(nums.map((x) => x.toLocaleString("zh"))));
// "8,888、9,999"
```

Using toLocaleString() on sparse arrays

`toLocaleString()` treats empty slots the same as `undefined` and produces an extra separator:

```js
console.log([1, , 3].toLocaleString()); // '1,,3'
```

Calling toLocaleString() on non-array objects

The `toLocaleString()` method reads the `length` property of `this` and then accesses each property whose key is a nonnegative integer less than `length`.

```js
const arrayLike = {
  length: 3,
  0: 1,
  1: 2,
  2: 3,
  3: 4, // ignored by toLocaleString() since length is 3
};
console.log(Array.prototype.toLocaleString.call(arrayLike));
// 1,2,3
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/indexed-collections.html#sec-array.prototype.tolocalestring`
ECMAScript® 2027 Internationalization API Specification `https://tc39.es/ecma402/#sup-array.prototype.tolocalestring`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`toLocaleString`	1	12	1	4	1	18	4	10.1	1	1.0	1	1	1.0.0	1.0	0.10.0
`locales` parameter	24	79	52	15	7	25	56	14	7	2.0	4.4	7	1.0.0	1.8[1,2]	13.0.0[3,4]
`options` parameter	24	79	52	15	7	25	56	14	7	2.0	4.4	7	1.0.0	1.0	0.12.0

1. Also from version 1.0–1.8: partial support.
2. Also from version 1.0–1.8: Only the locale data for `en-US` is available.
3. Also from version 0.12.0–13.0.0: partial support.
4. Also from version 0.12.0–13.0.0: Before version 13.0.0, only the locale data for `en-US` is available by default. When other locales are specified, the function silently falls back to `en-US`. To make full ICU (locale) data available before version 13, see Node.js documentation on the `--with-intl` option and how to provide the data.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Array.toLocaleString`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Indexed collections guide
- `Array`
- `Array.prototype.toString()`
- `TypedArray.prototype.toLocaleString()`
- `Intl`
- `Intl.ListFormat`
- `Object.prototype.toLocaleString()`
- `Number.prototype.toLocaleString()`
- `Temporal.PlainDate.prototype.toLocaleString()`

JavaScript › Reference › Standard built-in objects › Array

Array.prototype.toReversed()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/toReversed · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since July 2023.

The **`toReversed()`** method of `Array` instances is the copying counterpart of the `reverse()` method. It returns a new array with the elements in reversed order.

Syntax

```
toReversed()
```

Parameters

None.

Return value

A new array containing the elements in reversed order.

Description

The `toReversed()` method transposes the elements of the calling array object in reverse order and returns a new array.

When used on sparse arrays, the `toReversed()` method iterates empty slots as if they have the value `undefined`.

The `toReversed()` method is generic. It only expects the `this` value to have a `length` property and integer-keyed properties.

Examples

Reversing the elements in an array

The following example creates an array `items`, containing three elements, then creates a new array that's the reverse of `items`. The `items` array remains unchanged.

```js
const items = [1, 2, 3];
console.log(items); // [1, 2, 3]

const reversedItems = items.toReversed();
console.log(reversedItems); // [3, 2, 1]
console.log(items); // [1, 2, 3]
```

Using toReversed() on sparse arrays

The return value of `toReversed()` is never sparse. Empty slots become `undefined` in the returned array.

```js
console.log([1, , 3].toReversed()); // [3, undefined, 1]
console.log([1, , 3, 4].toReversed()); // [4, 3, undefined, 1]
```

Calling toReversed() on non-array objects

The `toReversed()` method reads the `length` property of `this`. It then visits each property having an integer key between `length - 1` and `0` in descending order, adding the value of the current property to the end of the array to be returned.

```js
const arrayLike = {
  length: 3,
  unrelated: "foo",
  2: 4,
};
console.log(Array.prototype.toReversed.call(arrayLike));
// [4, undefined, undefined]
// The '0' and '1' indices are not present so they become undefined
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/indexed-collections.html#sec-array.prototype.toreversed

Browser compatibility

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
toReversed	110	110	115	96	16	110	115	74	16	21.0	110	16	1.0.0	1.31	20.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Array.toReversed`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Polyfill of `Array.prototype.toReversed` in `core-js`
- es-shims polyfill of `Array.prototype.toReversed`
- Indexed collections guide
- `Array.prototype.reverse()`
- `Array.prototype.toSorted()`
- `Array.prototype.toSpliced()`
- `Array.prototype.with()`
- `TypedArray.prototype.toReversed()`

JavaScript › Reference › Standard built-in objects › Array

Array.prototype.toSorted()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/toSorted · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since July 2023.

The **`toSorted()`** method of `Array` instances is the copying version of the `sort()` method. It returns a new array with the elements sorted in ascending order.

Syntax

```js
toSorted()
toSorted(compareFn)
```

Parameters

`compareFn` Optional

A function that determines the order of the elements. If omitted, the array elements are converted to strings, then sorted according to each character's Unicode code point value. See `sort()` for more information.

Return value

A new array with the elements sorted in ascending order.

Description

See `sort()` for more information on the `compareFn` parameter.

When used on sparse arrays, the `toSorted()` method iterates empty slots as if they have the value `undefined`.

The `toSorted()` method is generic. It only expects the `this` value to have a `length` property and integer-keyed properties.

Examples

Sorting an array

```js
const months = ["Mar", "Jan", "Feb", "Dec"];
const sortedMonths = months.toSorted();
console.log(sortedMonths); // ['Dec', 'Feb', 'Jan', 'Mar']
console.log(months); // ['Mar', 'Jan', 'Feb', 'Dec']

const values = [1, 10, 21, 2];
const sortedValues = values.toSorted((a, b) => a - b);
console.log(sortedValues); // [1, 2, 10, 21]
console.log(values); // [1, 10, 21, 2]
```

For more usage examples, see `sort()`.

Using toSorted() on sparse arrays

Empty slots are sorted as if they have the value `undefined`. They are always sorted to the end of the array and `compareFn` is not called for them.

```js
console.log(["a", "c", , "b"].toSorted()); // ['a', 'b', 'c', undefined]
console.log([, undefined, "a", "b"].toSorted()); // ["a", "b", undefined, undefined]
```

Calling toSorted() on non-array objects

The `toSorted()` method reads the `length` property of `this`. It then collects all existing integer-keyed properties in the range of `0` to `length - 1`, sorts them, and writes them into a new array.

```js
const arrayLike = {
  length: 3,
  unrelated: "foo",
  0: 5,
  2: 4,
  3: 3, // ignored by toSorted() since length is 3
};
console.log(Array.prototype.toSorted.call(arrayLike));
// [4, 5, undefined]
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/indexed-collections.html#sec-array.prototype.tosorted

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`toSorted`	110	110	115	96	16	110	115	74	16	21.0	110	16	1.0.0	1.31	20.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Array.toSorted`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Polyfill of `Array.prototype.toSorted` in `core-js`
- es-shims polyfill of `Array.prototype.toSorted`
- Indexed collections guide
- `Array.prototype.sort()`
- `Array.prototype.toReversed()`
- `Array.prototype.toSpliced()`
- `Array.prototype.with()`
- `TypedArray.prototype.toSorted()`

JavaScript › Reference › Standard built-in objects › Array

Array.prototype.toSpliced()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/toSpliced · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since July 2023.

The **`toSpliced()`** method of `Array` instances is the copying version of the `splice()` method. It returns a new array with some elements removed and/or replaced at a given index.

Syntax

```js
toSpliced(start)
toSpliced(start, skipCount)
toSpliced(start, skipCount, item1)
toSpliced(start, skipCount, item1, item2)
toSpliced(start, skipCount, item1, item2, /* …, */ itemN)
```

Parameters

`start`

Zero-based index at which to start changing the array, converted to an integer.

- Negative index counts back from the end of the array — if `-array.length <= start < 0`, `start + array.length` is used.
- If `start < -array.length` or `start` is omitted, `0` is used.
- If `start >= array.length`, no element will be deleted, but the method will behave as an adding function, adding as many elements as provided.

`skipCount` Optional

An integer indicating the number of elements in the array to remove (or, to skip) from `start`.

If `skipCount` is omitted, or if its value is greater than or equal to the number of elements after the position specified by `start`, then all the elements from `start` to the end of the array will be deleted. However, if you wish to pass any `itemN` parameter, you should pass `Infinity` as `skipCount` to delete all elements after `start`, because an explicit `undefined` gets converted to `0`.

If `skipCount` is `0` or negative, no elements are removed. In this case, you should specify at least one new element (see below).

`item1, ..., itemN` Optional

The elements to add to the array, beginning from `start`.

If you do not specify any elements, `toSpliced()` will only remove elements from the array.

Return value

A new array that consists of all elements before `start`, `item1`, `item2`, ..., `itemN`, and all elements after `start + skipCount`.

Description

The `toSpliced()` method, like `splice()`, does multiple things at once: it removes the given number of elements from the array, starting at a given index, and then inserts the given elements at the same index. However, it returns a new array instead of modifying the original array. The deleted elements therefore are not returned from this method, but they remain accessible in the original array.

The `toSpliced()` method never produces a sparse array. If the source array is sparse, the empty slots will be replaced with `undefined` in the new array.

The `toSpliced()` method is generic. It only expects the `this` value to have a `length` property and integer-keyed properties.

Examples

Deleting, adding, and replacing elements

You can use `toSpliced()` to delete, add, and replace elements in an array and create a new array more efficiently than using `slice()` and `concat()`.

js

```
const months = ["Jan", "Mar", "Apr", "May"];

// Inserting an element at index 1
const months2 = months.toSpliced(1, 0, "Feb");
console.log(months2); // ["Jan", "Feb", "Mar", "Apr", "May"]

// Deleting two elements starting from index 2
const months3 = months2.toSpliced(2, 2);
console.log(months3); // ["Jan", "Feb", "May"]

// Replacing one element at index 1 with two new elements
const months4 = months3.toSpliced(1, 1, "Feb", "Mar");
console.log(months4); // ["Jan", "Feb", "Mar", "May"]
```

(code continues on the next page)

(code continued from the previous page)

```
// Original array is not modified
console.log(months); // ["Jan", "Mar", "Apr", "May"]
```

Using toSpliced() on sparse arrays

The `toSpliced()` method always creates a dense array.

js

```
const arr = [1, , 3, 4, , 6];
console.log(arr.toSpliced(1, 2)); // [1, 4, undefined, 6]
```

Calling toSpliced() on non-array objects

The `toSpliced()` method reads the `length` property of `this`. It then reads the integer-keyed properties needed and writes them into the new array.

js

```
const arrayLike = {
  length: 3,
  unrelated: "foo",
  0: 5,
  2: 4,
};
console.log(Array.prototype.toSpliced.call(arrayLike, 0, 1, 2, 3));
// [2, 3, undefined, 4]
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/indexed-collections.html#sec-array.prototype.tospliced

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
toSpliced	110	110	115	96	16	110	115	74	16	21.0	110	16	1.0.0	1.31	20.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Array.toSpliced`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Polyfill of `Array.prototype.toSpliced` in `core-js`
- es-shims polyfill of `Array.prototype.toSpliced`
- `Array.prototype.splice()`
- `Array.prototype.toReversed()`
- `Array.prototype.toSorted()`
- `Array.prototype.with()`

JavaScript › Reference › Standard built-in objects › Array

Array.prototype.toString()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/toString · MDN page last modified 2025-07-20

Baseline: Widely available — available across major browsers since July 2015.

The **`toString()`** method of `Array` instances returns a string representing the specified array and its elements.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Array.prototype.toString()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const array = [1, 2, "a", "1a"];

console.log(array.toString());
// Expected output: "1,2,a,1a"
```

Syntax

js

```
toString()
```

Parameters

None.

Return value

A string representing the elements of the array.

Description

The `Array` object overrides the `toString` method of `Object`. The `toString` method of arrays calls `join()` internally, which joins the array and returns one string containing each array element separated by commas. If the `join` method is unavailable or is not a function, `Object.prototype.toString` is used instead, returning `[object Array]`.

js

```
const arr = [];
arr.join = 1; // re-assign `join` with a non-function
console.log(arr.toString()); // [object Array]

console.log(Array.prototype.toString.call({ join: () => 1 })); // 1
```

JavaScript calls the `toString` method automatically when an array is to be represented as a text value or when an array is referred to in a string concatenation.

`Array.prototype.toString` recursively converts each element, including other arrays, to strings. Because the string returned by `Array.prototype.toString` does not have delimiters, nested arrays look like they are flattened.

```js
const matrix = [
  [1, 2, 3],
  [4, 5, 6],
  [7, 8, 9],
];

console.log(matrix.toString()); // 1,2,3,4,5,6,7,8,9
```

When an array is cyclic (it contains an element that is itself), browsers avoid infinite recursion by ignoring the cyclic reference.

```js
const arr = [];
arr.push(1, [3, arr, 4], 2);
console.log(arr.toString()); // 1,3,,4,2
```

Examples

Using toString()

```js
const array = [1, 2, "a", "1a"];

console.log(array.toString()); // "1,2,a,1a"
```

Using toString() on sparse arrays

Following the behavior of `join()`, `toString()` treats empty slots the same as `undefined` and produces an extra separator:

```js
console.log([1, , 3].toString()); // '1,,3'
```

Calling toString() on non-array objects

`toString()` is generic. It expects `this` to have a `join()` method; or, failing that, uses `Object.prototype.toString()` instead.

```js
console.log(Array.prototype.toString.call({ join: () => 1 }));
// 1; a number
console.log(Array.prototype.toString.call({ join: () => undefined }));
// undefined
console.log(Array.prototype.toString.call({ join: "not function" }));
// "[object Object]"
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/indexed-collections.html#sec-array.prototype.tostring

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`toString`	1	12	1	4	1	18	4	10.1	1	1.0	1	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Array.toString`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Indexed collections guide
- `Array`
- `Array.prototype.join()`
- `Array.prototype.toLocaleString()`
- `TypedArray.prototype.toString()`
- `String.prototype.toString()`

JavaScript › Reference › Standard built-in objects › Array

Array.prototype.unshift()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/unshift · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since July 2015.

The **`unshift()`** method of `Array` instances adds the specified elements to the beginning of an array and returns the new length of the array.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Array.prototype.unshift()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const array = [1, 2, 3];

console.log(array.unshift(4, 5));
// Expected output: 5

console.log(array);
// Expected output: Array [4, 5, 1, 2, 3]
```

Syntax

js

```
unshift()
unshift(element1)
unshift(element1, element2)
unshift(element1, element2, /* …, */ elementN)
```

Parameters

`element1, ..., elementN`

The elements to add to the front of the `arr`.

Return value

The new `length` property of the object upon which the method was called.

Description

The `unshift()` method inserts the given values to the beginning of an array-like object.

`Array.prototype.push()` has similar behavior to `unshift()`, but applied to the end of an array.

Please note that, if multiple elements are passed as parameters, they're inserted in chunk at the beginning of the object, in the exact same order they were passed as parameters. Hence, calling `unshift()` with `n` arguments **once**, or calling it `n` times with **1** argument (with a loop, for example), don't yield the same results.

See example:

```js
let arr = [4, 5, 6];

arr.unshift(1, 2, 3);
console.log(arr);
// [1, 2, 3, 4, 5, 6]

arr = [4, 5, 6]; // resetting the array

arr.unshift(1);
arr.unshift(2);
arr.unshift(3);

console.log(arr);
// [3, 2, 1, 4, 5, 6]
```

The `unshift()` method is generic. It only expects the `this` value to have a `length` property and integer-keyed properties. Although strings are also array-like, this method is not suitable to be applied on them, as strings are immutable.

Examples

Using unshift()

```js
const arr = [1, 2];

arr.unshift(0); // result of the call is 3, which is the new array length
// arr is [0, 1, 2]

arr.unshift(-2, -1); // the new array length is 5
// arr is [-2, -1, 0, 1, 2]

arr.unshift([-4, -3]); // the new array length is 6
// arr is [[-4, -3], -2, -1, 0, 1, 2]

arr.unshift([-7, -6], [-5]); // the new array length is 8
// arr is [ [-7, -6], [-5], [-4, -3], -2, -1, 0, 1, 2 ]
```

Calling unshift() on non-array objects

The `unshift()` method reads the `length` property of `this`. It shifts all indices in the range `0` to `length - 1` right by the number of arguments (incrementing their values by this number). Then, it sets each index starting at `0` with the arguments passed to `unshift()`. Finally, it sets the `length` to the previous length plus the number of prepended elements.

```js
const arrayLike = {
  length: 3,
  unrelated: "foo",
  2: 4,
};
Array.prototype.unshift.call(arrayLike, 1, 2);
console.log(arrayLike);
// { '0': 1, '1': 2, '4': 4, length: 5, unrelated: 'foo' }

const plainObj = {};
// There's no length property, so the length is 0
Array.prototype.unshift.call(plainObj, 1, 2);
console.log(plainObj);
// { '0': 1, '1': 2, length: 2 }
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/indexed-collections.html#sec-array.prototype.unshift

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
unshift	1	12	1	4	1	18	4	10.1	1	1.0	1	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Array.unshift`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Array.prototype.unshift` in `core-js` with fixes of this method
- es-shims polyfill of `Array.prototype.unshift`
- Indexed collections guide
- `Array`
- `Array.prototype.push()`
- `Array.prototype.pop()`
- `Array.prototype.shift()`
- `Array.prototype.concat()`
- `Array.prototype.splice()`

JavaScript › Reference › Standard built-in objects › Array

Array.prototype.values()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/values · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since May 2018.

The **`values()`** method of `Array` instances returns a new *array iterator* object that iterates the value of each item in the array.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Array.prototype.values()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const array = ["a", "b", "c"];
const iterator = array.values();

for (const value of iterator) {
  console.log(value);
}

// Expected output: "a"
// Expected output: "b"
// Expected output: "c"
```

Syntax

js

```
values()
```

Parameters

None.

Return value

A new iterable iterator object.

Description

`Array.prototype.values()` is the default implementation of `Array.prototype[Symbol.iterator]()`.

```js
Array.prototype.values === Array.prototype[Symbol.iterator]; // true
```

When used on sparse arrays, the `values()` method iterates empty slots as if they have the value `undefined`.

The `values()` method is generic. It only expects the `this` value to have a `length` property and integer-keyed properties.

Examples

Iteration using for...of loop

Because `values()` returns an iterable iterator, you can use a `for...of` loop to iterate it.

```js
const arr = ["a", "b", "c", "d", "e"];
const iterator = arr.values();

for (const letter of iterator) {
  console.log(letter);
} // "a" "b" "c" "d" "e"
```

Iteration using next()

Because the return value is also an iterator, you can directly call its `next()` method.

```js
const arr = ["a", "b", "c", "d", "e"];
const iterator = arr.values();
iterator.next(); // { value: "a", done: false }
iterator.next(); // { value: "b", done: false }
iterator.next(); // { value: "c", done: false }
iterator.next(); // { value: "d", done: false }
iterator.next(); // { value: "e", done: false }
iterator.next(); // { value: undefined, done: true }
console.log(iterator.next().value); // undefined
```

Reusing the iterable

> **Warning:** The array iterator object should be a one-time use object. Do not reuse it.

The iterable returned from `values()` is not reusable. When `next().done = true` or `currentIndex > length`, the `for...of` loop ends, and further iterating it has no effect.

```js
const arr = ["a", "b", "c", "d", "e"];
const values = arr.values();
for (const letter of values) {
  console.log(letter);
}
// "a" "b" "c" "d" "e"
for (const letter of values) {
  console.log(letter);
}
// No output
```

If you use a `break` statement to end the iteration early, the iterator can resume from the current position when continuing to iterate it.

```js
const arr = ["a", "b", "c", "d", "e"];
const values = arr.values();
for (const letter of values) {
  console.log(letter);
  if (letter === "b") {
    break;
  }
}
// "a" "b"

for (const letter of values) {
  console.log(letter);
}
// "c" "d" "e"
```

Mutations during iteration

There are no values stored in the array iterator object returned from `values()`; instead, it stores the address of the array used in its creation, and reads the currently visited index on each iteration. Therefore, its iteration output depends on the value stored in that index at the time of stepping. If the values in the array changed, the array iterator object's values change too.

```js
const arr = ["a", "b", "c", "d", "e"];
const iterator = arr.values();
console.log(iterator); // Array Iterator { }
console.log(iterator.next().value); // "a"
arr[1] = "n";
console.log(iterator.next().value); // "n"
```

Unlike iterative methods, the array iterator object does not save the array's length at the time of its creation, but reads it once on each iteration. Therefore, if the array grows during iteration, the iterator will visit the new elements too. This may lead to infinite loops.

```js
const arr = [1, 2, 3];
for (const e of arr) {
  arr.push(e * 10);
}
// RangeError: invalid array length
```

Iterating sparse arrays

`values()` will visit empty slots as if they are `undefined`.

```js
for (const element of [, "a"].values()) {
  console.log(element);
}
// undefined
// 'a'
```

Calling values() on non-array objects

The `values()` method reads the `length` property of `this` and then accesses each property whose key is a nonnegative integer less than `length`.

```js
const arrayLike = {
  length: 3,
  0: "a",
  1: "b",
  2: "c",
  3: "d", // ignored by values() since length is 3
};
for (const entry of Array.prototype.values.call(arrayLike)) {
  console.log(entry);
}
// a
// b
// c
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/indexed-collections.html#sec-array.prototype.values`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`values`	66	14	60	53	9	66	60	47	9	9.0	66	9	1.0.0	1.0	10.9.0[1]

1. Also from version 0.12.0–4.0.0: supported.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Array.values`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Array.prototype.values` in `core-js`
- es-shims polyfill of `Array.prototype.values`
- Indexed collections guide
- `Array`
- `Array.prototype.entries()`
- `Array.prototype.keys()`
- `Array.prototype[Symbol.iterator]()`
- `TypedArray.prototype.values()`
- Iteration protocols

JavaScript › Reference › Standard built-in objects › Array

Array.prototype.with()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/with · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since July 2023.

The **`with()`** method of `Array` instances is the copying version of using the bracket notation to change the value of a given index. It returns a new array with the element at the given index replaced with the given value.

Syntax

```js
arrayInstance.with(index, value)
```

Parameters

`index`

Zero-based index at which to change the array, converted to an integer.

- Negative index counts back from the end of the array — if `-array.length <= index < 0`, `index + array.length` is used.
- If the index after normalization is out of bounds, a `RangeError` is thrown.

`value`

Any value to be assigned to the given index.

Return value

A new array with the element at `index` replaced with `value`.

Exceptions

`RangeError`

Thrown if `index >= array.length` or `index < -array.length`.

Description

The `with()` method changes the value of a given index in the array, returning a new array with the element at the given index replaced with the given value. The original array is not modified. This allows you to chain array methods while doing manipulations.

By combining `with()` with `at()`, you can both write and read (respectively) an array using negative indices.

The `with()` method never produces a sparse array. If the source array is sparse, the empty slots will be replaced with `undefined` in the new array.

The `with()` method is generic. It only expects the `this` value to have a `length` property and integer-keyed properties.

Examples

Creating a new array with a single element changed

```js
const arr = [1, 2, 3, 4, 5];
console.log(arr.with(2, 6)); // [1, 2, 6, 4, 5]
console.log(arr); // [1, 2, 3, 4, 5]
```

Chaining array methods

With the `with()` method, you can update a single element in an array and then apply other array methods.

```js
const arr = [1, 2, 3, 4, 5];
console.log(arr.with(2, 6).map((x) => x ** 2)); // [1, 4, 36, 16, 25]
```

Using with() on sparse arrays

The `with()` method always creates a dense array.

```js
const arr = [1, , 3, 4, , 6];
console.log(arr.with(0, 2)); // [2, undefined, 3, 4, undefined, 6]
```

Calling with() on non-array objects

The `with()` method creates and returns a new array. It reads the `length` property of `this` and then accesses each property whose key is a nonnegative integer less than `length`. As each property of `this` is accessed, the array element having an index equal to the key of the property is set to the value of the property. Finally, the array value at `index` is set to `value`.

```js
const arrayLike = {
  length: 3,
  unrelated: "foo",
  0: 5,
  2: 4,
  3: 3, // ignored by with() since length is 3
};
console.log(Array.prototype.with.call(arrayLike, 0, 1));
// [ 1, undefined, 4 ]
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/indexed-collections.html#sec-array.prototype.with

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
with	110	110	115	96	16	110	115	74	16	21.0	110	16	1.0.0	1.31	20.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Array.with`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Array.prototype.with` in `core-js`
- es-shims polyfill of `Array.prototype.with`
- Indexed collections guide
- `Array.prototype.toReversed()`
- `Array.prototype.toSorted()`
- `Array.prototype.toSpliced()`
- `Array.prototype.at()`
- `TypedArray.prototype.with()`

JavaScript › Reference › Standard built-in objects › Array

Array.prototype[Symbol.iterator]()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/Symbol.iterator · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since September 2016.

The **`[Symbol.iterator]()`** method of `Array` instances implements the iterable protocol and allows arrays to be consumed by most syntaxes expecting iterables, such as the spread syntax and `for...of` loops. It returns an array iterator object that yields the value of each index in the array.

The initial value of this property is the same function object as the initial value of the `Array.prototype.values` property.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Array.prototype[Symbol.iterator]()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const array = ["a", "b", "c"];
const iterator = array[Symbol.iterator]();

for (const value of iterator) {
  console.log(value);
}

// Expected output: "a"
// Expected output: "b"
// Expected output: "c"
```

Syntax

js

```
array[Symbol.iterator]()
```

Parameters

None.

Return value

The same return value as `Array.prototype.values()`: a new iterable iterator object that yields the value of each index in the array.

Examples

Iteration using for...of loop

Note that you seldom need to call this method directly. The existence of the `[Symbol.iterator]()` method makes arrays iterable, and iterating syntaxes like the `for...of` loop automatically call this method to obtain the iterator to loop over.

HTML

```html
<ul id="letterResult"></ul>
```

JavaScript

```js
const arr = ["a", "b", "c"];
const letterResult = document.getElementById("letterResult");
for (const letter of arr) {
  const li = document.createElement("li");
  li.textContent = letter;
  letterResult.appendChild(li);
}
```

Result

▷ Live sample output ("Iteration using for...of loop sample") is rendered in the browser on MDN and cannot be reproduced on paper. See it running at developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/Symbol.iterator#iteration_using_for...of_loop .

Manually hand-rolling the iterator

You may still manually call the `next()` method of the returned iterator object to achieve maximum control over the iteration process.

```js
const arr = ["a", "b", "c", "d", "e"];
const arrIter = arr[Symbol.iterator]();
console.log(arrIter.next().value); // a
console.log(arrIter.next().value); // b
console.log(arrIter.next().value); // c
console.log(arrIter.next().value); // d
console.log(arrIter.next().value); // e
```

Handling strings and string arrays with the same function

Because both strings and arrays implement the iterable protocol, a generic function can be designed to handle both inputs in the same fashion. This is better than calling `Array.prototype.values()` directly, which requires the input to be an array, or at least an object with such a method.

```js
function logIterable(it) {
  if (typeof it[Symbol.iterator] !== "function") {
    console.log(it, "is not iterable.");
    return;
  }
  for (const letter of it) {
    console.log(letter);
  }
}

// Array
logIterable(["a", "b", "c"]);
// a
// b
// c
```

(code continues on the next page)

(code continued from the previous page)

```
// String
logIterable("abc");
// a
// b
// c

// Number
logIterable(123);
// 123 is not iterable.
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/indexed-collections.html#sec-array.prototype-%symbol.iterator%`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
[Symbol.iterator]	38	12	36[1,2,3,4]	25	10	38	36[1,2,3,4]	25	10	3.0	38	10	1.0.0	1.0	0.12.0

1. Also from version 27–36: uses the non-standard name `@@iterator`.
2. Also from version 27–36: A placeholder property named `@@iterator` is used.
3. Also from version 17–27: uses the non-standard name `iterator`.
4. Also from version 17–27: A placeholder property named `iterator` is used.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Array.@@iterator`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Array.prototype[Symbol.iterator]` in `core-js`
- Indexed collections guide
- `Array`
- `Array.prototype.keys()`
- `Array.prototype.entries()`
- `Array.prototype.values()`
- `TypedArray.prototype[Symbol.iterator]()`
- `String.prototype[Symbol.iterator]()`
- `Symbol.iterator`
- Iteration protocols

JavaScript › Reference › Standard built-in objects › Array

Array.prototype[Symbol.unscopables]

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/Symbol.unscopables · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since September 2016.

The **`[Symbol.unscopables]`** data property of `Array.prototype` is shared by all `Array` instances. It contains property names that were not included in the ECMAScript standard prior to the ES2015 version and that are ignored for `with` statement-binding purposes.

Value

A `null`-prototype object with property names given below and their values set to `true`.

Property attributes of `Array.prototype[Symbol.unscopables]`	
Writable	no
Enumerable	no
Configurable	yes

Description

The default `Array` properties that are ignored for `with` statement-binding purposes are:

- `at()`
- `copyWithin()`
- `entries()`
- `fill()`
- `find()`
- `findIndex()`
- `findLast()`
- `findLastIndex()`
- `flat()`
- `flatMap()`
- `includes()`
- `keys()`
- `toReversed()`
- `toSorted()`
- `toSpliced()`
- `values()`

`Array.prototype[Symbol.unscopables]` is an empty object only containing all the above property names with the value `true`. Its prototype is `null`, so `Object.prototype` properties like `toString` won't accidentally be made unscopable, and a `toString()` within the `with` statement will continue to be called on the array.

See `Symbol.unscopables` for how to set unscopable properties for your own objects.

Examples

Imagine the `values.push('something')` call below is in code that was written prior to ECMAScript 2015.

```js
var values = [];

with (values) {
  values.push("something");
}
```

When ECMAScript 2015 introduced the `Array.prototype.values()` method, the `with` statement in the above code started to interpret `values` as the `values.values` array method instead of the external `values` variable. The `values.push('something')` call would break because it's now accessing `push` on the `values.values` method. This caused a bug to be reported to Firefox (Firefox Bug 883914).

So the `[Symbol.unscopables]` data property for `Array.prototype` causes the `Array` properties introduced in ECMAScript 2015 to be ignored for `with` statement-binding purposes — allowing code that was written prior to ECMAScript 2015 to continue working as expected, rather than breaking.

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/indexed-collections.html#sec-array.prototype-%symbol.unscopables%`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
[Symbol.unscopables]	38	12	48	25	10	38	48	25	10	3.0	38	10	1.0.0	1.0	0.12.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Array.@@unscopables`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Array.prototype[Symbol.unscopables]` in `core-js`
- Indexed collections guide
- `Array`
- `with`
- `Symbol.unscopables`

JavaScript › Reference › Standard built-in objects › Array

Array: length

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/length · MDN page last modified 2026-05-02

Baseline: Widely available — available across major browsers since July 2015.

The **`length`** data property of an `Array` instance represents the number of slots in that array. The value is an unsigned, 32-bit integer that is always numerically greater than the highest index in the array. It may be greater than the number of elements if the array is sparse.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Array: length". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const clothing = ["shoes", "shirts", "socks", "sweaters"];

console.log(clothing.length);
// Expected output: 4
```

Value

A nonnegative integer less than 2^{32}.

Property attributes of `Array: length`	
Writable	yes
Enumerable	no
Configurable	no

Description

The value of the `length` property is a nonnegative integer with a value less than 2^{32}.

js

```
const listA = [1, 2, 3];
const listB = new Array(6);

console.log(listA.length);
// 3

console.log(listB.length);
// 6

listB.length = 2 ** 32; // 4294967296
// RangeError: Invalid array length

const listC = new Array(-100); // Negative numbers are not allowed
// RangeError: Invalid array length
```

The array object observes the `length` property, and automatically syncs the `length` value with the array's content. This means:

- Setting `length` to a value smaller than the current length truncates the array — elements beyond the new `length` are deleted.

- Setting any array index (a nonnegative integer smaller than 2^{32}) beyond the current `length` extends the array — the `length` property is increased to reflect the new highest index.
- Setting `length` to an invalid value (e.g., a negative number or a non-integer) throws a `RangeError` exception.

When `length` is set to a bigger value than the current length, the array is extended by adding empty slots, not actual `undefined` values. Empty slots have some special interactions with array methods; see array methods and empty slots.

```js
const arr = [1, 2];
console.log(arr);
// [ 1, 2 ]

arr.length = 5; // set array length to 5 while currently 2.
console.log(arr);
// [ 1, 2, <3 empty items> ]

arr.forEach((element) => console.log(element));
// 1
// 2
```

See also Relationship between `length` and numerical properties.

Examples

Iterating over an array

In the following example, the array `numbers` is iterated through by looking at the `length` property. The value in each element is then doubled.

```js
const numbers = [1, 2, 3, 4, 5];
const length = numbers.length;
for (let i = 0; i < length; i++) {
  numbers[i] *= 2;
}
// numbers is now [2, 4, 6, 8, 10]
```

Shortening an array

The following example shortens the array `numbers` to a length of 3 if the current length is greater than 3.

```js
const numbers = [1, 2, 3, 4, 5];

if (numbers.length > 3) {
  numbers.length = 3;
}

console.log(numbers); // [1, 2, 3]
console.log(numbers.length); // 3
console.log(numbers[3]); // undefined; the extra elements are deleted
```

Create empty array of fixed length

Setting `length` to a value greater than the current length creates a sparse array.

```js
const numbers = [];
numbers.length = 3;
console.log(numbers); // [empty x 3]
```

Array with non-writable length

The `length` property is automatically updated by the array when elements are added beyond the current length. If the `length` property is made non-writable, the array will not be able to update it. This causes an error in strict mode.

```js
"use strict";

const numbers = [1, 2, 3, 4, 5];
Object.defineProperty(numbers, "length", { writable: false });
numbers[5] = 6; // TypeError: Cannot assign to read only property 'length' of object
'[object Array]'
numbers.push(5); // // TypeError: Cannot assign to read only property 'length' of object
'[object Array]'
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/indexed-collections.html#sec-properties-of-array-instances-length`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`length`	1	12	1	4	1	18	4	10.1	1	1.0	1	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Array.length`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Indexed collections guide
- `Array`
- `TypedArray.prototype.length`
- `String: length`
- RangeError: invalid array length

JavaScript › Reference › Standard built-in objects

ArrayBuffer

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/ArrayBuffer · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since July 2015.

The **`ArrayBuffer`** object is used to represent a generic raw binary data buffer.

It is an array of bytes, often referred to in other languages as a "byte array". You cannot directly manipulate the contents of an `ArrayBuffer`; instead, you create one of the typed array objects or a `DataView` object which represents the buffer in a specific format, and use that to read and write the contents of the buffer.

The `ArrayBuffer()` constructor creates a new `ArrayBuffer` of the given length in bytes. You can also get an array buffer from existing data, for example, from a Base64 string or from a local file.

`ArrayBuffer` is a transferable object.

Description

Resizing ArrayBuffers

`ArrayBuffer` objects can be made resizable by including the `maxByteLength` option when calling the `ArrayBuffer()` constructor. You can query whether an `ArrayBuffer` is resizable and what its maximum size is by accessing its `resizable` and `maxByteLength` properties, respectively. You can assign a new size to a resizable `ArrayBuffer` with a `resize()` call. New bytes are initialized to 0.

These features make resizing `ArrayBuffers` more efficient — otherwise, you have to make a copy of the buffer with a new size. It also gives JavaScript parity with WebAssembly in this regard (Wasm linear memory can be resized with `WebAssembly.Memory.prototype.grow()`).

Transferring ArrayBuffers

`ArrayBuffer` objects can be transferred between different execution contexts, like Web Workers or Service Workers, using the structured clone algorithm. This is done by passing the `ArrayBuffer` as a transferable object in a call to `Worker.postMessage()` or `ServiceWorker.postMessage()`. In pure JavaScript, you can also transfer the ownership of memory from one `ArrayBuffer` to another using its `transfer()` or `transferToFixedLength()` method.

When an `ArrayBuffer` is transferred, its original copy becomes *detached* — this means it is no longer usable. At any moment, there will only be one copy of the `ArrayBuffer` that actually has access to the underlying memory. Detached buffers have the following behaviors:

- `byteLength` becomes 0 (in both the buffer and the associated typed array views).
- Methods, such as `resize()` and `slice()`, throw a `TypeError` when invoked. The associated typed array views' methods also throw a `TypeError`.

You can check whether an `ArrayBuffer` is detached by its `detached` property.

Constructor

`ArrayBuffer()`

Creates a new `ArrayBuffer` object.

Static properties

`ArrayBuffer[Symbol.species]`

The constructor function that is used to create derived objects.

Static methods

`ArrayBuffer.isView()`

Returns `true` if `arg` is one of the ArrayBuffer views, such as typed array objects or a `DataView`. Returns `false` otherwise.

Instance properties

These properties are defined on `ArrayBuffer.prototype` and shared by all `ArrayBuffer` instances.

`ArrayBuffer.prototype.byteLength`

The size, in bytes, of the `ArrayBuffer`. This is established when the array is constructed and can only be changed using the `ArrayBuffer.prototype.resize()` method if the `ArrayBuffer` is resizable.

`ArrayBuffer.prototype.constructor`

The constructor function that created the instance object. For `ArrayBuffer` instances, the initial value is the `ArrayBuffer` constructor.

`ArrayBuffer.prototype.detached`

Read-only. Returns `true` if the `ArrayBuffer` has been detached (transferred), or `false` if not.

`ArrayBuffer.prototype.maxByteLength`

The read-only maximum length, in bytes, that the `ArrayBuffer` can be resized to. This is established when the array is constructed and cannot be changed.

`ArrayBuffer.prototype.resizable`

Read-only. Returns `true` if the `ArrayBuffer` can be resized, or `false` if not.

`ArrayBuffer.prototype[Symbol.toStringTag]`

The initial value of the `[Symbol.toStringTag]` property is the string `"ArrayBuffer"`. This property is used in `Object.prototype.toString()`.

Instance methods

`ArrayBuffer.prototype.resize()`

Resizes the `ArrayBuffer` to the specified size, in bytes.

`ArrayBuffer.prototype.slice()`

Returns a new `ArrayBuffer` whose contents are a copy of this `ArrayBuffer`'s bytes from `begin` (inclusive) up to `end` (exclusive). If either `begin` or `end` is negative, it refers to an index from the end of the array, as opposed to from the beginning.

`ArrayBuffer.prototype.transfer()`

Creates a new `ArrayBuffer` with the same byte content as this buffer, then detaches this buffer.

`ArrayBuffer.prototype.transferToFixedLength()`

Creates a new non-resizable `ArrayBuffer` with the same byte content as this buffer, then detaches this buffer.

Examples

Creating an ArrayBuffer

In this example, we create a 8-byte buffer with an `Int32Array` view referring to the buffer:

```js
const buffer = new ArrayBuffer(8);
const view = new Int32Array(buffer);
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/structured-data.html#sec-arraybuffer-objects

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`ArrayBuffer`	7	12	4	11.6	5.1	18	4	12	4.2	1.0	4	4.2	1.0.0	1.0	0.10.0
[Symbol.species]	51	13	48	38	10	51	48	41	10	5.0	51	10	1.0.0	1.0	6.5.0
`ArrayBuffer()` constructor	7	12	4	11.6	5.1	18	4	12	4.2	1.0	4	4.2	1.0.0	1.0	0.10.0
`maxByteLength` option	111	111	128	97	16.4	111	128	75	16.4	22.0	111	16.4	1.0.0	1.33	20.0.0
`byteLength`	7	12	4	11.6	5.1	18	4	12	4.2	1.0	4	4.2	1.0.0	1.0	0.10.0
`detached`	114	114	122	100	17.4	114	122	76	17.4	23.0	114	17.4	1.0.19	1.33	21.0.0
`isView`	32	12	29	19	7	32	29	19	7	2.0	4.4.3	7	1.0.0	1.0	4.0.0
`maxByteLength`	111	111	128	97	16.4	111	128	75	16.4	22.0	111	16.4	1.0.0	1.33	20.0.0
`resizable`	111	111	128	97	16.4	111	128	75	16.4	22.0	111	16.4	1.0.0	1.33	20.0.0
`resize`	111	111	128	97	16.4	111	128	75	16.4	22.0	111	16.4	1.0.0	1.33	20.0.0
`slice`	17	12	12[1]	12.1	5.1	18	14[2]	12.1	6	1.0	4.4	6	1.0.0	1.0	0.12.0
`transfer`	114	114	122	100	17.4	114	122	76	17.4	23.0	114	17.4	1.0.19	1.33	21.0.0
`transferToFixedLength`	114	114	122	100	17.4	114	122	76	17.4	23.0	114	17.4	1.0.19	1.33	21.0.0

1. The non-standard `ArrayBuffer.slice()` method has been removed in Firefox 53 (but the standardized version `ArrayBuffer.prototype.slice()` is kept.
2. The non-standard `ArrayBuffer.slice()` method has been removed in Firefox for Android 53 (but the standardized version `ArrayBuffer.prototype.slice()` is kept.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.ArrayBuffer`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Polyfill of `ArrayBuffer` in `core-js`
- JavaScript typed arrays guide
- `SharedArrayBuffer`
- RangeError: invalid array length

JavaScript › Reference › Standard built-in objects › ArrayBuffer

ArrayBuffer() constructor

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/ArrayBuffer/ArrayBuffer · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since July 2015.

The **`ArrayBuffer()`** constructor creates `ArrayBuffer` objects.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: ArrayBuffer() constructor". It runs live in the browser there; its source code follows.*

js · interactive example source

```
// Create an ArrayBuffer with a size in bytes
const buffer = new ArrayBuffer(8);

console.log(buffer.byteLength);
// Expected output: 8
```

Syntax

js

```
new ArrayBuffer(length)
new ArrayBuffer(length, options)
```

> **Note:** `ArrayBuffer()` can only be constructed with `new`. Attempting to call it without `new` throws a `TypeError`.

Parameters

`length`

The size, in bytes, of the array buffer to create.

`options` Optional

An object, which can contain the following properties:

`maxByteLength` Optional

The maximum size, in bytes, that the array buffer can be resized to.

Return value

A new `ArrayBuffer` object of the specified size, with its `maxByteLength` property set to the specified `maxByteLength` if one was specified. Its contents are initialized to 0.

Exceptions

`RangeError`

Thrown in one of the following cases:

- `length` or `maxByteLength` is larger than `Number.MAX_SAFE_INTEGER` ($\geq 2^{53}$) or negative.
- `length` is larger than `maxByteLength`.

Examples

Creating an ArrayBuffer

In this example, we create a 8-byte buffer with an `Int32Array` view referring to the buffer:

```js
const buffer = new ArrayBuffer(8);
const view = new Int32Array(buffer);
```

Creating a resizable ArrayBuffer

In this example, we create a 8-byte buffer that is resizable to a max length of 16 bytes, then `resize()` it to 12 bytes:

```js
const buffer = new ArrayBuffer(8, { maxByteLength: 16 });

buffer.resize(12);
```

> **Note:** It is recommended that `maxByteLength` is set to the smallest value possible for your use case. It should never exceed `1073741824` (1GB) to reduce the risk of out-of-memory errors.

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/structured-data.html#sec-arraybuffer-constructor`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Nodejs
`ArrayBuffer()` constructor	7	12	4	11.6	5.1	18	4	12	4.2	1.0	4	4.2	1.0.0	1.0	0.10.0
`maxByteLength` option	111	111	128	97	16.4	111	128	75	16.4	22.0	111	16.4	1.0.0	1.33	20.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.ArrayBuffer.ArrayBuffer`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `ArrayBuffer` in `core-js`
- JavaScript typed arrays guide
- `SharedArrayBuffer`

JavaScript › Reference › Standard built-in objects › ArrayBuffer

ArrayBuffer.isView()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/ArrayBuffer/isView · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **`ArrayBuffer.isView()`** static method determines whether the passed value is one of the `ArrayBuffer` views, such as typed array objects or a `DataView`.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: ArrayBuffer.isView()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
// Create an ArrayBuffer with a size in bytes
const buffer = new ArrayBuffer(16);

console.log(ArrayBuffer.isView(new Int32Array()));
// Expected output: true
```

Syntax

js

```
ArrayBuffer.isView(value)
```

Parameters

`value`

The value to be checked.

Return value

`true` if the given argument is one of the `ArrayBuffer` views; otherwise, `false`.

Examples

Using isView

js

```
ArrayBuffer.isView(); // false
ArrayBuffer.isView([]); // false
ArrayBuffer.isView({}); // false
ArrayBuffer.isView(null); // false
ArrayBuffer.isView(undefined); // false
ArrayBuffer.isView(new ArrayBuffer(10)); // false

ArrayBuffer.isView(new Uint8Array()); // true
ArrayBuffer.isView(new Float32Array()); // true
ArrayBuffer.isView(new Int8Array(10).subarray(0, 3)); // true

const buffer = new ArrayBuffer(2);
const dv = new DataView(buffer);
ArrayBuffer.isView(dv); // true
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/structured-data.html#sec-arraybuffer.isview`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`isView`	32	12	29	19	7	32	29	19	7	2.0	4.4.3	7	1.0.0	1.0	4.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.ArrayBuffer.isView`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- JavaScript typed arrays guide

JavaScript › Reference › Standard built-in objects › ArrayBuffer

ArrayBuffer[Symbol.species]

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/ArrayBuffer/Symbol.species · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since September 2016.

The **`ArrayBuffer[Symbol.species]`** static accessor property returns the constructor used to construct return values from array buffer methods.

> **Warning:** The existence of `[Symbol.species]` allows execution of arbitrary code and may create security vulnerabilities. It also makes certain optimizations much harder. Engine implementers are investigating whether to remove this feature. Avoid relying on it if possible.

Syntax

```
ArrayBuffer[Symbol.species]
```

Return value

The value of the constructor (`this`) on which `get [Symbol.species]` was called. The return value is used to construct return values from array buffer methods that create new array buffers.

Description

The `[Symbol.species]` accessor property returns the default constructor for `ArrayBuffer` objects. Subclass constructors may override it to change the constructor assignment. The default implementation is basically:

```js
// Hypothetical underlying implementation for illustration
class ArrayBuffer {
  static get [Symbol.species]() {
    return this;
  }
}
```

Because of this polymorphic implementation, `[Symbol.species]` of derived subclasses would also return the constructor itself by default.

```js
class SubArrayBuffer extends ArrayBuffer {}
SubArrayBuffer[Symbol.species] === SubArrayBuffer; // true
```

When calling array buffer methods that do not mutate the existing object but return a new array buffer instance (for example, `slice()`), the object's `constructor[Symbol.species]` will be accessed. The returned constructor will be used to construct the return value of the array buffer method.

Examples

Species in ordinary objects

The `[Symbol.species]` property returns the default constructor function, which is the `ArrayBuffer` constructor for `ArrayBuffer`.

```js
ArrayBuffer[Symbol.species]; // function ArrayBuffer()
```

Species in derived objects

In an instance of a custom `ArrayBuffer` subclass, such as `MyArrayBuffer`, the `MyArrayBuffer` species is the `MyArrayBuffer` constructor. However, you might want to overwrite this, in order to return parent `ArrayBuffer` objects in your derived class methods:

```js
class MyArrayBuffer extends ArrayBuffer {
  // Overwrite MyArrayBuffer species to the parent ArrayBuffer constructor
  static get [Symbol.species]() {
    return ArrayBuffer;
  }
}
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/structured-data.html#sec-get-arraybuffer-%symbol.species%

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
[Symbol.species]	51	13	48	38	10	51	48	41	10	5.0	51	10	1.0.0	1.0	6.5.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.ArrayBuffer.@@species`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `ArrayBuffer`
- `Symbol.species`

JavaScript › Reference › Standard built-in objects › ArrayBuffer

ArrayBuffer.prototype.resize()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/ArrayBuffer/resize · MDN page last modified 2025-07-10

Baseline 2024: Newly available — works across the latest browser versions since July 2024; may not work in older ones.

The **`resize()`** method of `ArrayBuffer` instances resizes the `ArrayBuffer` to the specified size, in bytes.

Try it

▷ Interactive example on MDN: "JavaScript Demo: ArrayBuffer.prototype.resize()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const buffer = new ArrayBuffer(8, { maxByteLength: 16 });

console.log(buffer.byteLength);
// Expected output: 8

buffer.resize(12);

console.log(buffer.byteLength);
// Expected output: 12
```

Syntax

js

```
resize(newLength)
```

Parameters

`newLength`

The new length, in bytes, to resize the `ArrayBuffer` to.

Return value

None (`undefined`).

Exceptions

`TypeError`

Thrown if the `ArrayBuffer` is detached or is not resizable.

`RangeError`

Thrown if `newLength` is larger than the `maxByteLength` of the `ArrayBuffer`.

Description

The `resize()` method resizes an `ArrayBuffer` to the size specified by the `newLength` parameter, provided that the `ArrayBuffer` is resizable and the new size is less than or equal to the `maxByteLength` of the `ArrayBuffer`. New bytes are initialized to 0.

Note that you can use `resize()` to shrink as well as grow an `ArrayBuffer` — it is permissible for `newLength` to be smaller than the `ArrayBuffer`'s current `byteLength`.

Examples

Using resize()

In this example, we create a 8-byte buffer that is resizable to a max length of 16 bytes, then check its `resizable` property, resizing it if `resizable` returns `true`:

```js
const buffer = new ArrayBuffer(8, { maxByteLength: 16 });

if (buffer.resizable) {
  console.log("Buffer is resizable!");
  buffer.resize(12);
}
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/structured-data.html#sec-arraybuffer.prototype.resize

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`resize`	111	111	128	97	16.4	111	128	75	16.4	22.0	111	16.4	1.0.0	1.33	20.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.ArrayBuffer.resize`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `ArrayBuffer`
- `ArrayBuffer.prototype.resizable`
- `ArrayBuffer.prototype.maxByteLength`

JavaScript › Reference › Standard built-in objects › ArrayBuffer

ArrayBuffer.prototype.slice()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/ArrayBuffer/slice · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **`slice()`** method of `ArrayBuffer` instances returns a new `ArrayBuffer` whose contents are a copy of this `ArrayBuffer`'s bytes from `start`, inclusive, up to `end`, exclusive. If either `start` or `end` is negative, it refers to an index from the end of the array, as opposed to from the beginning.

Try it

▷ Interactive example on MDN: "JavaScript Demo: ArrayBuffer.prototype.slice()". It runs live in the browser there; its source code follows.

js · interactive example source

```
// Create an ArrayBuffer with a size in bytes
const buffer = new ArrayBuffer(16);
const int32View = new Int32Array(buffer);
// Produces Int32Array [0, 0, 0, 0]

int32View[1] = 42;
const sliced = new Int32Array(buffer.slice(4, 12));
// Produces Int32Array [42, 0]

console.log(sliced[0]);
// Expected output: 42
```

Syntax

js

```
slice()
slice(start)
slice(start, end)
```

Parameters

`start` Optional

Zero-based index at which to start extraction, converted to an integer.

- Negative index counts back from the end of the buffer — if `-buffer.length <= start < 0`, `start + buffer.length` is used.
- If `start < -buffer.length` or `start` is omitted, `0` is used.
- If `start >= buffer.length`, an empty buffer is returned.

end Optional

Zero-based index at which to end extraction, converted to an integer. `slice()` extracts up to but not including `end`.

- Negative index counts back from the end of the buffer — if `-buffer.length <= end < 0`, `end + buffer.length` is used.
- If `end < -buffer.length`, `0` is used.
- If `end >= buffer.length` or `end` is omitted is `undefined`, `buffer.length` is used, causing all elements until the end to be extracted.
- If `end` implies a position before or at the position that `start` implies, an empty buffer is returned.

Return value

A new `ArrayBuffer` containing the extracted elements. It is not resizable, even if the original was.

Examples

Copying an ArrayBuffer

```js
const buf1 = new ArrayBuffer(8);
const buf2 = buf1.slice(0);
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/structured-data.html#sec-arraybuffer.prototype.slice`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`slice`	17	12	12[1]	12.1	5.1	18	14[2]	12.1	6	1.0	4.4	6	1.0.0	1.0	0.12.0

1. The non-standard `ArrayBuffer.slice()` method has been removed in Firefox 53 (but the standardized version `ArrayBuffer.prototype.slice()` is kept.
2. The non-standard `ArrayBuffer.slice()` method has been removed in Firefox for Android 53 (but the standardized version `ArrayBuffer.prototype.slice()` is kept.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.ArrayBuffer.slice`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `ArrayBuffer`
- `SharedArrayBuffer.prototype.slice()`

JavaScript › Reference › Standard built-in objects › ArrayBuffer

ArrayBuffer.prototype.transfer()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/ArrayBuffer/transfer · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since March 2024.

The **`transfer()`** method of `ArrayBuffer` instances creates a new `ArrayBuffer` with the same byte content as this buffer, then detaches this buffer.

Syntax

```js
transfer()
transfer(newByteLength)
```

Parameters

`newByteLength` Optional

The `byteLength` of the new `ArrayBuffer`. Defaults to the `byteLength` of this `ArrayBuffer`.

- If `newByteLength` is smaller than the `byteLength` of this `ArrayBuffer`, the "overflowing" bytes are dropped.
- If `newByteLength` is larger than the `byteLength` of this `ArrayBuffer`, the extra bytes are filled with zeros.
- If this `ArrayBuffer` is resizable, `newByteLength` must not be greater than its `maxByteLength`.

Return value

A new `ArrayBuffer` object. Its contents are initialized to the contents of this `ArrayBuffer`, and extra bytes, if any, are filled with zeros. The new `ArrayBuffer` is resizable if and only if this `ArrayBuffer` is resizable, in which case its `maxByteLength` is the same as this `ArrayBuffer`'s. The original `ArrayBuffer` is detached.

Exceptions

`RangeError`

Thrown if this `ArrayBuffer` is resizable and `newByteLength` is greater than the `maxByteLength` of this `ArrayBuffer`.

`TypeError`

Thrown if this `ArrayBuffer` is already detached, or if it can only be detached by designated operations. Currently, only certain web APIs are capable of creating `ArrayBuffer` objects with designated detaching methods, such as `GPUBuffer.getMappedRange()` and `WebAssembly.Memory.buffer`.

Description

The `transfer()` method performs the same operation as the structured clone algorithm. It copies the bytes of this `ArrayBuffer` into a new `ArrayBuffer` object, then detaches this `ArrayBuffer` object. See transferring ArrayBuffers for more information.

`transfer()` preserves the resizability of this `ArrayBuffer`. If you want the new `ArrayBuffer` to be non-resizable, use `transferToFixedLength()` instead. There's no way to transfer a buffer that makes a fixed-length buffer become resizable.

`transfer()` is very efficient because implementations may implement this method as a zero-copy move or a `realloc` — there does not need to be an actual copy of the data.

Examples

Transferring an ArrayBuffer

```js
// Create an ArrayBuffer and write a few bytes
const buffer = new ArrayBuffer(8);
const view = new Uint8Array(buffer);
view[1] = 2;
view[7] = 4;

// Copy the buffer to the same size
const buffer2 = buffer.transfer();
console.log(buffer.detached); // true
console.log(buffer2.byteLength); // 8
const view2 = new Uint8Array(buffer2);
console.log(view2[1]); // 2
console.log(view2[7]); // 4

// Copy the buffer to a smaller size
const buffer3 = buffer2.transfer(4);
console.log(buffer3.byteLength); // 4
const view3 = new Uint8Array(buffer3);
console.log(view3[1]); // 2
console.log(view3[7]); // undefined

// Copy the buffer to a larger size
const buffer4 = buffer3.transfer(8);
console.log(buffer4.byteLength); // 8
const view4 = new Uint8Array(buffer4);
console.log(view4[1]); // 2
console.log(view4[7]); // 0

// Already detached, throws TypeError
buffer.transfer(); // TypeError: Cannot perform ArrayBuffer.prototype.transfer on a detached
ArrayBuffer
```

Transferring a resizable ArrayBuffer

```js
const buffer = new ArrayBuffer(8, { maxByteLength: 16 });
const view = new Uint8Array(buffer);
view[1] = 2;
view[7] = 4;
```

(code continues on the next page)

(code continued from the previous page)

```
// Copy the buffer to a smaller size
const buffer2 = buffer.transfer(4);
console.log(buffer2.byteLength); // 4
console.log(buffer2.maxByteLength); // 16
const view2 = new Uint8Array(buffer2);
console.log(view2[1]); // 2
console.log(view2[7]); // undefined
buffer2.resize(8);
console.log(view2[7]); // 0

// Copy the buffer to a larger size within maxByteLength
const buffer3 = buffer2.transfer(12);
console.log(buffer3.byteLength); // 12

// Copy the buffer to a larger size than maxByteLength
buffer3.transfer(20); // RangeError: Invalid array buffer length
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/structured-data.html#sec-arraybuffer.prototype.transfer`

Browser compatibility

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`transfer`	114	114	122	100	17.4	114	122	76	17.4	23.0	114	17.4	1.0.19	1.33	21.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.ArrayBuffer.transfer`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `ArrayBuffer.prototype.transfer` in `core-js`
- es-shims polyfill of `ArrayBuffer.prototype.transfer`
- `ArrayBuffer`
- `ArrayBuffer.prototype.detached`
- `ArrayBuffer.prototype.transferToFixedLength()`

JavaScript › Reference › Standard built-in objects › ArrayBuffer

ArrayBuffer.prototype.transferToFixedLength()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/ArrayBuffer/transferToFixedLength · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since March 2024.

The **`transferToFixedLength()`** method of `ArrayBuffer` instances creates a new non-resizable `ArrayBuffer` with the same byte content as this buffer, then detaches this buffer.

Syntax

```
transferToFixedLength()
transferToFixedLength(newByteLength)
```

Parameters

`newByteLength`

The `byteLength` of the new `ArrayBuffer`. Defaults to the `byteLength` of this `ArrayBuffer`.

- If `newByteLength` is smaller than the `byteLength` of this `ArrayBuffer`, the "overflowing" bytes are dropped.
- If `newByteLength` is larger than the `byteLength` of this `ArrayBuffer`, the extra bytes are filled with zeros.

Return value

A new `ArrayBuffer` object. Its contents are initialized to the contents of this `ArrayBuffer`, and extra bytes, if any, are filled with zeros. The new `ArrayBuffer` is always non-resizable. The original `ArrayBuffer` is detached.

Exceptions

`TypeError`

Thrown if this `ArrayBuffer` is already detached, or if it can only be detached by designated operations. Currently, only certain web APIs are capable of creating `ArrayBuffer` objects with designated detaching methods, such as `GPUBuffer.getMappedRange()` and `WebAssembly.Memory.buffer`.

Description

Unlike `transfer()`, `transferToFixedLength()` always creates a non-resizable `ArrayBuffer`. This means `newByteLength` can be larger than the `maxByteLength`, even if this `ArrayBuffer` is resizable. See transferring ArrayBuffers for more information.

Examples

Transferring a resizable ArrayBuffer to fixed-length

```js
const buffer = new ArrayBuffer(8, { maxByteLength: 16 });
const view = new Uint8Array(buffer);
view[1] = 2;
view[7] = 4;

const buffer2 = buffer.transferToFixedLength();
console.log(buffer2.byteLength); // 8
console.log(buffer2.resizable); // false
const view2 = new Uint8Array(buffer2);
console.log(view2[1]); // 2
console.log(view2[7]); // 4
```

Using `transferToFixedLength`, `newByteLength` can be larger than the `maxByteLength` of the original `ArrayBuffer`.

```js
const buffer = new ArrayBuffer(8, { maxByteLength: 16 });
const view = new Uint8Array(buffer);
view[1] = 2;
view[7] = 4;

const buffer2 = buffer.transferToFixedLength(20);
console.log(buffer2.byteLength); // 20
console.log(buffer2.resizable); // false
const view2 = new Uint8Array(buffer2);
console.log(view2[1]); // 2
console.log(view2[7]); // 4
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/structured-data.html#sec-arraybuffer.prototype.transfertofixedlength`

Browser compatibility

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`transferToFixedLength`	114	114	122	100	17.4	114	122	76	17.4	23.0	114	17.4	1.0.19	1.33	21.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.ArrayBuffer.transferToFixedLength`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `ArrayBuffer.prototype.transferToFixedLength` in `core-js`
- es-shims polyfill of `ArrayBuffer.prototype.transferToFixedLength`
- `ArrayBuffer`

- `ArrayBuffer.prototype.detached`
- `ArrayBuffer.prototype.transfer()`

JavaScript › Reference › Standard built-in objects › ArrayBuffer

ArrayBuffer.prototype.byteLength

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/ArrayBuffer/byteLength · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **`byteLength`** accessor property of `ArrayBuffer` instances returns the length (in bytes) of this array buffer.

Try it

▷ Interactive example on MDN: "JavaScript Demo: ArrayBuffer.prototype.byteLength". It runs live in the browser there; its source code follows.

js · interactive example source

```
// Create an ArrayBuffer with a size in bytes
const buffer = new ArrayBuffer(8);

// Use byteLength to check the size
const bytes = buffer.byteLength;

console.log(bytes);
// Expected output: 8
```

Description

The `byteLength` property is an accessor property whose set accessor function is `undefined`, meaning that you can only read this property. The value is established when the array is constructed and cannot be changed. This property returns 0 if this `ArrayBuffer` has been detached.

Examples

Using byteLength

js

```
const buffer = new ArrayBuffer(8);
buffer.byteLength; // 8
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/structured-data.html#sec-get-arraybuffer.prototype.bytelength`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
byteLength	7	12	4	11.6	5.1	18	4	12	4.2	1.0	4	4.2	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.ArrayBuffer.byteLength`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `ArrayBuffer`

JavaScript › Reference › Standard built-in objects › ArrayBuffer

ArrayBuffer.prototype.detached

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/ArrayBuffer/detached · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since March 2024.

The **`detached`** accessor property of `ArrayBuffer` instances returns a boolean indicating whether or not this buffer has been detached (transferred).

Description

The `detached` property is an accessor property whose set accessor function is `undefined`, meaning that you can only read this property. The value is `false` when the `ArrayBuffer` is first created. The value becomes `true` if the `ArrayBuffer` is transferred, which detaches the instance from its underlying memory. Once a buffer becomes detached, it is no longer usable.

Examples

Using detached

```js
const buffer = new ArrayBuffer(8);
console.log(buffer.detached); // false
const newBuffer = buffer.transfer();
console.log(buffer.detached); // true
console.log(newBuffer.detached); // false
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/structured-data.html#sec-get-arraybuffer.prototype.detached`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`detached`	114	114	122	100	17.4	114	122	76	17.4	23.0	114	17.4	1.0.19	1.33	21.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.ArrayBuffer.detached`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `ArrayBuffer.prototype.detached` in `core-js`
- es-shims polyfill of `ArrayBuffer.prototype.detached`
- `ArrayBuffer`
- `ArrayBuffer.prototype.transfer()`
- `ArrayBuffer.prototype.transferToFixedLength()`

JavaScript › Reference › Standard built-in objects › ArrayBuffer

ArrayBuffer.prototype.maxByteLength

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/ArrayBuffer/maxByteLength · MDN page last modified 2025-07-10

Baseline 2024: Newly available — works across the latest browser versions since July 2024; may not work in older ones.

The **`maxByteLength`** accessor property of `ArrayBuffer` instances returns the maximum length (in bytes) that this array buffer can be resized to.

Try it

▷ Interactive example on MDN: "JavaScript Demo: ArrayBuffer.prototype.maxByteLength". It runs live in the browser there; its source code follows.

js · interactive example source

```
const buffer = new ArrayBuffer(8, { maxByteLength: 16 });

console.log(buffer.byteLength);
// Expected output: 8

console.log(buffer.maxByteLength);
// Expected output: 16
```

Description

The `maxByteLength` property is an accessor property whose set accessor function is `undefined`, meaning that you can only read this property. The value is established when the array is constructed, set via the `maxByteLength` option of the `ArrayBuffer()` constructor, and cannot be changed.

This property returns 0 if this `ArrayBuffer` has been detached. If this `ArrayBuffer` was constructed without specifying a `maxByteLength` value, this property returns a value equal to the value of the

`ArrayBuffer`'s `byteLength`.

Examples

Using maxByteLength

In this example, we create an 8-byte buffer that is resizable to a max length of 16 bytes, then return its `maxByteLength`:

```js
const buffer = new ArrayBuffer(8, { maxByteLength: 16 });

buffer.maxByteLength; // 16
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/structured-data.html#sec-get-arraybuffer.prototype.maxbytelength`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`maxByteLength`	111	111	128	97	16.4	111	128	75	16.4	22.0	111	16.4	1.0.0	1.33	20.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.ArrayBuffer.maxByteLength`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `ArrayBuffer`
- `ArrayBuffer.prototype.byteLength`
- `ArrayBuffer.prototype.resize()`

JavaScript › Reference › Standard built-in objects › ArrayBuffer

ArrayBuffer.prototype.resizable

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/ArrayBuffer/resizable · MDN page last modified 2025-07-10

Baseline 2024: Newly available — works across the latest browser versions since July 2024; may not work in older ones.

The **`resizable`** accessor property of `ArrayBuffer` instances returns whether this array buffer can be resized or not.

Try it

▷ Interactive example on MDN: "JavaScript Demo: ArrayBuffer.prototype.resizable". It runs live in the browser there; its source code follows.

js · interactive example source

```
const buffer1 = new ArrayBuffer(8, { maxByteLength: 16 });
const buffer2 = new ArrayBuffer(8);

console.log(buffer1.resizable);
// Expected output: true

console.log(buffer2.resizable);
// Expected output: false
```

Description

The `resizable` property is an accessor property whose set accessor function is `undefined`, meaning that you can only read this property. The value is established when the array is constructed. If the `maxByteLength` option was set in the constructor, `resizable` will return `true`; if not, it will return `false`.

Examples

Using resizable

In this example, we create a 8-byte buffer that is resizable to a max length of 16 bytes, then check its `resizable` property, resizing it if `resizable` returns `true`:

js

```
const buffer = new ArrayBuffer(8, { maxByteLength: 16 });

if (buffer.resizable) {
  console.log("Buffer is resizable!");
  buffer.resize(12);
}
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/structured-data.html#sec-get-arraybuffer.prototype.resizable`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`resizable`	111	111	128	97	16.4	111	128	75	16.4	22.0	111	16.4	1.0.0	1.33	20.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.ArrayBuffer.resizable`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `ArrayBuffer`
- `ArrayBuffer.prototype.maxByteLength`
- `ArrayBuffer.prototype.resize()`

JavaScript › Reference › Standard built-in objects

AsyncDisposableStack

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/AsyncDisposableStack · MDN page last modified 2026-10-26

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`AsyncDisposableStack`** object represents a stack of async disposers to run when the stack itself is disposed. Disposer functions are executed in reverse order of registration, with strong error handling guarantees. Calling its `move()` method will transfer responsibility for calling the current registered disposers to a new `AsyncDisposableStack` and prevent registering any additional disposers.

See `DisposableStack` for general information about using disposable stacks.

Constructor

`AsyncDisposableStack()`

Creates a new `AsyncDisposableStack` object.

Instance properties

These properties are defined on `AsyncDisposableStack.prototype` and shared by all `AsyncDisposableStack` instances.

`AsyncDisposableStack.prototype.constructor`

The constructor function that created the instance object. For `AsyncDisposableStack` instances, the initial value is the `AsyncDisposableStack` constructor.

`AsyncDisposableStack.prototype.disposed`

Read-only. Returns `true` if the `AsyncDisposableStack` has been disposed, or `false` if not.

`AsyncDisposableStack.prototype[Symbol.toStringTag]`

The initial value of the `[Symbol.toStringTag]` property is the string `"AsyncDisposableStack"`. This property is used in `Object.prototype.toString()`.

Instance methods

`AsyncDisposableStack.prototype.adopt()`

Registers a value that doesn't implement the async disposable protocol to the stack by providing a custom disposer function.

`AsyncDisposableStack.prototype.disposeAsync()`

Disposes this stack by calling all disposers registered to it in reverse order of registration.

`AsyncDisposableStack.prototype.defer()`

Takes a callback function to be called when the stack is disposed.

`AsyncDisposableStack.prototype.move()`

Creates a new `AsyncDisposableStack` instance that contains the same disposers as this stack, and then marks this stack as disposed, without calling any disposers.

`AsyncDisposableStack.prototype.use()`

Registers a value that implements the async disposable protocol to the stack.

`AsyncDisposableStack.prototype[Symbol.asyncDispose]`

An alias for the `disposeAsync()` method.

Specifications

Specification
ECMAScript Async Explicit Resource Management `https://tc39.es/proposal-async-explicit-resource-management/#sec-asyncdisposablestack-objects`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`AsyncDisposableStack`	134	134	141	119	Prev.	134	141	88	**No**	29.0	134	**No**	1.3.0	2.2.10	24.0.0
`@@asyncDispose`	134	134	141	119	Prev.	134	141	88	**No**	29.0	134	**No**	1.3.0	2.2.10	24.0.0
`AsyncDisposableStack`	134	134	141	119	Prev.	134	141	88	**No**	29.0	134	**No**	1.3.0	2.2.10	24.0.0
`adopt`	134	134	141	119	Prev.	134	141	88	**No**	29.0	134	**No**	1.3.0	2.2.10	24.0.0
`defer`	134	134	141	119	Prev.	134	141	88	**No**	29.0	134	**No**	1.3.0	2.2.10	24.0.0
`disposeAsync`	134	134	141	119	Prev.	134	141	88	**No**	29.0	134	**No**	1.3.0	2.2.10	24.0.0
`disposed`	134	134	141	119	Prev.	134	141	88	**No**	29.0	134	**No**	1.3.0	2.2.10	24.0.0
`move`	134	134	141	119	Prev.	134	141	88	**No**	29.0	134	**No**	1.3.0	2.2.10	24.0.0
`use`	134	134	141	119	Prev.	134	141	88	**No**	29.0	134	**No**	1.3.0	2.2.10	24.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.AsyncDisposableStack`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `AsyncDisposableStack` in `core-js`
- JavaScript resource management
- `Symbol.asyncDispose`
- `await using`
- `DisposableStack`

JavaScript › Reference › Standard built-in objects › AsyncDisposableStack

AsyncDisposableStack() constructor

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/AsyncDisposableStack/AsyncDisposableStack · MDN page last modified 2026-10-26

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`AsyncDisposableStack()`** constructor creates `AsyncDisposableStack` objects.

Syntax

```
new AsyncDisposableStack()
```

> **Note:** `AsyncDisposableStack()` can only be constructed with `new`. Attempting to call it without `new` throws a `TypeError`.

Parameters

None.

Return value

A new `AsyncDisposableStack` object.

Examples

Creating an AsyncDisposableStack

```js
const disposer = new AsyncDisposableStack();
disposer.defer(() => console.log("Disposed!"));
await disposer.disposeAsync();
// Logs: Disposed!
```

Specifications

Specification
ECMAScript Async Explicit Resource Management `https://tc39.es/proposal-async-explicit-resource-management/#sec-properties-of-the-asyncdisposablestack-constructor`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`AsyncDisposableStack`	134	134	141	119	Prev.	134	141	88	**No**	29.0	134	**No**	1.3.0	2.2.10	24.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.AsyncDisposableStack.AsyncDisposableStack`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Polyfill of `AsyncDisposableStack` in `core-js`
- JavaScript resource management

JavaScript › Reference › Standard built-in objects › AsyncDisposableStack

AsyncDisposableStack.prototype.adopt()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/AsyncDisposableStack/adopt · MDN page last modified 2026-10-26

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`adopt()`** method of `AsyncDisposableStack` instances registers a value that doesn't implement the async disposable protocol to the stack by providing a custom disposer function.

See `DisposableStack.prototype.adopt()` for general information about the `adopt()` method.

Syntax

```js
adopt(value, onDispose)
```

Parameters

`value`

Any value to be registered to the stack.

`onDispose`

A function that will be called when the stack is disposed. The function receives `value` as its only argument, and it can return a promise which gets awaited.

Return value

The same `value` that was passed in.

Exceptions

`TypeError`

Thrown if `onDispose` is not a function.

`ReferenceError`

Thrown if the stack is already disposed.

Examples

Using adopt()

This function creates a file handle (as a Node.js `FileHandle`), that gets closed when the function completes. We suppose that the file handle does not implement the async disposable protocol (in reality it does), so we use `adopt()` to register it to the stack. Because the `handle.close()` method returns a promise, we need to use an `AsyncDisposableStack` so that the disposal gets awaited.

```js
async function readFile(path) {
  await using disposer = new AsyncDisposableStack();
  const handle = disposer.adopt(
    await fs.open(path),
    async (handle) => await handle.close(),
  );
  const data = await handle.read();
  // The handle.close() method is called and awaited here before exiting
  return data;
}
```

Specifications

Specification
ECMAScript Async Explicit Resource Management https://tc39.es/proposal-async-explicit-resource-management/#sec-asyncdisposablestack.prototype.adopt

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
adopt	134	134	141	119	Prev.	134	141	88	**No**	29.0	134	**No**	1.3.0	2.2.10	24.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.AsyncDisposableStack.adopt`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `AsyncDisposableStack.prototype.adopt` in `core-js`
- JavaScript resource management
- `AsyncDisposableStack`
- `AsyncDisposableStack.prototype.defer()`
- `AsyncDisposableStack.prototype.use()`

JavaScript › Reference › Standard built-in objects › AsyncDisposableStack

AsyncDisposableStack.prototype.defer()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/AsyncDisposableStack/defer · MDN page last modified 2026-10-26

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`defer()`** method of `AsyncDisposableStack` instances takes a callback function to be called and awaited when the stack is disposed.

See `DisposableStack.prototype.defer()` for general information about the `defer()` method.

Syntax

```js
defer(onDispose)
```

Parameters

`onDispose`

A function that will be called when the stack is disposed. The function receives no arguments and can return a promise which gets awaited.

Return value

None (`undefined`).

Exceptions

`TypeError`

Thrown if `onDispose` is not a function.

`ReferenceError`

Thrown if the stack is already disposed.

Examples

Using defer()

One use case of `defer()` is to do something unrelated to resource freeing during scope exit, such as logging a message.

```js
async function doSomething() {
  await using disposer = new AsyncDisposableStack();
  disposer.defer(async () => {
    await fs.writeFile("log.txt", "All resources freed successfully");
  });
  // Other code that claims and frees more data
}
```

Specifications

Specification
ECMAScript Async Explicit Resource Management `https://tc39.es/proposal-async-explicit-resource-management/#sec-asyncdisposablestack.prototype.defer`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`defer`	134	134	141	119	Prev.	134	141	88	**No**	29.0	134	**No**	1.3.0	2.2.10	24.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.AsyncDisposableStack.defer`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `AsyncDisposableStack.prototype.defer` in `core-js`
- JavaScript resource management
- `AsyncDisposableStack`
- `AsyncDisposableStack.prototype.adopt()`
- `AsyncDisposableStack.prototype.use()`

JavaScript › Reference › Standard built-in objects › AsyncDisposableStack

AsyncDisposableStack.prototype.disposeAsync()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/AsyncDisposableStack/disposeAsync · MDN page last modified 2026-10-26

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`disposeAsync()`** method of `AsyncDisposableStack` instances disposes this stack by calling all disposers registered to it in reverse order of registration, awaiting for each one's completion before calling the next one. If the stack is already disposed, this method does nothing.

It performs the same action as `await using disposer = new AsyncDisposableStack()` at scope exit. It can be used if you need to clean up at a point other than scope exit.

Syntax

```
disposeAsync()
```

Parameters

None.

Return value

A new `Promise` that resolves with `undefined` when all registered disposers have completed in sequence.

Exceptions

`disposeAsync()` never synchronously throws an error. The returned promise may reject with one of the following errors:

`SuppressedError`

Thrown if multiple disposers in the stack threw an error. If only one error is thrown, it is rethrown as-is. Otherwise, for each additional error, a new `SuppressedError` is created, with the original error as the `suppressed` property, and the new error as the `error` property.

Examples

Disposing a stack

Here we push three disposers to the stack, using the `use()`, `adopt()`, and `defer()` methods. When `disposeAsync()` is called, the disposers are called in reverse order of registration.

Note that usually you don't need to call `disposeAsync()` manually. Declare the stack with `await using`, and its `[Symbol.asyncDispose]()` method will be automatically called when the stack goes out of scope.

```js
class Resource {
  #doDisposal() {
    // Imagine more meaningful disposal logic here
    return new Promise((resolve) => {
      setTimeout(resolve, 1000);
    });
  }
  async dispose() {
    await this.#doDisposal();
    console.log("Resource disposed");
  }
  async [Symbol.asyncDispose]() {
    await this.#doDisposal();
    console.log("Resource disposed via Symbol.asyncDispose");
  }
}

async function doSomething() {
  const disposer = new AsyncDisposableStack();
  const resource = disposer.use(new Resource());
  const resource2 = disposer.adopt(new Resource(), (resource) =>
    resource.dispose(),
  );
  disposer.defer(() => console.log("Deferred disposer"));
  disposer.disposeAsync();
  // Logs in order:
  // Deferred disposer
  // Resource disposed
  // Resource disposed via Symbol.dispose
}

doSomething();
```

Specifications

Specification
ECMAScript Async Explicit Resource Management `https://tc39.es/proposal-async-explicit-resource-management/#sec-asyncdisposablestack.prototype.disposeAsync`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`disposeAsync`	134	134	141	119	Prev.	134	141	88	**No**	29.0	134	**No**	1.3.0	2.2.10	24.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.AsyncDisposableStack.disposeAsync`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Polyfill of `AsyncDisposableStack.prototype.disposeAsync` in `core-js`
- JavaScript resource management
- `AsyncDisposableStack`
- `AsyncDisposableStack.prototype.adopt()`
- `AsyncDisposableStack.prototype.defer()`
- `AsyncDisposableStack.prototype.use()`
- `AsyncDisposableStack.prototype[Symbol.asyncDispose]()`

AsyncDisposableStack.prototype.move()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/AsyncDisposableStack/move · MDN page last modified 2026-10-26

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`move()`** method of `AsyncDisposableStack` instances creates a new `AsyncDisposableStack` instance that contains the same disposers as this stack, and then marks this stack as disposed, without calling any disposers.

Syntax

```js
move()
```

Parameters

None.

Return value

A new `AsyncDisposableStack` instance.

Exceptions

`ReferenceError`

Thrown if the stack is already disposed.

Examples

Claiming ownership of a stack

```js
async function consumeStack(stack) {
  await using newStack = stack.move(); // newStack now owns the disposers
  console.log(stack.disposed); // true
  console.log(newStack.disposed); // false
  // newStack is disposed here immediately before the function exits
}

const stack = new AsyncDisposableStack();
console.log(stack.disposed); // false
await consumeStack(stack);
console.log(stack.disposed); // true
```

Allowing resources to be disposed within two code paths

The major use case of `move()` is when you have one or more resources which could either be disposed right here or could be persisted for later use. In this case, you can put the resources in an `AsyncDisposableStack` and then call `move()` when you need to persist the resources for later usage.

```js
class PluginHost {
  #disposed = false;
  #disposables;
  #channel;
  #socket;

  static async init() {
    // Create an AsyncDisposableStack that is disposed when init exits.
    // If construction succeeds, we move everything out of `stack` and into
    // `#disposables` to be disposed later.
    await using stack = new AsyncDisposableStack();

    const channel = stack.use(await getChannel());

    const socket = stack.use(await getSocket());

    // If we made it here, then there were no errors during construction and
    // we can safely move the disposables out of `stack`.
    return new PluginHost(channel, socket, stack.move());

    // If construction failed, then `stack` would be disposed before reaching
    // the line above, which would dispose `channel` and `socket` in turn.
  }

  constructor(channel, socket, disposables) {
    this.#channel = channel;
    this.#socket = socket;
    this.#disposables = disposables;
  }
```

(code continues on the next page)

(code continued from the previous page)

```
  async [Symbol.asyncDispose]() {
    if (this.#disposed) {
      return;
    }
    this.#disposed = true;
    // Put `this.#disposables` into a `using` variable, so it is automatically
    // disposed when the function exits.
    await using disposables = this.#disposables;

    // NOTE: we can free `#socket` and `#channel` here since they will be
    // disposed by the call to `disposables[Symbol.asyncDispose]()`, below.
    // This isn't strictly a requirement for every disposable, but is
    // good housekeeping since these objects will no longer be useable.
    this.#socket = undefined;
    this.#channel = undefined;
    this.#disposables = undefined;
  }
}
```

Specifications

Specification
ECMAScript Async Explicit Resource Management `https://tc39.es/proposal-async-explicit-resource-management/#sec-asyncdisposablestack.prototype.move`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`move`	134	134	141	119	Prev.	134	141	88	**No**	29.0	134	**No**	1.3.0	2.2.10	24.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.AsyncDisposableStack.move`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `AsyncDisposableStack.prototype.move` in `core-js`
- JavaScript resource management
- `AsyncDisposableStack`
- `AsyncDisposableStack.prototype.disposeAsync()`

JavaScript › Reference › Standard built-in objects › AsyncDisposableStack

AsyncDisposableStack.prototype.use()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/AsyncDisposableStack/use · MDN page last modified 2026-10-26

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`use()`** method of `AsyncDisposableStack` instances registers a value that implements the async disposable protocol to the stack.

See `DisposableStack.prototype.use()` for general information about the `use()` method.

Syntax

```js
use(value)
```

Parameters

`value`

The value to register to the stack. Must either contain a `[Symbol.asyncDispose]()` or `[Symbol.dispose]()` method, or be `null` or `undefined`.

Return value

The same `value` that was passed in.

Exceptions

`TypeError`

Thrown if `value` is not `null` or `undefined`, and does not contain a `[Symbol.asyncDispose]()` or `[Symbol.dispose]()` method.

`ReferenceError`

Thrown if the stack is already disposed.

Examples

Using use()

This function reads a file (as a Node.js `FileHandle`) and returns its contents. The file handle is automatically closed when the function completes, given that the `FileHandle` class implements a `[Symbol.asyncDispose]()` method that asynchronously closes the file.

```js
async function readFileContents(path) {
  await using disposer = new AsyncDisposableStack();
  const handle = disposer.use(await fs.open(path));
  const data = await handle.read();
  return data;
  // The disposer is disposed here, which causes handle to be closed too
}
```

Specifications

Specification
ECMAScript Async Explicit Resource Management `https://tc39.es/proposal-async-explicit-resource-management/#sec-asyncdisposablestack.prototype.use`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
use	134	134	141	119	Prev.	134	141	88	**No**	29.0	134	**No**	1.3.0	2.2.10	24.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.AsyncDisposableStack.use`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `AsyncDisposableStack.prototype.use` in `core-js`
- JavaScript resource management
- `AsyncDisposableStack`
- `AsyncDisposableStack.prototype.adopt()`
- `AsyncDisposableStack.prototype.defer()`

JavaScript › Reference › Standard built-in objects › AsyncDisposableStack

AsyncDisposableStack.prototype[Symbol.asyncDispose]()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/AsyncDisposableStack/Symbol.asyncDispose · MDN page last modified 2026-10-26

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`[Symbol.asyncDispose]()`** method of `AsyncDisposableStack` instances implements the *async disposable protocol* and allows it to be disposed when used with `await using`. It is an alias for the `disposeAsync()` method.

Syntax

```js
asyncDisposableStack[Symbol.asyncDispose]()
```

Parameters

None.

Return value

None (`undefined`).

Examples

Declaring a stack with `await using`

The `Symbol.asyncDispose` method is intended to be automatically called in an `await using` declaration.

```js
async function doSomething() {
  await using disposer = new AsyncDisposableStack();
  const resource = disposer.use(new Resource());
  resource.doSomething();
  // disposer is disposed here immediately before the function exits
  // which causes the resource to be disposed
}
```

Specifications

Specification
ECMAScript Async Explicit Resource Management https://tc39.es/proposal-async-explicit-resource-management/#sec-asyncdisposablestack.prototype-@@asyncDispose

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`@@asyncDispose`	134	134	141	119	Prev.	134	141	88	**No**	29.0	134	**No**	1.3.0	2.2.10	24.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.AsyncDisposableStack.@@asyncDispose`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Polyfill of `AsyncDisposableStack.prototype[Symbol.asyncDispose]` in `core-js`
- JavaScript resource management
- `AsyncDisposableStack`
- `AsyncDisposableStack.prototype.disposeAsync()`

JavaScript › Reference › Standard built-in objects › AsyncDisposableStack

AsyncDisposableStack.prototype.disposed

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/AsyncDisposableStack/disposed · MDN page last modified 2026-10-26

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`disposed`** accessor property of `AsyncDisposableStack` instances returns a boolean indicating whether or not this `AsyncDisposableStack` has been disposed or moved by doing any of the following:

- Calling its `disposeAsync()` method
- Calling its `move()` method
- Declaring it with `await using` and letting the variable go out of scope, which automatically calls the `[Symbol.asyncDispose]()` method.

Examples

Checking if a stack is disposed

```js
const disposer = new AsyncDisposableStack();
console.log(disposer.disposed); // false
await disposer.disposeAsync();
console.log(disposer.disposed); // true
```

Specifications

Specification
ECMAScript Async Explicit Resource Management `https://tc39.es/proposal-async-explicit-resource-management/#sec-get-asyncdisposablestack.prototype.disposed`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`disposed`	134	134	141	119	Prev.	134	141	88	**No**	29.0	134	**No**	1.3.0	2.2.10	24.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.AsyncDisposableStack.disposed`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Polyfill of `AsyncDisposableStack.prototype.disposed` in `core-js`
- JavaScript resource management

JavaScript › Reference › Standard built-in objects

AsyncFunction

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/AsyncFunction · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since April 2017.

The **`AsyncFunction`** object provides methods for async functions. In JavaScript, every async function is actually an `AsyncFunction` object.

Note that `AsyncFunction` is *not* a global object. It can be obtained with the following code:

```js
const AsyncFunction = async function () {}.constructor;
```

`AsyncFunction` is a subclass of `Function`.

Constructor

`AsyncFunction()`

Creates a new `AsyncFunction` object.

Instance properties

Also inherits instance properties from its parent `Function`.

These properties are defined on `AsyncFunction.prototype` and shared by all `AsyncFunction` instances.

`AsyncFunction.prototype.constructor`

The constructor function that created the instance object. For `AsyncFunction` instances, the initial value is the `AsyncFunction` constructor.

`AsyncFunction.prototype[Symbol.toStringTag]`

The initial value of the `[Symbol.toStringTag]` property is the string `"AsyncFunction"`. This property is used in `Object.prototype.toString()`.

> **Note:** `AsyncFunction` instances do not have the `prototype` property.

Instance methods

Inherits instance methods from its parent `Function`.

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/control-abstraction-objects.html#sec-async-function-objects`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`AsyncFunction`	55	15	52	42	10.1	55	52	42	10.3	6.0	55	10.3	1.0.0	1.0	7.6.0
`AsyncFunction()` constructor	55	15	52	42	10.1	55	52	42	10.3	6.0	55	10.3	1.0.0	1.0	7.6.0
Requires arguments to be `TrustedScript` instance when trusted types are enforced	**No**	**No**	Prev.	**No**	26	**No**	**No**	**No**	26	**No**	**No**	26	?	?	?

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.AsyncFunction`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `async function`
- `async function` expression
- `Function`
- `AsyncGeneratorFunction`
- `GeneratorFunction`
- Functions

JavaScript › Reference › Standard built-in objects › AsyncFunction

AsyncFunction() constructor

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/AsyncFunction/AsyncFunction · MDN page last modified 2026-01-21

Baseline: Widely available — available across major browsers since April 2017.

> **Warning:** The arguments passed to this constructor are dynamically parsed and executed as JavaScript. APIs like this are known as injection sinks, and are potentially a vector for cross-site-scripting (XSS) attacks.
>
> You can mitigate this risk by always passing `TrustedScript` objects instead of strings and enforcing trusted types.
>
> See Security considerations in the `Function()` constructor reference for more information.

The **`AsyncFunction()`** constructor creates `AsyncFunction` objects.

Note that `AsyncFunction` is *not* a global object. It can be obtained with the following code:

```js
const AsyncFunction = async function () {}.constructor;
```

The `AsyncFunction()` constructor is not intended to be used directly, and all caveats mentioned in the `Function()` description apply to `AsyncFunction()`.

Syntax

```
new AsyncFunction(functionBody)
new AsyncFunction(arg1, functionBody)
new AsyncFunction(arg1, arg2, functionBody)
new AsyncFunction(arg1, arg2, /* …, */ argN, functionBody)

AsyncFunction(functionBody)
AsyncFunction(arg1, functionBody)
AsyncFunction(arg1, arg2, functionBody)
AsyncFunction(arg1, arg2, /* …, */ argN, functionBody)
```

Note: `AsyncFunction()` can be called with or without `new`. Both create a new `AsyncFunction` instance.

Parameters

See `Function()`.

Examples

Note that these examples omit the use of trusted types for brevity. For code showing the recommended approach, see Using `TrustedScript` in `eval()`.

Creating an async function from an AsyncFunction() constructor

```
function resolveAfter2Seconds(x) {
  return new Promise((resolve) => {
    setTimeout(() => {
      resolve(x);
    }, 2000);
  });
}

const AsyncFunction = async function () {}.constructor;

const fn = new AsyncFunction(
  "a",
  "b",
  "return await resolveAfter2Seconds(a) + await resolveAfter2Seconds(b);",
);

fn(10, 20).then((v) => {
  console.log(v); // prints 30 after 4 seconds
});
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/control-abstraction-objects.html#sec-async-function-constructor

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`AsyncFunction()` constructor	55	15	52	42	10.1	55	52	42	10.3	6.0	55	10.3	1.0.0	1.0	7.6.0
Requires arguments to be `TrustedScript` instance when trusted types are enforced	**No**	**No**	Prev.	**No**	26	**No**	**No**	**No**	26	**No**	**No**	26	?	?	?

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.AsyncFunction.AsyncFunction`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `async function`
- `async function` expression
- `Function()` constructor

JavaScript › Reference › Standard built-in objects

AsyncGenerator

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/AsyncGenerator · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since January 2020.

The **`AsyncGenerator`** object is returned by an async generator function and it conforms to both the async iterable protocol and the async iterator protocol.

Async generator methods always yield `Promise` objects.

`AsyncGenerator` is a subclass of the hidden `AsyncIterator` class.

Constructor

There's no JavaScript entity that corresponds to the `AsyncGenerator` constructor. Instances of `AsyncGenerator` must be returned from async generator functions:

```js
async function* createAsyncGenerator() {
  yield Promise.resolve(1);
  yield await Promise.resolve(2);
  yield 3;
}
const asyncGen = createAsyncGenerator();
asyncGen.next().then((res) => console.log(res.value)); // 1
asyncGen.next().then((res) => console.log(res.value)); // 2
asyncGen.next().then((res) => console.log(res.value)); // 3
```

There's only a hidden object which is the prototype object shared by all objects created by async generator functions. This object is often stylized as `AsyncGenerator.prototype` to make it look like a class, but it should be more appropriately called `AsyncGeneratorFunction.prototype.prototype`, because `AsyncGeneratorFunction` is an actual JavaScript entity. To understand the prototype chain of `AsyncGenerator` instances, see `AsyncGeneratorFunction.prototype.prototype`.

Instance properties

These properties are defined on `AsyncGenerator.prototype` and shared by all `AsyncGenerator` instances.

`AsyncGenerator.prototype.constructor`

The constructor function that created the instance object. For `AsyncGenerator` instances, the initial value is `AsyncGeneratorFunction.prototype`.

> **Note:** `AsyncGenerator` objects do not store a reference to the async generator function that created them.

`AsyncGenerator.prototype[Symbol.toStringTag]`

The initial value of the `[Symbol.toStringTag]` property is the string `"AsyncGenerator"`. This property is used in `Object.prototype.toString()`.

Instance methods

Also inherits instance methods from its parent `AsyncIterator`.

`AsyncGenerator.prototype.next()`

Returns a `Promise` which will be resolved with the given value yielded by the `yield` expression.

`AsyncGenerator.prototype.return()`

Acts as if a `return` statement is inserted in the generator's body at the current suspended position, which finishes the generator and allows the generator to perform any cleanup tasks when combined with a `try...finally` block.

`AsyncGenerator.prototype.throw()`

Acts as if a `throw` statement is inserted in the generator's body at the current suspended position, which informs the generator of an error condition and allows it to handle the error, or perform cleanup and close itself.

Examples

Async generator iteration

The following example iterates over an async generator, logging values 1–6 to the console at decreasing time intervals. Notice how each time a Promise is yielded, but it's automatically resolved within the `for await...of` loop.

```js
// An async task. Pretend it's doing something more useful
// in practice.
function delayedValue(time, value) {
  return new Promise((resolve /*, reject */) => {
    setTimeout(() => resolve(value), time);
  });
}

async function* generate() {
  yield delayedValue(2000, 1);
  yield delayedValue(1000, 2);
  yield delayedValue(500, 3);
  yield delayedValue(250, 4);
  yield delayedValue(125, 5);
  yield delayedValue(50, 6);
  console.log("All done!");
}

async function main() {
  for await (const value of generate()) {
    console.log("value", value);
  }
}

main().catch((e) => console.error(e));
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/control-abstraction-objects.html#sec-asyncgenerator-objects`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`AsyncGenerator`	63	79	55	50	12	63	55	46	12	8.0	63	12	1.0.0	1.0	10.0.0
`next`	63	79	55	50	12	63	55	46	12	8.0	63	12	1.0.0	1.0	10.0.0
`return`	63	79	55	50	12	63	55	46	12	8.0	63	12	1.0.0	1.0	10.0.0
`throw`	63	79	55	50	12	63	55	46	12	8.0	63	12	1.0.0	1.0	10.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.AsyncGenerator`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- function*
- async function*
- `function*` expression
- Generator Function
- Async Generator Function
- Iterators and generators guide

JavaScript › Reference › Standard built-in objects › AsyncGenerator

AsyncGenerator.prototype.next()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/AsyncGenerator/next · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since January 2020.

The **`next()`** method of `AsyncGenerator` instances returns the next value in the sequence.

Syntax

```js
next()
next(value)
```

Parameters

`value` Optional

An optional value used to modify the internal state of the generator. A value passed to the `next()` method will be received by `yield`

Return value

A `Promise` which when resolved returns an `Object` with two properties:

`done`

A boolean value:

- `true` if the generator is past the end of its control flow. In this case `value` specifies the *return value* of the generator (which may be undefined).
- `false` if the generator is able to produce more values.

`value`

Any JavaScript value yielded or returned by the generator.

Examples

Using next()

The following example shows a generator and the object that the `next` method returns:

```js
// An async task. Pretend it's doing something more useful
// in practice.
function delayedValue(time, value) {
  return new Promise((resolve, reject) => {
    setTimeout(() => resolve(value), time);
  });
}

async function* createAsyncGenerator() {
  yield delayedValue(500, 1);
  yield delayedValue(500, 2);
  yield delayedValue(500, 3);
}

const asyncGen = createAsyncGenerator();
asyncGen.next().then((res) => console.log(res)); // { value: 1, done: false }
asyncGen.next().then((res) => console.log(res)); // { value: 2, done: false }
asyncGen.next().then((res) => console.log(res)); // { value: 3, done: false }
asyncGen.next().then((res) => console.log(res)); // { value: undefined, done: true }
```

Sending values to the generator

In this example, `next` is called with a value.

> **Note:** The first call does not log anything, because the generator was not yielding anything initially.

```js
// An async task. Pretend it's doing something more useful
// in practice.
function sleep(time) {
  return new Promise((resolve, reject) => {
    setTimeout(resolve, time);
  });
}
```

(code continues on the next page)

(code continued from the previous page)

```
async function* createAsyncGenerator() {
  while (true) {
    await sleep(500);
    const value = yield;
    console.log(value);
  }
}

async function main() {
  const asyncGen = createAsyncGenerator();
  // No log at this step: the first value sent through `next` is lost
  console.log(await asyncGen.next(1)); // { value: undefined, done: false }
  // Logs 2: the value sent through `next`
  console.log(await asyncGen.next(2)); // { value: undefined, done: false }
}

main();
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/control-abstraction-objects.html#sec-asyncgenerator-prototype-next

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Nodejs
next	63	79	55	50	12	63	55	46	12	8.0	63	12	1.0.0	1.0	10.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.AsyncGenerator.next`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `async function*`
- Iterators and generators guide

JavaScript › Reference › Standard built-in objects › AsyncGenerator

AsyncGenerator.prototype.return()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/AsyncGenerator/return · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since January 2020.

The **`return()`** method of `AsyncGenerator` instances acts as if a `return` statement is inserted in the generator's body at the current suspended position, which finishes the generator and allows the generator to perform any cleanup tasks when combined with a `try...finally` block.

Syntax

```js
asyncGeneratorInstance.return()
asyncGeneratorInstance.return(value)
```

Parameters

`value` Optional

The value to return.

Return value

A `Promise` which resolves with an `Object` with two properties:

`done`

A boolean value:

- `true` if the generator function's control flow has reached the end.
- `false` if the generator function's control flow hasn't reached the end and can produce more values. This can only happen if the `return` is captured in a `try...finally` and there are more `yield` expressions in the `finally` block.

`value`

The value that is given as an argument, or, if the `yield` expression is wrapped in a `try...finally`, the value yielded/returned from the `finally` block.

Description

The `return()` method, when called, can be seen as if a `return value;` statement is inserted in the generator's body at the current suspended position, where `value` is the value passed to the `return()` method. Therefore, in a typical flow, calling `return(value)` will return `{ done: true, value: value }`. However, if the `yield` expression is wrapped in a `try...finally` block, the control flow doesn't exit the function body, but proceeds to the `finally` block instead. In this case, the value returned may be different, and `done` may even be `false`, if there are more `yield` expressions within the `finally` block.

Examples

Using return()

The following example shows an async generator and the `return` method.

```js
// An async task. Pretend it's doing something more useful
// in practice.
function delayedValue(time, value) {
  return new Promise((resolve, reject) => {
    setTimeout(() => resolve(value), time);
  });
}
```

(code continues on the next page)

(code continued from the previous page)

```
async function* createAsyncGenerator() {
  yield delayedValue(500, 1);
  yield delayedValue(500, 2);
  yield delayedValue(500, 3);
}

const asyncGen = createAsyncGenerator();
asyncGen.next().then((res) => console.log(res)); // { value: 1, done: false }
asyncGen.return("foo").then((res) => console.log(res)); // { value: "foo", done: true }
asyncGen.next().then((res) => console.log(res)); // { value: undefined, done: true }
```

Using return() once a generator is complete

If no `value` argument is passed into the `return()` method, the promise will resolve as if the next() method has been called. In this example the generator has completed, so the value returned is `undefined`.

`return()` can still be called after the generator is in a "completed" state, however the generator will stay in this state.

js

```
async function* createAsyncGenerator() {
  yield Promise.resolve(1);
  yield await Promise.resolve(2);
  yield 3;
}
const asyncGen = createAsyncGenerator();
asyncGen.next().then((res) => console.log(res)); // { value: 1, done: false }
asyncGen.next().then((res) => console.log(res)); // { value: 2, done: false }
asyncGen.next().then((res) => console.log(res)); // { value: 3, done: false }
// value is returned undefined, as no value is passed and generator is 'done'
asyncGen.return().then((res) => console.log(res)); // { value: undefined, done: true }
// we can still return a value once the generator is complete
asyncGen.return(1).then((res) => console.log(res)); // { value: 1, done: true }
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/control-abstraction-objects.html#sec-asyncgenerator-prototype-return`

Browser compatibility

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`return`	63	79	55	50	12	63	55	46	12	8.0	63	12	1.0.0	1.0	10.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.AsyncGenerator.return`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- async function*
- Iterators and generators guide

JavaScript › Reference › Standard built-in objects › AsyncGenerator

AsyncGenerator.prototype.throw()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/AsyncGenerator/throw · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since January 2020.

The **`throw()`** method of AsyncGenerator instances acts as if a `throw` statement is inserted in the generator's body at the current suspended position, which informs the generator of an error condition and allows it to handle the error, or perform cleanup and close itself.

Syntax

```js
asyncGeneratorInstance.throw(exception)
```

Parameters

`exception`

The exception to throw. For debugging purposes, it is useful to make it an `instanceof` Error.

Return value

If the thrown error is not caught, it will return a Promise which rejects with the exception passed in.

If the exception is caught by a try...catch and the generator resumes to yield more values, it will return a Promise which resolves with an Object with two properties:

`done`

A boolean value:

- `true` if the generator function's control flow has reached the end.
- `false` if the generator function is able to produce more values.

`value`

The value yielded from the next `yield` expression.

Examples

Using throw()

The following example shows a generator and an error that is thrown using the `throw` method. An error can be caught by a try...catch block as usual.

```js
// An async task. Pretend it's doing something more useful
// in practice.
function sleep(time) {
  return new Promise((resolve, reject) => {
    setTimeout(resolve, time);
  });
}

async function* createAsyncGenerator() {
  while (true) {
    try {
      await sleep(500);
      yield 42;
    } catch (e) {
      console.error(e);
    }
  }
}

const asyncGen = createAsyncGenerator();
asyncGen.next(1).then((res) => console.log(res)); // { value: 42, done: false }
asyncGen
  .throw(new Error("Something went wrong")) // Error: Something went wrong
  .then((res) => console.log(res)); // { value: 42, done: false }
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/control-abstraction-objects.html#sec-asyncgenerator-prototype-throw

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
throw	63	79	55	50	12	63	55	46	12	8.0	63	12	1.0.0	1.0	10.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.AsyncGenerator.throw`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `async function*`
- Iterators and generators guide

JavaScript › Reference › Standard built-in objects

AsyncGeneratorFunction

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/AsyncGeneratorFunction · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since January 2020.

The **`AsyncGeneratorFunction`** object provides methods for async generator functions. In JavaScript, every async generator function is actually an `AsyncGeneratorFunction` object.

Note that `AsyncGeneratorFunction` is *not* a global object. It can be obtained with the following code:

js

```
const AsyncGeneratorFunction = async function* () {}.constructor;
```

`AsyncGeneratorFunction` is a subclass of `Function`.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: AsyncGeneratorFunction". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const AsyncGeneratorFunction = async function* () {}.constructor;

const foo = new AsyncGeneratorFunction(`
  yield await Promise.resolve('a');
  yield await Promise.resolve('b');
  yield await Promise.resolve('c');
`);

let str = "";

async function generate() {
  for await (const val of foo()) {
    str += val;
  }
  console.log(str);
}

generate();
// Expected output: "abc"
```

Constructor

`AsyncGeneratorFunction()`

Creates a new `AsyncGeneratorFunction` object.

Instance properties

Also inherits instance properties from its parent `Function`.

These properties are defined on `AsyncGeneratorFunction.prototype` and shared by all `AsyncGeneratorFunction` instances.

`AsyncGeneratorFunction.prototype.constructor`

The constructor function that created the instance object. For `AsyncGeneratorFunction` instances, the initial value is the `AsyncGeneratorFunction` constructor.

`AsyncGeneratorFunction.prototype.prototype`

All async generator functions share the same `prototype` property, which is `AsyncGenerator.prototype`. Each async generator function created with the `async function*` syntax or the `AsyncGeneratorFunction()` constructor also has its own `prototype` property, whose prototype is `AsyncGeneratorFunction.prototype.prototype`. When the async generator function is called, its `prototype` property becomes the prototype of the returned async generator object.

`AsyncGeneratorFunction.prototype[Symbol.toStringTag]`

The initial value of the `[Symbol.toStringTag]` property is the string `"AsyncGeneratorFunction"`. This property is used in `Object.prototype.toString()`.

These properties are own properties of each `AsyncGeneratorFunction` instance.

`prototype`

Used when the function is used as a constructor with the `new` operator. It will become the new object's prototype.

Instance methods

Inherits instance methods from its parent `Function`.

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/control-abstraction-objects.html#sec-asyncgeneratorfunction-objects`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`AsyncGeneratorFunction`	63	79	55	50	12	63	55	46	12	8.0	63	12	1.0.0	1.0	10.0.0
`AsyncGeneratorFunction()` constructor	63	79	55	50	12	63	55	46	12	8.0	63	12	1.0.0	1.0	10.0.0
Requires arguments to be `TrustedScript` instance when trusted types are enforced	**No**	**No**	Prev.	**No**	26	**No**	**No**	**No**	26	**No**	**No**	26	?	?	?

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.AsyncGeneratorFunction`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `async function*`
- `async function*` expression
- `Function`
- `AsyncFunction`
- `GeneratorFunction`

- Functions

JavaScript › Reference › Standard built-in objects › AsyncGeneratorFunction

AsyncGeneratorFunction() constructor

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/AsyncGeneratorFunction/AsyncGeneratorFunction · MDN page last modified 2026-01-21

Baseline: Widely available — available across major browsers since January 2020.

> **Warning:** The arguments passed to this constructor are dynamically parsed and executed as JavaScript. APIs like this are known as injection sinks, and are potentially a vector for cross-site-scripting (XSS) attacks.
>
> You can mitigate this risk by always passing `TrustedScript` objects instead of strings and enforcing trusted types.
>
> See Security considerations in the `Function()` constructor reference for more information.

The **`AsyncGeneratorFunction()`** constructor creates `AsyncGeneratorFunction` objects.

Note that `AsyncGeneratorFunction` is not a global object. It could be obtained by evaluating the following code.

```js
const AsyncGeneratorFunction = async function* () {}.constructor;
```

The `AsyncGeneratorFunction()` constructor is not intended to be used directly, and all caveats mentioned in the `Function()` description apply to `AsyncGeneratorFunction()`.

Syntax

```js
new AsyncGeneratorFunction(functionBody)
new AsyncGeneratorFunction(arg1, functionBody)
new AsyncGeneratorFunction(arg1, arg2, functionBody)
new AsyncGeneratorFunction(arg1, arg2, /* …, */ argN, functionBody)

AsyncGeneratorFunction(functionBody)
AsyncGeneratorFunction(arg1, functionBody)
AsyncGeneratorFunction(arg1, arg2, functionBody)
AsyncGeneratorFunction(arg1, arg2, /* …, */ argN, functionBody)
```

> **Note:** `AsyncGeneratorFunction()` can be called with or without `new`. Both create a new `AsyncGeneratorFunction` instance.

Parameters

See `Function()`.

Examples

Note that these examples omit the use of trusted types for brevity. For code showing the recommended approach, see Using `TrustedScript` in `eval()`.

Using the constructor

The following example uses the `AsyncGeneratorFunction` constructor to create an async generator function.

```js
const AsyncGeneratorFunction = async function* () {}.constructor;
const createAsyncGenerator = new AsyncGeneratorFunction("a", "yield a * 2");
const asyncGen = createAsyncGenerator(10);
asyncGen.next().then((res) => console.log(res.value)); // 20
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/control-abstraction-objects.html#sec-asyncgeneratorfunction-constructor`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`AsyncGeneratorFunction()` constructor	63	79	55	50	12	63	55	46	12	8.0	63	12	1.0.0	1.0	10.0.0
Requires arguments to be `TrustedScript` instance when trusted types are enforced	**No**	**No**	Prev.	**No**	26	**No**	**No**	**No**	26	**No**	**No**	26	?	?	?

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.AsyncGeneratorFunction.AsyncGeneratorFunction`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `async function*`
- `async function*` expression
- `Function()` constructor
- Iterators and generators guide
- Functions

JavaScript › Reference › Standard built-in objects › AsyncGeneratorFunction

AsyncGeneratorFunction.prototype.prototype

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/AsyncGeneratorFunction/prototype · MDN page last modified 2025-07-10

The **`prototype`** property of `AsyncGeneratorFunction.prototype` is shared by all async generator functions. Its value is `AsyncGenerator.prototype`. Each async generator function created with the `async function*` syntax or the `AsyncGeneratorFunction()` constructor also has its own `prototype` property, whose prototype is `AsyncGeneratorFunction.prototype.prototype`. When the async generator function is called, its `prototype` property becomes the prototype of the returned async generator object.

Value

The same object as `AsyncGenerator.prototype`. `AsyncGeneratorFunction.prototype.prototype` is the technically more accurate name, but `AsyncGenerator.prototype` appeals to the intuition that it's the prototype of async generator objects.

Property attributes of `AsyncGeneratorFunction.prototype.prototype`	
Writable	no
Enumerable	no
Configurable	yes

The `prototype` property of each `AsyncGeneratorFunction` instance is an empty object with no properties, whose prototype is `AsyncGeneratorFunction.prototype.prototype`. It has the following property attributes:

Property attributes of `AsyncGeneratorFunction.prototype.prototype`	
Writable	yes
Enumerable	no
Configurable	no

Description

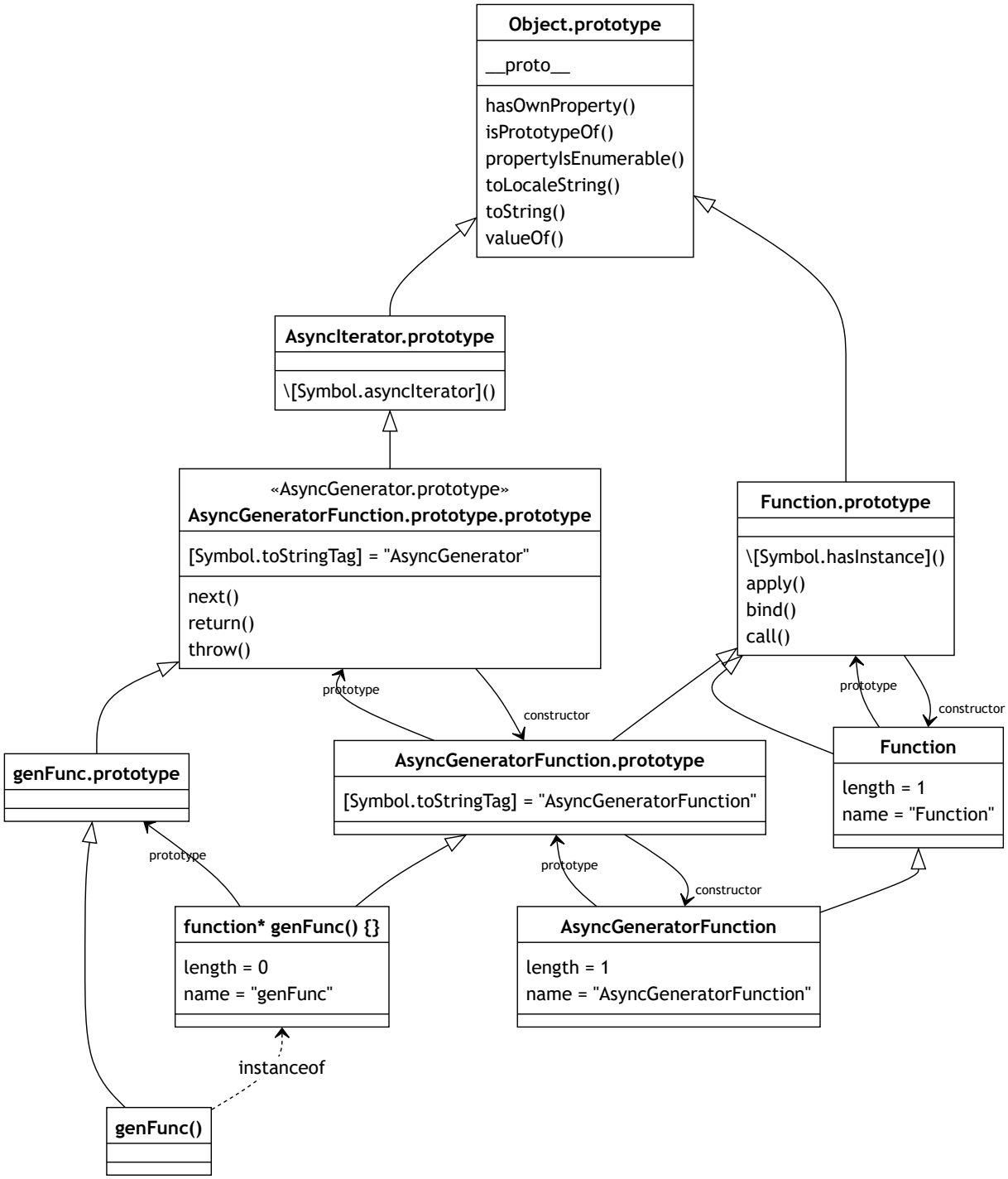
An async generator function instance has two `prototype` properties. The first one is its own `prototype` property. The second one is the `prototype` property on its prototype, which is `AsyncGeneratorFunction.prototype`. (Remember that every async generator function is an instance of `AsyncGeneratorFunction`, so it has `AsyncGeneratorFunction.prototype` as its prototype.)

```js
async function* genFunc() {}
const AsyncGeneratorFunctionPrototype = Object.getPrototypeOf(genFunc);
console.log(Object.hasOwn(genFunc, "prototype")); // true
console.log(Object.hasOwn(AsyncGeneratorFunctionPrototype, "prototype")); // true
```

When an async generator function is called, the async generator function's `prototype` property becomes the prototype of the returned async generator object.

```js
const gen = genFunc();
const proto = Object.getPrototypeOf;
console.log(proto(gen) === genFunc.prototype); // true
console.log(proto(proto(gen)) === AsyncGeneratorFunctionPrototype.prototype); // true
```

The following diagram illustrates the prototype chain of an async generator function and its instances. Each hollow arrow indicates an inheritance relationship (i.e., a prototype link), and each solid arrow indicates a property relationship. Note that there's no way to access `genFunc` from `gen` — they only have an `instanceof` relationship.



Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/control-abstraction-objects.html#sec-asyncgeneratorfunction-prototype-prototype
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/control-abstraction-objects.html#sec-asyncgeneratorfunction-instances-prototype

See also

- `async function*`
- `async function*` expression
- `AsyncGeneratorFunction`
- `GeneratorFunction`
- Inheritance and the prototype chain
- Iterators and generators

JavaScript › Reference › Standard built-in objects

AsyncIterator

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/AsyncIterator · MDN page last modified 2025-10-30

Baseline: Widely available — available across major browsers since January 2020.

An **`AsyncIterator`** object is an object that conforms to the async iterator protocol by providing a `next()` method that returns a promise fulfilling to an iterator result object. The `AsyncIterator.prototype` object is a hidden global object that all built-in async iterators inherit from. It provides a `[Symbol.asyncIterator]()` method that returns the async iterator object itself, making the async iterator also async iterable.

Note that `AsyncIterator` is *not* a global object, although it will be in the future with the async iterator helpers proposal. The `AsyncIterator.prototype` object shared by all built-in async iterators can be obtained with the following code:

```js
const AsyncIteratorPrototype = Object.getPrototypeOf(
  Object.getPrototypeOf(Object.getPrototypeOf((async function* () {})())),
);
```

Description

Currently, the only built-in JavaScript async iterator is the `AsyncGenerator` object returned by async generator functions. There are some other built-in async iterators in web API, such as the one of a `ReadableStream`.

Each of these async iterators have a distinct prototype object, which defines the `next()` method used by the particular async iterator. All of these prototype objects inherit from `AsyncIterator.prototype`, which provides a `[Symbol.asyncIterator]()` method that returns the async iterator object itself, making the async iterator also async iterable.

> **Note:** `AsyncIterator.prototype` does not implement `[Symbol.iterator]()`, so async iterators are not sync iterable by default.

Instance methods

`AsyncIterator.prototype[Symbol.asyncDispose]()`

Calls and awaits the `return()` method of `this`, if it exists. This implements the *async disposable protocol* and allows it to be disposed when used with `await using`.

`AsyncIterator.prototype[Symbol.asyncIterator]()`

Returns the async iterator object itself. This allows async iterator objects to also be async iterable.

Examples

Using an async iterator as an async iterable

All built-in async iterators are also async iterable, so you can use them in a `for await...of` loop:

```js
const asyncIterator = (async function* () {
  yield 1;
  yield 2;
  yield 3;
})();
(async () => {
  for await (const value of asyncIterator) {
    console.log(value);
  }
})();
// Logs: 1, 2, 3
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/control-abstraction-objects.html#sec-asynciteratorprototype

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
AsyncIterator	63	79	57	50	11.1	63	57	46	11.3	8.0	63	11.3	1.0.0	1.0	10.0.0
@@asyncDispose	134	134	141	119	Prev.	134	141	88	**No**	29.0	134	**No**	1.3.0	2.2.10	24.0.0
[Symbol.asyncIterator]	63	79	57	50	11.1	63	57	46	11.3	8.0	63	11.3	1.0.0	1.0	10.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.AsyncIterator`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `async function*`
- Iteration protocols

JavaScript › Reference › Standard built-in objects › AsyncIterator

AsyncIterator.prototype[Symbol.asyncDispose]()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/AsyncIterator/Symbol.asyncDispose · MDN page last modified 2026-10-26

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`[Symbol.asyncDispose]()`** method of `AsyncIterator` instances implements the *async disposable protocol* and allows it to be disposed when used with `await using`. It calls and awaits the `return()` method of `this`, if it exists.

Syntax

```js
asyncIterator[Symbol.asyncDispose]()
```

Parameters

None.

Return value

None (`undefined`).

Examples

Declaring an async iterator with `await using`

The `Symbol.asyncDispose` method is intended to be automatically called in an `await using` declaration. This is useful if you have an async iterator that you manually iterate over by calling its `next()` method; if you iterate it with `for await...of` or something similar, then error handling and cleanup is done automatically.

```js
async function* generateNumbers() {
  try {
    yield 1;
    yield 2;
    yield 3;
  } finally {
    console.log("Cleaning up");
  }
}

async function doSomething() {
  await using numbers = generateNumbers();
  const res1 = await numbers.next();
  // Not iterating the rest of the numbers
  // Before the function exits, the async iterator is disposed
  // Logs "Cleaning up"
}

doSomething();
```

Specifications

Specification
ECMAScript Async Explicit Resource Management https://tc39.es/proposal-async-explicit-resource-management/#sec-asyncdisposablestack.prototype-@@asyncDispose

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`@@asyncDispose`	134	134	141	119	Prev.	134	141	88	**No**	29.0	134	**No**	1.3.0	2.2.10	24.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.AsyncIterator.@@asyncDispose`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `AsyncIterator.prototype[Symbol.asyncDispose]` in `core-js`
- JavaScript resource management
- `Symbol.asyncDispose`
- `await using`

JavaScript › Reference › Standard built-in objects › AsyncIterator

AsyncIterator.prototype[Symbol.asyncIterator]()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/AsyncIterator/Symbol.asyncIterator · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since January 2020.

The **`[Symbol.asyncIterator]()`** method of `AsyncIterator` instances implements the async iterable protocol and allows built-in async iterators to be consumed by most syntaxes expecting async iterables, such as `for await...of` loops. It returns the value of `this`, which is the async iterator object itself.

Syntax

```
asyncIterator[Symbol.asyncIterator]()
```

Parameters

None.

Return value

The value of `this`, which is the async iterator object itself.

Examples

Iteration using for await...of loop

Note that you seldom need to call this method directly. The existence of the `[Symbol.asyncIterator]()` method makes all built-in async iterators async iterable, and iterating syntaxes like the `for await...of` loop automatically calls this method to obtain the async iterator to loop over.

```js
const asyncIterator = (async function* () {
  yield 1;
  yield 2;
  yield 3;
})();
(async () => {
  for await (const value of asyncIterator) {
    console.log(value);
  }
})();
// Logs: 1, 2, 3
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/control-abstraction-objects.html#sec-%asynciteratorprototype%-%symbol.asynciterator%

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
[Symbol.asyncIterator]	63	79	57	50	11.1	63	57	46	11.3	8.0	63	11.3	1.0.0	1.0	10.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.AsyncIterator.@@asyncIterator`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `for await...of`

JavaScript › Reference › Standard built-in objects

Atomics

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Atomics · MDN page last modified 2026-10-28

Baseline: Widely available — available across major browsers since December 2021.

The **`Atomics`** namespace object contains static methods for carrying out atomic operations. They are used with `SharedArrayBuffer` and `ArrayBuffer` objects.

Description

Unlike most global objects, `Atomics` is not a constructor. You cannot use it with the `new` operator or invoke the `Atomics` object as a function. All properties and methods of `Atomics` are static (just like the `Math` object).

Static properties

`Atomics[Symbol.toStringTag]`

The initial value of the `[Symbol.toStringTag]` property is the string `"Atomics"`. This property is used in `Object.prototype.toString()`.

Static methods

`Atomics.add()`

Adds the provided value to the existing value at the specified index of the array. Returns the old value at that index.

`Atomics.and()`

Computes a bitwise AND on the value at the specified index of the array with the provided value. Returns the old value at that index.

`Atomics.compareExchange()`

Stores a value at the specified index of the array, if it equals a value. Returns the old value.

`Atomics.exchange()`

Stores a value at the specified index of the array. Returns the old value.

`Atomics.isLockFree()`

An optimization primitive that can be used to determine whether to use locks or atomic operations. Returns `true` if an atomic operation on arrays of the given element size will be implemented using a hardware atomic operation (as opposed to a lock). Experts only.

`Atomics.load()`

Returns the value at the specified index of the array.

`Atomics.notify()`

Notifies agents that are waiting on the specified index of the array. Returns the number of agents that were notified.

`Atomics.or()`

Computes a bitwise OR on the value at the specified index of the array with the provided value. Returns the old value at that index.

`Atomics.pause()`

Provides a micro-wait primitive that hints to the CPU that the caller is spinning while waiting on access to a shared resource. This allows the system to reduce the resources allocated to the core (such as power) or thread, without yielding the current thread.

`Atomics.store()`

Stores a value at the specified index of the array. Returns the value.

`Atomics.sub()`

Subtracts a value at the specified index of the array. Returns the old value at that index.

`Atomics.wait()`

Verifies that the specified index of the array still contains a value and sleeps awaiting or times out. Returns either `"ok"`, `"not-equal"`, or `"timed-out"`. If waiting is not allowed in the calling agent then it throws an exception. (Most browsers will not allow `wait()` on the browser's main thread.)

`Atomics.waitAsync()`

Waits asynchronously (i.e., without blocking, unlike `Atomics.wait`) on a shared memory location and returns an object representing the result of the operation.

`Atomics.xor()`

Computes a bitwise XOR on the value at the specified index of the array with the provided value. Returns the old value at that index.

Examples

Note that these examples cannot be run directly from the console or an arbitrary web page, because `SharedArrayBuffer` is not defined unless its security requirements are met.

Using Atomics

js

```
const sab = new SharedArrayBuffer(1024);
const ta = new Uint8Array(sab);

ta[0]; // 0
ta[0] = 5; // 5

Atomics.add(ta, 0, 12); // 5
Atomics.load(ta, 0); // 17

Atomics.and(ta, 0, 1); // 17
Atomics.load(ta, 0); // 1

Atomics.compareExchange(ta, 0, 5, 12); // 1
Atomics.load(ta, 0); // 1

Atomics.exchange(ta, 0, 12); // 1
Atomics.load(ta, 0); // 12

Atomics.isLockFree(1); // true
Atomics.isLockFree(2); // true
Atomics.isLockFree(3); // false
Atomics.isLockFree(4); // true

Atomics.or(ta, 0, 1); // 12
Atomics.load(ta, 0); // 13
```

(code continues on the next page)

(code continued from the previous page)

```
Atomics.store(ta, 0, 12); // 12

Atomics.sub(ta, 0, 2); // 12
Atomics.load(ta, 0); // 10

Atomics.xor(ta, 0, 1); // 10
Atomics.load(ta, 0); // 11
```

Waiting and notifying

Given a shared `Int32Array`:

js

```
const sab = new SharedArrayBuffer(1024);
const int32 = new Int32Array(sab);
```

A reading thread is sleeping and waiting on location 0 because the provided value matches what is stored at the provided index. The reading thread will not move on until the writing thread has called `Atomics.notify()` on position 0 of the provided typed array. Note that if, after being woken up, the value of location 0 has not been changed by the writing thread, the reading thread will **not** go back to sleep, but will continue on.

js

```
Atomics.wait(int32, 0, 0);
console.log(int32[0]); // 123
```

A writing thread stores a new value and notifies the waiting thread once it has written:

js

```
console.log(int32[0]); // 0;
Atomics.store(int32, 0, 123);
Atomics.notify(int32, 0, 1);
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/structured-data.html#sec-atomics-object

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`Atomics`	68	79	78	55	15.2[1]	89	79	63	15.2[2]	15.0	89	15.2[3]	1.0.0	1.0	8.10.0
Atomic operations on non-shared `ArrayBuffer` objects	**No**	**No**	79	**No**	**No**	**No**	79	**No**	**No**	**No**	**No**	**No**	1.0.0	**No**	**No**
`add`	68	79	78	55	15.2	89	79	63	15.2	15.0	89	15.2	1.0.0	1.0	8.10.0
`and`	68	79	78	55	15.2	89	79	63	15.2	15.0	89	15.2	1.0.0	1.0	8.10.0
`compareExchange`	68	79	78	55	15.2	89	79	63	15.2	15.0	89	15.2	1.0.0	1.0	8.10.0
`exchange`	68	79	78	55	15.2	89	79	63	15.2	15.0	89	15.2	1.0.0	1.0	8.10.0

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Nodejs
`isLockFree`	68	79	78	55	15.2	89	79	63	15.2	15.0	89	15.2	1.0.0	1.0	8.10.0
`load`	68	79	78	55	15.2	89	79	63	15.2	15.0	89	15.2	1.0.0	1.0	8.10.0
`notify`	68	79	78	55	15.2	89	79	63	15.2	15.0	89	15.2	1.0.0	1.0	8.10.0
`or`	68	79	78	55	15.2	89	79	63	15.2	15.0	89	15.2	1.0.0	1.0	8.10.0
`pause`	133	133	137	118	18.4	133	137	88	18.4	29.0	133	18.4	1.1.31	2.2	24.0.0
`store`	68	79	78	55	15.2	89	79	63	15.2	15.0	89	15.2	1.0.0	1.0	8.10.0
`sub`	68	79	78	55	15.2	89	79	63	15.2	15.0	89	15.2	1.0.0	1.0	8.10.0
`wait`	68	79	78	55	15.2	89	79	63	15.2	15.0	89	15.2	1.0.0	1.0	8.10.0
`waitAsync`	90[4,5]	90[4,5]	145	75	16.4	90[6,7]	145	64[8,9]	16.4	15.0	90[6,7]	16.4	1.0.0	1.4	16.0.0
`xor`	68	79	78	55	15.2	89	79	63	15.2	15.0	89	15.2	1.0.0	1.0	8.10.0

1. Before Safari 16.4, `Atomics` is gated behind COOP/COEP. For more detail, read Making your website "cross-origin isolated" using COOP and COEP.
2. Before Safari on iOS 16.4, `Atomics` is gated behind COOP/COEP. For more detail, read Making your website "cross-origin isolated" using COOP and COEP.
3. Before WebView on iOS 16.4, `Atomics` is gated behind COOP/COEP. For more detail, read Making your website "cross-origin isolated" using COOP and COEP.
4. Also from version 87–90: partial support.
5. Also from version 87–90: The `Atomics.waitAsync()` method never times out. See bug 40742782.
6. Also from version 89–90: partial support.
7. Also from version 89–90: The `Atomics.waitAsync()` method never times out. See bug 40742782.
8. Also from version 63–64: partial support.
9. Also from version 63–64: The `Atomics.waitAsync()` method never times out. See bug 40742782.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Atomics`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Agent communication and coordination guide
- `ArrayBuffer`
- JavaScript typed arrays guide
- Web Workers
- Shared Memory – a brief tutorial in the TC39 ecmascript-sharedmem proposal
- A Taste of JavaScript's New Parallel Primitives on hacks.mozilla.org (2016)

JavaScript › Reference › Standard built-in objects › Atomics

Atomics.add()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Atomics/add · MDN page last modified 2026-02-10

Baseline: Widely available — available across major browsers since December 2021.

The **`Atomics.add()`** static method adds a given value at a given position in the array, and returns the old value at that position. This atomic operation guarantees that no other write happens until the modified value is written back.

Syntax

```js
Atomics.add(typedArray, index, value)
```

Parameters

`typedArray`

An integer typed array. One of `Int8Array`, `Uint8Array`, `Int16Array`, `Uint16Array`, `Int32Array`, `Uint32Array`, `BigInt64Array`, or `BigUint64Array`.

`index`

The position in the `typedArray` to add a `value` to.

`value`

The number to add.

Return value

The old value at the given position (`typedArray[index]`).

Exceptions

`TypeError`

Thrown if `typedArray` is not one of the allowed integer types.

`RangeError`

Thrown if `index` is out of bounds in the `typedArray`.

Examples

Note that these examples cannot be run directly from the console or an arbitrary web page, because `SharedArrayBuffer` is not defined unless its security requirements are met.

Using Atomics.add()

```js
// Create a SharedArrayBuffer with a size in bytes
const sab = new SharedArrayBuffer(1024);
// Create a view and set the value of the 0 index
const ta = new Uint8Array(sab);
ta[0] = 7;

// 7 + 12 = 19
console.log(Atomics.add(ta, 0, 12)); // 7, the old value
console.log(Atomics.load(ta, 0)); // 19, the new/current value
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/structured-data.html#sec-atomics.add

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
add	68	79	78	55	15.2	89	79	63	15.2	15.0	89	15.2	1.0.0	1.0	8.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Atomics.add`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Atomics`
- `Atomics.sub()`

JavaScript › Reference › Standard built-in objects › Atomics

Atomics.and()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Atomics/and · MDN page last modified 2026-02-10

Baseline: Widely available — available across major browsers since December 2021.

The **`Atomics.and()`** static method computes a bitwise AND with a given value at a given position in the array, and returns the old value at that position. This atomic operation guarantees that no other write happens until the modified value is written back.

Syntax

```js
Atomics.and(typedArray, index, value)
```

Parameters

`typedArray`

An integer typed array. One of `Int8Array`, `Uint8Array`, `Int16Array`, `Uint16Array`, `Int32Array`, `Uint32Array`, `BigInt64Array`, or `BigUint64Array`.

`index`

The position in the `typedArray` to compute the bitwise AND.

`value`

The number to compute the bitwise AND with.

Return value

The old value at the given position (`typedArray[index]`).

Exceptions

`TypeError`

Thrown if `typedArray` is not one of the allowed integer types.

`RangeError`

Thrown if `index` is out of bounds in the `typedArray`.

Description

The bitwise AND operation only yields 1, if both `a` and `b` are 1. The truth table for the AND operation is:

a	b	a & b
0	0	0
0	1	0
1	0	0
1	1	1

For example, a bitwise AND of `5 & 1` results in `0001` which is 1 in decimal.

```
5  0101
1  0001
   ----
1  0001
```

Examples

Note that these examples cannot be run directly from the console or an arbitrary web page, because `SharedArrayBuffer` is not defined unless its security requirements are met.

Using Atomics.and()

js

```
// Create a SharedArrayBuffer with a size in bytes
const sab = new SharedArrayBuffer(1024);
// Create a view and set the value of the 0 index
const ta = new Uint8Array(sab);
ta[0] = 7;

// 7 (0111) AND 10 (1010) = 2 (0010)
console.log(Atomics.and(ta, 0, 10)); // 7, the old value
console.log(Atomics.load(ta, 0)); // 2, the new/current value
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/structured-data.html#sec-atomics.and

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
and	68	79	78	55	15.2	89	79	63	15.2	15.0	89	15.2	1.0.0	1.0	8.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Atomics.and`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Atomics`
- `Atomics.or()`
- `Atomics.xor()`

JavaScript › Reference › Standard built-in objects › Atomics

Atomics.compareExchange()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Atomics/compareExchange · MDN page last modified 2026-02-10

Baseline: Widely available — available across major browsers since December 2021.

The **`Atomics.compareExchange()`** static method exchanges a given replacement value at a given position in the array, if a given expected value equals the old value. It returns the old value at that position whether it was equal to the expected value or not. This atomic operation guarantees that no other write happens until the modified value is written back.

Syntax

```js
Atomics.compareExchange(typedArray, index, expectedValue, replacementValue)
```

Parameters

`typedArray`

An integer typed array. One of `Int8Array`, `Uint8Array`, `Int16Array`, `Uint16Array`, `Int32Array`, `Uint32Array`, `BigInt64Array`, or `BigUint64Array`.

`index`

The position in the `typedArray` to exchange a `replacementValue`.

`expectedValue`

The value to check for equality.

`replacementValue`

The number to exchange.

Return value

The old value at the given position (`typedArray[index]`). If the return value is equal to `expectedValue`, the exchange was successful; otherwise, the exchange failed.

Exceptions

`TypeError`

Thrown if `typedArray` is not one of the allowed integer types.

`RangeError`

Thrown if `index` is out of bounds in the `typedArray`.

Examples

Note that these examples cannot be run directly from the console or an arbitrary web page, because `SharedArrayBuffer` is not defined unless its security requirements are met.

Using Atomics.compareExchange()

```js
// Create a SharedArrayBuffer with a size in bytes
const sab = new SharedArrayBuffer(1024);
// Create a view and set the value of the 0 index
const ta = new Uint8Array(sab);
ta[0] = 7;

console.log(Atomics.compareExchange(ta, 0, 7, 12)); // 7, the old value
console.log(Atomics.load(ta, 0)); // 12, the new/current value
```

Checking the return value

Compare-and-swap guarantees that the new value is calculated based on up-to-date information; if the value had been updated by another thread in the meantime, the write would fail. Therefore, you should check the return value of `compareExchange()` to check if it has failed, and retry if necessary.

Here is one example of an atomic adder (same functionality as `Atomics.add()`), adapted from the linked Wikipedia article:

```js
function add(mem, index, a) {
  let done = false;
  while (!done) {
    const value = Atomics.load(mem, index);
    done = Atomics.compareExchange(mem, index, value, value + a) === value;
  }
  return value + a;
}
```

It first reads the value at the given index, then tries to update it with the new value. It keeps retrying until it successfully updates the value.

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/structured-data.html#sec-atomics.compareexchange`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`compareExchange`	68	79	78	55	15.2	89	79	63	15.2	15.0	89	15.2	1.0.0	1.0	8.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Atomics.compareExchange`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Atomics`
- `Atomics.exchange()`

JavaScript › Reference › Standard built-in objects › Atomics

Atomics.exchange()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Atomics/exchange · MDN page last modified 2026-02-10

Baseline: Widely available — available across major browsers since December 2021.

The **`Atomics.exchange()`** static method exchanges a given value at a given position in the array, and returns the old value at that position. This atomic operation guarantees that no other write happens between the read of the old value and the write of the new value.

Syntax

```js
Atomics.exchange(typedArray, index, value)
```

Parameters

`typedArray`

An integer typed array. One of `Int8Array`, `Uint8Array`, `Int16Array`, `Uint16Array`, `Int32Array`, `Uint32Array`, `BigInt64Array`, or `BigUint64Array`.

`index`

The position in the `typedArray` to exchange a `value`.

`value`

The number to exchange.

Return value

The old value at the given position (`typedArray[index]`).

Exceptions

`TypeError`

Thrown if `typedArray` is not one of the allowed integer types.

`RangeError`

Thrown if `index` is out of bounds in the `typedArray`.

Examples

Note that these examples cannot be run directly from the console or an arbitrary web page, because `SharedArrayBuffer` is not defined unless its security requirements are met.

Using Atomics.exchange()

```js
// Create a SharedArrayBuffer with a size in bytes
const sab = new SharedArrayBuffer(1024);
// Create a view and set the value of the 0 index
const ta = new Uint8Array(sab);
ta[0] = 7;

console.log(Atomics.exchange(ta, 0, 12)); // 7, the old value
console.log(Atomics.load(ta, 0)); // 12, the new/current value
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/structured-data.html#sec-atomics.exchange`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
exchange	68	79	78	55	15.2	89	79	63	15.2	15.0	89	15.2	1.0.0	1.0	8.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Atomics.exchange`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Atomics`
- `Atomics.compareExchange()`
- `Atomics.store()`

JavaScript › Reference › Standard built-in objects › Atomics

Atomics.isLockFree()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Atomics/isLockFree · MDN page last modified 2026-02-10

Baseline: Widely available — available across major browsers since December 2021.

The **`Atomics.isLockFree()`** static method is used to determine whether the `Atomics` methods use locks or atomic hardware operations when applied to typed arrays with the given element byte size. It is intended as an optimization primitive, so that high-performance algorithms can determine whether to use locks or atomic operations in critical sections. If an atomic primitive is not lock-free, it is often more efficient for an algorithm to provide its own locking.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Atomics.isLockFree()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
console.log(Atomics.isLockFree(3));
// 3 is not one of the BYTES_PER_ELEMENT values
// Expected output: false

console.log(Atomics.isLockFree(4));
// 4 is one of the BYTES_PER_ELEMENT values
// Expected output: true
```

Syntax

js

```
Atomics.isLockFree(size)
```

Parameters

`size`

The size in bytes to check.

Return value

A `true` or `false` value indicating whether the operation is lock free.

- Always `true` if `size` is 4, because all known platforms support 4-byte atomic operations.
- Always `false` if the given size is not one of the `BYTES_PER_ELEMENT` property of integer TypedArray types.

Examples

Using Atomics.isLockFree()

```js
Atomics.isLockFree(1); // true (platform-dependent)
Atomics.isLockFree(2); // true (platform-dependent)
Atomics.isLockFree(3); // false
Atomics.isLockFree(4); // true
Atomics.isLockFree(5); // false
Atomics.isLockFree(6); // false
Atomics.isLockFree(7); // false
Atomics.isLockFree(8); // true (platform-dependent)
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/structured-data.html#sec-atomics.islockfree

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
isLockFree	68	79	78	55	15.2	89	79	63	15.2	15.0	89	15.2	1.0.0	1.0	8.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Atomics.isLockFree`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Atomics`

JavaScript › Reference › Standard built-in objects › Atomics

Atomics.load()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Atomics/load · MDN page last modified 2026-02-10

Baseline: Widely available — available across major browsers since December 2021.

The **`Atomics.load()`** static method returns a value at a given position in the array. This atomic operation guarantees that the read is tear-free, and that all atomic reads are sequentially consistent.

Syntax

```js
Atomics.load(typedArray, index)
```

Parameters

`typedArray`

An integer typed array. One of `Int8Array`, `Uint8Array`, `Int16Array`, `Uint16Array`, `Int32Array`, `Uint32Array`, `BigInt64Array`, or `BigUint64Array`.

`index`

The position in the `typedArray` to load from.

Return value

The value at the given position (`typedArray[index]`).

Exceptions

`TypeError`

Thrown if `typedArray` is not one of the allowed integer types.

`RangeError`

Thrown if `index` is out of bounds in the `typedArray`.

Examples

Note that these examples cannot be run directly from the console or an arbitrary web page, because `SharedArrayBuffer` is not defined unless its security requirements are met.

Using Atomics.load()

```js
// Create a SharedArrayBuffer with a size in bytes
const sab = new SharedArrayBuffer(1024);
// Create a view and set the value of the 0 index
const ta = new Uint8Array(sab);
ta[0] = 7;

Atomics.add(ta, 0, 12); // Add 12 to index 0
console.log(Atomics.load(ta, 0)); // 19, the new/current value
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/structured-data.html#sec-atomics.load`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`load`	68	79	78	55	15.2	89	79	63	15.2	15.0	89	15.2	1.0.0	1.0	8.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Atomics.load`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Atomics
- Atomics.store()

JavaScript › Reference › Standard built-in objects › Atomics

Atomics.notify()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Atomics/notify · MDN page last modified 2026-02-10

Baseline: Widely available — available across major browsers since December 2021.

The **`Atomics.notify()`** static method notifies up some agents that are sleeping in the wait queue.

> **Note:** This operation only works with an `Int32Array` or `BigInt64Array` that views a `SharedArrayBuffer`. It will return `0` on non-shared `ArrayBuffer` objects.

Syntax

```js
Atomics.notify(typedArray, index, count)
```

Parameters

`typedArray`

An `Int32Array` or `BigInt64Array` that views a `SharedArrayBuffer`.

`index`

The position in the `typedArray` to wake up on.

`count` Optional

The number of sleeping agents to notify. Defaults to `Infinity`.

Return value

Returns the number of woken up agents, or `0` if `typedArray` is a view on a non-shared `ArrayBuffer`.

Exceptions

`TypeError`

Thrown if `typedArray` is not an `Int32Array` or `BigInt64Array`.

`RangeError`

Thrown if `index` is out of bounds in the `typedArray`.

Examples

Note that these examples cannot be run directly from the console or an arbitrary web page, because `SharedArrayBuffer` is not defined unless its security requirements are met.

Using Atomics.notify()

Given a shared `Int32Array`:

```js
const sab = new SharedArrayBuffer(1024);
const int32 = new Int32Array(sab);
```

A reading thread is sleeping and waiting on location 0 because the provided `value` matches what is stored at the provided `index`. The reading thread will not move on until the writing thread has called `Atomics.notify()` on position 0 of the provided `typedArray`. Note that if, after being woken up, the value of location 0 has not been changed by the writing thread, the reading thread will **not** go back to sleep, but will continue on.

```js
Atomics.wait(int32, 0, 0);
console.log(int32[0]); // 123
```

A writing thread stores a new value and notifies the waiting thread once it has written:

```js
console.log(int32[0]); // 0;
Atomics.store(int32, 0, 123);
Atomics.notify(int32, 0, 1);
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/structured-data.html#sec-atomics.notify`

Browser compatibility

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`notify`	68	79	78	55	15.2	89	79	63	15.2	15.0	89	15.2	1.0.0	1.0	8.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Atomics.notify`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Atomics`
- `Atomics.wait()`
- `Atomics.waitAsync()`

JavaScript › Reference › Standard built-in objects › Atomics

Atomics.or()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Atomics/or · MDN page last modified 2026-09-05

Baseline: Widely available — available across major browsers since December 2021.

The **`Atomics.or()`** static method computes a bitwise OR with a given value at a given position in the array, and returns the old value at that position. This atomic operation guarantees that no other write happens until the modified value is written back.

Syntax

```js
Atomics.or(typedArray, index, value)
```

Parameters

`typedArray`

An integer typed array. One of `Int8Array`, `Uint8Array`, `Int16Array`, `Uint16Array`, `Int32Array`, `Uint32Array`, `BigInt64Array`, or `BigUint64Array`.

`index`

The position in the `typedArray` to compute the bitwise OR.

`value`

The number to compute the bitwise OR with.

Return value

The old value at the given position (`typedArray[index]`).

Exceptions

`TypeError`

Thrown if `typedArray` is not one of the allowed integer types.

`RangeError`

Thrown if `index` is out of bounds in the `typedArray`.

Description

The bitwise OR operation yields 1, if either `a` or `b` is 1. The truth table for the OR operation is:

a	b	a | b
0	0	0
0	1	1
1	0	1
1	1	1

For example, a bitwise OR of `5 | 1` results in `0101` which is 5 in decimal.

```
5  0101
1  0001
   ----
```

(code continues on the next page)

(code continued from the previous page)

```
5  0101
```

Examples

Note that these examples cannot be run directly from the console or an arbitrary web page, because `SharedArrayBuffer` is not defined unless its security requirements are met.

Using Atomics.or()

js

```
// Create a SharedArrayBuffer with a size in bytes
const sab = new SharedArrayBuffer(1024);
// Create a view and set the value of the 0 index
const ta = new Uint8Array(sab);
ta[0] = 7;

// 7 (0111) OR 10 (1010) = 15 (1111)
console.log(Atomics.or(ta, 0, 10)); // 7, the old value
console.log(Atomics.load(ta, 0)); // 15, the new/current value
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/structured-data.html#sec-atomics.or`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
or	68	79	78	55	15.2	89	79	63	15.2	15.0	89	15.2	1.0.0	1.0	8.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Atomics.or`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Atomics`
- `Atomics.and()`
- `Atomics.xor()`

JavaScript › Reference › Standard built-in objects › Atomics

Atomics.pause()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Atomics/pause · MDN page last modified 2026-10-28

Baseline 2025: Newly available — works across the latest browser versions since April 2025; may not work in older ones.

The **`Atomics.pause()`** static method provides a micro-wait primitive that hints to the CPU that the caller is spinning while waiting on access to a shared resource. This allows the system to reduce the resources allocated to the core (such as power) or thread, without yielding the current thread.

`pause()` has no observable behavior other than timing. The exact behavior is dependent on the CPU architecture and the operating system. For example, in Intel x86, it may be a `pause` instruction as per Intel's optimization manual. It could be a no-op in certain platforms.

Syntax

```js
Atomics.pause()
```

Parameters

None.

Return value

None (`undefined`).

Examples

Note that these examples cannot be run directly from the console or an arbitrary web page, because `SharedArrayBuffer` is not defined unless its security requirements are met.

Using Atomics.pause()

Calling `Atomics.wait()` or `Atomics.waitAsync()` in order to wait for access to shared memory causes the thread to be scheduled out of the core and then back in again after the wait. This is efficient during times of high contention, where access to the shared memory could take some time. When contention is low, then it is often more efficient to poll on the lock without yielding the thread: this approach is known as busy waiting or spinlocking. The `pause()` method allows you to spinlock more efficiently while waiting, by providing a hint to the CPU about what the thread is doing, and hence its low need for resources.

To cater for both conditions, a common approach is to first spinlock in the hope that contention is low, and then wait if the lock is not gained after a short time. If we acquired the lock via spinlocking already, then the `wait()` call is skipped.

The example below shows how this approach can be used with `Atomics.pause()` and `Atomics.wait()`.

> **Warning:** Using spinlocking on the main thread is not recommended, as it will freeze the entire page. In general, unless designed very carefully, spinlocks may not actually be more performant than a regular wait.

```js
// Imagine another thread also has access to this shared memory
const sab = new SharedArrayBuffer(1024);
const i32 = new Int32Array(sab);

function lock() {
  while (true) {
    // Fast path: spin the CPU for a short while
    let spin = 0;
    do {
      if (Atomics.compareExchange(i32, 0, 0, 1) === 0) {
        return;
      }
      Atomics.pause();
      spin++;
    } while (spin < 10);

    // Slow path: wait for the lock
    // This can only be called in a worker thread,
    // because the main thread cannot be blocked
    Atomics.wait(i32, 0, 1);
  }
}
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/structured-data.html#sec-atomics.pause`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
pause	133	133	137	118	18.4	133	137	88	18.4	29.0	133	18.4	1.1.31	2.2	24.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Atomics.pause`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Atomics`
- `Atomics.wait()`
- `Atomics.waitAsync()`

JavaScript › Reference › Standard built-in objects › Atomics

Atomics.store()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Atomics/store · MDN page last modified 2026-02-10

Baseline: Widely available — available across major browsers since December 2021.

The **`Atomics.store()`** static method stores a given value at a given position in the array and returns that value. This atomic operation guarantees that the write is tear-free, and that all atomic writes are sequentially consistent.

Note that unlike many other `Atomic` methods, `store()` returns the new value instead of the old value. To get the old value, use `Atomics.exchange()` instead.

Syntax

```
Atomics.store(typedArray, index, value)
```

Parameters

`typedArray`

An integer typed array. One of `Int8Array`, `Uint8Array`, `Int16Array`, `Uint16Array`, `Int32Array`, `Uint32Array`, `BigInt64Array`, or `BigUint64Array`.

`index`

The position in the `typedArray` to store a `value` in.

`value`

The number to store.

Return value

The value that has been stored.

Exceptions

`TypeError`

Thrown if `typedArray` is not one of the allowed integer types.

`RangeError`

Thrown if `index` is out of bounds in the `typedArray`.

Examples

Note that these examples cannot be run directly from the console or an arbitrary web page, because `SharedArrayBuffer` is not defined unless its security requirements are met.

Using Atomics.store()

```
// Create a SharedArrayBuffer with a size in bytes
const sab = new SharedArrayBuffer(1024);
const ta = new Uint8Array(sab);

console.log(Atomics.store(ta, 0, 12)); // 12, the new/current value
console.log(Atomics.load(ta, 0)); // 12, the new/current value
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/structured-data.html#sec-atomics.store`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`store`	68	79	78	55	15.2	89	79	63	15.2	15.0	89	15.2	1.0.0	1.0	8.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Atomics.store`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Atomics`
- `Atomics.load()`
- `Atomics.exchange()`

Atomics.sub()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Atomics/sub · MDN page last modified 2026-02-10

Baseline: Widely available — available across major browsers since December 2021.

The **`Atomics.sub()`** static method subtracts a given value at a given position in the array, and returns the old value at that position. This atomic operation guarantees that no other write happens until the modified value is written back.

Syntax

```js
Atomics.sub(typedArray, index, value)
```

Parameters

`typedArray`

An integer typed array. One of `Int8Array`, `Uint8Array`, `Int16Array`, `Uint16Array`, `Int32Array`, `Uint32Array`, `BigInt64Array`, or `BigUint64Array`.

`index`

The position in the `typedArray` to subtract a `value` from.

`value`

The number to subtract.

Return value

The old value at the given position (`typedArray[index]`).

Exceptions

`TypeError`

Thrown if `typedArray` is not one of the allowed integer types.

`RangeError`

Thrown if `index` is out of bounds in the `typedArray`.

Examples

Note that these examples cannot be run directly from the console or an arbitrary web page, because `SharedArrayBuffer` is not defined unless its security requirements are met.

Using Atomics.sub()

```js
// Create a SharedArrayBuffer with a size in bytes
const sab = new SharedArrayBuffer(1024);
// Create a view and set the value of the 0 index
const ta = new Uint8Array(sab);
ta[0] = 48;

// 48 - 12 = 36
console.log(Atomics.sub(ta, 0, 12)); // 48, the old value
console.log(Atomics.load(ta, 0)); // 36, the new/current value
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/structured-data.html#sec-atomics.sub`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
sub	68	79	78	55	15.2	89	79	63	15.2	15.0	89	15.2	1.0.0	1.0	8.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Atomics.sub`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Atomics`
- `Atomics.add()`

JavaScript › Reference › Standard built-in objects › Atomics

Atomics.wait()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Atomics/wait · MDN page last modified 2026-08-31

Baseline: Widely available — available across major browsers since December 2021.

The **`Atomics.wait()`** static method verifies that a shared memory location contains a given value and if so sleeps, awaiting a wake-up notification or a timeout. It returns a string which is `"not-equal"` if the memory location does not match the given value, `"ok"` if woken by `Atomics.notify()`, or `"timed-out"` if the timeout expires.

`Atomics.wait()` and `Atomics.notify()` are used together to enable thread synchronization based on a value in shared memory. A thread can proceed immediately if the synchronization value has changed, or it can wait for notification from another thread when it reaches the synchronization point.

This method only works with an `Int32Array` or `BigInt64Array` that views a `SharedArrayBuffer`. It is blocking and cannot be used in the main thread. For a non-blocking, asynchronous version of this method, see `Atomics.waitAsync()`.

Syntax

```js
Atomics.wait(typedArray, index, value)
Atomics.wait(typedArray, index, value, timeout)
```

Parameters

`typedArray`

An `Int32Array` or `BigInt64Array` that views a `SharedArrayBuffer`.

`index`

The position in the `typedArray` to wait on.

`value`

The expected value to test.

`timeout` Optional

Time to wait in milliseconds. `NaN` (and values that get converted to `NaN`, such as `undefined`) becomes `Infinity`. Negative values become `0`.

Return value

A string which is either `"not-equal"`, `"ok"`, or `"timed-out"`.

- `"not-equal"` is returned immediately if the initial `value` does not equal what is stored at `index`.
- `"ok"` is returned if woken up by a call to `Atomics.notify()`, **regardless of whether the expected value has changed**.
- `"timed-out"` is returned if a sleeping wait exceeds the specified `timeout` without being woken up by `Atomics.notify()`.

Exceptions

`TypeError`

Thrown in one of the following cases:

- If `typedArray` is not an `Int32Array` or `BigInt64Array` that views a `SharedArrayBuffer`.
- If the current thread cannot be blocked (for example, because it's the main thread).

`RangeError`

Thrown if `index` is out of bounds in the `typedArray`.

Examples

Note that these examples cannot be run directly from the console or an arbitrary web page, because `SharedArrayBuffer` is not defined unless its security requirements are met.

Using Atomics.wait()

Given a shared `Int32Array`:

```js
// Create a SharedArrayBuffer with a size in bytes
const sab = new SharedArrayBuffer(1024);
const int32 = new Int32Array(sab);
```

A reading thread is sleeping and waiting on location 0 because the provided `value` matches what is stored at the provided `index`. The reading thread will not move on until the writing thread has called `Atomics.notify()` on position 0 of the provided `typedArray`. Note that if, after being woken up, the value of location 0 has not been changed by the writing thread, the reading thread will **not** go back to sleep, but will continue on.

```js
Atomics.wait(int32, 0, 0);
console.log(int32[0]); // 123
```

A writing thread stores a new value and notifies the waiting thread once it has written:

```js
console.log(int32[0]); // 0;
Atomics.store(int32, 0, 123);
Atomics.notify(int32, 0, 1);
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/structured-data.html#sec-atomics.wait`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`wait`	68	79	78	55	15.2	89	79	63	15.2	15.0	89	15.2	1.0.0	1.0	8.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Atomics.wait`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Atomics`
- `Atomics.waitAsync()`
- `Atomics.notify()`

JavaScript › Reference › Standard built-in objects › Atomics

Atomics.waitAsync()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Atomics/waitAsync · MDN page last modified 2026-02-10

Baseline 2025: Newly available — works across the latest browser versions since November 2025; may not work in older ones.

The **`Atomics.waitAsync()`** static method verifies that a shared memory location contains a given value, immediately returning an object with the `value` property containing the string `"not-equal"` if the memory location does not match the given value, or `"timed-out"` if the timeout was set to zero. Otherwise the method returns an object where the `value` property is a `Promise` that fulfills with either `"ok"` when `Atomics.notify()` is called, or `"timed-out"` if the timeout expires.

`Atomics.waitAsync()` and `Atomics.notify()` are used together to enable thread synchronization based on a value in shared memory. A thread can proceed immediately if the synchronization value has changed, or it can wait for notification from another thread when it reaches the synchronization point.

This method only works with an `Int32Array` or `BigInt64Array` that views a `SharedArrayBuffer`. It is non-blocking and, unlike `Atomics.wait()`, can be used on the main thread. Because it does not block the whole thread, you still need to be careful not to access the shared memory before the promise settles.

Syntax

```js
Atomics.waitAsync(typedArray, index, value)
Atomics.waitAsync(typedArray, index, value, timeout)
```

Parameters

`typedArray`

An `Int32Array` or `BigInt64Array` that views a `SharedArrayBuffer`.

`index`

The position in the `typedArray` to wait on.

`value`

The expected value to test.

`timeout` Optional

Time to wait in milliseconds. `NaN` (and values that get converted to `NaN`, such as `undefined`) becomes `Infinity`. Negative values become `0`.

Return value

An `Object` with the following properties:

async

A boolean indicating whether the `value` property is a `Promise` or not.

value

If `async` is `false`, it will be a string which is either `"not-equal"` or `"timed-out"` (only when the `timeout` parameter is `0`). If `async` is `true`, it will be a `Promise` which is fulfilled with a string value, either `"ok"` or `"timed-out"`. The promise is never rejected.

Exceptions

`TypeError`

Thrown if `typedArray` is not an `Int32Array` or `BigInt64Array` that views a `SharedArrayBuffer`.

`RangeError`

Thrown if `index` is out of bounds in the `typedArray`.

Examples

Note that these examples cannot be run directly from the console or an arbitrary web page, because `SharedArrayBuffer` is not defined unless its security requirements are met.

Using Atomics.waitAsync()

Given a shared `Int32Array`:

```js
// Create a SharedArrayBuffer with a size in bytes
const sab = new SharedArrayBuffer(1024);
const int32 = new Int32Array(sab);
```

A reading thread is sleeping and waiting on location 0 which is expected to be 0. The `result.value` will be a promise.

```js
const result = Atomics.waitAsync(int32, 0, 0, 1000);
// { async: true, value: Promise {<pending>} }
```

In the reading thread or in another thread, the memory location 0 is called and the promise can be resolved with `"ok"`.

```js
Atomics.notify(int32, 0);
// { async: true, value: Promise {<fulfilled>: 'ok'} }
```

If it isn't resolving to `"ok"`, the value in the shared memory location wasn't the expected (the `value` would be `"not-equal"` instead of a promise) or the timeout was reached (the promise will resolve to `"time-out"`).

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/structured-data.html#sec-atomics.waitasync

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`waitAsync`	90[1,2]	90[1,2]	145	75	16.4	90[3,4]	145	64[5,6]	16.4	15.0	90[3,4]	16.4	1.0.0	1.4	16.0.0

1. Also from version 87–90: partial support.
2. Also from version 87–90: The `Atomics.waitAsync()` method never times out. See bug 40742782.
3. Also from version 89–90: partial support.
4. Also from version 89–90: The `Atomics.waitAsync()` method never times out. See bug 40742782.
5. Also from version 63–64: partial support.
6. Also from version 63–64: The `Atomics.waitAsync()` method never times out. See bug 40742782.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Atomics.waitAsync`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Atomics`
- `Atomics.wait()`
- `Atomics.notify()`

JavaScript › Reference › Standard built-in objects › Atomics

Atomics.xor()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Atomics/xor · MDN page last modified 2026-02-10

Baseline: Widely available — available across major browsers since December 2021.

The **`Atomics.xor()`** static method computes a bitwise XOR with a given value at a given position in the array, and returns the old value at that position. This atomic operation guarantees that no other write happens until the modified value is written back.

Syntax

```js
Atomics.xor(typedArray, index, value)
```

Parameters

`typedArray`

An integer typed array. One of `Int8Array`, `Uint8Array`, `Int16Array`, `Uint16Array`, `Int32Array`, `Uint32Array`, `BigInt64Array`, or `BigUint64Array`.

`index`

The position in the `typedArray` to compute the bitwise XOR.

`value`

The number to compute the bitwise XOR with.

Return value

The old value at the given position (`typedArray[index]`).

Exceptions

`TypeError`

Thrown if `typedArray` is not one of the allowed integer types.

`RangeError`

Thrown if `index` is out of bounds in the `typedArray`.

Description

The bitwise XOR operation yields 1, if `a` and `b` are different. The truth table for the XOR operation is:

a	b	a ^ b
0	0	0
0	1	1
1	0	1
1	1	0

For example, a bitwise XOR of `5 ^ 1` results in `0100` which is 4 in decimal.

```
5  0101
1  0001
   ----
4  0100
```

Examples

Note that these examples cannot be run directly from the console or an arbitrary web page, because `SharedArrayBuffer` is not defined unless its security requirements are met.

Using Atomics.xor()

js

```
// Create a SharedArrayBuffer with a size in bytes
const sab = new SharedArrayBuffer(1024);
// Create a view and set the value of the 0 index
const ta = new Uint8Array(sab);
ta[0] = 7;

// 7 (0111) XOR 2 (0010) = 5 (0101)
console.log(Atomics.xor(ta, 0, 2)); // 7, the old value
console.log(Atomics.load(ta, 0)); // 5, the new/current value
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/structured-data.html#sec-atomics.xor

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`xor`	68	79	78	55	15.2	89	79	63	15.2	15.0	89	15.2	1.0.0	1.0	8.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Atomics.xor`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Atomics`
- `Atomics.and()`
- `Atomics.or()`

JavaScript › Reference › Standard built-in objects

BigInt

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/BigInt · MDN page last modified 2026-09-02

Baseline: Widely available — available across major browsers since September 2020.

`BigInt` values represent integer values which are too high or too low to be represented by the `number` primitive.

Description

A **BigInt value**, also sometimes just called a **BigInt**, is a `bigint` primitive, created by appending `n` to the end of an integer literal, or by calling the `BigInt()` function (without the `new` operator) and giving it an integer value or string value.

```js
const previouslyMaxSafeInteger = 9007199254740991n;

const alsoHuge = BigInt(9007199254740991);
// 9007199254740991n

const hugeString = BigInt("9007199254740991");
// 9007199254740991n

const hugeHex = BigInt("0x1fffffffffffff");
// 9007199254740991n

const hugeOctal = BigInt("0o37777777777777777");
// 9007199254740991n

const hugeBin = BigInt(
  "0b11111111111111111111111111111111111111111111111111111",
);
// 9007199254740991n
```

BigInt values are similar to Number values in some ways, but also differ in a few key matters: A BigInt value cannot be used with methods in the built-in `Math` object and cannot be mixed with a Number value in operations; they must be coerced to the same type. Be careful coercing values back and forth, however, as the precision of a BigInt value may be lost when it is coerced to a Number value.

Type information

When tested against `typeof`, a BigInt value (`bigint` primitive) will give `"bigint"`:

```js
typeof 1n === "bigint"; // true
typeof BigInt("1") === "bigint"; // true
```

A BigInt value can also be wrapped in an `Object`:

```js
typeof Object(1n) === "object"; // true
```

Operators

Most operators support BigInts, however most do not permit operands to be of mixed types — both operands must be BigInt or neither:

- Arithmetic operators: +, -, *, /, %, **
- Bitwise operators: >>, <<, &, |, ^, ~
- Unary negation (-)
- Increment/decrement: ++, --

The boolean-returning operators allow mixing numbers and BigInts as operands:

- Relational operators and equality operators: >, <, >=, <=, ==, !=, ===, !==
- Logical operators only rely on the truthiness of operands

A couple of operators do not support BigInt at all:

- Unary plus (+) cannot be supported due to conflicting usage in asm.js, so it has been left out in order to not break asm.js.
- Unsigned right shift (>>>) is the only bitwise operator that's unsupported, as every BigInt value is signed.

Special cases:

- Addition (+) involving a string and a BigInt returns a string.
- Division (/) truncates fractional components towards zero, since BigInt is unable to represent fractional quantities.

```js
const previousMaxSafe = BigInt(Number.MAX_SAFE_INTEGER); // 9007199254740991n
const maxPlusOne = previousMaxSafe + 1n; // 9007199254740992n
const theFuture = previousMaxSafe + 2n; // 9007199254740993n, this works now!
const prod = previousMaxSafe * 2n; // 18014398509481982n
const diff = prod - 10n; // 18014398509481972n
const mod = prod % 10n; // 2n
const bigN = 2n ** 54n; // 18014398509481984n
bigN * -1n; // -18014398509481984n
const expected = 4n / 2n; // 2n
const truncated = 5n / 2n; // 2n, not 2.5n
```

Comparisons

A BigInt value is not strictly equal to a Number value, but it *is* loosely so:

```js
0n === 0; // false
0n == 0; // true
```

A Number value and a BigInt value may be compared as usual:

```js
1n < 2; // true
2n > 1; // true
2 > 2; // false
2n > 2; // false
2n >= 2; // true
```

BigInt values and Number values may be mixed in arrays and sorted:

```js
const mixed = [4n, 6, -12n, 10, 4, 0, 0n];
// [4n, 6, -12n, 10, 4, 0, 0n]

mixed.sort(); // default sorting behavior
// [ -12n, 0, 0n, 10, 4n, 4, 6 ]

mixed.sort((a, b) => a - b);
// won't work since subtraction will not work with mixed types
// TypeError: can't convert BigInt value to Number value

// sort with an appropriate numeric comparator
mixed.sort((a, b) => (a < b ? -1 : a > b ? 1 : 0));
// [ -12n, 0, 0n, 4n, 4, 6, 10 ]
```

Note that comparisons with `Object`-wrapped BigInt values act as with other objects, only indicating equality when the same object instance is compared:

```js
Object(0n) === 0n; // false
Object(0n) === Object(0n); // false

const o = Object(0n);
o === o; // true
```

Because coercing between Number values and BigInt values can lead to loss of precision, the following are recommended:

- Only use a BigInt value when values greater than 2^{53} are reasonably expected.
- Don't coerce between BigInt values and Number values.

Conditionals

A BigInt value follows the same conversion rules as Numbers when:

- it is converted to a `Boolean`: via the `Boolean` function;
- when used with logical operators `||`, `&&`, and `!`; or
- within a conditional test like an `if` statement.

Namely, only `0n` is falsy; everything else is truthy.

```js
if (0n) {
  console.log("Hello from the if!");
} else {
  console.log("Hello from the else!");
}
// "Hello from the else!"

0n || 12n; // 12n
0n && 12n; // 0n
Boolean(0n); // false
Boolean(12n); // true
!12n; // false
!0n; // true
```

Cryptography

The operations supported on BigInt values are not constant-time and are thus open to timing attacks. JavaScript BigInts therefore could be dangerous for use in cryptography without mitigating factors. As a very generic example, an attacker could measure the time difference between `101n ** 65537n` and `17n ** 9999n`, and deduce the magnitude of secrets, such as private keys, based on the time elapsed. If you still have to use BigInts, take a look at the Timing attack FAQ for general advice regarding the issue.

Use within JSON

Using `JSON.stringify()` with any BigInt value will raise a `TypeError`, as BigInt values aren't serialized in JSON by default. However, `JSON.stringify()` specifically leaves a backdoor for BigInt values: it would try to call the BigInt's `toJSON()` method. (It doesn't do so for any other primitive values.) Therefore, you can implement your own `toJSON()` method (which is one of the few cases where patching built-in objects is not explicitly discouraged):

```js
BigInt.prototype.toJSON = function () {
  return { $bigint: this.toString() };
};
```

Instead of throwing, `JSON.stringify()` now produces a string like this:

```js
console.log(JSON.stringify({ a: 1n }));
// {"a":{"$bigint":"1"}}
```

If you do not wish to patch `BigInt.prototype`, you can use the `replacer` parameter of `JSON.stringify` to serialize BigInt values:

```js
const replacer = (key, value) =>
  typeof value === "bigint" ? { $bigint: value.toString() } : value;

const data = {
  number: 1,
  big: 18014398509481982n,
};
const stringified = JSON.stringify(data, replacer);

console.log(stringified);
// {"number":1,"big":{"$bigint":"18014398509481982"}}
```

You can then use the `reviver` parameter of `JSON.parse` to handle them:

```js
const reviver = (key, value) =>
  value !== null &&
  typeof value === "object" &&
  "$bigint" in value &&
  typeof value.$bigint === "string"
    ? BigInt(value.$bigint)
    : value;

const payload = '{"number":1,"big":{"$bigint":"18014398509481982"}}';
const parsed = JSON.parse(payload, reviver);

console.log(parsed);
// { number: 1, big: 18014398509481982n }
```

> **Note:** While it's possible to make the replacer of `JSON.stringify()` generic and properly serialize BigInt values for all objects as shown above, the reviver of `JSON.parse()` has to be used with caution, because the serialization is *irreversible*: it's not possible to distinguish between an object that happens to have a property called `$bigint` and an actual BigInt.
>
> In addition, the example above creates an entire object during replacing and reviving, which may have performance or storage implications for larger objects containing many BigInts. If you know the shape of the payload, it may be better to just serialize them as strings and revive them based on the property key's name instead.

In fact, JSON allows number literals that are arbitrarily long; they just cannot be parsed to full precision in JavaScript. If you are communicating with another program in a language that supports longer integers (such as 64-bit integers), and you want to transmit the BigInt as a JSON number instead of a JSON string, see Lossless number serialization.

BigInt coercion

Many built-in operations that expect BigInts first coerce their arguments to BigInts. The operation can be summarized as follows:

- BigInts are returned as-is.
- `undefined` and `null` throw a `TypeError`.
- `true` turns into `1n`; `false` turns into `0n`.
- Strings are converted by parsing them as if they contain an integer literal. Any parsing failure results in a `SyntaxError`. The syntax is a subset of string numeric literals, where decimal points or exponent indicators are not allowed.
- Numbers throw a `TypeError` to prevent unintended implicit coercion causing loss of precision.
- Symbols throw a `TypeError`.
- Objects are first converted to a primitive by calling their `[Symbol.toPrimitive]()` (with `"number"` as hint), `valueOf()`, and `toString()` methods, in that order. The resulting primitive is then converted to a BigInt.

The best way to achieve nearly the same effect in JavaScript is through the `BigInt()` function: `BigInt(x)` uses the same algorithm to convert `x`, except that Numbers don't throw a `TypeError`, but are converted to BigInts if they are integers.

Note that built-in operations expecting BigInts often truncate the BigInt to a fixed width after coercion. This includes `BigInt.asIntN()`, `BigInt.asUintN()`, and methods of `BigInt64Array` and

`BigUint64Array`.

Constructor

`BigInt()`

Returns primitive values of type BigInt. Throws an error when called with `new`.

Static methods

`BigInt.asIntN()`

Truncates a `BigInt` value to the given number of least significant bits and returns that value as a signed integer.

`BigInt.asUintN()`

Truncates a `BigInt` value to the given number of least significant bits and returns that value as an unsigned integer.

Instance properties

These properties are defined on `BigInt.prototype` and shared by all `BigInt` instances.

`BigInt.prototype.constructor`

The constructor function that created the instance object. For `BigInt` instances, the initial value is the `BigInt` constructor.

`BigInt.prototype[Symbol.toStringTag]`

The initial value of the `[Symbol.toStringTag]` property is the string `"BigInt"`. This property is used in `Object.prototype.toString()`. However, because `BigInt` also has its own `toString()` method, this property is not used unless you call `Object.prototype.toString.call()` with a BigInt as `thisArg`.

Instance methods

`BigInt.prototype.toLocaleString()`

Returns a string with a language-sensitive representation of this BigInt value. Overrides the `Object.prototype.toLocaleString()` method.

`BigInt.prototype.toString()`

Returns a string representing this BigInt value in the specified radix (base). Overrides the `Object.prototype.toString()` method.

`BigInt.prototype.valueOf()`

Returns this BigInt value. Overrides the `Object.prototype.valueOf()` method.

Examples

Calculating Primes

js

```
function isPrime(n) {
  if (n < 2n) {
    return false;
  }
  if (n % 2n === 0n) {
```

(code continues on the next page)

(code continued from the previous page)

```
    return n === 2n;
  }
  for (let factor = 3n; factor * factor <= n; factor += 2n) {
    if (n % factor === 0n) {
      return false;
    }
  }
  return true;
}

// Takes a BigInt value as an argument, returns nth prime number as a BigInt value
function nthPrime(nth) {
  let maybePrime = 2n;
  let prime = 0n;

  while (nth >= 0n) {
    if (isPrime(maybePrime)) {
      nth--;
      prime = maybePrime;
    }
    maybePrime++;
  }

  return prime;
}

nthPrime(20n);
// 73n
```

> **Note:** The `isPrime()` implementation is for demonstration only. For a real-world application, you would want to use a heavily memoized algorithm such as the Sieve of Eratosthenes to avoid repeated calculations.

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-bigint-objects`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`BigInt`	67	79	68	54	14	67	68	48	14	9.0	67	14	1.0.0	1.0	10.4.0
`BigInt()` constructor	67	79	68	54	14	67	68	48	14	9.0	67	14	1.0.0	1.0	10.4.0
`asIntN`	67	79	68	54	14	67	68	48	14	9.0	67	14	1.0.0	1.0	10.4.0
`asUintN`	67	79	68	54	14	67	68	48	14	9.0	67	14	1.0.0	1.0	10.4.0
`toLocaleString`	67	79	68	54	14	67	68	48	14	9.0	67	14	1.0.0	1.0	10.4.0
`locales` parameter	76	79	70	**No**	14	76	79	54	14	12.0	76	14	1.0.0	1.8[1,2]	12.9.0

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Nodejs
`options` parameter	76	79	70	**No**	14	76	79	54	14	12.0	76	14	1.0.0	1.0	12.9.0
`toString`	67	79	68	54	14	67	68	48	14	9.0	67	14	1.0.0	1.0	10.4.0
`valueOf`	67	79	68	54	14	67	68	48	14	9.0	67	14	1.0.0	1.0	10.4.0

1. Also from version 1.0–1.8: partial support.
2. Also from version 1.0–1.8: Only the locale data for `en-US` is available.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.BigInt`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Number`
- `Number.MAX_SAFE_INTEGER`

JavaScript › Reference › Standard built-in objects › BigInt

BigInt() constructor

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/BigInt/BigInt · MDN page last modified 2026-09-03

Baseline: Widely available — available across major browsers since September 2020.

The **`BigInt()`** function returns primitive values of type BigInt.

Syntax

```
BigInt(value)
```

> **Note:** `BigInt()` can only be called without `new`. Attempting to construct it with `new` throws a `TypeError`.

Parameters

`value`

The value to be converted to a BigInt value. It may be a string, an integer, a boolean, or another `BigInt`.

Return value

A `BigInt` value. Number values must be integers and are converted to BigInts. The boolean value `true` becomes `1n`, and `false` becomes `0n`. Strings are parsed as if they are source text for integer literals, which means they can have leading and trailing whitespaces and can be prefixed with `0b`, `0o`, or `0x`.

Exceptions

`RangeError`

Thrown if the parameter is a non-integral number.

`TypeError`

Thrown in one of the following cases:

- The parameter cannot be converted to a primitive.
- After conversion to a primitive, the result is `undefined`, `null`, or `Symbol`.

`SyntaxError`

Thrown if the parameter is a string that cannot be parsed as a `BigInt`.

Examples

Using BigInt() to convert a number to a BigInt

`BigInt()` is the only case where a number can be converted to a BigInt without throwing, because it's very explicit. However, only integers are allowed.

```js
BigInt(123); // 123n
BigInt(123.3); // RangeError: The number 123.3 cannot be converted to a BigInt because it is not an integer
```

Using string values

```js
BigInt("123"); // 123n
BigInt("0b10101"); // 21n, which is 10101 in binary
BigInt("0o123"); // 83n, which is 123 in octal
BigInt("0x123"); // 291n, which is 123 in hexadecimal
BigInt("  123  "); // 123n, leading and trailing whitespaces are allowed
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-bigint-constructor

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`BigInt()` constructor	67	79	68	54	14	67	68	48	14	9.0	67	14	1.0.0	1.0	10.4.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.BigInt.BigInt`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `BigInt`

JavaScript › Reference › Standard built-in objects › BigInt

BigInt.asIntN()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/BigInt/asIntN · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since September 2020.

The **`BigInt.asIntN()`** static method truncates a `BigInt` value to the given number of least significant bits and returns that value as a signed integer.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: BigInt.asIntN()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const I64_CEIL = 2n ** 63n;

console.log(BigInt.asIntN(64, I64_CEIL - 1n));
// 9223372036854775807n (2n ** 63n - 1n, the maximum non-wrapping value)
console.log(BigInt.asIntN(64, I64_CEIL));
// -9223372036854775808n (wraps to min value)
console.log(BigInt.asIntN(64, I64_CEIL + 1n));
// -9223372036854775807n (min value + 1n)
console.log(BigInt.asIntN(64, I64_CEIL * 2n));
// 0n (wrapped around to zero)
console.log(BigInt.asIntN(64, -I64_CEIL * -42n));
// 0n (also wraps on negative multiples)
```

Syntax

js

```
BigInt.asIntN(bits, bigint)
```

Parameters

`bits`

The number of bits available for the returned BigInt. Should be an integer between 0 and 2^{53} - 1, inclusive.

`bigint`

The BigInt value to truncate to fit into the supplied bits.

Return value

A BigInt containing the bit representation of `bigint` truncated to `bits` bits, interpreted as a signed integer.

Exceptions

`RangeError`

Thrown if `bits` is negative or greater than 2^{53} - 1.

Description

The `BigInt.asIntN` method truncates a `BigInt` value to the given number of bits, and interprets the result as a signed integer. For example, for `BigInt.asIntN(3, 25n)`, the value `25n` is truncated to `1n`:

```
25n = 00011001 (base 2)
          ^=== Use only the three remaining bits
===>       001 (base 2) = 1n
```

If the leading bit of the remaining number is `1`, the result is negative. For example, `BigInt.asIntN(4, 25n)` yields `-7n`, because `1001` is the encoding of `-7` under two's complement:

```
25n = 00011001 (base 2)
         ^==== Use only the four remaining bits
===>      1001 (base 2) = -7n
```

> **Note:** `BigInt` values are always encoded as two's complement in binary.

Unlike similar language APIs such as `Number.prototype.toExponential()`, `asIntN` is a static property of `BigInt`, so you always use it as `BigInt.asIntN()`, rather than as a method of a BigInt value. Exposing `asIntN()` as a "standard library function" allows interop with asm.js.

Examples

Staying in 64-bit ranges

The `BigInt.asIntN()` method can be useful to stay in the range of 64-bit arithmetic.

js

```
const max = 2n ** (64n - 1n) - 1n;

BigInt.asIntN(64, max); // 9223372036854775807n

BigInt.asIntN(64, max + 1n); // -9223372036854775808n
// negative because the 64th bit of 2^63 is 1
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-bigint.asintn

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
asIntN	67	79	68	54	14	67	68	48	14	9.0	67	14	1.0.0	1.0	10.4.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.BigInt.asIntN`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `BigInt`
- `BigInt.asUintN()`

JavaScript › Reference › Standard built-in objects › BigInt

BigInt.asUintN()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/BigInt/asUintN · MDN page last modified 2026-09-04

Baseline: Widely available — available across major browsers since September 2020.

The **`BigInt.asUintN()`** static method truncates a `BigInt` value to the given number of least significant bits and returns that value as an unsigned integer.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: BigInt.asUintN()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const U64_CEIL = 2n ** 64n;

console.log(BigInt.asUintN(64, U64_CEIL - 1n));
// 18446744073709551615n (2n ** 64n - 1n, the maximum non-wrapping value)
console.log(BigInt.asUintN(64, U64_CEIL));
// 0n (wraps to zero)
console.log(BigInt.asUintN(64, U64_CEIL + 1n));
// 1n
console.log(BigInt.asUintN(64, U64_CEIL * 2n));
// 0n (wraps on multiples)
console.log(BigInt.asUintN(64, U64_CEIL * -42n));
// 0n (also wraps on negative multiples)
```

Syntax

js

```
BigInt.asUintN(bits, bigint)
```

Parameters

`bits`

The number of bits available for the returned BigInt. Should be an integer between 0 and 2^{53} - 1, inclusive.

`bigint`

The BigInt value to truncate to fit into the supplied bits.

Return value

A BigInt containing the bit representation of `bigint` truncated to `bits` bits, interpreted as an unsigned integer.

Exceptions

`RangeError`

Thrown if `bits` is negative or greater than $2^{53} - 1$.

Description

The `BigInt.asUintN` method truncates a `BigInt` value to the given number of bits, and interprets the result as an unsigned integer. Unsigned integers have no sign bits and are always non-negative. For example, for `BigInt.asUintN(4, 25n)`, the value `25n` is truncated to `9n`:

```
25n = 00011001 (base 2)
         ^==== Use only the four remaining bits
===>      1001 (base 2) = 9n
```

> **Note:** `BigInt` values are always encoded as two's complement in binary.

Unlike similar language APIs such as `Number.prototype.toExponential()`, `asUintN` is a static property of `BigInt`, so you always use it as `BigInt.asUintN()`, rather than as a method of a BigInt value. Exposing `asUintN()` as a "standard library function" allows interop with asm.js.

Examples

Staying in 64-bit ranges

The `BigInt.asUintN()` method can be useful to stay in the range of 64-bit arithmetic.

js

```
const max = 2n ** 64n - 1n;

BigInt.asUintN(64, max); // 18446744073709551615n

BigInt.asUintN(64, max + 1n); // 0n
// zero because of overflow: the lowest 64 bits are all zeros
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-bigint.asuintn`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`asUintN`	67	79	68	54	14	67	68	48	14	9.0	67	14	1.0.0	1.0	10.4.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.BigInt.asUintN`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `BigInt`
- `BigInt.asIntN()`

JavaScript › Reference › Standard built-in objects › BigInt

BigInt.prototype.toLocaleString()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/BigInt/toLocaleString · MDN page last modified 2025-10-30

Baseline: Widely available — available across major browsers since September 2020.

The **`toLocaleString()`** method of `BigInt` values returns a string with a language-sensitive representation of this BigInt. In implementations with `Intl.NumberFormat` API support, this method delegates to `Intl.NumberFormat`.

Every time `toLocaleString` is called, it has to perform a search in a big database of localization strings, which is potentially inefficient. When the method is called many times with the same arguments, it is better to create an `Intl.NumberFormat` object and use its `format()` method, because a `NumberFormat` object remembers the arguments passed to it and may decide to cache a slice of the database, so future `format` calls can search for localization strings within a more constrained context.

Try it

▷ Interactive example on MDN: "JavaScript Demo: BigInt.prototype.toLocaleString()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const bigint = 123456789123456789n;

// German uses period for thousands
console.log(bigint.toLocaleString("de-DE"));
// Expected output: "123.456.789.123.456.789"

// Request a currency format
console.log(
  bigint.toLocaleString("de-DE", { style: "currency", currency: "EUR" }),
);
// Expected output: "123.456.789.123.456.789,00 €"
```

Syntax

js

```
toLocaleString()
toLocaleString(locales)
toLocaleString(locales, options)
```

Parameters

The `locales` and `options` parameters customize the behavior of the function and let applications specify the language whose formatting conventions should be used.

In implementations that support the `Intl.NumberFormat` API, these parameters correspond exactly to the `Intl.NumberFormat()` constructor's parameters. Implementations without `Intl.NumberFormat`

support are asked to ignore both parameters, making the locale used and the form of the string returned entirely implementation-dependent.

`locales` Optional

A string with a BCP 47 language tag, or an array of such strings. Corresponds to the `locales` parameter of the `Intl.NumberFormat()` constructor.

In implementations without `Intl.NumberFormat` support, this parameter is ignored and the host's locale is usually used.

`options` Optional

An object adjusting the output format. Corresponds to the `options` parameter of the `Intl.NumberFormat()` constructor.

In implementations without `Intl.NumberFormat` support, this parameter is ignored.

See the `Intl.NumberFormat()` constructor for details on these parameters and how to use them.

Return value

A string representing the given BigInt according to language-specific conventions.

In implementations with `Intl.NumberFormat`, this is equivalent to `new Intl.NumberFormat(locales, options).format(number)`.

> **Note:** Most of the time, the formatting returned by `toLocaleString()` is consistent. However, the output may vary between implementations, even within the same locale — output variations are by design and allowed by the specification. It may also not be what you expect. For example, the string may use non-breaking spaces or be surrounded by bidirectional control characters. You should not compare the results of `toLocaleString()` to hardcoded constants.

Examples

Using toLocaleString()

Basic use of this method without specifying a `locale` returns a formatted string in the default locale and with default options.

```js
const bigint = 3500n;

console.log(bigint.toLocaleString());
// "3,500" if in U.S. English locale
```

Checking for support for locales and options parameters

The `locales` and `options` parameters may not be supported in all implementations, because support for the internationalization API is optional, and some systems may not have the necessary data. For implementations without internationalization support, `toLocaleString()` always uses the system's locale, which may not be what you want. Because any implementation that supports the `locales` and `options` parameters must support the `Intl` API, you can check the existence of the latter for support:

```js
function toLocaleStringSupportsLocales() {
  return (
    typeof Intl === "object" &&
    !!Intl &&
    typeof Intl.NumberFormat === "function"
  );
}
```

Using locales

This example shows some of the variations in localized number formats. In order to get the format of the language used in the user interface of your application, make sure to specify that language (and possibly some fallback languages) using the `locales` argument:

```js
const bigint = 123456789123456789n;

// German uses period for thousands
console.log(bigint.toLocaleString("de-DE"));
// 123.456.789.123.456.789

// Arabic in most Arabic speaking countries uses Eastern Arabic digits
console.log(bigint.toLocaleString("ar-EG"));
// ١٢٣,٤٥٦,٧٨٩,١٢٣,٤٥٦,٧٨٩

// India uses thousands/lakh/crore separators
console.log(bigint.toLocaleString("en-IN"));
// 1,23,45,67,89,12,34,56,789

// the nu extension key requests a numbering system, e.g. Chinese decimal
console.log(bigint.toLocaleString("zh-Hans-CN-u-nu-hanidec"));
// 一二三,四五六,七八九,一二三,四五六,七八九

// when requesting a language that may not be supported, such as
// Balinese, include a fallback language, in this case Indonesian
console.log(bigint.toLocaleString(["ban", "id"]));
// 123.456.789.123.456.789
```

Using options

The results provided by `toLocaleString()` can be customized using the `options` parameter:

```js
const bigint = 123456789123456789n;

// request a currency format
console.log(
  bigint.toLocaleString("de-DE", { style: "currency", currency: "EUR" }),
);
// 123.456.789.123.456.789,00 €

// the Japanese yen doesn't use a minor unit
console.log(
  bigint.toLocaleString("ja-JP", { style: "currency", currency: "JPY" }),
);
// ￥123,456,789,123,456,789
```

(code continues on the next page)

(code continued from the previous page)

```
// limit to three significant digits
console.log(bigint.toLocaleString("en-IN", { maximumSignificantDigits: 3 }));
// 1,23,00,00,00,00,00,00,000
```

Specifications

Specification
ECMAScript® 2027 Internationalization API Specification `https://tc39.es/ecma402/#sup-bigint.prototype.tolocalestring`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`toLocaleString`	67	79	68	54	14	67	68	48	14	9.0	67	14	1.0.0	1.0	10.4.0
`locales` parameter	76	79	70	**No**	14	76	79	54	14	12.0	76	14	1.0.0	1.8[1,2]	12.9.0
`options` parameter	76	79	70	**No**	14	76	79	54	14	12.0	76	14	1.0.0	1.0	12.9.0

1. Also from version 1.0–1.8: partial support.
2. Also from version 1.0–1.8: Only the locale data for `en-US` is available.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.BigInt.toLocaleString`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Intl.NumberFormat`
- `BigInt.prototype.toString()`

JavaScript › Reference › Standard built-in objects › BigInt

BigInt.prototype.toString()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/BigInt/toString · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since September 2020.

The **`toString()`** method of `BigInt` values returns a string representing the specified `BigInt` value. The trailing "n" is not part of the string.

Try it

▷ Interactive example on MDN: "JavaScript Demo: BigInt.prototype.toString()". It runs live in the browser there; its source code follows.

js · interactive example source

```
console.log(1024n.toString());
// Expected output: "1024"

console.log(1024n.toString(2));
// Expected output: "10000000000"

console.log(1024n.toString(16));
// Expected output: "400"
```

Syntax

js

```
toString()
toString(radix)
```

Parameters

`radix` Optional

An integer in the range 2 through 36 specifying the base to use for representing the BigInt value. Defaults to 10.

Return value

A string representing the specified `BigInt` value.

Exceptions

`RangeError`

Thrown if `radix` is less than 2 or greater than 36.

Description

The `BigInt` object overrides the `toString` method of `Object`; it does not inherit `Object.prototype.toString()`. For `BigInt` values, the `toString()` method returns a string representation of the value in the specified radix.

For radixes above 10, the letters of the alphabet indicate digits greater than 9. For example, for hexadecimal numbers (base 16) `a` through `f` are used.

If the specified BigInt value is negative, the sign is preserved. This is the case even if the radix is 2; the string returned is the positive binary representation of the BigInt value preceded by a `-` sign, **not** the two's complement of the BigInt value.

The `toString()` method requires its `this` value to be a `BigInt` primitive or wrapper object. It throws a `TypeError` for other `this` values without attempting to coerce them to BigInt values.

Because `BigInt` doesn't have a `[Symbol.toPrimitive]()` method, JavaScript calls the `toString()` method automatically when a `BigInt` *object* is used in a context expecting a string, such as in a template literal. However, BigInt *primitive* values do not consult the `toString()` method to be coerced to strings

— rather, they are directly converted using the same algorithm as the initial `toString()` implementation.

```js
BigInt.prototype.toString = () => "Overridden";
console.log(`${1n}`); // "1"
console.log(`${Object(1n)}`); // "Overridden"
```

Examples

Using toString()

```js
17n.toString(); // "17"
66n.toString(2); // "1000010"
254n.toString(16); // "fe"
(-10n).toString(2); // "-1010"
(-0xffn).toString(2); // "-11111111"
```

Negative-zero BigInt

There is no negative-zero `BigInt` as there are no negative zeros in integers. `-0.0` is an IEEE floating-point concept that only appears in the JavaScript `Number` type.

```js
(-0n).toString(); // "0"
BigInt(-0).toString(); // "0"
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-bigint.prototype.tostring

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`toString`	67	79	68	54	14	67	68	48	14	9.0	67	14	1.0.0	1.0	10.4.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.BigInt.toString`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `BigInt.prototype.toLocaleString()`
- `BigInt.prototype.valueOf()`
- `Number.prototype.toString()`

JavaScript › Reference › Standard built-in objects › BigInt

BigInt.prototype.valueOf()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/BigInt/valueOf · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since September 2020.

The **`valueOf()`** method of `BigInt` values returns the wrapped primitive value of a `BigInt` object.

Try it

▷ Interactive example on MDN: "JavaScript Demo: BigInt.prototype.valueOf()". It runs live in the browser there; its source code follows.

js · interactive example source

```
console.log(typeof Object(1n));
// Expected output: "object"

console.log(typeof Object(1n).valueOf());
// Expected output: "bigint"
```

Syntax

js

```
valueOf()
```

Parameters

None.

Return value

A BigInt representing the primitive value of the specified `BigInt` object.

Examples

Using `valueOf`

js

```
typeof Object(1n); // object
typeof Object(1n).valueOf(); // bigint
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-bigint.prototype.valueof

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`valueOf`	67	79	68	54	14	67	68	48	14	9.0	67	14	1.0.0	1.0	10.4.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.BigInt.valueOf`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `BigInt.prototype.toString()`

JavaScript › Reference › Standard built-in objects

BigInt64Array

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/BigInt64Array · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since September 2021.

The **`BigInt64Array`** typed array represents an array of 64-bit signed integers in the platform byte order. If control over byte order is needed, use `DataView` instead. The contents are initialized to `0n` unless initialization data is explicitly provided. Once established, you can reference elements in the array using the object's methods, or using standard array index syntax (that is, using bracket notation).

`BigInt64Array` is a subclass of the hidden `TypedArray` class.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: BigInt64Array". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const buffer = new ArrayBuffer(24);
const bigint64 = new BigInt64Array(buffer);
bigint64[0] = 5886014448488689n;
bigint64[1] = 1881938909131133n;
bigint64[2] = 1898875537769492n;

bigint64[0] = 6118793953620967n;
console.log(bigint64);
// Expected Output: BigInt64Array [6118793953620967n, 1881938909131133n, 1898875537769492n]

console.log(bigint64[2]);
// Expected Output: 1898875537769492n

console.log("Array length:", bigint64.length);
// Expected Output: Array length: 3

console.log("Array byte length:", bigint64.byteLength);
// Expected Output: Array byte length: 24

console.log("Array byte offset:", bigint64.byteOffset);
// Expected Output: Array byte offset: 0

bigint64.set([100n, 200n], 1);
console.log(bigint64);
// Expected Output: BigInt64Array [6118793953620967n, 100n, 200n]
```

Constructor

`BigInt64Array()`

Creates a new `BigInt64Array` object.

Static properties

Also inherits static properties from its parent `TypedArray`.

`BigInt64Array.BYTES_PER_ELEMENT`

Returns a number value of the element size. `8` in the case of `BigInt64Array`.

Static methods

Inherits static methods from its parent `TypedArray`.

Instance properties

Also inherits instance properties from its parent `TypedArray`.

These properties are defined on `BigInt64Array.prototype` and shared by all `BigInt64Array` instances.

`BigInt64Array.prototype.BYTES_PER_ELEMENT`

Returns a number value of the element size. `8` in the case of a `BigInt64Array`.

`BigInt64Array.prototype.constructor`

The constructor function that created the instance object. For `BigInt64Array` instances, the initial value is the `BigInt64Array` constructor.

Instance methods

Inherits instance methods from its parent `TypedArray`.

Examples

Different ways to create a BigInt64Array

```js
// From a length
const bigint64 = new BigInt64Array(2);
bigint64[0] = 42n;
console.log(bigint64[0]); // 42n
console.log(bigint64.length); // 2
console.log(bigint64.BYTES_PER_ELEMENT); // 8

// From an array
const x = new BigInt64Array([21n, 31n]);
console.log(x[1]); // 31n

// From another TypedArray
const y = new BigInt64Array(x);
console.log(y[0]); // 21n

// From an ArrayBuffer
const buffer = new ArrayBuffer(64);
const z = new BigInt64Array(buffer, 8, 4);
console.log(z.byteOffset); // 8

// From an iterable
const iterable = (function* () {
  yield* [1n, 2n, 3n];
})();
const bigint64FromIterable = new BigInt64Array(iterable);
console.log(bigint64FromIterable);
// BigInt64Array [1n, 2n, 3n]
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/indexed-collections.html#sec-typedarray-objects`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`BigInt64Array`	67	79	68	54	15	67	68	48	15	9.0	67	15	1.0.0	1.0	10.4.0
`BigInt64Array()` constructor	67	79	68	54	15	67	68	48	15	9.0	67	15	1.0.0	1.0	10.4.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.BigInt64Array`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- JavaScript typed arrays guide
- `TypedArray`
- `ArrayBuffer`
- `DataView`

JavaScript › Reference › Standard built-in objects › BigInt64Array

BigInt64Array() constructor

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/BigInt64Array/BigInt64Array · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since September 2021.

The **`BigInt64Array()`** constructor creates `BigInt64Array` objects. The contents are initialized to `0n` unless initialization data is explicitly provided.

Syntax

```js
new BigInt64Array()
new BigInt64Array(length)
new BigInt64Array(typedArray)
new BigInt64Array(object)

new BigInt64Array(buffer)
new BigInt64Array(buffer, byteOffset)
new BigInt64Array(buffer, byteOffset, length)
```

> **Note:** `BigInt64Array()` can only be constructed with `new`. Attempting to call it without `new` throws a `TypeError`.

Parameters

See TypedArray.

Exceptions

See TypedArray.

Examples

Different ways to create a BigInt64Array

```js
// From a length
const bigint64 = new BigInt64Array(2);
bigint64[0] = 42n;
console.log(bigint64[0]); // 42n
console.log(bigint64.length); // 2
console.log(bigint64.BYTES_PER_ELEMENT); // 8

// From an array
const x = new BigInt64Array([21n, 31n]);
console.log(x[1]); // 31n

// From another TypedArray
const y = new BigInt64Array(x);
console.log(y[0]); // 21n

// From an ArrayBuffer
const buffer = new ArrayBuffer(64);
const z = new BigInt64Array(buffer, 8, 4);
console.log(z.byteOffset); // 8

// From an iterable
const iterable = (function* () {
  yield* [1n, 2n, 3n];
})();
const bigint64FromIterable = new BigInt64Array(iterable);
console.log(bigint64FromIterable);
// BigInt64Array [1n, 2n, 3n]
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/indexed-collections.html#sec-typedarray-constructors

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`BigInt64Array()` constructor	67	79	68	54	15	67	68	48	15	9.0	67	15	1.0.0	1.0	10.4.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.BigInt64Array.BigInt64Array`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- JavaScript typed arrays guide
- `TypedArray`
- `ArrayBuffer`
- `DataView`

JavaScript › Reference › Standard built-in objects

BigUint64Array

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/BigUint64Array · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since September 2021.

The **`BigUint64Array`** typed array represents an array of 64-bit unsigned integers in the platform byte order. If control over byte order is needed, use `DataView` instead. The contents are initialized to `0n` unless initialization data is explicitly provided. Once established, you can reference elements in the array using the object's methods, or using standard array index syntax (that is, using bracket notation).

`BigUint64Array` is a subclass of the hidden `TypedArray` class.

Constructor

`BigUint64Array()`

Creates a new `BigUint64Array` object.

Static properties

Also inherits static properties from its parent `TypedArray`.

`BigUint64Array.BYTES_PER_ELEMENT`

Returns a number value of the element size. `8` in the case of `BigUint64Array`.

Static methods

Inherits static methods from its parent `TypedArray`.

Instance properties

Also inherits instance properties from its parent `TypedArray`.

These properties are defined on `BigUint64Array.prototype` and shared by all `BigUint64Array` instances.

`BigUint64Array.prototype.BYTES_PER_ELEMENT`

Returns a number value of the element size. `8` in the case of a `BigUint64Array`.

`BigUint64Array.prototype.constructor`

The constructor function that created the instance object. For `BigUint64Array` instances, the initial value is the `BigUint64Array` constructor.

Instance methods

Inherits instance methods from its parent `TypedArray`.

Examples

Different ways to create a BigUint64Array

```js
// From a length
const biguint64 = new BigUint64Array(2);
biguint64[0] = 42n;
console.log(biguint64[0]); // 42n
console.log(biguint64.length); // 2
console.log(biguint64.BYTES_PER_ELEMENT); // 8

// From an array
const x = new BigUint64Array([21n, 31n]);
console.log(x[1]); // 31n

// From another TypedArray
const y = new BigUint64Array(x);
console.log(y[0]); // 21n

// From an ArrayBuffer
const buffer = new ArrayBuffer(64);
const z = new BigUint64Array(buffer, 8, 4);
console.log(z.byteOffset); // 8

// From an iterable
const iterable = (function* () {
  yield* [1n, 2n, 3n];
})();
const biguint64FromIterable = new BigUint64Array(iterable);
console.log(biguint64FromIterable);
// BigUint64Array [1n, 2n, 3n]
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/indexed-collections.html#sec-typedarray-objects`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`BigUint64Array`	67	79	68	54	15	67	68	48	15	9.0	67	15	1.0.0	1.0	10.4.0
`BigUint64Array()` constructor	67	79	68	54	15	67	68	48	15	9.0	67	15	1.0.0	1.0	10.4.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.BigUint64Array`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- JavaScript typed arrays guide
- `TypedArray`
- `ArrayBuffer`

- DataView

JavaScript › Reference › Standard built-in objects › BigUint64Array

BigUint64Array() constructor

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/BigUint64Array/BigUint64Array · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since September 2021.

The **`BigUint64Array()`** constructor creates BigUint64Array objects. The contents are initialized to `0n` unless initialization data is explicitly provided.

Syntax

```js
new BigUint64Array()
new BigUint64Array(length)
new BigUint64Array(typedArray)
new BigUint64Array(object)

new BigUint64Array(buffer)
new BigUint64Array(buffer, byteOffset)
new BigUint64Array(buffer, byteOffset, length)
```

> **Note:** `BigUint64Array()` can only be constructed with `new`. Attempting to call it without `new` throws a `TypeError`.

Parameters

See TypedArray.

Exceptions

See TypedArray.

Examples

Different ways to create a BigUint64Array

```js
// From a length
const biguint64 = new BigUint64Array(2);
biguint64[0] = 42n;
console.log(biguint64[0]); // 42n
console.log(biguint64.length); // 2
console.log(biguint64.BYTES_PER_ELEMENT); // 8

// From an array
const x = new BigUint64Array([21n, 31n]);
console.log(x[1]); // 31n

// From another TypedArray
const y = new BigUint64Array(x);
console.log(y[0]); // 21n
```

(code continues on the next page)

(code continued from the previous page)

```
// From an ArrayBuffer
const buffer = new ArrayBuffer(64);
const z = new BigUint64Array(buffer, 8, 4);
console.log(z.byteOffset); // 8

// From an iterable
const iterable = (function* () {
  yield* [1n, 2n, 3n];
})();
const biguint64FromIterable = new BigUint64Array(iterable);
console.log(biguint64FromIterable);
// BigUint64Array [1n, 2n, 3n]
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/indexed-collections.html#sec-typedarray-constructors`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`BigUint64Array()` constructor	67	79	68	54	15	67	68	48	15	9.0	67	15	1.0.0	1.0	10.4.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.BigUint64Array.BigUint64Array`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- JavaScript typed arrays guide
- `TypedArray`
- `ArrayBuffer`
- `DataView`

JavaScript › Reference › Standard built-in objects

Boolean

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Boolean · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

`Boolean` values can be one of two values: `true` or `false`, representing the truth value of a logical proposition.

Description

Boolean values are typically produced by relational operators, equality operators, and logical NOT (`!`). They can also be produced by functions that represent conditions, such as `Array.isArray()`. Note that binary logical operators such as `&&` and `||` return the values of the operands, which may or may not be boolean values.

Boolean values are typically used in conditional testing, such as the condition for `if...else` and `while` statements, the conditional operator (`? :`), or the predicate return value of `Array.prototype.filter()`.

You would rarely need to explicitly convert something to a boolean value, as JavaScript does this automatically in boolean contexts, so you can use any value as if it's a boolean, based on its truthiness. You are also encouraged to use `if (condition)` and `if (!condition)` instead of `if (condition === true)` or `if (condition === false)` in your own code so you can take advantage of this convention. However, making sure that values representing conditions are always booleans can help clarify the intent of your code.

js

```
// Do this:
// This always returns a boolean value
const isObject = (obj) => !!obj && typeof obj === "object";

// Or this:
const isObject = (obj) => Boolean(obj) && typeof obj === "object";

// Or this:
const isObject = (obj) => obj !== null && typeof obj === "object";

// Instead of this:
// This may return falsy values that are not equal to false
const isObject = (obj) => obj && typeof obj === "object";
```

Boolean primitives and Boolean objects

For converting non-boolean values to boolean, use `Boolean` as a function or use the double NOT operator. Do not use the `Boolean()` constructor with `new`.

js · ✓ good example

```
const good = Boolean(expression);
const good2 = !!expression;
```

js · ✗ bad example

```
const bad = new Boolean(expression); // don't use this!
```

This is because *all* objects, including a `Boolean` object whose wrapped value is `false`, are truthy and evaluate to `true` in places such as conditional statements. (See also the boolean coercion section below.)

```js
if (new Boolean(true)) {
  console.log("This log is printed.");
}

if (new Boolean(false)) {
  console.log("This log is ALSO printed.");
}

const myFalse = new Boolean(false); // myFalse is a Boolean object (not the primitive value
false)
const g = Boolean(myFalse); // g is true
const myString = new String("Hello"); // myString is a String object
const s = Boolean(myString); // s is true
```

> **Warning:** You should rarely find yourself using `Boolean` as a constructor.

Boolean coercion

Many built-in operations that expect booleans first coerce their arguments to booleans. The operation can be summarized as follows:

- Booleans are returned as-is.
- `undefined` turns into `false`.
- `null` turns into `false`.
- `0`, `-0`, and `NaN` turn into `false`; other numbers turn into `true`.
- `0n` turns into `false`; other BigInts turn into `true`.
- The empty string `""` turns into `false`; other strings turn into `true`.
- Symbols turn into `true`.
- All objects become `true`.

> **Note:** A legacy behavior makes `document.all` return `false` when used as a boolean, despite it being an object. This property is legacy and non-standard and should not be used.

> **Note:** Unlike other type conversions like string coercion or number coercion, boolean coercion does not attempt to convert objects to primitives by calling user methods.

In other words, there are only a handful of values that get coerced to `false` — these are called falsy values. All other values are called truthy values. A value's truthiness is important when used with logical operators, conditional statements, or any boolean context.

There are two ways to achieve the same effect in JavaScript.

- Double NOT: `!!x` negates `x` twice, which converts `x` to a boolean using the same algorithm as above.
- The `Boolean()` function: `Boolean(x)` uses the same algorithm as above to convert `x`.

Note that truthiness is not the same as being loosely equal to `true` or `false`.

```js
if ([]) {
  console.log("[] is truthy");
}
if ([] == false) {
  console.log("[] == false");
}
// [] is truthy
// [] == false
```

`[]` is truthy, but it's also loosely equal to `false`. It's truthy, because all objects are truthy. However, when comparing with `false`, which is a primitive, `[]` is also converted to a primitive, which is `""` via `Array.prototype.toString()`. Comparing strings and booleans results in both being converted to numbers, and they both become `0`, so `[] == false` is `true`. In general, falsiness and `== false` differ in the following cases:

- `NaN`, `undefined`, and `null` are falsy but not loosely equal to `false`.
- `"0"` (and other string literals that are not `""` but get coerced to 0) is truthy but loosely equal to `false`.
- Objects are always truthy, but their primitive representation may be loosely equal to `false`.

Truthy values are even more unlikely to be loosely equal to `true`. All values are either truthy or falsy, but most values are loosely equal to neither `true` nor `false`.

Constructor

`Boolean()`

Creates `Boolean` objects. When called as a function, it returns primitive values of type Boolean.

Instance properties

These properties are defined on `Boolean.prototype` and shared by all `Boolean` instances.

`Boolean.prototype.constructor`

The constructor function that created the instance object. For `Boolean` instances, the initial value is the `Boolean` constructor.

Instance methods

`Boolean.prototype.toString()`

Returns a string of either `true` or `false` depending upon the value of the object. Overrides the `Object.prototype.toString()` method.

`Boolean.prototype.valueOf()`

Returns the primitive value of the `Boolean` object. Overrides the `Object.prototype.valueOf()` method.

Examples

Creating false values

```js
const bNoParam = Boolean();
const bZero = Boolean(0);
const bNull = Boolean(null);
const bEmptyString = Boolean("");
const bfalse = Boolean(false);
```

Creating true values

```js
const btrue = Boolean(true);
const btrueString = Boolean("true");
const bfalseString = Boolean("false");
const bSuLin = Boolean("Su Lin");
const bArrayProto = Boolean([]);
const bObjProto = Boolean({});
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-boolean-objects`

Browser compatibility

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`Boolean`	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`Boolean()` constructor	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`toString`	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`valueOf`	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Boolean`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Boolean
- Boolean primitives
- Boolean data type on Wikipedia

JavaScript › Reference › Standard built-in objects › Boolean

Boolean() constructor

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Boolean/Boolean · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **`Boolean()`** constructor creates `Boolean` objects. When called as a function, it returns primitive values of type Boolean.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Boolean() constructor". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const flag = new Boolean();
console.log(typeof flag);
// Expected output: object
console.log(flag === false);
// Expected output: false

const flag2 = Boolean();
console.log(typeof flag2);
// Expected output: boolean
console.log(flag2 === false);
// Expected output: true
```

Syntax

js

```
new Boolean(value)
Boolean(value)
```

> **Note:** `Boolean()` can be called with or without `new`, but with different effects. See Return value.

Parameters

`value`

The initial value of the `Boolean` object.

Return value

When `Boolean()` is called as a function (without `new`), it returns `value` coerced to a boolean primitive.

When `Boolean()` is called as a constructor (with `new`), it coerces `value` to a boolean primitive and returns a wrapping `Boolean` object, which is **not** a primitive.

> **Warning:** You should rarely find yourself using `Boolean` as a constructor.

Description

The value passed as the first parameter is converted to a boolean value. If the value is omitted or is `0`, `-0`, `0n`, `null`, `false`, `NaN`, `undefined`, or the empty string (`""`), then the object has an initial value of `false`.

All other values, including any object, an empty array ([]), or the string "false", create an object with an initial value of true.

> **Note:** When the non-standard property document.all is used as an argument for this constructor, the result is a Boolean object with the value false. This property is legacy and non-standard and should not be used.

Examples

Creating Boolean objects with an initial value of false

```js
const bZero = new Boolean(0);
const bNull = new Boolean(null);
const bEmptyString = new Boolean("");
const bfalse = new Boolean(false);

typeof bfalse; // "object"
Boolean(bfalse); // true
```

Note how converting a Boolean object to a primitive with Boolean() always yields true, even if the object holds a value of false. You are therefore always advised to avoid constructing Boolean wrapper objects.

If you need to take the primitive value out from the wrapper object, instead of using the Boolean() function, use the object's valueOf() method instead.

```js
const bfalse = new Boolean(false);

bfalse.valueOf(); // false
```

Creating Boolean objects with an initial value of true

```js
const btrue = new Boolean(true);
const btrueString = new Boolean("true");
const bfalseString = new Boolean("false");
const bSuLin = new Boolean("Su Lin");
const bArrayProto = new Boolean([]);
const bObjProto = new Boolean({});
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-boolean-constructor

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`Boolean()` constructor	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Boolean.Boolean`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Boolean

JavaScript › Reference › Standard built-in objects › Boolean

Boolean.prototype.toString()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Boolean/toString · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **`toString()`** method of Boolean values returns a string representing the specified boolean value.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Boolean.prototype.toString()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const flag1 = new Boolean(true);

console.log(flag1.toString());
// Expected output: "true"

const flag2 = new Boolean(1);

console.log(flag2.toString());
// Expected output: "true"
```

Syntax

js

```
toString()
```

Parameters

None.

Return value

A string representing the specified boolean value.

Description

The `Boolean` object overrides the `toString` method of `Object`; it does not inherit `Object.prototype.toString()`. For `Boolean` values, the `toString` method returns a string representation of the boolean value, which is either `"true"` or `"false"`.

The `toString()` method requires its `this` value to be a `Boolean` primitive or wrapper object. It throws a `TypeError` for other `this` values without attempting to coerce them to boolean values.

Because `Boolean` doesn't have a `[Symbol.toPrimitive]()` method, JavaScript calls the `toString()` method automatically when a `Boolean` *object* is used in a context expecting a string, such as in a template literal. However, boolean *primitive* values do not consult the `toString()` method to be coerced to strings — rather, they are directly converted using the same algorithm as the initial `toString()` implementation.

```js
Boolean.prototype.toString = () => "Overridden";
console.log(`${true}`); // "true"
console.log(`${new Boolean(true)}`); // "Overridden"
```

Examples

Using toString()

```js
const flag = new Boolean(true);
console.log(flag.toString()); // "true"
console.log(false.toString()); // "false"
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-boolean.prototype.tostring

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`toString`	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Boolean.toString`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Object.prototype.toString()`

JavaScript › Reference › Standard built-in objects › Boolean

Boolean.prototype.valueOf()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Boolean/valueOf · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **`valueOf()`** method of `Boolean` values returns the primitive value of a `Boolean` object.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Boolean.prototype.valueOf()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const x = new Boolean();

console.log(x.valueOf());
// Expected output: false

const y = new Boolean("Mozilla");

console.log(y.valueOf());
// Expected output: true
```

Syntax

js

```
valueOf()
```

Parameters

None.

Return value

The primitive value of the given `Boolean` object.

Description

The `valueOf()` method of `Boolean` returns the primitive value of a `Boolean` object or literal `Boolean` as a Boolean data type.

This method is usually called internally by JavaScript and not explicitly in code.

Examples

Using `valueOf()`

js

```
const x = new Boolean();
const myVar = x.valueOf(); // assigns false to myVar
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-boolean.prototype.valueof`

Browser compatibility

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`valueOf`	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Boolean.valueOf`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Object.prototype.valueOf()`

JavaScript › Reference › Standard built-in objects

DataView

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/DataView · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since July 2015.

The **`DataView`** view provides a low-level interface for reading and writing multiple number types in a binary `ArrayBuffer`, without having to care about the platform's endianness.

Description

Endianness

Multi-byte number formats are represented in memory differently depending on machine architecture — see Endianness for an explanation. `DataView` accessors provide explicit control of how data is accessed, regardless of the executing computer's endianness. For example, WebAssembly memory is always little-endian, so you should use `DataView` instead of typed arrays to read and write multi-byte values. See `WebAssembly.Memory` for an example.

```js
const littleEndian = (() => {
  const buffer = new ArrayBuffer(2);
  new DataView(buffer).setInt16(0, 256, true /* littleEndian */);
  // Int16Array uses the platform's endianness.
  return new Int16Array(buffer)[0] === 256;
})();
console.log(littleEndian); // true or false
```

> **Note:** `DataView` defaults to big-endian read and write, but most platforms use little-endian.

Constructor

`DataView()`

Creates a new `DataView` object.

Instance properties

These properties are defined on `DataView.prototype` and shared by all `DataView` instances.

`DataView.prototype.buffer`

Returns the `ArrayBuffer` referenced by the `DataView`.

`DataView.prototype.byteLength`

Returns the length (in bytes) of the `DataView`.

`DataView.prototype.byteOffset`

Returns the offset (in bytes) of the `DataView` from the start of its `ArrayBuffer`.

`DataView.prototype.constructor`

The constructor function that created the instance object. For `DataView` instances, the initial value is the `DataView` constructor.

`DataView.prototype[Symbol.toStringTag]`

The initial value of the `[Symbol.toStringTag]` property is the string `"DataView"`. This property is used in `Object.prototype.toString()`.

Instance methods

`DataView.prototype.getBigInt64()`

Reads 8 bytes starting at the specified byte offset of this `DataView` and interprets them as a 64-bit signed integer.

`DataView.prototype.getBigUint64()`

Reads 8 bytes starting at the specified byte offset of this `DataView` and interprets them as a 64-bit unsigned integer.

`DataView.prototype.getFloat16()`

Reads 2 bytes starting at the specified byte offset of this `DataView` and interprets them as a 16-bit floating point number.

`DataView.prototype.getFloat32()`

Reads 4 bytes starting at the specified byte offset of this `DataView` and interprets them as a 32-bit floating point number.

`DataView.prototype.getFloat64()`

Reads 8 bytes starting at the specified byte offset of this `DataView` and interprets them as a 64-bit floating point number.

`DataView.prototype.getInt16()`

Reads 2 bytes starting at the specified byte offset of this `DataView` and interprets them as a 16-bit signed integer.

`DataView.prototype.getInt32()`

Reads 4 bytes starting at the specified byte offset of this `DataView` and interprets them as a 32-bit signed integer.

`DataView.prototype.getInt8()`

Reads 1 byte at the specified byte offset of this `DataView` and interprets it as an 8-bit signed integer.

`DataView.prototype.getUint16()`

Reads 2 bytes starting at the specified byte offset of this `DataView` and interprets them as a 16-bit unsigned integer.

`DataView.prototype.getUint32()`

Reads 4 bytes starting at the specified byte offset of this `DataView` and interprets them as a 32-bit unsigned integer.

`DataView.prototype.getUint8()`

Reads 1 byte at the specified byte offset of this `DataView` and interprets it as an 8-bit unsigned integer.

`DataView.prototype.setBigInt64()`

Takes a BigInt and stores it as a 64-bit signed integer in the 8 bytes starting at the specified byte offset of this `DataView`.

`DataView.prototype.setBigUint64()`

Takes a BigInt and stores it as a 64-bit unsigned integer in the 8 bytes starting at the specified byte offset of this `DataView`.

`DataView.prototype.setFloat16()`

Takes a number and stores it as a 16-bit float in the 2 bytes starting at the specified byte offset of this `DataView`.

`DataView.prototype.setFloat32()`

Takes a number and stores it as a 32-bit float in the 4 bytes starting at the specified byte offset of this `DataView`.

`DataView.prototype.setFloat64()`

Takes a number and stores it as a 64-bit float in the 8 bytes starting at the specified byte offset of this `DataView`.

`DataView.prototype.setInt16()`

Takes a number and stores it as a 16-bit signed integer in the 2 bytes at the specified byte offset of this `DataView`.

`DataView.prototype.setInt32()`

Takes a number and stores it as a 32-bit signed integer in the 4 bytes at the specified byte offset of this `DataView`.

`DataView.prototype.setInt8()`

Takes a number and stores it as an 8-bit signed integer in the byte at the specified byte offset of this `DataView`.

`DataView.prototype.setUint16()`

Takes a number and stores it as a 16-bit unsigned integer in the 2 bytes at the specified byte offset of this `DataView`.

`DataView.prototype.setUint32()`

Takes a number and stores it as a 32-bit unsigned integer in the 4 bytes at the specified byte offset of this `DataView`.

`DataView.prototype.setUint8()`

Takes a number and stores it as an 8-bit unsigned integer in the byte at the specified byte offset of this `DataView`.

Examples

Using DataView

```js
const buffer = new ArrayBuffer(16);
const view = new DataView(buffer, 0);

view.setInt16(1, 42);
view.getInt16(1); // 42
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/structured-data.html#sec-dataview-objects`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`DataView`	9	12	15	12.1	5.1	18	15	12.1	5	1.0	4	5	1.0.0	1.0	0.10.0
`DataView()` constructor	9	12	15	12.1	5.1	18	15	12.1	5	1.0	4	5	1.0.0	1.0	0.10.0
`SharedArrayBuffer` accepted as buffer	68	79	79	55	15.2	89	79	63	15.2	15.0	89	15.2	1.0.0	1.0	8.10.0
`buffer`	9	12	15	12.1	5.1	18	15	12.1	5	1.0	4	5	1.0.0	1.0	0.10.0
`byteLength`	9	12	15	12.1	5.1	18	15	12.1	5	1.0	4	5	1.0.0	1.0	0.10.0
`byteOffset`	9	12	15	12.1	5.1	18	15	12.1	5	1.0	4	5	1.0.0	1.0	0.10.0
`getBigInt64`	67	79	68	54	15	67	68	48	15	9.0	67	15	1.0.0	1.0	10.4.0
`getBigUint64`	67	79	68	54	15	67	68	48	15	9.0	67	15	1.0.0	1.0	10.4.0
`getFloat16`	135	135	129	120	18.2	135	129	89	18.2	29.0	135	18.2	1.1.23	1.43	24.0.0
`getFloat32`	9	12	15	12.1	5.1	18	15	12.1	5	1.0	4	5	1.0.0	1.0	0.10.0
`getFloat64`	9	12	15	12.1	5.1	18	15	12.1	5	1.0	4	5	1.0.0	1.0	0.10.0
`getInt16`	9	12	15	12.1	5.1	18	15	12.1	5	1.0	4	5	1.0.0	1.0	0.10.0
`getInt32`	9	12	15	12.1	5.1	18	15	12.1	5	1.0	4	5	1.0.0	1.0	0.10.0
`getInt8`	9	12	15	12.1	5.1	18	15	12.1	5	1.0	4	5	1.0.0	1.0	0.10.0
`getUint16`	9	12	15	12.1	5.1	18	15	12.1	5	1.0	4	5	1.0.0	1.0	0.10.0
`getUint32`	9	12	15	12.1	5.1	18	15	12.1	5	1.0	4	5	1.0.0	1.0	0.10.0
`getUint8`	9	12	15	12.1	5.1	18	15	12.1	5	1.0	4	5	1.0.0	1.0	0.10.0
`setBigInt64`	67	79	68	54	15	67	68	48	15	9.0	67	15	1.0.0	1.0	10.4.0
`setBigUint64`	67	79	68	54	15	67	68	48	15	9.0	67	15	1.0.0	1.0	10.4.0
`setFloat16`	135	135	129	120	18.2	135	129	89	18.2	29.0	135	18.2	1.1.23	1.43	24.0.0
`setFloat32`	9	12	15	12.1	5.1	18	15	12.1	5	1.0	4	5	1.0.0	1.0	0.10.0
`setFloat64`	9	12	15	12.1	5.1	18	15	12.1	5	1.0	4	5	1.0.0	1.0	0.10.0
`setInt16`	9	12	15	12.1	5.1	18	15	12.1	5	1.0	4	5	1.0.0	1.0	0.10.0
`setInt32`	9	12	15	12.1	5.1	18	15	12.1	5	1.0	4	5	1.0.0	1.0	0.10.0
`setInt8`	9	12	15	12.1	5.1	18	15	12.1	5	1.0	4	5	1.0.0	1.0	0.10.0
`setUint16`	9	12	15	12.1	5.1	18	15	12.1	5	1.0	4	5	1.0.0	1.0	0.10.0
`setUint32`	9	12	15	12.1	5.1	18	15	12.1	5	1.0	4	5	1.0.0	1.0	0.10.0
`setUint8`	9	12	15	12.1	5.1	18	15	12.1	5	1.0	4	5	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.DataView`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `DataView` in `core-js`
- `ArrayBuffer`
- `SharedArrayBuffer`

JavaScript › Reference › Standard built-in objects › DataView

DataView() constructor

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/DataView/DataView · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since July 2015.

The **`DataView()`** constructor creates `DataView` objects.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: DataView() constructor". It runs live in the browser there; its source code follows.*

js · interactive example source

```
// Create an ArrayBuffer with a size in bytes
const buffer = new ArrayBuffer(16);

// Create a couple of views
const view1 = new DataView(buffer);
const view2 = new DataView(buffer, 12, 4); // From byte 12 for the next 4 bytes
view1.setInt8(12, 42); // Put 42 in slot 12

console.log(view2.getInt8(0));
// Expected output: 42
```

Syntax

js

```
new DataView(buffer)
new DataView(buffer, byteOffset)
new DataView(buffer, byteOffset, byteLength)
```

> **Note:** `DataView()` can only be constructed with `new`. Attempting to call it without `new` throws a `TypeError`.

Parameters

`buffer`

An existing `ArrayBuffer` or `SharedArrayBuffer` to use as the storage backing the new `DataView` object.

`byteOffset` Optional

The offset, in bytes, to the first byte in the above buffer for the new view to reference. If unspecified, the buffer view starts with the first byte.

`byteLength` Optional

The number of elements in the byte array. If unspecified, the view's length will match the buffer's length.

Return value

A new `DataView` object representing the specified data buffer.

Exceptions

`RangeError`

Thrown if the `byteOffset` or `byteLength` parameter values result in the view extending past the end of the buffer. In other words, `byteOffset + byteLength > buffer.byteLength`.

Examples

Using DataView

```js
const buffer = new ArrayBuffer(16);
const view = new DataView(buffer, 0);

view.setInt16(1, 42);
view.getInt16(1); // 42
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/structured-data.html#sec-dataview-constructor

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`DataView()` constructor	9	12	15	12.1	5.1	18	15	12.1	5	1.0	4	5	1.0.0	1.0	0.10.0
`SharedArrayBuffer` accepted as buffer	68	79	79	55	15.2	89	79	63	15.2	15.0	89	15.2	1.0.0	1.0	8.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.DataView.DataView`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Polyfill of `DataView` in `core-js`
- `DataView`

JavaScript › Reference › Standard built-in objects › DataView

DataView.prototype.getBigInt64()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/DataView/getBigInt64 · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since September 2021.

The **`getBigInt64()`** method of `DataView` instances reads 8 bytes starting at the specified byte offset of this `DataView` and interprets them as a 64-bit signed integer. There is no alignment constraint; multi-byte values may be fetched from any offset within bounds.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: DataView.prototype.getBigInt64()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
// Create an ArrayBuffer with a size in bytes
const buffer = new ArrayBuffer(16);

// Highest possible BigInt value that fits in a signed 64-bit integer
const max = 2n ** (64n - 1n) - 1n;

const view = new DataView(buffer);
view.setBigInt64(1, max);

console.log(view.getBigInt64(1));
// Expected output: 9223372036854775807n
```

Syntax

js

```
getBigInt64(byteOffset)
getBigInt64(byteOffset, littleEndian)
```

Parameters

`byteOffset`

The offset, in bytes, from the start of the view to read the data from.

`littleEndian` Optional

Indicates whether the data is stored in little- or big-endian format. If `false` or `undefined`, a big-endian value is read.

Return value

A `BigInt` from -2^{63} to $2^{63}-1$, inclusive.

Exceptions

`RangeError`

Thrown if the `byteOffset` is set such that it would read beyond the end of the view.

Examples

Using getBigInt64()

```js
const { buffer } = new Uint8Array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9]);
const dataview = new DataView(buffer);
console.log(dataview.getBigInt64(1)); // 72623859790382856n
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/structured-data.html#sec-dataview.prototype.getbigint64

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
getBigInt64	67	79	68	54	15	67	68	48	15	9.0	67	15	1.0.0	1.0	10.4.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.DataView.getBigInt64`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- JavaScript typed arrays guide
- `DataView`
- `ArrayBuffer`
- `BigInt64Array`

JavaScript › Reference › Standard built-in objects › DataView

DataView.prototype.getBigUint64()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/DataView/getBigUint64 · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since September 2021.

The **`getBigUint64()`** method of `DataView` instances reads 8 bytes starting at the specified byte offset of this `DataView` and interprets them as a 64-bit unsigned integer. There is no alignment constraint; multi-byte values may be fetched from any offset within bounds.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: DataView.prototype.getBigUint64()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
// Create an ArrayBuffer with a size in bytes
const buffer = new ArrayBuffer(16);

// Highest possible BigInt value that fits in an unsigned 64-bit integer
const max = 2n ** 64n - 1n;

const view = new DataView(buffer);
view.setBigUint64(1, max);

console.log(view.getBigUint64(1));
// Expected output: 18446744073709551615n
```

Syntax

js

```
getBigUint64(byteOffset)
getBigUint64(byteOffset, littleEndian)
```

Parameters

`byteOffset`

The offset, in bytes, from the start of the view to read the data from.

`littleEndian` Optional

Indicates whether the data is stored in little- or big-endian format. If `false` or `undefined`, a big-endian value is read.

Return value

A `BigInt` from 0 to 2^{64}-1, inclusive.

Exceptions

`RangeError`

Thrown if the `byteOffset` is set such that it would read beyond the end of the view.

Examples

Using getBigUint64()

js

```
const { buffer } = new Uint8Array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9]);
const dataview = new DataView(buffer);
console.log(dataview.getBigUint64(1)); // 72623859790382856n
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/structured-data.html#sec-dataview.prototype.getbiguint64`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`getBigUint64`	67	79	68	54	15	67	68	48	15	9.0	67	15	1.0.0	1.0	10.4.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.DataView.getBigUint64`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- JavaScript typed arrays guide
- `DataView`
- `ArrayBuffer`
- `BigUint64Array`

JavaScript › Reference › Standard built-in objects › DataView

DataView.prototype.getFloat16()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/DataView/getFloat16 · MDN page last modified 2026-10-26

Baseline 2025: Newly available — works across the latest browser versions since April 2025; may not work in older ones.

The **`getFloat16()`** method of `DataView` instances reads 2 bytes starting at the specified byte offset of this `DataView` and interprets them as a 16-bit floating point number. There is no alignment constraint; multi-byte values may be fetched from any offset within bounds.

Try it

▷ Interactive example on MDN: "JavaScript Demo: DataView.prototype.getFloat16()". It runs live in the browser there; its source code follows.

js · interactive example source

```
// Create an ArrayBuffer with a size in bytes
const buffer = new ArrayBuffer(16);

const view = new DataView(buffer);
view.setFloat16(1, Math.PI);

console.log(view.getFloat16(1));
// Expected output: 3.140625
```

Syntax

js

```
getFloat16(byteOffset)
getFloat16(byteOffset, littleEndian)
```

Parameters

`byteOffset`

The offset, in bytes, from the start of the view to read the data from.

`littleEndian` Optional

Indicates whether the data is stored in little- or big-endian format. If `false` or `undefined`, a big-endian value is read.

Return value

A floating point number from `-65504` to `65504`.

Exceptions

`RangeError`

Thrown if the `byteOffset` is set such that it would read beyond the end of the view.

Examples

Using getFloat16()

```js
const { buffer } = new Uint8Array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9]);
const dataview = new DataView(buffer);
console.log(dataview.getFloat16(1)); // 0.00012207031250000000
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/structured-data.html#sec-dataview.prototype.getfloat16

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`getFloat16`	135	135	129	120	18.2	135	129	89	18.2	29.0	135	18.2	1.1.23	1.43	24.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.DataView.getFloat16`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Polyfill of `DataView.prototype.getFloat16` in `core-js`
- JavaScript typed arrays guide
- `DataView`
- `ArrayBuffer`
- `Float16Array`

JavaScript › Reference › Standard built-in objects › DataView

DataView.prototype.getFloat32()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/DataView/getFloat32 · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **`getFloat32()`** method of `DataView` instances reads 4 bytes starting at the specified byte offset of this `DataView` and interprets them as a 32-bit floating point number. There is no alignment constraint; multi-byte values may be fetched from any offset within bounds.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: DataView.prototype.getFloat32()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
// Create an ArrayBuffer with a size in bytes
const buffer = new ArrayBuffer(16);

const view = new DataView(buffer);
view.setFloat32(1, Math.PI);

console.log(view.getFloat32(1));
// Expected output: 3.1415927410125732
```

Syntax

js

```
getFloat32(byteOffset)
getFloat32(byteOffset, littleEndian)
```

Parameters

`byteOffset`

The offset, in bytes, from the start of the view to read the data from.

`littleEndian` Optional

Indicates whether the data is stored in little- or big-endian format. If `false` or `undefined`, a big-endian value is read.

Return value

A floating point number from `-3.4e38` to `3.4e38`.

Exceptions

`RangeError`

Thrown if the `byteOffset` is set such that it would read beyond the end of the view.

Examples

Using getFloat32()

```js
const { buffer } = new Uint8Array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9]);
const dataview = new DataView(buffer);
console.log(dataview.getFloat32(1)); // 2.387939260590663e-38
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/structured-data.html#sec-dataview.prototype.getfloat32

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
getFloat32	9	12	15	12.1	5.1	18	15	12.1	5	1.0	4	5	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.DataView.getFloat32`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- JavaScript typed arrays guide
- `DataView`
- `ArrayBuffer`
- `Float32Array`

JavaScript › Reference › Standard built-in objects › DataView

DataView.prototype.getFloat64()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/DataView/getFloat64 · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **`getFloat64()`** method of `DataView` instances reads 8 bytes starting at the specified byte offset of this `DataView` and interprets them as a 64-bit floating point number. There is no alignment constraint; multi-byte values may be fetched from any offset within bounds.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: DataView.prototype.getFloat64()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
// Create an ArrayBuffer with a size in bytes
const buffer = new ArrayBuffer(16);

const view = new DataView(buffer);
view.setFloat64(1, Math.PI);

console.log(view.getFloat64(1));
// Expected output: 3.141592653589793
```

Syntax

js

```
getFloat64(byteOffset)
getFloat64(byteOffset, littleEndian)
```

Parameters

`byteOffset`

The offset, in bytes, from the start of the view to read the data from.

`littleEndian` Optional

Indicates whether the data is stored in little- or big-endian format. If `false` or `undefined`, a big-endian value is read.

Return value

Any number value.

Exceptions

`RangeError`

Thrown if the `byteOffset` is set such that it would read beyond the end of the view.

Examples

Using getFloat64()

js

```
const { buffer } = new Uint8Array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9]);
const dataview = new DataView(buffer);
console.log(dataview.getFloat64(1)); // 8.20788039913184e-304
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/structured-data.html#sec-dataview.prototype.getfloat64`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
getFloat64	9	12	15	12.1	5.1	18	15	12.1	5	1.0	4	5	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.DataView.getFloat64`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- JavaScript typed arrays guide
- `DataView`
- `ArrayBuffer`
- `Float64Array`

JavaScript › Reference › Standard built-in objects › DataView

DataView.prototype.getInt16()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/DataView/getInt16 · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **`getInt16()`** method of `DataView` instances reads 2 bytes starting at the specified byte offset of this `DataView` and interprets them as a 16-bit signed integer. There is no alignment constraint; multi-byte values may be fetched from any offset within bounds.

Try it

▷ Interactive example on MDN: "JavaScript Demo: DataView.prototype.getInt16()". It runs live in the browser there; its source code follows.

js · interactive example source

```
// Create an ArrayBuffer with a size in bytes
const buffer = new ArrayBuffer(16);

const view = new DataView(buffer);
view.setInt16(1, 32767); // Max signed 16-bit integer

console.log(view.getInt16(1));
// Expected output: 32767
```

Syntax

js

```
getInt16(byteOffset)
getInt16(byteOffset, littleEndian)
```

Parameters

`byteOffset`

The offset, in bytes, from the start of the view to read the data from.

`littleEndian` Optional

Indicates whether the data is stored in little- or big-endian format. If `false` or `undefined`, a big-endian value is read.

Return value

An integer from -32768 to 32767, inclusive.

Exceptions

`RangeError`

Thrown if the `byteOffset` is set such that it would read beyond the end of the view.

Examples

Using getInt16()

```js
const { buffer } = new Uint8Array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9]);
const dataview = new DataView(buffer);
console.log(dataview.getInt16(1)); // 258
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/structured-data.html#sec-dataview.prototype.getint16

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`getInt16`	9	12	15	12.1	5.1	18	15	12.1	5	1.0	4	5	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.DataView.getInt16`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- JavaScript typed arrays guide
- `DataView`
- `ArrayBuffer`
- `Int16Array`

JavaScript › Reference › Standard built-in objects › DataView

DataView.prototype.getInt32()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/DataView/getInt32 · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **`getInt32()`** method of `DataView` instances reads 4 bytes starting at the specified byte offset of this `DataView` and interprets them as a 32-bit signed integer. There is no alignment constraint; multi-byte values may be fetched from any offset within bounds.

Try it

▷ Interactive example on MDN: "JavaScript Demo: DataView.prototype.getInt32()". It runs live in the browser there; its source code follows.

js · interactive example source

```
// Create an ArrayBuffer with a size in bytes
const buffer = new ArrayBuffer(16);

const view = new DataView(buffer);
view.setInt32(1, 2147483647); // Max signed 32-bit integer

console.log(view.getInt32(1));
// Expected output: 2147483647
```

Syntax

js

```
getInt32(byteOffset)
getInt32(byteOffset, littleEndian)
```

Parameters

`byteOffset`

The offset, in bytes, from the start of the view to read the data from.

`littleEndian` Optional

Indicates whether the data is stored in little- or big-endian format. If `false` or `undefined`, a big-endian value is read.

Return value

An integer from -2147483648 to 2147483647, inclusive.

Exceptions

`RangeError`

Thrown if the `byteOffset` is set such that it would read beyond the end of the view.

Examples

Using getInt32()

```js
const { buffer } = new Uint8Array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9]);
const dataview = new DataView(buffer);
console.log(dataview.getInt32(1)); // 16909060
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/structured-data.html#sec-dataview.prototype.getint32

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
getInt32	9	12	15	12.1	5.1	18	15	12.1	5	1.0	4	5	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.DataView.getInt32`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- JavaScript typed arrays guide
- `DataView`
- `ArrayBuffer`
- `Int32Array`

JavaScript › Reference › Standard built-in objects › DataView

DataView.prototype.getInt8()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/DataView/getInt8 · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **`getInt8()`** method of `DataView` instances reads 1 byte at the specified byte offset of this `DataView` and interprets it as an 8-bit signed integer.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: DataView.prototype.getInt8()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
// Create an ArrayBuffer with a size in bytes
const buffer = new ArrayBuffer(16);

const view = new DataView(buffer);
view.setInt8(1, 127); // Max signed 8-bit integer

console.log(view.getInt8(1));
// Expected output: 127
```

Syntax

js

```
getInt8(byteOffset)
```

Parameters

`byteOffset`

The offset, in bytes, from the start of the view to read the data from.

Return value

An integer from -128 to 127, inclusive.

Exceptions

`RangeError`

Thrown if the `byteOffset` is set such that it would read beyond the end of the view.

Examples

Using getInt8()

js

```
const { buffer } = new Uint8Array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9]);
const dataview = new DataView(buffer);
console.log(dataview.getInt8(1)); // 1
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/structured-data.html#sec-dataview.prototype.getint8`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`getInt8`	9	12	15	12.1	5.1	18	15	12.1	5	1.0	4	5	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.DataView.getInt8`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- JavaScript typed arrays guide
- `DataView`
- `ArrayBuffer`
- `Int8Array`

JavaScript › Reference › Standard built-in objects › DataView

DataView.prototype.getUint16()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/DataView/getUint16 · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **`getUint16()`** method of `DataView` instances reads 2 bytes starting at the specified byte offset of this `DataView` and interprets them as a 16-bit unsigned integer. There is no alignment constraint; multi-byte values may be fetched from any offset within bounds.

Try it

▷ Interactive example on MDN: "JavaScript Demo: DataView.prototype.getUint16()". It runs live in the browser there; its source code follows.

js · interactive example source

```
// Create an ArrayBuffer with a size in bytes
const buffer = new ArrayBuffer(16);

const view = new DataView(buffer);
view.setUint16(1, 65535); // Max unsigned 16-bit integer

console.log(view.getUint16(1));
// Expected output: 65535
```

Syntax

js

```
getUint16(byteOffset)
getUint16(byteOffset, littleEndian)
```

Parameters

`byteOffset`

The offset, in bytes, from the start of the view to read the data from.

`littleEndian` Optional

Indicates whether the data is stored in little- or big-endian format. If `false` or `undefined`, a big-endian value is read.

Return value

An integer from 0 to 65535, inclusive.

Exceptions

`RangeError`

Thrown if the `byteOffset` is set such that it would read beyond the end of the view.

Examples

Using getUint16()

```js
const { buffer } = new Uint8Array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9]);
const dataview = new DataView(buffer);
console.log(dataview.getUint16(1)); // 258
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/structured-data.html#sec-dataview.prototype.getuint16

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`getUint16`	9	12	15	12.1	5.1	18	15	12.1	5	1.0	4	5	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.DataView.getUint16`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- JavaScript typed arrays guide
- `DataView`
- `ArrayBuffer`
- `Uint16Array`

JavaScript › Reference › Standard built-in objects › DataView

DataView.prototype.getUint32()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/DataView/getUint32 · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **`getUint32()`** method of `DataView` instances reads 4 bytes starting at the specified byte offset of this `DataView` and interprets them as a 32-bit unsigned integer. There is no alignment constraint; multi-byte values may be fetched from any offset within bounds.

Try it

▷ Interactive example on MDN: "JavaScript Demo: DataView.prototype.getUint32()". It runs live in the browser there; its source code follows.

js · interactive example source

```
// Create an ArrayBuffer with a size in bytes
const buffer = new ArrayBuffer(16);

const view = new DataView(buffer);
view.setUint32(1, 4294967295); // Max unsigned 32-bit integer

console.log(view.getUint32(1));
// Expected output: 4294967295
```

Syntax

js

```
getUint32(byteOffset)
getUint32(byteOffset, littleEndian)
```

Parameters

`byteOffset`

The offset, in bytes, from the start of the view to read the data from.

`littleEndian` Optional

Indicates whether the data is stored in little- or big-endian format. If `false` or `undefined`, a big-endian value is read.

Return value

An integer from 0 to 4294967295, inclusive.

Exceptions

`RangeError`

Thrown if the `byteOffset` is set such that it would read beyond the end of the view.

Examples

Using getUint32()

```js
const { buffer } = new Uint8Array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9]);
const dataview = new DataView(buffer);
console.log(dataview.getUint32(1)); // 16909060
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/structured-data.html#sec-dataview.prototype.getuint32

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
getUint32	9	12	15	12.1	5.1	18	15	12.1	5	1.0	4	5	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.DataView.getUint32`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- JavaScript typed arrays guide
- `DataView`
- `ArrayBuffer`
- `Uint32Array`

JavaScript › Reference › Standard built-in objects › DataView

DataView.prototype.getUint8()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/DataView/getUint8 · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **`getUint8()`** method of `DataView` instances reads 1 byte at the specified byte offset of this `DataView` and interprets it as an 8-bit unsigned integer.

Try it

▷ Interactive example on MDN: "JavaScript Demo: DataView.prototype.getUint8()". It runs live in the browser there; its source code follows.

js · interactive example source

```
// Create an ArrayBuffer with a size in bytes
const buffer = new ArrayBuffer(16);

const view = new DataView(buffer);
view.setUint8(1, 255); // Max unsigned 8-bit integer

console.log(view.getUint8(1));
// Expected output: 255
```

Syntax

js

```
getUint8(byteOffset)
```

Parameters

`byteOffset`

The offset, in bytes, from the start of the view to read the data from.

Return value

An integer from 0 to 255, inclusive.

Exceptions

`RangeError`

Thrown if the `byteOffset` is set such that it would read beyond the end of the view.

Examples

Using getUint8()

js

```
const { buffer } = new Uint8Array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9]);
const dataview = new DataView(buffer);
console.log(dataview.getUint8(1)); // 1
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/structured-data.html#sec-dataview.prototype.getuint8`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
getUint8	9	12	15	12.1	5.1	18	15	12.1	5	1.0	4	5	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.DataView.getUint8`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- JavaScript typed arrays guide
- `DataView`
- `ArrayBuffer`
- `Uint8Array`

JavaScript › Reference › Standard built-in objects › DataView

DataView.prototype.setBigInt64()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/DataView/setBigInt64 · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since September 2021.

The **`setBigInt64()`** method of `DataView` instances takes a BigInt and stores it as a 64-bit signed integer in the 8 bytes starting at the specified byte offset of this `DataView`. There is no alignment constraint; multi-byte values may be stored at any offset within bounds.

Try it

▷ Interactive example on MDN: "JavaScript Demo: DataView.prototype.setBigInt64()". It runs live in the browser there; its source code follows.

js · interactive example source

```
// Create an ArrayBuffer with a size in bytes
const buffer = new ArrayBuffer(16);

// Highest possible BigInt value that fits in a signed 64-bit integer
const max = 2n ** (64n - 1n) - 1n;

const view = new DataView(buffer);
view.setBigInt64(1, max);

console.log(view.getBigInt64(1));
// Expected output: 9223372036854775807n
```

Syntax

js

```
setBigInt64(byteOffset, value)
setBigInt64(byteOffset, value, littleEndian)
```

Parameters

`byteOffset`

The offset, in bytes, from the start of the view to store the data in.

`value`

The value to set as a `BigInt`. For how the value is encoded in bytes, see Value encoding and normalization.

`littleEndian` Optional

Indicates whether the data is stored in little- or big-endian format. If `false` or `undefined`, a big-endian value is written.

Return value

`undefined`.

Exceptions

`RangeError`

Thrown if the `byteOffset` is set such that it would store beyond the end of the view.

Examples

Using setBigInt64()

```js
const buffer = new ArrayBuffer(10);
const dataview = new DataView(buffer);
dataview.setBigInt64(0, 3n);
dataview.getBigInt64(1); // 768n
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/structured-data.html#sec-dataview.prototype.setbigint64`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`setBigInt64`	67	79	68	54	15	67	68	48	15	9.0	67	15	1.0.0	1.0	10.4.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.DataView.setBigInt64`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- JavaScript typed arrays guide
- `DataView`
- `ArrayBuffer`
- `BigInt64Array`

DataView.prototype.setBigUint64()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/DataView/setBigUint64 · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since September 2021.

The **`setBigUint64()`** method of `DataView` instances takes a BigInt and stores it as a 64-bit unsigned integer in the 8 bytes starting at the specified byte offset of this `DataView`. There is no alignment constraint; multi-byte values may be stored at any offset within bounds.

Try it

▷ Interactive example on MDN: "JavaScript Demo: DataView.prototype.setBigUint64()". It runs live in the browser there; its source code follows.

js · interactive example source

```
// Create an ArrayBuffer with a size in bytes
const buffer = new ArrayBuffer(16);

// Highest possible BigInt value that fits in an unsigned 64-bit integer
const max = 2n ** 64n - 1n;

const view = new DataView(buffer);
view.setBigUint64(1, max);

console.log(view.getBigUint64(1));
// Expected output: 18446744073709551615n
```

Syntax

js

```
setBigUint64(byteOffset, value)
setBigUint64(byteOffset, value, littleEndian)
```

Parameters

`byteOffset`

The offset, in bytes, from the start of the view to store the data in.

`value`

The value to set as a `BigInt`. For how the value is encoded in bytes, see Value encoding and normalization.

`littleEndian` Optional

Indicates whether the data is stored in little- or big-endian format. If `false` or `undefined`, a big-endian value is written.

Return value

undefined.

Exceptions

RangeError

Thrown if the byteOffset is set such that it would store beyond the end of the view.

Examples

Using setBigUint64()

```js
const buffer = new ArrayBuffer(10);
const dataview = new DataView(buffer);
dataview.setBigUint64(0, 3n);
dataview.getBigUint64(1); // 768n
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/structured-data.html#sec-dataview.prototype.setbiguint64

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
setBigUint64	67	79	68	54	15	67	68	48	15	9.0	67	15	1.0.0	1.0	10.4.0

Compatibility data: MDN browser-compat-data 8.1.8, javascript.builtins.DataView.setBigUint64*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- JavaScript typed arrays guide
- DataView
- ArrayBuffer
- BigUint64Array

JavaScript › Reference › Standard built-in objects › DataView

DataView.prototype.setFloat16()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/DataView/setFloat16 · MDN page last modified 2026-10-26

Baseline 2025: Newly available — works across the latest browser versions since April 2025; may not work in older ones.

The **`setFloat16()`** method of `DataView` instances takes a number and stores it as a 16-bit floating point number in the 2 bytes starting at the specified byte offset of this `DataView`. There is no alignment constraint; multi-byte values may be stored at any offset within bounds.

Try it

▷ Interactive example on MDN: "JavaScript Demo: DataView.prototype.setFloat16()". It runs live in the browser there; its source code follows.

js · interactive example source

```
// Create an ArrayBuffer with a size in bytes
const buffer = new ArrayBuffer(16);

const view = new DataView(buffer);
view.setFloat16(1, Math.PI);

console.log(view.getFloat16(1));
// Expected output: 3.140625
```

Syntax

js

```
setFloat16(byteOffset, value)
setFloat16(byteOffset, value, littleEndian)
```

Parameters

`byteOffset`

The offset, in bytes, from the start of the view to store the data in.

`value`

The value to set. For how the value is encoded in bytes, see Value encoding and normalization.

`littleEndian` Optional

Indicates whether the data is stored in little- or big-endian format. If `false` or `undefined`, a big-endian value is written.

Return value

`undefined`.

Exceptions

`RangeError`

Thrown if the `byteOffset` is set such that it would store beyond the end of the view.

Examples

Using setFloat16()

```js
const buffer = new ArrayBuffer(10);
const dataview = new DataView(buffer);
dataview.setFloat16(0, 3);
dataview.getFloat16(1); // 0
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/structured-data.html#sec-dataview.prototype.setfloat16`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`setFloat16`	135	135	129	120	18.2	135	129	89	18.2	29.0	135	18.2	1.1.23	1.43	24.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.DataView.setFloat16`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Polyfill of `DataView.prototype.setFloat16` in `core-js`
- JavaScript typed arrays guide
- `DataView`
- `ArrayBuffer`
- `Float16Array`

JavaScript › Reference › Standard built-in objects › DataView

DataView.prototype.setFloat32()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/DataView/setFloat32 · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **`setFloat32()`** method of `DataView` instances takes a number and stores it as a 32-bit floating point number in the 4 bytes starting at the specified byte offset of this `DataView`. There is no alignment constraint; multi-byte values may be stored at any offset within bounds.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: DataView.prototype.setFloat32()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
// Create an ArrayBuffer with a size in bytes
const buffer = new ArrayBuffer(16);

const view = new DataView(buffer);
view.setFloat32(1, Math.PI);

console.log(view.getFloat32(1));
// Expected output: 3.1415927410125732
```

Syntax

js

```
setFloat32(byteOffset, value)
setFloat32(byteOffset, value, littleEndian)
```

Parameters

`byteOffset`

The offset, in bytes, from the start of the view to store the data in.

`value`

The value to set. For how the value is encoded in bytes, see Value encoding and normalization.

`littleEndian` Optional

Indicates whether the data is stored in little- or big-endian format. If `false` or `undefined`, a big-endian value is written.

Return value

`undefined`.

Exceptions

`RangeError`

Thrown if the `byteOffset` is set such that it would store beyond the end of the view.

Examples

Using setFloat32()

js

```
const buffer = new ArrayBuffer(10);
const dataview = new DataView(buffer);
dataview.setFloat32(0, 3);
dataview.getFloat32(1); // 2
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/structured-data.html#sec-dataview.prototype.setfloat32`

Browser compatibility

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`setFloat32`	9	12	15	12.1	5.1	18	15	12.1	5	1.0	4	5	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.DataView.setFloat32`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- JavaScript typed arrays guide
- `DataView`
- `ArrayBuffer`
- `Float32Array`

JavaScript › Reference › Standard built-in objects › DataView

DataView.prototype.setFloat64()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/DataView/setFloat64 · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **`setFloat64()`** method of `DataView` instances takes a number and stores it as a 64-bit floating point number in the 8 bytes starting at the specified byte offset of this `DataView`. There is no alignment constraint; multi-byte values may be stored at any offset within bounds.

Try it

▷ Interactive example on MDN: "JavaScript Demo: DataView.prototype.setFloat64()". It runs live in the browser there; its source code follows.

js · interactive example source

```
// Create an ArrayBuffer with a size in bytes
const buffer = new ArrayBuffer(16);

const view = new DataView(buffer);
view.setFloat64(1, Math.PI);

console.log(view.getFloat64(1));
// Expected output: 3.141592653589793
```

Syntax

```js
setFloat64(byteOffset, value)
setFloat64(byteOffset, value, littleEndian)
```

Parameters

`byteOffset`

The offset, in bytes, from the start of the view to store the data in.

`value`

The value to set. For how the value is encoded in bytes, see Value encoding and normalization.

`littleEndian` Optional

Indicates whether the data is stored in little- or big-endian format. If `false` or `undefined`, a big-endian value is written.

Return value

`undefined`.

Exceptions

`RangeError`

Thrown if the `byteOffset` is set such that it would store beyond the end of the view.

Examples

Using setFloat64()

```js
const buffer = new ArrayBuffer(10);
const dataview = new DataView(buffer);
dataview.setFloat64(0, 3);
dataview.getFloat64(1); // 3.785766995733679e-270
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/structured-data.html#sec-dataview.prototype.setfloat64`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`setFloat64`	9	12	15	12.1	5.1	18	15	12.1	5	1.0	4	5	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.DataView.setFloat64`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- JavaScript typed arrays guide
- `DataView`
- `ArrayBuffer`
- `Float64Array`

JavaScript › Reference › Standard built-in objects › DataView

DataView.prototype.setInt16()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/DataView/setInt16 · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **`setInt16()`** method of `DataView` instances takes a number and stores it as a 16-bit signed integer in the 2 bytes starting at the specified byte offset of this `DataView`. There is no alignment constraint; multi-byte values may be stored at any offset within bounds.

Try it

▷ Interactive example on MDN: "JavaScript Demo: DataView.prototype.setInt16()". It runs live in the browser there; its source code follows.

js · interactive example source

```
// Create an ArrayBuffer with a size in bytes
const buffer = new ArrayBuffer(16);

const view = new DataView(buffer);
view.setInt16(1, 32767); // Max signed 16-bit integer

console.log(view.getInt16(1));
// Expected output: 32767
```

Syntax

js

```
setInt16(byteOffset, value)
setInt16(byteOffset, value, littleEndian)
```

Parameters

`byteOffset`

The offset, in bytes, from the start of the view to store the data in.

`value`

The value to set. For how the value is encoded in bytes, see Value encoding and normalization.

`littleEndian` Optional

Indicates whether the data is stored in little- or big-endian format. If `false` or `undefined`, a big-endian value is written.

Return value

`undefined`.

Exceptions

`RangeError`

Thrown if the `byteOffset` is set such that it would store beyond the end of the view.

Examples

Using setInt16()

```js
const buffer = new ArrayBuffer(10);
const dataview = new DataView(buffer);
dataview.setInt16(0, 3);
dataview.getInt16(1); // 768
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/structured-data.html#sec-dataview.prototype.setint16`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`setInt16`	9	12	15	12.1	5.1	18	15	12.1	5	1.0	4	5	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.DataView.setInt16`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- JavaScript typed arrays guide
- `DataView`
- `ArrayBuffer`
- `Int16Array`

JavaScript › Reference › Standard built-in objects › DataView

DataView.prototype.setInt32()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/DataView/setInt32 · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **`setInt32()`** method of `DataView` instances takes a number and stores it as a 32-bit signed integer in the 4 bytes starting at the specified byte offset of this `DataView`. There is no alignment constraint; multi-byte values may be stored at any offset within bounds.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: DataView.prototype.setInt32()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
// Create an ArrayBuffer with a size in bytes
const buffer = new ArrayBuffer(16);

const view = new DataView(buffer);
view.setInt32(1, 2147483647); // Max signed 32-bit integer

console.log(view.getInt32(1));
// Expected output: 2147483647
```

Syntax

js

```
setInt32(byteOffset, value)
setInt32(byteOffset, value, littleEndian)
```

Parameters

`byteOffset`

The offset, in bytes, from the start of the view to store the data in.

`value`

The value to set. For how the value is encoded in bytes, see Value encoding and normalization.

`littleEndian` Optional

Indicates whether the data is stored in little- or big-endian format. If `false` or `undefined`, a big-endian value is written.

Return value

`undefined`.

Exceptions

`RangeError`

Thrown if the `byteOffset` is set such that it would store beyond the end of the view.

Examples

Using setInt32()

```js
const buffer = new ArrayBuffer(10);
const dataview = new DataView(buffer);
dataview.setInt32(0, 3);
dataview.getInt32(1); // 768
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/structured-data.html#sec-dataview.prototype.setint32

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
setInt32	9	12	15	12.1	5.1	18	15	12.1	5	1.0	4	5	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.DataView.setInt32`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- JavaScript typed arrays guide
- `DataView`
- `ArrayBuffer`
- `Int32Array`

JavaScript › Reference › Standard built-in objects › DataView

DataView.prototype.setInt8()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/DataView/setInt8 · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **`setInt8()`** method of `DataView` instances takes a number and stores it as an 8-bit signed integer in the byte at the specified byte offset of this `DataView`.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: DataView.prototype.setInt8()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
// Create an ArrayBuffer with a size in bytes
const buffer = new ArrayBuffer(16);

const view = new DataView(buffer);
view.setInt8(1, 127); // Max signed 8-bit integer

console.log(view.getInt8(1));
// Expected output: 127
```

Syntax

js

```
setInt8(byteOffset, value)
```

Parameters

`byteOffset`

The offset, in bytes, from the start of the view to store the data in.

`value`

The value to set. For how the value is encoded in bytes, see Value encoding and normalization.

Return value

`undefined`.

Exceptions

`RangeError`

Thrown if the `byteOffset` is set such that it would store beyond the end of the view.

Examples

Using setInt8()

js

```
const buffer = new ArrayBuffer(10);
const dataview = new DataView(buffer);
dataview.setInt8(0, 3);
dataview.getInt8(0); // 3
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/structured-data.html#sec-dataview.prototype.setint8`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
setInt8	9	12	15	12.1	5.1	18	15	12.1	5	1.0	4	5	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.DataView.setInt8`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- JavaScript typed arrays guide
- `DataView`
- `ArrayBuffer`
- `Int8Array`

JavaScript › Reference › Standard built-in objects › DataView

DataView.prototype.setUint16()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/DataView/setUint16 · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **`setUint16()`** method of `DataView` instances takes a number and stores it as a 16-bit unsigned integer in the 2 bytes starting at the specified byte offset of this `DataView`. There is no alignment constraint; multi-byte values may be stored at any offset within bounds.

Try it

▷ Interactive example on MDN: "JavaScript Demo: DataView.prototype.setUint16()". It runs live in the browser there; its source code follows.

js · interactive example source

```
// Create an ArrayBuffer with a size in bytes
const buffer = new ArrayBuffer(16);

const view = new DataView(buffer);
view.setUint16(1, 65535); // Max unsigned 16-bit integer

console.log(view.getUint16(1));
// Expected output: 65535
```

Syntax

js

```
setUint16(byteOffset, value)
setUint16(byteOffset, value, littleEndian)
```

Parameters

`byteOffset`

The offset, in bytes, from the start of the view to store the data in.

`value`

The value to set. For how the value is encoded in bytes, see Value encoding and normalization.

`littleEndian` Optional

Indicates whether the data is stored in little- or big-endian format. If `false` or `undefined`, a big-endian value is written.

Return value

`undefined`.

Exceptions

`RangeError`

Thrown if the `byteOffset` is set such that it would store beyond the end of the view.

Examples

Using setUint16()

```js
const buffer = new ArrayBuffer(10);
const dataview = new DataView(buffer);
dataview.setUint16(0, 3);
dataview.getUint16(1); // 768
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/structured-data.html#sec-dataview.prototype.setuint16`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`setUint16`	9	12	15	12.1	5.1	18	15	12.1	5	1.0	4	5	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.DataView.setUint16`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- JavaScript typed arrays guide
- `DataView`

- ArrayBuffer
- Uint16Array

JavaScript › Reference › Standard built-in objects › DataView

DataView.prototype.setUint32()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/DataView/setUint32 · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **`setUint32()`** method of `DataView` instances takes a number and stores it as a 32-bit unsigned integer in the 4 bytes starting at the specified byte offset of this `DataView`. There is no alignment constraint; multi-byte values may be stored at any offset within bounds.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: DataView.prototype.setUint32()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
// Create an ArrayBuffer with a size in bytes
const buffer = new ArrayBuffer(16);

const view = new DataView(buffer);
view.setUint32(1, 4294967295); // Max unsigned 32-bit integer

console.log(view.getUint32(1));
// Expected output: 4294967295
```

Syntax

js

```
setUint32(byteOffset, value)
setUint32(byteOffset, value, littleEndian)
```

Parameters

`byteOffset`

The offset, in bytes, from the start of the view to store the data in.

`value`

The value to set. For how the value is encoded in bytes, see Value encoding and normalization.

`littleEndian` Optional

Indicates whether the data is stored in little- or big-endian format. If `false` or `undefined`, a big-endian value is written.

Return value

`undefined`.

Exceptions

`RangeError`

Thrown if the `byteOffset` is set such that it would store beyond the end of the view.

Examples

Using setUint32()

```js
const buffer = new ArrayBuffer(10);
const dataview = new DataView(buffer);
dataview.setUint32(0, 3);
dataview.getUint32(1); // 768
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/structured-data.html#sec-dataview.prototype.setuint32

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
setUint32	9	12	15	12.1	5.1	18	15	12.1	5	1.0	4	5	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.DataView.setUint32`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- JavaScript typed arrays guide
- `DataView`
- `ArrayBuffer`
- `Uint32Array`

JavaScript › Reference › Standard built-in objects › DataView

DataView.prototype.setUint8()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/DataView/setUint8 · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **`setUint8()`** method of `DataView` instances takes a number and stores it as an 8-bit unsigned integer in the byte at the specified byte offset of this `DataView`.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: DataView.prototype.setUint8()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
// Create an ArrayBuffer with a size in bytes
const buffer = new ArrayBuffer(16);

const view = new DataView(buffer);
view.setUint8(1, 255); // Max unsigned 8-bit integer

console.log(view.getUint8(1));
// Expected output: 255
```

Syntax

js

```
setUint8(byteOffset, value)
```

Parameters

`byteOffset`

The offset, in bytes, from the start of the view to store the data in.

`value`

The value to set. For how the value is encoded in bytes, see Value encoding and normalization.

Return value

`undefined`.

Exceptions

`RangeError`

Thrown if the `byteOffset` is set such that it would store beyond the end of the view.

Examples

Using setUint8()

js

```
const buffer = new ArrayBuffer(10);
const dataview = new DataView(buffer);
dataview.setUint8(0, 3);
dataview.getUint8(0); // 3
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/structured-data.html#sec-dataview.prototype.setuint8`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`setUint8`	9	12	15	12.1	5.1	18	15	12.1	5	1.0	4	5	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.DataView.setUint8`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- JavaScript typed arrays guide
- `DataView`
- `ArrayBuffer`
- `Uint8Array`

JavaScript › Reference › Standard built-in objects › DataView

DataView.prototype.buffer

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/DataView/buffer · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **`buffer`** accessor property of `DataView` instances returns the `ArrayBuffer` or `SharedArrayBuffer` referenced by this view at construction time.

Try it

▷ Interactive example on MDN: "JavaScript Demo: DataView.prototype.buffer". It runs live in the browser there; its source code follows.

js · interactive example source

```
// Create an ArrayBuffer
const buffer = new ArrayBuffer(123);

// Create a view
const view = new DataView(buffer);

console.log(view.buffer.byteLength);
// Expected output: 123
```

Description

The `buffer` property is an accessor property whose set accessor function is `undefined`, meaning that you can only read this property. The value is established when the `DataView` is constructed and cannot be changed.

Examples

Using the buffer property

```js
const buffer = new ArrayBuffer(8);
const dataview = new DataView(buffer);
dataview.buffer; // ArrayBuffer { byteLength: 8 }
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/structured-data.html#sec-get-dataview.prototype.buffer

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
buffer	9	12	15	12.1	5.1	18	15	12.1	5	1.0	4	5	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.DataView.buffer`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `DataView`
- `ArrayBuffer`
- `SharedArrayBuffer`

JavaScript › Reference › Standard built-in objects › DataView

DataView.prototype.byteLength

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/DataView/byteLength · MDN page last modified 2025-09-25

Baseline: Widely available — available across major browsers since July 2015.

The **`byteLength`** accessor property of `DataView` instances returns the length (in bytes) of this view.

Try it

▷ Interactive example on MDN: "JavaScript Demo: DataView.prototype.byteLength". It runs live in the browser there; its source code follows.

js · interactive example source

```
// Create an ArrayBuffer with a size in bytes
const buffer = new ArrayBuffer(16);

const view1 = new DataView(buffer);
const view2 = new DataView(buffer, 12, 4); // From byte 12 for the next 4 bytes

console.log(view1.byteLength + view2.byteLength); // 16 + 4
// Expected output: 20
```

Description

The `byteLength` property is an accessor property whose set accessor function is `undefined`, meaning that you can only read this property. If the `DataView` is length-tracking, then its length depends on the length of the underlying buffer, and may change if the buffer is resized. Otherwise, the value is established when the `DataView` is constructed and cannot be changed. Whether length-tracking or not, the `byteLength` becomes 0 if the underlying buffer is resized such that the viewed range is no longer valid.

Examples

Using the byteLength property

js

```
const buffer = new ArrayBuffer(8);
const dataview = new DataView(buffer);
dataview.byteLength; // 8 (matches the byteLength of the buffer)

const dataview2 = new DataView(buffer, 1, 5);
dataview2.byteLength; // 5 (as specified when constructing the DataView)

const dataview3 = new DataView(buffer, 2);
dataview3.byteLength; // 6 (due to the offset of the constructed DataView)

const buffer2 = new ArrayBuffer(16, { maxByteLength: 32 });
const dataviewLengthTracking = new DataView(buffer2, 4);
dataviewLengthTracking.byteLength; // 12 (16 - 4)
buffer2.resize(20);
dataviewLengthTracking.byteLength; // 16 (20 - 4)
buffer2.resize(3);
dataviewLengthTracking.byteLength; // 0 (viewed range is no longer valid)
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/structured-data.html#sec-get-dataview.prototype.bytelength`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
byteLength	9	12	15	12.1	5.1	18	15	12.1	5	1.0	4	5	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.DataView.byteLength`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- JavaScript typed arrays guide
- `DataView`
- `ArrayBuffer`
- `SharedArrayBuffer`

JavaScript › Reference › Standard built-in objects › DataView

DataView.prototype.byteOffset

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/DataView/byteOffset · MDN page last modified 2025-09-25

Baseline: Widely available — available across major browsers since July 2015.

The **`byteOffset`** accessor property of `DataView` instances returns the offset (in bytes) of this view from the start of its `ArrayBuffer` or `SharedArrayBuffer`.

Try it

▷ Interactive example on MDN: "JavaScript Demo: DataView.prototype.byteOffset". It runs live in the browser there; its source code follows.

js · interactive example source

```
// Create an ArrayBuffer with a size in bytes
const buffer = new ArrayBuffer(16);

const view = new DataView(buffer, 12, 4); // From byte 12 for the next 4 bytes

console.log(view.byteOffset);
// Expected output: 12
```

Description

The `byteOffset` property is an accessor property whose set accessor function is `undefined`, meaning that you can only read this property. The value is established when the `DataView` is constructed and cannot be changed. However, the `byteOffset` becomes 0 if the underlying buffer is resized such that the viewed range is no longer valid.

Examples

Using the byteOffset property

```js
const buffer = new ArrayBuffer(8);
const dataview = new DataView(buffer);
dataview.byteOffset; // 0 (no offset specified)

const dataview2 = new DataView(buffer, 3);
dataview2.byteOffset; // 3 (as specified when constructing the DataView)

const buffer2 = new ArrayBuffer(16, { maxByteLength: 32 });
const dataviewLengthTracking = new DataView(buffer2, 4);
dataviewLengthTracking.byteOffset; // 4
buffer2.resize(3);
dataviewLengthTracking.byteOffset; // 0 (viewed range is no longer valid)
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/structured-data.html#sec-get-dataview.prototype.byteoffset

Browser compatibility

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
byteOffset	9	12	15	12.1	5.1	18	15	12.1	5	1.0	4	5	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.DataView.byteOffset`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- JavaScript typed arrays guide
- `DataView`
- `ArrayBuffer`
- `SharedArrayBuffer`

JavaScript › Reference › Standard built-in objects

Date

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Date · MDN page last modified 2025-12-08

Baseline: Widely available — available across major browsers since July 2015.

JavaScript **`Date`** objects represent a single moment in time in a platform-independent format. `Date` objects encapsulate an integral number that represents milliseconds since the midnight at the beginning of January 1, 1970, UTC (the *epoch*).

> **Note:** With the introduction of the `Temporal` API, the `Date` object is considered a legacy feature. Consider using `Temporal` for new code and migrate existing code over to it if possible (check the browser compatibility. We will be writing a usage guide soon!

Description

The epoch, timestamps, and invalid date

A JavaScript date is fundamentally specified as the time in milliseconds that has elapsed since the epoch, which is defined as the midnight at the beginning of January 1, 1970, UTC (equivalent to the UNIX epoch). This timestamp is *timezone-agnostic* and uniquely defines an instant in history.

> **Note:** While the time value at the heart of a Date object is UTC, the basic methods to fetch the date and time or its components all work in the local (i.e., host system) time zone and offset.

The maximum timestamp representable by a `Date` object is slightly smaller than the maximum safe integer (`Number.MAX_SAFE_INTEGER`, which is 9,007,199,254,740,991). A `Date` object can represent a maximum of ±8,640,000,000,000,000 milliseconds, or ±100,000,000 (one hundred million) days, relative to the epoch. This is the range from April 20, 271821 BC to September 13, 275760 AD. Any attempt to represent a time outside this range results in the `Date` object holding a timestamp value of `NaN`, which is an "Invalid Date".

```js
console.log(new Date(8.64e15).toString()); // "Sat Sep 13 275760 00:00:00 GMT+0000
(Coordinated Universal Time)"
console.log(new Date(8.64e15 + 1).toString()); // "Invalid Date"
```

There are various methods that allow you to interact with the timestamp stored in the date:

- You can interact with the timestamp value directly using the `getTime()` and `setTime()` methods.
- The `valueOf()` and `[Symbol.toPrimitive]()` (when passed `"number"`) methods — which are automatically called in number coercion — return the timestamp, causing `Date` objects to behave like their timestamps when used in number contexts.
- All static methods (`Date.now()`, `Date.parse()`, and `Date.UTC()`) return timestamps instead of `Date` objects.
- The `Date()` constructor can be called with a timestamp as the only argument.

Date components and time zones

A date is represented internally as a single number, the *timestamp*. When interacting with it, the timestamp needs to be interpreted as a structured date-and-time representation. There are always two ways to interpret a timestamp: as a local time or as a Coordinated Universal Time (UTC), the global

standard time defined by the World Time Standard. The local timezone is not stored in the date object, but is determined by the host environment (user's device).

> **Note:** UTC should not be confused with the Greenwich Mean Time (GMT), because they are not always equal — this is explained in more detail in the linked Wikipedia page.

For example, the timestamp 0 represents a unique instant in history, but it can be interpreted in two ways:

- As a UTC time, it is midnight at the beginning of January 1, 1970, UTC,
- As a local time in New York (UTC-5), it is 19:00:00 on December 31, 1969.

The `getTimezoneOffset()` method returns the difference between UTC and the local time in minutes. Note that the timezone offset does not only depend on the current timezone, but also on the time represented by the `Date` object, because of daylight saving time and historical changes. In essence, the timezone offset is the offset from UTC time, at the time represented by the `Date` object and at the location of the host environment.

There are two groups of `Date` methods: one group gets and sets various date components by interpreting the timestamp as a local time, while the other uses UTC.

Component	Local		UTC	
	Get	Set	Get	Set
Year	`getFullYear()`	`setFullYear()`	`getUTCFullYear()`	`setUTCFullYear()`
Month	`getMonth()`	`setMonth()`	`getUTCMonth()`	`setUTCMonth()`
Date (of month)	`getDate()`	`setDate()`	`getUTCDate()`	`setUTCDate()`
Hours	`getHours()`	`setHours()`	`getUTCHours()`	`setUTCHours()`
Minutes	`getMinutes()`	`setMinutes()`	`getUTCMinutes()`	`setUTCMinutes()`
Seconds	`getSeconds()`	`setSeconds()`	`getUTCSeconds()`	`setUTCSeconds()`
Milliseconds	`getMilliseconds()`	`setMilliseconds()`	`getUTCMilliseconds()`	`setUTCMilliseconds()`
Day (of week)	`getDay()`	N/A	`getUTCDay()`	N/A

The `Date()` constructor can be called with two or more arguments, in which case they are interpreted as the year, month, day, hour, minute, second, and millisecond, respectively, in local time. `Date.UTC()` works similarly, but it interprets the components as UTC time and also accepts a single argument representing the year.

> **Note:** Some methods, including the `Date()` constructor, `Date.UTC()`, and the deprecated `getYear()`/`setYear()` methods, interpret a two-digit year as a year in the 1900s. For example, `new Date(99, 5, 24)` is interpreted as June 24, 1999, not June 24, 99. See Interpretation of two-digit years for more information.

When a segment overflows or underflows its expected range, it usually "carries over to" or "borrows from" the higher segment. For example, if the month is set to 12 (months are zero-based, so December is 11), it becomes the January of the next year. If the day of month is set to 0, it becomes the last day of the previous month. This also applies to dates specified with the date time string format.

When attempting to set the local time to a time falling within an offset transition (usually daylight saving time), the exact time is derived using the same behavior as `Temporal`'s `disambiguation:`

"compatible" option. That is, if the local time corresponds to two instants, the earlier one is chosen; if the local time does not exist (there is a gap), we go forward by the gap duration.

```js
// Assume America/New_York local time zone
// 2024-03-10 02:30 is within the spring-forward transition and does not exist
// 01:59 (UTC-5) jumps to 03:00 (UTC-4), so 02:30 moves forward by one hour
console.log(new Date(2024, 2, 10, 2, 30).toString());
// Sun Mar 10 2024 03:30:00 GMT-0400 (Eastern Daylight Time)

// 2024-11-03 01:30 is within the fall-back transition and exists twice
// 01:59 (UTC-4) jumps to 01:00 (UTC-5), so the earlier 01:30 (UTC-4) is chosen
console.log(new Date(2024, 10, 3, 1, 30).toString());
// Sun Nov 03 2024 01:30:00 GMT-0400 (Eastern Daylight Time)
```

Date time string format

There are many ways to format a date as a string. The JavaScript specification only specifies one format to be universally supported: the *date time string format*, a simplification of the ISO 8601 calendar date extended format. The format is as follows:

```
YYYY-MM-DDTHH:mm:ss.sssZ
```

- `YYYY` is the year, with four digits (`0000` to `9999`), or as an *expanded year* of `+` or `-` followed by six digits. The sign is required for expanded years. `-000000` is explicitly disallowed as a valid year.
- `MM` is the month, with two digits (`01` to `12`). Defaults to `01`.
- `DD` is the day of the month, with two digits (`01` to `31`). Defaults to `01`.
- `T` is a literal character, which indicates the beginning of the *time* part of the string. The `T` is required when specifying the time part.
- `HH` is the hour, with two digits (`00` to `23`). As a special case, `24:00:00` is allowed, and is interpreted as midnight at the beginning of the next day. Defaults to `00`.
- `mm` is the minute, with two digits (`00` to `59`). Defaults to `00`.
- `ss` is the second, with two digits (`00` to `59`). Defaults to `00`.
- `sss` is the millisecond, with three digits (`000` to `999`). Defaults to `000`.
- `Z` is the timezone offset, which can either be the literal character `Z` (indicating UTC), or `+` or `-` followed by `HH:mm`, the offset in hours and minutes from UTC.

Various components can be omitted, so the following are all valid:

- Date-only form: `YYYY`, `YYYY-MM`, `YYYY-MM-DD`
- Date-time form: one of the above date-only forms, followed by `T`, followed by `HH:mm`, `HH:mm:ss`, or `HH:mm:ss.sss`. Each combination can be followed by a time zone offset.

For example, `"2011-10-10"` (*date-only* form), `"2011-10-10T14:48:00"` (*date-time* form), or `"2011-10-10T14:48:00.000+09:00"` (*date-time* form with milliseconds and time zone) are all valid date time strings.

When the time zone offset is absent, **date-only forms are interpreted as a UTC time and date-time forms are interpreted as a local time.** The interpretation as a UTC time is due to a historical spec error that was not consistent with ISO 8601 but could not be changed due to web compatibility. See Broken Parser – A Web Reality Issue.

`Date.parse()` and the `Date()` constructor both accept strings in the date time string format as input. Furthermore, implementations are allowed to support other date formats when the input fails to match this format.

The `toISOString()` method returns a string representation of the date in the date time string format, with the time zone offset always set to `Z` (UTC).

> **Note:** You are encouraged to make sure your input conforms to the date time string format above for maximum compatibility, because support for other formats is not guaranteed. However, there are some formats that are supported in all major implementations — like RFC 2822 format — in which case their usage can be acceptable. Always conduct cross-browser tests to ensure your code works in all target browsers. A library can help if many different formats are to be accommodated.

Non-standard strings can be parsed in any way as desired by the implementation, including the time zone — most implementations use the local time zone by default. Implementations are not required to return invalid date for out-of-bounds date components, although they usually do. A string may have in-bounds date components (with the bounds defined above), but does not represent a date in reality (for example, "February 30"). Implementations behave inconsistently in this case. The `Date.parse()` page offers more examples about these non-standard cases.

Other ways to format a date

- `toISOString()` returns a string in the format `1970-01-01T00:00:00.000Z` (the date time string format introduced above, which is simplified ISO 8601). `toJSON()` calls `toISOString()` and returns the result.
- `toString()` returns a string in the format `Thu Jan 01 1970 00:00:00 GMT+0000 (Coordinated Universal Time)`, while `toDateString()` and `toTimeString()` return the date and time parts of the string, respectively. `[Symbol.toPrimitive]()` (when passed `"string"` or `"default"`) calls `toString()` and returns the result.
- `toUTCString()` returns a string in the format `Thu, 01 Jan 1970 00:00:00 GMT` (generalized RFC 7231).
- `toLocaleDateString()`, `toLocaleTimeString()`, and `toLocaleString()` use locale-specific date and time formats, usually provided by the `Intl` API.

See the Formats of `toString` method return values section for examples.

Constructor

`Date()`

When called as a constructor, returns a new `Date` object. When called as a function, returns a string representation of the current date and time.

Static methods

`Date.now()`

Returns the numeric value corresponding to the current time—the number of milliseconds since January 1, 1970 00:00:00 UTC, with leap seconds ignored.

`Date.parse()`

Parses a string representation of a date and returns the number of milliseconds since January 1, 1970 00:00:00 UTC, with leap seconds ignored.

`Date.UTC()`

Accepts the same parameters as the longest form of the constructor (i.e., 2 to 7) and returns the number of milliseconds since January 1, 1970 00:00:00 UTC, with leap seconds ignored.

Instance properties

These properties are defined on `Date.prototype` and shared by all `Date` instances.

`Date.prototype.constructor`

The constructor function that created the instance object. For `Date` instances, the initial value is the `Date` constructor.

Instance methods

`Date.prototype.getDate()`

Returns the day of the month (`1` – `31`) for the specified date according to local time.

`Date.prototype.getDay()`

Returns the day of the week (`0` – `6`) for the specified date according to local time.

`Date.prototype.getFullYear()`

Returns the year (4 digits for 4-digit years) of the specified date according to local time.

`Date.prototype.getHours()`

Returns the hour (`0` – `23`) in the specified date according to local time.

`Date.prototype.getMilliseconds()`

Returns the milliseconds (`0` – `999`) in the specified date according to local time.

`Date.prototype.getMinutes()`

Returns the minutes (`0` – `59`) in the specified date according to local time.

`Date.prototype.getMonth()`

Returns the month (`0` – `11`) in the specified date according to local time.

`Date.prototype.getSeconds()`

Returns the seconds (`0` – `59`) in the specified date according to local time.

`Date.prototype.getTime()`

Returns the numeric value of the specified date as the number of milliseconds since January 1, 1970 00:00:00 UTC. (Negative values are returned for prior times.)

`Date.prototype.getTimezoneOffset()`

Returns the time-zone offset in minutes for the current locale.

`Date.prototype.getUTCDate()`

Returns the day (date) of the month (`1` – `31`) in the specified date according to universal time.

`Date.prototype.getUTCDay()`

Returns the day of the week (`0` – `6`) in the specified date according to universal time.

`Date.prototype.getUTCFullYear()`

Returns the year (4 digits for 4-digit years) in the specified date according to universal time.

`Date.prototype.getUTCHours()`

Returns the hours (`0` – `23`) in the specified date according to universal time.

`Date.prototype.getUTCMilliseconds()`

Returns the milliseconds (`0` – `999`) in the specified date according to universal time.

`Date.prototype.getUTCMinutes()`

Returns the minutes (`0` – `59`) in the specified date according to universal time.

`Date.prototype.getUTCMonth()`

Returns the month (`0` – `11`) in the specified date according to universal time.

`Date.prototype.getUTCSeconds()`

Returns the seconds (`0` – `59`) in the specified date according to universal time.

`Date.prototype.getYear()` Deprecated

Returns the year (usually 2–3 digits) in the specified date according to local time. Use `getFullYear()` instead.

`Date.prototype.setDate()`

Sets the day of the month for a specified date according to local time.

`Date.prototype.setFullYear()`

Sets the full year (e.g., 4 digits for 4-digit years) for a specified date according to local time.

`Date.prototype.setHours()`

Sets the hours for a specified date according to local time.

`Date.prototype.setMilliseconds()`

Sets the milliseconds for a specified date according to local time.

`Date.prototype.setMinutes()`

Sets the minutes for a specified date according to local time.

`Date.prototype.setMonth()`

Sets the month for a specified date according to local time.

`Date.prototype.setSeconds()`

Sets the seconds for a specified date according to local time.

`Date.prototype.setTime()`

Sets the `Date` object to the time represented by the number of milliseconds since January 1, 1970 00:00:00 UTC. Use negative numbers for times prior.

`Date.prototype.setUTCDate()`

Sets the day of the month for a specified date according to universal time.

`Date.prototype.setUTCFullYear()`

Sets the full year (e.g., 4 digits for 4-digit years) for a specified date according to universal time.

`Date.prototype.setUTCHours()`

Sets the hour for a specified date according to universal time.

`Date.prototype.setUTCMilliseconds()`

Sets the milliseconds for a specified date according to universal time.

`Date.prototype.setUTCMinutes()`

Sets the minutes for a specified date according to universal time.

`Date.prototype.setUTCMonth()`

Sets the month for a specified date according to universal time.

`Date.prototype.setUTCSeconds()`

Sets the seconds for a specified date according to universal time.

`Date.prototype.setYear()` Deprecated

Sets the year (usually 2–3 digits) for a specified date according to local time. Use `setFullYear()` instead.

`Date.prototype.toDateString()`

Returns the "date" portion of the `Date` as a human-readable string like `'Thu Apr 12 2018'`.

`Date.prototype.toISOString()`

Converts a date to a string following the ISO 8601 Extended Format.

`Date.prototype.toJSON()`

Returns a string representing the `Date` using `toISOString()`. Intended to be implicitly called by `JSON.stringify()`.

`Date.prototype.toLocaleDateString()`

Returns a string with a locality sensitive representation of the date portion of this date based on system settings.

`Date.prototype.toLocaleString()`

Returns a string with a locality-sensitive representation of this date. Overrides the `Object.prototype.toLocaleString()` method.

`Date.prototype.toLocaleTimeString()`

Returns a string with a locality-sensitive representation of the time portion of this date, based on system settings.

`Date.prototype.toString()`

Returns a string representing the specified `Date` object. Overrides the `Object.prototype.toString()` method.

`Date.prototype.toTemporalInstant()`

Returns a new `Temporal.Instant` object with the same `epochMilliseconds` value as this date's timestamp.

`Date.prototype.toTimeString()`

Returns the "time" portion of the `Date` as a human-readable string.

`Date.prototype.toUTCString()`

Converts a date to a string using the UTC timezone.

`Date.prototype.valueOf()`

Returns the primitive value of a `Date` object. Overrides the `Object.prototype.valueOf()` method.

`Date.prototype[Symbol.toPrimitive]()`

Converts this `Date` object to a primitive value.

Examples

Several ways to create a Date object

The following examples show several ways to create JavaScript dates:

> **Note:** Creating a date from a string has a lot of behavior inconsistencies. See date time string format for caveats on using different formats.

```js
const today = new Date();
const birthday = new Date("December 17, 1995 03:24:00"); // DISCOURAGED: may not work in all runtimes
const birthday2 = new Date("1995-12-17T03:24:00"); // This is standardized and will work reliably
const birthday3 = new Date(1995, 11, 17); // the month is 0-indexed
const birthday4 = new Date(1995, 11, 17, 3, 24, 0);
const birthday5 = new Date(628021800000); // passing epoch timestamp
```

Formats of toString method return values

```js
const date = new Date("2020-05-12T23:50:21.817Z");
date.toString(); // Tue May 12 2020 18:50:21 GMT-0500 (Central Daylight Time)
date.toDateString(); // Tue May 12 2020
date.toTimeString(); // 18:50:21 GMT-0500 (Central Daylight Time)
date[Symbol.toPrimitive]("string"); // Tue May 12 2020 18:50:21 GMT-0500 (Central Daylight Time)

date.toISOString(); // 2020-05-12T23:50:21.817Z
date.toJSON(); // 2020-05-12T23:50:21.817Z

date.toUTCString(); // Tue, 12 May 2020 23:50:21 GMT

date.toLocaleString(); // 5/12/2020, 6:50:21 PM
date.toLocaleDateString(); // 5/12/2020
date.toLocaleTimeString(); // 6:50:21 PM
```

To get Date, Month and Year or Time

```js
const date = new Date("2000-01-17T16:45:30");
const [month, day, year] = [
  date.getMonth(),
  date.getDate(),
  date.getFullYear(),
];
// [0, 17, 2000] as month are 0-indexed
const [hour, minutes, seconds] = [
  date.getHours(),
  date.getMinutes(),
  date.getSeconds(),
];
// [16, 45, 30]
```

Interpretation of two-digit years

`new Date()` exhibits legacy undesirable, inconsistent behavior with two-digit year values; specifically, when a `new Date()` call is given a two-digit year value, that year value does not get treated as a literal year and used as-is but instead gets interpreted as a relative offset — in some cases as an offset from the year `1900`, but in other cases, as an offset from the year `2000`.

```js
let date = new Date(98, 1); // Sun Feb 01 1998 00:00:00 GMT+0000 (GMT)
date = new Date(22, 1); // Wed Feb 01 1922 00:00:00 GMT+0000 (GMT)
date = new Date("2/1/22"); // Tue Feb 01 2022 00:00:00 GMT+0000 (GMT)

// Legacy method; always interprets two-digit year values as relative to 1900
date.setYear(98);
date.toString(); // Sun Feb 01 1998 00:00:00 GMT+0000 (GMT)
date.setYear(22);
date.toString(); // Wed Feb 01 1922 00:00:00 GMT+0000 (GMT)
```

So, to create and get dates between the years `0` and `99`, instead use the preferred `setFullYear()` and `getFullYear()` methods:.

```js
// Preferred method; never interprets any value as being a relative offset,
// but instead uses the year value as-is
date.setFullYear(98);
date.getFullYear(); // 98 (not 1998)
date.setFullYear(22);
date.getFullYear(); // 22 (not 1922, not 2022)
```

Calculating elapsed time

The following examples show how to determine the elapsed time between two JavaScript dates in milliseconds.

Due to the differing lengths of days (due to daylight saving changeover), months, and years, expressing elapsed time in units greater than hours, minutes, and seconds requires addressing a number of issues, and should be thoroughly researched before being attempted.

```js
// Using Date objects
const start = Date.now();

// The event to time goes here:
doSomethingForALongTime();
const end = Date.now();
const elapsed = end - start; // elapsed time in milliseconds
```

```js
// Using built-in methods
const start = new Date();

// The event to time goes here:
doSomethingForALongTime();
const end = new Date();
const elapsed = end.getTime() - start.getTime(); // elapsed time in milliseconds
```

```js
// To test a function and get back its return
function printElapsedTime(testFn) {
  const startTime = Date.now();
  const result = testFn();
  const endTime = Date.now();

  console.log(`Elapsed time: ${String(endTime - startTime)} milliseconds`);
  return result;
}

const yourFunctionReturn = printElapsedTime(yourFunction);
```

> **Note:** In browsers that support the Performance API's high-resolution time feature, `Performance.now()` can provide more reliable and precise measurements of elapsed time than `Date.now()`.

Get the number of seconds since the ECMAScript Epoch

```js
const seconds = Math.floor(Date.now() / 1000);
```

In this case, it's important to return only an integer—so a simple division won't do. It's also important to only return actually elapsed seconds. (That's why this code uses `Math.floor()`, and *not* `Math.round()`.)

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-date-objects

Browser compatibility

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`Date`	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
[Symbol.toPrimitive]	47	15	44	34	10	47	44	34	10	5.0	47	10	1.0.0	1.0	6.0.0
`Date()` constructor	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`UTC`	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`monthIndex` defaults to 0	≤15	≤15	≤4	15	≤10.1	18	4	14	≤10.3	1.0	4.4	≤10.3	1.0.0	1.0	8.0.0
`getDate`	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`getDay`	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`getFullYear`	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`getHours`	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`getMilliseconds`	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`getMinutes`	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`getMonth`	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`getSeconds`	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`getTime`	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`getTimezoneOffset`	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`getUTCDate`	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`getUTCDay`	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`getUTCFullYear`	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`getUTCHours`	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`getUTCMilliseconds`	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`getUTCMinutes`	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`getUTCMonth`	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`getUTCSeconds`	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`getYear` Deprecated	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`now`	1	12	1	10.5	4	18	4	14	4	1.0	4.4	4	1.0.0	1.0	0.10.0
`parse`	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
ISO 8601 format	6	12	4	6	5.1	18	4	10.1	5	1.0	4.4	5	1.0.0	1.0	0.12.0
`setDate`	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`setFullYear`	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`setHours`	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`setMilliseconds`	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`setMinutes`	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`setMonth`	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`setSeconds`	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`setTime`	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`setUTCDate`	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`setUTCFullYear`	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`setUTCHours`	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`setUTCMilliseconds`	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`setUTCMinutes`	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`setUTCMonth`	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`setUTCSeconds`	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`setYear` Deprecated	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`toDateString`	1	12	1	5	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`toGMTString` Deprecated	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`toISOString`	3	12	1	10.5	4	18	4	11	3.2	1.0	4.4	3.2	1.0.0	1.0	0.10.0
`toJSON`	3	12	1	10.5	4	18	4	11	3.2	1.0	4.4	3.2	1.0.0	1.0	0.10.0
`toLocaleDateString`	1	12	1	5	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
IANA time zone names in `timeZone` option	24	14	52	15	7	25	56	14	7	1.5	4.4	7	1.0.0	1.8	0.12.0
`locales` parameter	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8[1,2]	13.0.0[3,4]
`options` parameter	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.0	0.12.0
`toLocaleString`	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
IANA time zone names in `timeZone` option	24	14	52	15	7	25	56	14	7	1.5	4.4	7	1.0.0	1.8	0.12.0

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`locales` parameter	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8[1,2]	13.0.0[3,4]
`options` parameter	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.0	0.12.0
`toLocaleTimeString`	1	12	1	5	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
IANA time zone names in `timeZone` option	24	14	52	15	7	25	56	14	7	1.5	4.4	7	1.0.0	1.8	0.12.0
`locales` parameter	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8[1,2]	13.0.0[3,4]
`options` parameter	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.0	0.12.0
`toString`	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`toTemporalInstant`	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0
`toTimeString`	1	12	1	5	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`toUTCString`	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`valueOf`	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

1. Also from version 1.0–1.8: partial support.
2. Also from version 1.0–1.8: Only the locale data for `en-US` is available.
3. Also from version 0.12.0–13.0.0: partial support.
4. Also from version 0.12.0–13.0.0: Before version 13.0.0, only the locale data for `en-US` is available by default. When other locales are specified, the function silently falls back to `en-US`. To make full ICU (locale) data available before version 13, see Node.js documentation on the `--with-intl` option and how to provide the data.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Date`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Date()`

JavaScript › Reference › Standard built-in objects › Date

Date() constructor

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Date/Date · MDN page last modified 2026-09-01

Baseline: Widely available — available across major browsers since July 2015.

The **`Date()`** constructor creates `Date` objects. When called as a function, it returns a string representing the current time.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Date() constructor". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const date1 = new Date("December 17, 1995 03:24:00");
// Sun Dec 17 1995 03:24:00 GMT...

const date2 = new Date("1995-12-17T03:24:00");
// Sun Dec 17 1995 03:24:00 GMT...

console.log(date1.getTime() === date2.getTime());
// Expected output: true
```

Syntax

js

```
new Date()
new Date(value)
new Date(dateString)
new Date(dateObject)

new Date(year, monthIndex)
new Date(year, monthIndex, day)
new Date(year, monthIndex, day, hours)
new Date(year, monthIndex, day, hours, minutes)
new Date(year, monthIndex, day, hours, minutes, seconds)
new Date(year, monthIndex, day, hours, minutes, seconds, milliseconds)

Date()
```

> **Note:** `Date()` can be called with or without `new`, but with different effects. See Return value.

Parameters

There are five basic forms for the `Date()` constructor:

No parameters

When no parameters are provided, the newly-created `Date` object represents the current date and time as of the time of instantiation. The returned date's timestamp is the same as the number returned by `Date.now()`.

Time value or timestamp number

`value`

An integer value representing the timestamp (the number of milliseconds since midnight at the beginning of January 1, 1970, UTC — a.k.a. the epoch).

Date string

`dateString`

A string value representing a date, parsed and interpreted using the same algorithm implemented by `Date.parse()`. See date time string format for caveats on using different formats.

Date object

`dateObject`

An existing `Date` object. This effectively makes a copy of the existing `Date` object with the same date and time. This is equivalent to `new Date(dateObject.valueOf())`, except the `valueOf()` method is not called.

When one parameter is passed to the `Date()` constructor, `Date` instances are specially treated. All other values are converted to primitives. If the result is a string, it will be parsed as a date string. Otherwise, the resulting primitive is further coerced to a number and treated as a timestamp.

Individual date and time component values

Given at least a year and month, this form of `Date()` returns a `Date` object whose component values (year, month, day, hour, minute, second, and millisecond) all come from the following parameters. Any missing fields are given the lowest possible value (`1` for `day` and `0` for every other component). The parameter values are all evaluated against the local time zone, rather than UTC. `Date.UTC()` accepts similar parameters but interprets the components as UTC and returns a timestamp.

If any parameter overflows its defined bounds, it "carries over". For example, if a `monthIndex` greater than `11` is passed in, those months will cause the year to increment; if a `minutes` greater than `59` is passed in, `hours` will increment accordingly, etc. Therefore, `new Date(1990, 12, 1)` will return January 1st, 1991; `new Date(2020, 5, 19, 25, 65)` will return 2:05 A.M. June 20th, 2020.

Similarly, if any parameter underflows, it "borrows" from the higher positions. For example, `new Date(2020, 5, 0)` will return May 31st, 2020.

`year`

Integer value representing the year. Values from `0` to `99` map to the years `1900` to `1999`. All other values are the actual year. See the example.

`monthIndex`

Integer value representing the month, beginning with `0` for January to `11` for December.

`day` Optional

Integer value representing the day of the month. Defaults to `1`.

`hours` Optional

Integer value between `0` and `23` representing the hour of the day. Defaults to `0`.

`minutes` Optional

Integer value representing the minute segment of a time. Defaults to `0`.

`seconds` Optional

Integer value representing the second segment of a time. Defaults to `0`.

`milliseconds` Optional

Integer value representing the millisecond segment of a time. Defaults to `0`.

Return value

Calling `new Date()` (the `Date()` constructor) returns a `Date` object. If called with an invalid date string, or if the date to be constructed will have a timestamp less than `-8,640,000,000,000,000` or greater than `8,640,000,000,000,000` milliseconds, it returns an invalid date (a `Date` object whose `toString()` method returns `"Invalid Date"` and `valueOf()` method returns `NaN`).

Calling the `Date()` function (without the `new` keyword) returns a string representation of the current date and time, exactly as `new Date().toString()` does. Any arguments given in a `Date()` function call (without the `new` keyword) are ignored; regardless of whether it's called with an invalid date string — or even called with any arbitrary object or other primitive as an argument — it always returns a string representation of the current date and time.

Description

Reduced time precision

To offer protection against timing attacks and fingerprinting, the precision of `new Date()` may be reduced depending on browser settings.

When a timestamp, date string, date object, or date component values are supplied to the constructor, the browser does not apply timer rounding to the supplied time.

When called without arguments, the constructor obtains the current time in the same way as `Date.now()`. It inherits the precision of that clock reading without introducing additional inaccuracy.

Examples

Several ways to create a Date object

The following examples show several ways to create JavaScript dates:

```js
const today = new Date();
const birthday = new Date("December 17, 1995 03:24:00"); // DISCOURAGED: may not work in all runtimes
const birthday = new Date("1995-12-17T03:24:00"); // This is standardized and will work reliably
const birthday = new Date(1995, 11, 17); // the month is 0-indexed
const birthday = new Date(1995, 11, 17, 3, 24, 0);
const birthday = new Date(628021800000); // passing epoch timestamp
```

Passing a non-Date, non-string, non-number value

If the `Date()` constructor is called with one parameter which is not a `Date` instance, it will be coerced to a primitive and then checked whether it's a string. For example, `new Date(undefined)` is different from `new Date()`:

```js
console.log(new Date(undefined)); // Invalid Date
```

This is because `undefined` is already a primitive but not a string, so it will be coerced to a number, which is `NaN` and therefore not a valid timestamp. On the other hand, `null` will be coerced to `0`.

```js
console.log(new Date(null)); // 1970-01-01T00:00:00.000Z
```

Arrays would be coerced to a string via `Array.prototype.toString()`, which joins the elements with commas. However, the resulting string for any array with more than one element is not a valid ISO 8601 date string, so its parsing behavior would be implementation-defined. **Do not pass arrays to the `Date()` constructor.**

```js
console.log(new Date(["2020-06-19", "17:13"]));
// 2020-06-19T17:13:00.000Z in Chrome, since it recognizes "2020-06-19,17:13"
// "Invalid Date" in Firefox
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-date-constructor

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`Date()` constructor	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Date.Date`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Date`

Date.now()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Date/now · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since July 2015.

The **`Date.now()`** static method returns the number of milliseconds elapsed since the epoch, which is defined as the midnight at the beginning of January 1, 1970, UTC.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Date.now()". It runs live in the browser there; its source code follows.

js · interactive example source

```
// This example takes 2 seconds to run
const start = Date.now();

console.log("starting timer...");
// Expected output: "starting timer..."

setTimeout(() => {
  const ms = Date.now() - start;

  console.log(`seconds elapsed = ${Math.floor(ms / 1000)}`);
  // Expected output: "seconds elapsed = 2"
}, 2000);
```

Syntax

```js
Date.now()
```

Parameters

None.

Return value

A number representing the timestamp, in milliseconds, of the current time.

Description

Reduced time precision

To offer protection against timing attacks and fingerprinting, the precision of `Date.now()` may be reduced depending on browser settings.

The timestamp is always an integer number of milliseconds, so its resolution is limited to 1 ms in all contexts. This already satisfies certain basic security and privacy needs.

In Firefox, the `privacy.reduceTimerPrecision` preference is enabled by default. With default settings, the timestamp has 1 ms resolution. If `privacy.resistFingerprinting` is enabled, the rounding interval is 16.667 ms or the interval configured by `privacy.resistFingerprinting.reduceTimerPrecision.microseconds`, whichever is larger. The final output is rounded to an integer.

For example, these are possible values in Firefox:

```js
// Reduced time precision (1 ms) with default settings
Date.now();
// Might be:
// 1519211809934
// 1519211810363
// 1519211811671
// …

// Reduced time precision with `privacy.resistFingerprinting` enabled
Date.now();
// Might be:
// 1519129853489
// 1519129853506
// 1519129853522
// …
```

Examples

Measuring time elapsed

You can use `Date.now()` to get the current time in milliseconds, then subtract a previous time to find out how much time elapsed between the two calls.

```js
const start = Date.now();
doSomeLongRunningProcess();
console.log(`Time elapsed: ${Date.now() - start} ms`);
```

For more complex scenarios, you may want to use the performance API instead.

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-date.now`

Browser compatibility

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
now	1	12	1	10.5	4	18	4	14	4	1.0	4.4	4	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Date.now`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Date.now` in `core-js`
- `Performance.now()`
- `console.time()`
- `console.timeEnd()`

JavaScript › Reference › Standard built-in objects › Date

Date.parse()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Date/parse · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **`Date.parse()`** static method parses a string representation of a date, and returns the date's timestamp.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Date.parse()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
// Standard date-time string format
const unixTimeZero = Date.parse("1970-01-01T00:00:00Z");
// Non-standard format resembling toUTCString()
const javaScriptRelease = Date.parse("04 Dec 1995 00:12:00 GMT");

console.log(unixTimeZero);
// Expected output: 0

console.log(javaScriptRelease);
// Expected output: 818035920000
```

Syntax

js

```
Date.parse(dateString)
```

Parameters

`dateString`

A string in the date time string format. See the linked reference for caveats on using different formats.

Return value

A number representing the timestamp of the given date. If `dateString` fails to be parsed as a valid date, `NaN` is returned.

Description

This function is useful for setting date values based on string values, for example in conjunction with the `setTime()` method.

The formats that `parse()` can handle are not explicitly specified, but there are a few invariants:

- The date time string format (produced by `toISOString()`) must be supported.
- If `x` is any Date whose milliseconds amount is zero, then `x.valueOf()` should be equal to any of the following: `Date.parse(x.toString())`, `Date.parse(x.toUTCString())`, `Date.parse(x.toISOString())`. This means the formats produced by `toString()` and `toUTCString()` should be supported too.
- The spec does *not* require support for the format produced by `toLocaleString()`. However, major engines all try to support `toLocaleString("en-US")` format.

Other formats are implementation-defined and may not work across all browsers. A library can help if many different formats are to be accommodated. In fact, the unreliability of `Date.parse()` is one of the motivations for the `Temporal` API to be introduced.

Because `parse()` is a static method of `Date`, you always use it as `Date.parse()`, rather than as a method of a `Date` object you created.

Examples

Using Date.parse()

The following calls all return `1546300800000`. The first will imply UTC time because it's date-only, and the others explicitly specify the UTC timezone.

```js
Date.parse("2019-01-01");
Date.parse("2019-01-01T00:00:00.000Z");
Date.parse("2019-01-01T00:00:00.000+00:00");
```

The following call, which does not specify a time zone will be set to 2019-01-01 at 00:00:00 in the local timezone of the system, because it has both date and time.

```js
Date.parse("2019-01-01T00:00:00");
```

toString() and toUTCString() formats

Apart from the standard date time string format, the `toString()` and `toUTCString()` formats are supported:

```js
// toString() format
Date.parse("Thu Jan 01 1970 00:00:00 GMT-0500 (Eastern Standard Time)");
// 18000000 in all implementations in all timezones

// toUTCString() format
Date.parse("Thu, 01 Jan 1970 00:00:00 GMT");
// 0 in all implementations in all timezones
```

Non-standard date strings

> **Note:** This section contains implementation-specific behavior that may be inconsistent across browsers or different versions of browsers. It is not meant to be a comprehensive browser compatibility table and you should always conduct your own tests before using any format in your code.

Implementations usually default to the local time zone when the date string is non-standard. For consistency, we will assume that the runtime uses the UTC timezone, and unless specified otherwise, the output will vary with the device's time zone. Daylight Saving Time (DST), of the local time zone, can also have an effect on this.

Here are some more examples of non-standard date strings. Browsers are very lenient when parsing date strings and may discard any part of a string that they cannot parse. For compatibility reasons, browsers often copy each other's behavior, so these handling patterns tend to propagate cross-browser. As previously stated, the following examples are for illustration only, and are not exhaustive by any means:

Description	Example	Chrome	Firefox	Safari
Single number	`0` (single-digit)	946684800000 (Jan 01 2000); NaN in Firefox ≤122		-62167219200000 (Jan 01 0000)
	`31` (two-digit)	NaN		-61188912000000 (Jan 01 0031)
	`999` (three-/four-digit)	-30641733102000 (Jan 01 0999)		
Date strings that use different separators	`1970-01-01` (standard)	0 in all time zones		
	`1970/01/01`	0		
	`1970,01,01`	0		NaN
	`1970 01 01`	0		NaN

Strings that look like `toString()`	`Thu Jan 01 1970 00:00:00`
	`Thu Jan 01 1970`
	`Jan 01 1970 00:00:00`
	`Jan 01 1970`
		0		

Strings that look like `toUTCString()`	`Thu, 01 Jan 1970 00:00:00`
	`Thu, 01 Jan 1970`
	`01 Jan 1970 00:00:00`
	`01 Jan 1970`
		0		

First date component is 2-digit	`01-02-03` (first segment can be valid month)	1041465600000 (Jan 02 2003)		-62132745600000 (Feb 03 0001) Note: Safari always assumes YY-MM-DD, but MM/DD/YY.
	`27-02-03` (first segment can be valid day but not month)	NaN		-61312291200000 (Feb 03 0027)
	`49-02-03` (first segment cannot be valid day and is <50)	2495923200000 (Feb 03 2049)		-60617980800000 (Feb 03 0049)
	`50-02-03` (first segment cannot be valid day and is ≥50)	-628300800000 (Feb 03 1950)		-60586444800000 (Feb 03 0050)

Out-of-bounds date components	`2014-25-23`
	`Mar 32, 2014`
	`2014/25/23`
		NaN		

	`2014-02-30`	1393718400000 (Mar 02 2014)		NaN
	`02/30/2014`	1393718400000		

Extraneous characters after the month name	`04 Dec 1995`
	`04 Decem 1995`
	`04 December 1995`
		818031600000		

	`04 DecFoo 1995`	818031600000 Only the first three characters are read. Firefox ≤121 reads up to the valid month name, thus returning NaN when it sees "F".		
	`04 De 1995`	NaN		

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-date.parse`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`parse`	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
ISO 8601 format	6	12	4	6	5.1	18	4	10.1	5	1.0	4.4	5	1.0.0	1.0	0.12.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Date.parse`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Date.UTC()`

JavaScript › Reference › Standard built-in objects › Date

Date.UTC()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Date/UTC · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **`Date.UTC()`** static method accepts parameters representing the date and time components similar to the `Date` constructor, but treats them as UTC. It returns the number of milliseconds since January 1, 1970, 00:00:00 UTC.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Date.UTC()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const utcDate1 = new Date(Date.UTC(96, 1, 2, 3, 4, 5));
const utcDate2 = new Date(Date.UTC(0, 0, 0, 0, 0, 0));

console.log(utcDate1.toUTCString());
// Expected output: "Fri, 02 Feb 1996 03:04:05 GMT"

console.log(utcDate2.toUTCString());
// Expected output: "Sun, 31 Dec 1899 00:00:00 GMT"
```

Syntax

```js
Date.UTC(year)
Date.UTC(year, monthIndex)
Date.UTC(year, monthIndex, day)
Date.UTC(year, monthIndex, day, hours)
Date.UTC(year, monthIndex, day, hours, minutes)
Date.UTC(year, monthIndex, day, hours, minutes, seconds)
Date.UTC(year, monthIndex, day, hours, minutes, seconds, milliseconds)
```

Parameters

`year`

Integer value representing the year. Values from `0` to `99` map to the years `1900` to `1999`. All other values are the actual year. See the example.

`monthIndex` Optional

Integer value representing the month, beginning with `0` for January to `11` for December. Defaults to `0`.

`day` Optional

Integer value representing the day of the month. Defaults to `1`.

`hours` Optional

Integer value between `0` and `23` representing the hour of the day. Defaults to `0`.

`minutes` Optional

Integer value representing the minute segment of a time. Defaults to `0`.

`seconds` Optional

Integer value representing the second segment of a time. Defaults to `0`.

`milliseconds` Optional

Integer value representing the millisecond segment of a time. Defaults to `0`.

Return value

A number representing the timestamp of the given date. Returns `NaN` if the date is invalid.

Description

Years between `0` and `99` are converted to a year in the 20th century (`1900 + year`). For example, `95` is converted to the year `1995`.

The `UTC()` method differs from the `Date()` constructor in three ways:

1. `Date.UTC()` uses universal time instead of the local time.
2. `Date.UTC()` returns a time value as a number instead of creating a `Date` object.
3. When passed a single number, `Date.UTC()` interprets it as a year instead of a timestamp.

If a parameter is outside of the expected range, the `UTC()` method updates the other parameters to accommodate the value. For example, if `15` is used for `monthIndex`, the year will be incremented by 1 (`year + 1`) and `3` will be used for the month.

Because `UTC()` is a static method of `Date`, you always use it as `Date.UTC()`, rather than as a method of a `Date` object you created.

Examples

Using Date.UTC()

The following statement creates a `Date` object with the arguments treated as UTC instead of local:

```js
const utcDate = new Date(Date.UTC(2018, 11, 1, 0, 0, 0));
```

Behavior of Date.UTC() with one argument

`Date.UTC()` when passed one argument used to have inconsistent behavior, because implementations only kept the behavior consistent with the `Date()` constructor, which does not interpret a single argument as the year number. Implementations are now required to treat omitted `monthIndex` as `0`, instead of coercing it to `NaN`.

```js
Date.UTC(2017); // 1483228800000
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-date.utc

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
UTC	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
monthIndex defaults to 0	≤15	≤15	≤4	15	≤10.1	18	4	14	≤10.3	1.0	4.4	≤10.3	1.0.0	1.0	8.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Date.UTC`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Date.parse()`
- `Date`

JavaScript › Reference › Standard built-in objects › Date

Date.prototype.getDate()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Date/getDate · MDN page last modified 2025-07-20

Baseline: Widely available — available across major browsers since July 2015.

The **`getDate()`** method of `Date` instances returns the day of the month for this date according to local time.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Date.prototype.getDate()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const birthday = new Date("August 19, 1975 23:15:30");
const date = birthday.getDate();

console.log(date);
// Expected output: 19
```

Syntax

js

```
getDate()
```

Parameters

None.

Return value

An integer, between 1 and 31, representing the day of the month for the given date according to local time. Returns `NaN` if the date is invalid.

Examples

Using getDate()

The `day` variable has value `25`, based on the value of the `Date` object `xmas95`.

js

```
const xmas95 = new Date("1995-12-25T23:15:30");
const day = xmas95.getDate();

console.log(day); // 25
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-date.prototype.getdate`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
getDate	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Date.getDate`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Date.prototype.getUTCDate()`
- `Date.prototype.getUTCDay()`
- `Date.prototype.setDate()`

JavaScript › Reference › Standard built-in objects › Date

Date.prototype.getDay()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Date/getDay · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **`getDay()`** method of `Date` instances returns the day of the week for this date according to local time, where 0 represents Sunday. For the day of the month, see `Date.prototype.getDate()`.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Date.prototype.getDay()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const birthday = new Date("August 19, 1975 23:15:30");
const day1 = birthday.getDay();
// Sunday - Saturday : 0 - 6

console.log(day1);
// Expected output: 2
```

Syntax

js

```
getDay()
```

Parameters

None.

Return value

An integer, between 0 and 6, representing the day of the week for the given date according to local time: 0 for Sunday, 1 for Monday, 2 for Tuesday, and so on. Returns `NaN` if the date is invalid.

Description

The return value of `getDay()` is zero-based, which is useful for indexing into arrays of days, for example:

```js
const valentines = new Date("1995-02-14");
const day = valentines.getDay();
const dayNames = ["Sunday", "Monday", "Tuesday" /* , … */];

console.log(dayNames[day]); // "Monday"
```

However, for the purpose of internationalization, you should prefer using `Intl.DateTimeFormat` with the `options` parameter instead.

```js
const options = { weekday: "long" };
console.log(new Intl.DateTimeFormat("en-US", options).format(valentines));
// "Monday"
console.log(new Intl.DateTimeFormat("de-DE", options).format(valentines));
// "Montag"
```

Examples

Using getDay()

The `weekday` variable has value `1`, based on the value of the `Date` object `xmas95`, because December 25, 1995 is a Monday.

```js
const xmas95 = new Date("1995-12-25T23:15:30");
const weekday = xmas95.getDay();

console.log(weekday); // 1
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-date.prototype.getday`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
getDay	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Date.getDay`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Date.prototype.getUTCDate()`
- `Date.prototype.getUTCDay()`
- `Date.prototype.setDate()`

JavaScript › Reference › Standard built-in objects › Date

Date.prototype.getFullYear()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Date/getFullYear · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **`getFullYear()`** method of `Date` instances returns the year for this date according to local time.

Use this method instead of the `getYear()` method.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Date.prototype.getFullYear()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const moonLanding = new Date("July 20, 69 00:20:18");

console.log(moonLanding.getFullYear());
// Expected output: 1969
```

Syntax

js

```
getFullYear()
```

Parameters

None.

Return value

An integer representing the year for the given date according to local time. Returns `NaN` if the date is invalid.

Description

Unlike `getYear()`, the value returned by `getFullYear()` is an absolute number. For dates between the years 1000 and 9999, `getFullYear()` returns a four-digit number, for example, 1995. Use this function to make sure a year is compliant with years after 2000.

Examples

Using getFullYear()

The `fullYear` variable has value `1995`, based on the value of the `Date` object `xmas95`.

js

```
const xmas95 = new Date("1995-12-25T23:15:30");
const fullYear = xmas95.getFullYear();

console.log(fullYear); // 1995
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-date.prototype.getfullyear

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
getFullYear	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Date.getFullYear`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Date.prototype.getUTCFullYear()`
- `Date.prototype.setFullYear()`
- `Date.prototype.getYear()`

JavaScript › Reference › Standard built-in objects › Date

Date.prototype.getHours()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Date/getHours · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **`getHours()`** method of `Date` instances returns the hours for this date according to local time.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Date.prototype.getHours()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const birthday = new Date("March 13, 08 04:20");

console.log(birthday.getHours());
// Expected output: 4
```

Syntax

```js
getHours()
```

Parameters

None.

Return value

An integer, between 0 and 23, representing the hours for the given date according to local time. Returns `NaN` if the date is invalid.

Examples

Using getHours()

The `hours` variable has value `23`, based on the value of the `Date` object `xmas95`.

```js
const xmas95 = new Date("1995-12-25T23:15:30");
const hours = xmas95.getHours();

console.log(hours); // 23
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-date.prototype.gethours`

Browser compatibility

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`getHours`	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Date.getHours`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Date.prototype.getUTCHours()`
- `Date.prototype.setHours()`

JavaScript › Reference › Standard built-in objects › Date

Date.prototype.getMilliseconds()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Date/getMilliseconds · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **`getMilliseconds()`** method of `Date` instances returns the milliseconds for this date according to local time.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Date.prototype.getMilliseconds()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const moonLanding = new Date("July 20, 69 00:20:18");
moonLanding.setMilliseconds(123);

console.log(moonLanding.getMilliseconds());
// Expected output: 123
```

Syntax

js

```
getMilliseconds()
```

Parameters

None.

Return value

An integer, between 0 and 999, representing the milliseconds for the given date according to local time. Returns `NaN` if the date is invalid.

Examples

Using getMilliseconds()

The `milliseconds` variable has value `0`, based on the value of the `Date` object `xmas95`, which doesn't specify the milliseconds component, so it defaults to 0.

js

```
const xmas95 = new Date("1995-12-25T23:15:30");
const milliseconds = xmas95.getMilliseconds();

console.log(milliseconds); // 0
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-date.prototype.getmilliseconds`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`getMilliseconds`	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Date.getMilliseconds`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Date.prototype.getUTCMilliseconds()`
- `Date.prototype.setMilliseconds()`

JavaScript › Reference › Standard built-in objects › Date

Date.prototype.getMinutes()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Date/getMinutes · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **`getMinutes()`** method of `Date` instances returns the minutes for this date according to local time.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Date.prototype.getMinutes()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const birthday = new Date("March 13, 08 04:20");

console.log(birthday.getMinutes());
// Expected output: 20
```

Syntax

js

```
getMinutes()
```

Parameters

None.

Return value

An integer, between 0 and 59, representing the minutes for the given date according to local time. Returns `NaN` if the date is invalid.

Examples

Using getMinutes()

The `minutes` variable has value `15`, based on the value of the `Date` object `xmas95`.

```js
const xmas95 = new Date("1995-12-25T23:15:30");
const minutes = xmas95.getMinutes();

console.log(minutes); // 15
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-date.prototype.getminutes

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
getMinutes	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Date.getMinutes`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Date.prototype.getUTCMinutes()`
- `Date.prototype.setMinutes()`

JavaScript › Reference › Standard built-in objects › Date

Date.prototype.getMonth()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Date/getMonth · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **`getMonth()`** method of `Date` instances returns the month for this date according to local time, as a zero-based value (where zero indicates the first month of the year).

Try it

▷ Interactive example on MDN: "JavaScript Demo: Date.prototype.getMonth()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const moonLanding = new Date("July 20, 69 00:20:18");

console.log(moonLanding.getMonth()); // (January gives 0)
// Expected output: 6
```

Syntax

js

```
getMonth()
```

Parameters

None.

Return value

An integer, between 0 and 11, representing the month for the given date according to local time: 0 for January, 1 for February, and so on. Returns `NaN` if the date is invalid.

Description

The return value of `getMonth()` is zero-based, which is useful for indexing into arrays of months, for example:

js

```
const valentines = new Date("1995-02-14");
const month = valentines.getMonth();
const monthNames = ["January", "February", "March" /* , … */];

console.log(monthNames[month]); // "February"
```

However, for the purpose of internationalization, you should prefer using `Intl.DateTimeFormat` with the `options` parameter instead.

js

```
const options = { month: "long" };
console.log(new Intl.DateTimeFormat("en-US", options).format(valentines));
// "February"
console.log(new Intl.DateTimeFormat("de-DE", options).format(valentines));
// "Februar"
```

Examples

Using getMonth()

The `month` variable has value `11`, based on the value of the `Date` object `xmas95`.

js

```
const xmas95 = new Date("1995-12-25T23:15:30");
const month = xmas95.getMonth();

console.log(month); // 11
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-date.prototype.getmonth

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
getMonth	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Date.getMonth`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Date.prototype.getUTCMonth()`
- `Date.prototype.setMonth()`

JavaScript › Reference › Standard built-in objects › Date

Date.prototype.getSeconds()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Date/getSeconds · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **`getSeconds()`** method of `Date` instances returns the seconds for this date according to local time.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Date.prototype.getSeconds()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const moonLanding = new Date("July 20, 69 00:20:18");

console.log(moonLanding.getSeconds());
// Expected output: 18
```

Syntax

```js
getSeconds()
```

Parameters

None.

Return value

An integer, between 0 and 59, representing the seconds for the given date according to local time. Returns `NaN` if the date is invalid.

Examples

Using getSeconds()

The `seconds` variable has value `30`, based on the value of the `Date` object `xmas95`.

```js
const xmas95 = new Date("1995-12-25T23:15:30");
const seconds = xmas95.getSeconds();

console.log(seconds); // 30
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-date.prototype.getseconds`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`getSeconds`	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Date.getSeconds`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Date.prototype.getUTCSeconds()`
- `Date.prototype.setSeconds()`

JavaScript › Reference › Standard built-in objects › Date

Date.prototype.getTime()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Date/getTime · MDN page last modified 2026-05-21

Baseline: Widely available — available across major browsers since July 2015.

The **`getTime()`** method of `Date` instances returns the number of milliseconds for this date since the epoch, which is defined as the midnight at the beginning of January 1, 1970, UTC.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Date.prototype.getTime()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const moonLanding = new Date("July 20, 69 20:17:40 GMT+00:00");

// Milliseconds since Jan 1, 1970, 00:00:00.000 GMT
console.log(moonLanding.getTime());
// Expected output: -14182940000
```

Syntax

js

```
getTime()
```

Parameters

None.

Return value

A number representing the timestamp, in milliseconds, of this date. Returns `NaN` if the date is invalid.

Description

`Date` objects are fundamentally represented by a timestamp, and this method allows you to retrieve the timestamp. You can use this method to help assign a date and time to another `Date` object. This method is functionally equivalent to the `valueOf()` method.

Examples

Using getTime() for copying dates

Constructing a date object with the identical time value.

js

```
// Since month is zero based, birthday will be January 10, 1995
const birthday = new Date(1994, 12, 10);
const copy = new Date();
copy.setTime(birthday.getTime());
```

Measuring execution time

Subtracting two subsequent `getTime()` calls on newly generated `Date` objects, give the time span between these two calls. This can be used to calculate the executing time of some operations. See also `Date.now()` to prevent instantiating unnecessary `Date` objects.

```js
let end, start;

start = new Date();
for (let i = 0; i < 1000; i++) {
  Math.sqrt(i);
}
end = new Date();

console.log(`Operation took ${end.getTime() - start.getTime()} msec`);
```

> **Note:** In browsers that support the Performance API's high-resolution time feature, `Performance.now()` can provide more reliable and precise measurements of elapsed time than `Date.now()`.

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-date.prototype.gettime`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`getTime`	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Date.getTime`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Date.prototype.setTime()`
- `Date.prototype.valueOf()`
- `Date.now()`

JavaScript › Reference › Standard built-in objects › Date

Date.prototype.getTimezoneOffset()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Date/getTimezoneOffset · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **`getTimezoneOffset()`** method of `Date` instances returns the difference, in minutes, between this date as evaluated in the UTC time zone, and the same date as evaluated in the local time zone.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Date.prototype.getTimezoneOffset()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const date1 = new Date("August 19, 1975 23:15:30 GMT+07:00");
const date2 = new Date("August 19, 1975 23:15:30 GMT-02:00");

console.log(date1.getTimezoneOffset());
// Expected output: your local timezone offset in minutes
// (e.g., -120). NOT the timezone offset of the date object.

console.log(date1.getTimezoneOffset() === date2.getTimezoneOffset());
// Expected output: true
```

Syntax

js

```
getTimezoneOffset()
```

Parameters

None.

Return value

A number representing the difference, in minutes, between the date as evaluated in the UTC time zone and as evaluated in the local time zone. The actual local time algorithm is implementation-defined, and the return value is allowed to be zero in runtimes without appropriate data. Returns `NaN` if the date is invalid.

Description

`date.getTimezoneOffset()` returns the difference, in minutes, between `date` as evaluated in the UTC time zone and as evaluated in the local time zone — that is, the time zone of the host system in which the browser is being used (if the code is run from the Web in a browser), or otherwise the host system of whatever JavaScript runtime (for example, a Node.js environment) the code is executed in.

Negative values and positive values

The number of minutes returned by `getTimezoneOffset()` is positive if the local time zone is behind UTC, and negative if the local time zone is ahead of UTC. For example, for UTC+10, `-600` will be returned.

Current time zone	Return value
UTC-8	480
UTC	0
UTC+3	-180

Varied results in Daylight Saving Time (DST) regions

In a region that annually shifts in and out of Daylight Saving Time (DST), as `date` varies, the number of minutes returned by calling `getTimezoneOffset()` can be non-uniform.

> **Note:** `getTimezoneOffset()`'s behavior will never differ based on the time when the code is run — its behavior is always consistent when running in the same region. Only the value of `date` affects the result.

> **Note:** Many countries have experimented with not changing the time twice a year and this has meant that DST has continued over the winter too. For example in the UK DST lasted from 2:00AM 18 February 1968 to 3:00AM 31 October 1971, so during the winter the clocks were not set back.

In most implementations, the IANA time zone database (tzdata) is used to precisely determine the offset of the local timezone at the moment of the `date`. However, if such information is unavailable, an implementation may return zero.

Examples

Using getTimezoneOffset()

```js
// Create a Date instance for the current time
const currentLocalDate = new Date();
// Create a Date instance for 03:24 GMT-0200 on May 1st in 2016
const laborDay2016at0324GMTminus2 = new Date("2016-05-01T03:24:00-02:00");
currentLocalDate.getTimezoneOffset() ===
  laborDay2016at0324GMTminus2.getTimezoneOffset();
// true, always, in any timezone that doesn't annually shift in and out of DST
// false, sometimes, in any timezone that annually shifts in and out of DST
```

getTimezoneOffset() and DST

In regions that use DST, the return value may change based on the time of the year `date` is in. Below is the output in a runtime in New York, where the timezone is UTC-05:00.

```js
const nyOffsetSummer = new Date("2022-02-01").getTimezoneOffset(); // 300
const nyOffsetWinter = new Date("2022-08-01").getTimezoneOffset(); // 240
```

getTimezoneOffset() and historical data

Due to historical reasons, the timezone a region is in can be constantly changing, even disregarding DST. For example, below is the output in a runtime in Shanghai, where the timezone is UTC+08:00.

```js
const shModernOffset = new Date("2022-01-27").getTimezoneOffset(); // -480
const shHistoricalOffset = new Date("1943-01-27").getTimezoneOffset(); // -540
```

This is because during the Sino-Japanese War when Shanghai was under Japanese control, the timezone was changed to UTC+09:00 to align with Japan's (in effect, it was a "year-round DST"), and this was recorded in the IANA database.

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-date.prototype.gettimezoneoffset`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`getTimezoneOffset`	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Date.getTimezoneOffset`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Date`

JavaScript › Reference › Standard built-in objects › Date

Date.prototype.getUTCDate()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Date/getUTCDate · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **`getUTCDate()`** method of `Date` instances returns the day of the month for this date according to universal time.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Date.prototype.getUTCDate()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const date1 = new Date("August 19, 1975 23:15:30 GMT+11:00");
const date2 = new Date("August 19, 1975 23:15:30 GMT-11:00");

console.log(date1.getUTCDate());
// Expected output: 19

console.log(date2.getUTCDate());
// Expected output: 20
```

Syntax

```js
getUTCDate()
```

Parameters

None.

Return value

An integer, between 1 and 31, representing day of month for the given date according to universal time. Returns `NaN` if the date is invalid.

Examples

Using getUTCDate()

The following example assigns the day of month of the current date to the variable `dayOfMonth`.

```js
const today = new Date();
const dayOfMonth = today.getUTCDate();
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-date.prototype.getutcdate

Browser compatibility

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
getUTCDate	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Date.getUTCDate`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Date.prototype.getUTCDay()`
- `Date.prototype.getDay()`
- `Date.prototype.setUTCDate()`

JavaScript › Reference › Standard built-in objects › Date

Date.prototype.getUTCDay()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Date/getUTCDay · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **`getUTCDay()`** method of `Date` instances returns the day of the week for this date according to universal time, where 0 represents Sunday.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Date.prototype.getUTCDay()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const date1 = new Date("August 19, 1975 23:15:30 GMT+11:00");
const date2 = new Date("August 19, 1975 23:15:30 GMT-11:00");

// Tuesday
console.log(date1.getUTCDay());
// Expected output: 2

// Wednesday
console.log(date2.getUTCDay());
// Expected output: 3
```

Syntax

js

```
getUTCDay()
```

Parameters

None.

Return value

An integer corresponding to the day of the week for the given date according to universal time: 0 for Sunday, 1 for Monday, 2 for Tuesday, and so on. Returns `NaN` if the date is invalid.

Examples

Using getUTCDay()

The following example assigns the weekday portion of the current date to the variable `weekday`.

js

```
const today = new Date();
const weekday = today.getUTCDay();
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-date.prototype.getutcday`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`getUTCDay`	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Date.getUTCDay`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Date.prototype.getUTCDate()`
- `Date.prototype.getDay()`
- `Date.prototype.setUTCDate()`

JavaScript › Reference › Standard built-in objects › Date

Date.prototype.getUTCFullYear()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Date/getUTCFullYear · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **`getUTCFullYear()`** method of `Date` instances returns the year for this date according to universal time.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Date.prototype.getUTCFullYear()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const date1 = new Date("December 31, 1975, 23:15:30 GMT+11:00");
const date2 = new Date("December 31, 1975, 23:15:30 GMT-11:00");

console.log(date1.getUTCFullYear());
// Expected output: 1975

console.log(date2.getUTCFullYear());
// Expected output: 1976
```

Syntax

```
getUTCFullYear()
```

Parameters

None.

Return value

An integer representing the year for the given date according to universal time. Returns `NaN` if the date is invalid.

Description

Unlike `getYear()`, the value returned by `getUTCFullYear()` is an absolute number. For dates between the years 1000 and 9999, `getFullYear()` returns a four-digit number, for example, 1995. Use this function to make sure a year is compliant with years after 2000.

Examples

Using getUTCFullYear()

The following example assigns the four-digit value of the current year to the variable `year`.

```
const today = new Date();
const year = today.getUTCFullYear();
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-date.prototype.getutcfullyear

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
getUTCFullYear	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Date.getUTCFullYear`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Date.prototype.getFullYear()`
- `Date.prototype.setFullYear()`

JavaScript › Reference › Standard built-in objects › Date

Date.prototype.getUTCHours()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Date/getUTCHours · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **`getUTCHours()`** method of `Date` instances returns the hours for this date according to universal time.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Date.prototype.getUTCHours()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const date1 = new Date("December 31, 1975, 23:15:30 GMT+11:00");
const date2 = new Date("December 31, 1975, 23:15:30 GMT-11:00");

console.log(date1.getUTCHours());
// Expected output: 12

console.log(date2.getUTCHours());
// Expected output: 10
```

Syntax

js

```
getUTCHours()
```

Parameters

None.

Return value

An integer, between 0 and 23, representing the hours for the given date according to universal time. Returns `NaN` if the date is invalid.

Examples

Using getUTCHours()

The following example assigns the hours portion of the current time to the variable `hours`.

js

```
const today = new Date();
const hours = today.getUTCHours();
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-date.prototype.getutchours

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
getUTCHours	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Date.getUTCHours`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Date.prototype.getHours()`
- `Date.prototype.setUTCHours()`

JavaScript › Reference › Standard built-in objects › Date

Date.prototype.getUTCMilliseconds()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Date/getUTCMilliseconds · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **`getUTCMilliseconds()`** method of `Date` instances returns the milliseconds for this date according to universal time.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Date.prototype.getUTCMilliseconds()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const exampleDate = new Date("2018-01-02T03:04:05.678Z"); // 2 January 2018, 03:04:05.678
(UTC)

console.log(exampleDate.getUTCMilliseconds());
// Expected output: 678
```

Syntax

js

```
getUTCMilliseconds()
```

Parameters

None.

Return value

An integer, between 0 and 999, representing the milliseconds for the given date according to universal time. Returns `NaN` if the date is invalid.

Not to be confused with the timestamp. To get the total milliseconds since the epoch, use the `getTime()` method.

Examples

Using getUTCMilliseconds()

The following example assigns the milliseconds portion of the current time to the variable `milliseconds`.

```js
const today = new Date();
const milliseconds = today.getUTCMilliseconds();
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-date.prototype.getutcmilliseconds

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`getUTCMilliseconds`	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Date.getUTCMilliseconds`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Date.prototype.getMilliseconds()`
- `Date.prototype.setUTCMilliseconds()`

JavaScript › Reference › Standard built-in objects › Date

Date.prototype.getUTCMinutes()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Date/getUTCMinutes · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **`getUTCMinutes()`** method of `Date` instances returns the minutes for this date according to universal time.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Date.prototype.getUTCMinutes()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const date1 = new Date("1 January 2000 03:15:30 GMT+07:00");
const date2 = new Date("1 January 2000 03:15:30 GMT+03:30");

console.log(date1.getUTCMinutes()); // 31 Dec 1999 20:15:30 GMT
// Expected output: 15

console.log(date2.getUTCMinutes()); // 31 Dec 1999 23:45:30 GMT
// Expected output: 45
```

Syntax

js

```
getUTCMinutes()
```

Parameters

None.

Return value

An integer, between 0 and 59, representing the minutes for the given date according to universal time. Returns `NaN` if the date is invalid.

Examples

Using getUTCMinutes()

The following example assigns the minutes portion of the current time to the variable `minutes`.

js

```
const today = new Date();
const minutes = today.getUTCMinutes();
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-date.prototype.getutcminutes

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`getUTCMinutes`	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Date.getUTCMinutes`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Date.prototype.getMinutes()`
- `Date.prototype.setUTCMinutes()`

JavaScript › Reference › Standard built-in objects › Date

Date.prototype.getUTCMonth()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Date/getUTCMonth · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **`getUTCMonth()`** method of `Date` instances returns the month for this date according to universal time, as a zero-based value (where zero indicates the first month of the year).

Try it

▷ Interactive example on MDN: "JavaScript Demo: Date.prototype.getUTCMonth()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const date1 = new Date("December 31, 1975, 23:15:30 GMT+11:00");
const date2 = new Date("December 31, 1975, 23:15:30 GMT-11:00");

// December
console.log(date1.getUTCMonth());
// Expected output: 11

// January
console.log(date2.getUTCMonth());
// Expected output: 0
```

Syntax

js

```
getUTCMonth()
```

Parameters

None.

Return value

An integer, between 0 and 11, representing the month for the given date according to universal time: 0 for January, 1 for February, and so on. Returns `NaN` if the date is invalid.

Examples

Using getUTCMonth()

The following example assigns the month portion of the current date to the variable `month`.

```js
const today = new Date();
const month = today.getUTCMonth();
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-date.prototype.getutcmonth

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
getUTCMonth	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Date.getUTCMonth`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Date.prototype.getMonth()`
- `Date.prototype.setUTCMonth()`

JavaScript › Reference › Standard built-in objects › Date

Date.prototype.getUTCSeconds()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Date/getUTCSeconds · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **`getUTCSeconds()`** method of `Date` instances returns the seconds in the specified date according to universal time.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Date.prototype.getUTCSeconds()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const moonLanding = new Date("July 20, 1969, 20:18:04 UTC");

console.log(moonLanding.getUTCSeconds());
// Expected output: 4
```

Syntax

js

```
getUTCSeconds()
```

Parameters

None.

Return value

An integer, between 0 and 59, representing the seconds for the given date according to universal time. Returns `NaN` if the date is invalid.

Examples

Using getUTCSeconds()

The following example assigns the seconds portion of the current time to the variable `seconds`.

js

```
const today = new Date();
const seconds = today.getUTCSeconds();
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-date.prototype.getutcseconds`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
getUTCSeconds	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Date.getUTCSeconds`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Date.prototype.getSeconds()`
- `Date.prototype.setUTCSeconds()`

Date.prototype.getYear()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Date/getYear · MDN page last modified 2026-10-26

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`getYear()`** method of `Date` instances returns the year for this date according to local time. Because `getYear()` does not return full years ("year 2000 problem"), it is deprecated and has been replaced by the `getFullYear()` method.

Syntax

```js
getYear()
```

Parameters

None.

Return value

An integer representing the year for the given date according to local time, minus 1900. Returns `NaN` if the date is invalid.

- For years greater than or equal to 2000, the value is 100 or greater. For example, if the year is 2026, `getYear()` returns 126.
- For years between and including 1900 and 1999, the value returned by `getYear()` is between 0 and 99. For example, if the year is 1976, `getYear()` returns 76.
- For years less than 1900, the value returned by `getYear()` is less than 0. For example, if the year is 1800, `getYear()` returns -100.

This method essentially returns the value of `getFullYear()` minus 1900. You should use `getFullYear()` instead, so that the year is specified in full.

Examples

Years between 1900 and 1999

The second statement assigns the value 95 to the variable `year`.

```js
const xmas = new Date("1995-12-25");
const year = xmas.getYear(); // returns 95
```

Years above 1999

The second statement assigns the value 100 to the variable `year`.

```js
const xmas = new Date("2000-12-25");
const year = xmas.getYear(); // returns 100
```

Years below 1900

The second statement assigns the value -100 to the variable `year`.

```js
const xmas = new Date("1800-12-25");
const year = xmas.getYear(); // returns -100
```

Setting and getting a year between 1900 and 1999

The third statement assigns the value 95 to the variable `year`, representing the year 1995.

```js
const xmas = new Date("2015-12-25");
xmas.setYear(95);
const year = xmas.getYear(); // returns 95
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/additional-ecmascript-features-for-web-browsers.html#sec-date.prototype.getyear`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`getYear` Deprecated	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Date.getYear`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Polyfill of `Date.prototype.getYear` in `core-js`
- es-shims polyfill of `Date.prototype.getYear`
- `Date.prototype.getFullYear()`
- `Date.prototype.getUTCFullYear()`
- `Date.prototype.setYear()`

JavaScript › Reference › Standard built-in objects › Date

Date.prototype.setDate()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Date/setDate · MDN page last modified 2025-07-30

Baseline: Widely available — available across major browsers since July 2015.

The **`setDate()`** method of `Date` instances changes the day of the month for this date according to local time.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Date.prototype.setDate()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const event = new Date("August 19, 1975 23:15:30");

event.setDate(24);

console.log(event.getDate());
// Expected output: 24

event.setDate(32);
// Only 31 days in August!

console.log(event.getDate());
// Expected output: 1
```

Syntax

js

```
setDate(dateValue)
```

Parameters

`dateValue`

An integer representing the day of the month.

Return value

Changes the `Date` object in place, and returns its new timestamp. If `dateValue` is `NaN` (or other values that get coerced to `NaN`, such as `undefined`), the date is set to Invalid Date and `NaN` is returned.

Description

If you specify a number outside the expected range, the date information in the `Date` object is updated accordingly. For example, if the `Date` object holds June 1st, a `dateValue` of 40 changes the date to July 10th, while a `dateValue` of 0 changes the date to the last day of the previous month, May 31st.

Because `setDate()` operates on the local time, crossing a Daylight Saving Time (DST) boundary may result in a different elapsed time than expected. For example, if setting the date crosses a spring-forward transition (losing an hour), the difference in timestamps between the new and old date is one hour less than the nominal day difference multiplied by 24 hours. Conversely, crossing a fall-back

transition (gaining an hour) result in an extra hour. If you need to adjust the date by a fixed amount of time, consider using `setUTCDate()` or `setTime()`.

If the new local time falls within an offset transition, the exact time is derived using the same behavior as `Temporal`'s `disambiguation: "compatible"` option. That is, if the local time corresponds to two instants, the earlier one is chosen; if the local time does not exist (there is a gap), we go forward by the gap duration.

Examples

Using setDate()

```js
const theBigDay = new Date(1962, 6, 7, 12); // noon of 1962-07-07 (7th of July 1962, month is 0-indexed)
const theBigDay2 = new Date(theBigDay).setDate(24); // 1962-07-24 (24th of July 1962)
const theBigDay3 = new Date(theBigDay).setDate(32); // 1962-08-01 (1st of August 1962)
const theBigDay4 = new Date(theBigDay).setDate(22); // 1962-07-22 (22nd of July 1962)
const theBigDay5 = new Date(theBigDay).setDate(0); // 1962-06-30 (30th of June 1962)
const theBigDay6 = new Date(theBigDay).setDate(98); // 1962-10-06 (6th of October 1962)
const theBigDay7 = new Date(theBigDay).setDate(-50); // 1962-05-11 (11th of May 1962)
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-date.prototype.setdate

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`setDate`	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Date.setDate`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Date()`
- `Date.prototype.getDate()`
- `Date.prototype.setUTCDate()`

JavaScript › Reference › Standard built-in objects › Date

Date.prototype.setFullYear()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Date/setFullYear · MDN page last modified 2025-07-30

Baseline: Widely available — available across major browsers since July 2015.

The **`setFullYear()`** method of `Date` instances changes the year, month, and/or day of month for this date according to local time.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Date.prototype.setFullYear()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const event = new Date("August 19, 1975 23:15:30");

event.setFullYear(1969);

console.log(event.getFullYear());
// Expected output: 1969

event.setFullYear(0);

console.log(event.getFullYear());
// Expected output: 0
```

Syntax

js

```
setFullYear(yearValue)
setFullYear(yearValue, monthValue)
setFullYear(yearValue, monthValue, dateValue)
```

Parameters

`yearValue`

An integer representing the year. For example, 1995.

`monthValue` Optional

An integer representing the month: 0 for January, 1 for February, and so on.

`dateValue` Optional

An integer between 1 and 31 representing the day of the month. If you specify `dateValue`, you must also specify `monthValue`.

Return value

Changes the `Date` object in place, and returns its new timestamp. If a parameter is `NaN` (or other values that get coerced to `NaN`, such as `undefined`), the date is set to Invalid Date and `NaN` is returned.

Description

If you do not specify the `monthValue` and `dateValue` parameters, the same values as what are returned by `getMonth()` and `getDate()` are used.

If a parameter you specify is outside of the expected range, other parameters and the date information in the `Date` object are updated accordingly. For example, if you specify 15 for `monthValue`, the year is incremented by 1 (`yearValue + 1`), and 3 is used for the month.

Because `setFullYear()` operates on the local time, crossing a Daylight Saving Time (DST) boundary may result in a different elapsed time than expected. For example, if setting the date crosses a spring-forward transition (losing an hour), the difference in timestamps between the new and old date is one hour less than the nominal day difference multiplied by 24 hours. Conversely, crossing a fall-back transition (gaining an hour) result in an extra hour. If you need to adjust the date by a fixed amount of time, consider using `setUTCFullYear()` or `setTime()`.

If the new local time falls within an offset transition, the exact time is derived using the same behavior as `Temporal`'s `disambiguation: "compatible"` option. That is, if the local time corresponds to two instants, the earlier one is chosen; if the local time does not exist (there is a gap), we go forward by the gap duration.

Examples

Using setFullYear()

```js
const theBigDay = new Date();
theBigDay.setFullYear(1997);
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-date.prototype.setfullyear`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`setFullYear`	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Date.setFullYear`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Date.prototype.getUTCFullYear()
- Date.prototype.setUTCFullYear()
- Date.prototype.setYear()

JavaScript › Reference › Standard built-in objects › Date

Date.prototype.setHours()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Date/setHours · MDN page last modified 2025-07-30

Baseline: Widely available — available across major browsers since July 2015.

The **setHours()** method of Date instances changes the hours, minutes, seconds, and/or milliseconds for this date according to local time.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Date.prototype.setHours()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const event = new Date("August 19, 1975 23:15:30");
event.setHours(20);

console.log(event);
// Expected output: "Tue Aug 19 1975 20:15:30 GMT+0200 (CEST)"
// Note: your timezone may vary

event.setHours(20, 21, 22);

console.log(event);
// Expected output: "Tue Aug 19 1975 20:21:22 GMT+0200 (CEST)"
```

Syntax

js

```
setHours(hoursValue)
setHours(hoursValue, minutesValue)
setHours(hoursValue, minutesValue, secondsValue)
setHours(hoursValue, minutesValue, secondsValue, msValue)
```

Parameters

hoursValue

An integer between 0 and 23 representing the hours.

minutesValue Optional

An integer between 0 and 59 representing the minutes.

secondsValue Optional

An integer between 0 and 59 representing the seconds. If you specify secondsValue, you must also specify minutesValue.

`msValue` Optional

An integer between 0 and 999 representing the milliseconds. If you specify `msValue`, you must also specify `minutesValue` and `secondsValue`.

Return value

Changes the `Date` object in place, and returns its new timestamp. If a parameter is `NaN` (or other values that get coerced to `NaN`, such as `undefined`), the date is set to Invalid Date and `NaN` is returned.

Description

If you do not specify the `minutesValue`, `secondsValue`, and `msValue` parameters, the same values as what are returned by `getMinutes()`, `getSeconds()`, and `getMilliseconds()` are used.

If a parameter you specify is outside of the expected range, other parameters and the date information in the `Date` object are updated accordingly. For example, if you specify 100 for `secondsValue`, the minutes are incremented by 1 (`minutesValue + 1`), and 40 is used for seconds.

Because `setHours()` operates on the local time, crossing a Daylight Saving Time (DST) boundary may result in a different elapsed time than expected. For example, if setting the hours crosses a spring-forward transition (losing an hour), the difference in timestamps between the new and old date is one hour less than the nominal hour difference. Conversely, crossing a fall-back transition (gaining an hour) result in an extra hour. If you need to adjust the date by a fixed amount of time, consider using `setUTCHours()` or `setTime()`.

If the new local time falls within an offset transition, the exact time is derived using the same behavior as `Temporal`'s `disambiguation: "compatible"` option. That is, if the local time corresponds to two instants, the earlier one is chosen; if the local time does not exist (there is a gap), we go forward by the gap duration.

Examples

Using setHours()

```js
const theBigDay = new Date();
theBigDay.setHours(7);
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-date.prototype.sethours

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`setHours`	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Date.setHours`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Date.prototype.getHours()`
- `Date.prototype.setUTCHours()`

JavaScript › Reference › Standard built-in objects › Date

Date.prototype.setMilliseconds()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Date/setMilliseconds · MDN page last modified 2025-07-30

Baseline: Widely available — available across major browsers since July 2015.

The **`setMilliseconds()`** method of `Date` instances changes the milliseconds for this date according to local time.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Date.prototype.setMilliseconds()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const event = new Date("August 19, 1975 23:15:30");

console.log(event.getMilliseconds());
// Expected output: 0

event.setMilliseconds(456);

console.log(event.getMilliseconds());
// Expected output: 456
```

Syntax

js

```
setMilliseconds(millisecondsValue)
```

Parameters

`millisecondsValue`

An integer between 0 and 999 representing the milliseconds.

Return value

Changes the `Date` object in place, and returns its new timestamp. If `millisecondsValue` is `NaN` (or other values that get coerced to `NaN`, such as `undefined`), the date is set to Invalid Date and `NaN` is returned.

Description

If you specify a number outside the expected range, the date information in the `Date` object is updated accordingly. For example, if you specify 1005, the number of seconds is incremented by 1, and 5 is used for the milliseconds.

Because `setMilliseconds()` operates on the local time, crossing a Daylight Saving Time (DST) boundary may result in a different elapsed time than expected. For example, if setting the milliseconds crosses a spring-forward transition (losing an hour), the difference in timestamps between the new and old date is one hour less than the nominal time difference. Conversely, crossing a fall-back transition (gaining an hour) result in an extra hour. If you need to adjust the date by a fixed amount of time, consider using `setUTCMilliseconds()` or `setTime()`.

If the new local time falls within an offset transition, the exact time is derived using the same behavior as `Temporal`'s `disambiguation: "compatible"` option. That is, if the local time corresponds to two instants, the earlier one is chosen; if the local time does not exist (there is a gap), we go forward by the gap duration.

Examples

Using setMilliseconds()

```js
const theBigDay = new Date();
theBigDay.setMilliseconds(100);
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-date.prototype.setmilliseconds

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`setMilliseconds`	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Date.setMilliseconds`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Date.prototype.getMilliseconds()
- Date.prototype.setUTCMilliseconds()

JavaScript › Reference › Standard built-in objects › Date

Date.prototype.setMinutes()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Date/setMinutes · MDN page last modified 2025-07-30

Baseline: Widely available — available across major browsers since July 2015.

The **`setMinutes()`** method of Date instances changes the minutes for this date according to local time.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Date.prototype.setMinutes()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const event = new Date("August 19, 1975 23:15:30");

event.setMinutes(45);

console.log(event.getMinutes());
// Expected output: 45

console.log(event);
// Expected output: "Tue Aug 19 1975 23:45:30 GMT+0200 (CEST)"
// Note: your timezone may vary
```

Syntax

js

```
setMinutes(minutesValue)
setMinutes(minutesValue, secondsValue)
setMinutes(minutesValue, secondsValue, msValue)
```

Parameters

`minutesValue`

An integer between 0 and 59 representing the minutes.

`secondsValue` Optional

An integer between 0 and 59 representing the seconds. If you specify `secondsValue`, you must also specify `minutesValue`.

`msValue` Optional

An integer between 0 and 999 representing the milliseconds. If you specify `msValue`, you must also specify `minutesValue` and `secondsValue`.

Return value

Changes the Date object in place, and returns its new timestamp. If a parameter is `NaN` (or other values that get coerced to `NaN`, such as `undefined`), the date is set to Invalid Date and `NaN` is returned.

Description

If you do not specify the `secondsValue` and `msValue` parameters, the same values as what are returned by `getSeconds()` and `getMilliseconds()` are used.

If a parameter you specify is outside of the expected range, other parameters and the date information in the `Date` object are updated accordingly. For example, if you specify 100 for `secondsValue`, the minutes is incremented by 1 (`minutesValue + 1`), and 40 is used for seconds.

Because `setMinutes()` operates on the local time, crossing a Daylight Saving Time (DST) boundary may result in a different elapsed time than expected. For example, if setting the minutes crosses a spring-forward transition (losing an hour), the difference in timestamps between the new and old date is one hour less than the nominal time difference. Conversely, crossing a fall-back transition (gaining an hour) result in an extra hour. If you need to adjust the date by a fixed amount of time, consider using `setUTCMinutes()` or `setTime()`.

If the new local time falls within an offset transition, the exact time is derived using the same behavior as `Temporal`'s `disambiguation: "compatible"` option. That is, if the local time corresponds to two instants, the earlier one is chosen; if the local time does not exist (there is a gap), we go forward by the gap duration.

Examples

Using setMinutes()

```js
const theBigDay = new Date();
theBigDay.setMinutes(45);
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-date.prototype.setminutes`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`setMinutes`	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Date.setMinutes`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Date.prototype.getMinutes()
- Date.prototype.setUTCMinutes()

JavaScript › Reference › Standard built-in objects › Date

Date.prototype.setMonth()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Date/setMonth · MDN page last modified 2025-07-30

Baseline: Widely available — available across major browsers since July 2015.

The **`setMonth()`** method of Date instances changes the month and/or day of the month for this date according to local time.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Date.prototype.setMonth()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const event = new Date("August 19, 1975 23:15:30");

event.setMonth(3);

console.log(event.getMonth());
// Expected output: 3

console.log(event);
// Expected output: "Sat Apr 19 1975 23:15:30 GMT+0100 (CET)"
// Note: your timezone may vary
```

Syntax

js

```
setMonth(monthValue)
setMonth(monthValue, dateValue)
```

Parameters

`monthValue`

An integer representing the month: 0 for January, 1 for February, and so on.

`dateValue` Optional

An integer from 1 to 31 representing the day of the month.

Return value

Changes the Date object in place, and returns its new timestamp. If a parameter is `NaN` (or other values that get coerced to `NaN`, such as `undefined`), the date is set to Invalid Date and `NaN` is returned.

Description

If you do not specify the `dateValue` parameter, the same value as what is returned by `getDate()` is used.

If a parameter you specify is outside of the expected range, other parameters and the date information in the `Date` object are updated accordingly. For example, if you specify 15 for `monthValue`, the year is incremented by 1, and 3 is used for month.

The current day of month will have an impact on the behavior of this method. Conceptually it will add the number of days given by the current day of the month to the 1st day of the new month specified as the parameter, to return the new date. For example, if the current value is 31st January 2016, calling setMonth with a value of 1 will return 2nd March 2016. This is because in 2016 February had 29 days.

Because `setMonth()` operates on the local time, crossing a Daylight Saving Time (DST) boundary may result in a different elapsed time than expected. For example, if setting the month crosses a spring-forward transition (losing an hour), the difference in timestamps between the new and old date is one hour less than the nominal day difference multiplied by 24 hours. Conversely, crossing a fall-back transition (gaining an hour) result in an extra hour. If you need to adjust the date by a fixed amount of time, consider using `setUTCMonth()` or `setTime()`.

If the new local time falls within an offset transition, the exact time is derived using the same behavior as `Temporal`'s `disambiguation: "compatible"` option. That is, if the local time corresponds to two instants, the earlier one is chosen; if the local time does not exist (there is a gap), we go forward by the gap duration.

Examples

Using setMonth()

js

```
const theBigDay = new Date();
theBigDay.setMonth(6);

// Watch out for end of month transitions
const endOfMonth = new Date(2016, 7, 31);
endOfMonth.setMonth(1);
console.log(endOfMonth); // Wed Mar 02 2016 00:00:00
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-date.prototype.setmonth

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`setMonth`	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Date.setMonth`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Date.prototype.getMonth()`
- `Date.prototype.setUTCMonth()`

JavaScript › Reference › Standard built-in objects › Date

Date.prototype.setSeconds()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Date/setSeconds · MDN page last modified 2026-10-09

Baseline: Widely available — available across major browsers since July 2015.

The **`setSeconds()`** method of `Date` instances changes the seconds and/or milliseconds for this date according to local time.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Date.prototype.setSeconds()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const event = new Date("August 19, 1975 23:15:30");

event.setSeconds(42);

console.log(event.getSeconds());
// Expected output: 42

console.log(event);
// Expected output: "Tue Aug 19 1975 23:15:42 GMT+0100 (CET)"
// Note: your timezone may vary
```

Syntax

js

```
setSeconds(secondsValue)
setSeconds(secondsValue, msValue)
```

Parameters

`secondsValue`

An integer between 0 and 59 representing the seconds.

`msValue` Optional

An integer between 0 and 999 representing the milliseconds.

Return value

Changes the `Date` object in place, and returns its new timestamp. If a parameter is `NaN` (or other values that get coerced to `NaN`, such as `undefined`), the date is set to Invalid Date and `NaN` is returned.

Description

If you do not specify the `msValue` parameter, the value returned from the `getMilliseconds()` method is used.

If a parameter you specify is outside of the expected range, `setSeconds()` attempts to update the date information in the `Date` object accordingly. For example, if you use 100 for `secondsValue`, the minutes stored in the `Date` object will be incremented by 1, and 40 will be used for seconds.

Because `setSeconds()` operates on the local time, crossing a Daylight Saving Time (DST) boundary may result in a different elapsed time than expected. For example, if setting the seconds crosses a spring-forward transition (losing an hour), the difference in timestamps between the new and old date is one hour less than the nominal time difference. Conversely, crossing a fall-back transition (gaining an hour) result in an extra hour. If you need to adjust the date by a fixed amount of time, consider using `setUTCSeconds()` or `setTime()`.

If the new local time falls within an offset transition, the exact time is derived using the same behavior as `Temporal`'s `disambiguation: "compatible"` option. That is, if the local time corresponds to two instants, the earlier one is chosen; if the local time does not exist (there is a gap), we go forward by the gap duration.

Examples

Using setSeconds()

```js
const theBigDay = new Date();
theBigDay.setSeconds(30);
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-date.prototype.setseconds`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`setSeconds`	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Date.setSeconds`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Date.prototype.getSeconds()`
- `Date.prototype.setUTCSeconds()`

JavaScript › Reference › Standard built-in objects › Date

Date.prototype.setTime()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Date/setTime · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **`setTime()`** method of `Date` instances changes the timestamp for this date, which is the number of milliseconds since the epoch, defined as the midnight at the beginning of January 1, 1970, UTC.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Date.prototype.setTime()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const launchDate = new Date("July 1, 1999, 12:00:00");
const futureDate = new Date();
futureDate.setTime(launchDate.getTime());

console.log(futureDate);
// Expected output: "Thu Jul 01 1999 12:00:00 GMT+0200 (CEST)"

const fiveMinutesInMs = 5 * 60 * 1000;
futureDate.setTime(futureDate.getTime() + fiveMinutesInMs);

console.log(futureDate);
// Expected output: "Thu Jul 01 1999 12:05:00 GMT+0200 (CEST)"
// Note: your timezone may vary
```

Syntax

js

```
setTime(timeValue)
```

Parameters

`timeValue`

An integer representing the new timestamp — the number of milliseconds since the midnight at the beginning of January 1, 1970, UTC.

Return value

Changes the `Date` object in place, and returns its new timestamp. If `timeValue` is `NaN` (or other values that get coerced to `NaN`, such as `undefined`), the date is set to Invalid Date and `NaN` is returned.

Examples

Using setTime()

```js
const theBigDay = new Date("1999-07-01");
const sameAsBigDay = new Date();
sameAsBigDay.setTime(theBigDay.getTime());
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-date.prototype.settime`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`setTime`	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Date.setTime`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Date.prototype.getTime()`
- `Date.prototype.setUTCHours()`

JavaScript › Reference › Standard built-in objects › Date

Date.prototype.setUTCDate()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Date/setUTCDate · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **`setUTCDate()`** method of `Date` instances changes the day of the month for this date according to universal time.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Date.prototype.setUTCDate()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const event = new Date("August 19, 1975 23:15:30 GMT-3:00");

console.log(event.getUTCDate());
// Expected output: 20

event.setUTCDate(19);

console.log(event.getUTCDate());
// Expected output: 19
```

Syntax

js

```
setUTCDate(dateValue)
```

Parameters

`dateValue`

An integer from 1 to 31 representing the day of the month.

Return value

Changes the `Date` object in place, and returns its new timestamp. If `dateValue` is `NaN` (or other values that get coerced to `NaN`, such as `undefined`), the date is set to Invalid Date and `NaN` is returned.

Description

If the `dateValue` is outside of the range of date values for the month, `setDate()` will update the `Date` object accordingly.

For example, if 0 is provided for `dateValue`, the date will be set to the last day of the previous month. If you use 40 for `dateValue`, and the month stored in the `Date` object is June, the day will be changed to 10 and the month will be incremented to July.

If a negative number is provided for `dateValue`, the date will be set counting backwards from the last day of the previous month. -1 would result in the date being set to 1 day before the last day of the previous month.

Examples

Using setUTCDate()

js

```
const theBigDay = new Date();
theBigDay.setUTCDate(20);
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-date.prototype.setutcdate`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`setUTCDate`	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Date.setUTCDate`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Date.prototype.getUTCDate()`
- `Date.prototype.setDate()`

JavaScript › Reference › Standard built-in objects › Date

Date.prototype.setUTCFullYear()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Date/setUTCFullYear · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **`setUTCFullYear()`** method of `Date` instances changes the year for this date according to universal time.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Date.prototype.setUTCFullYear()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const event = new Date("December 31, 1975 23:15:30 GMT-3:00");

console.log(event.getUTCFullYear());
// Expected output: 1976

console.log(event.toUTCString());
// Expected output: "Thu, 01 Jan 1976 02:15:30 GMT"

event.setUTCFullYear(1975);

console.log(event.toUTCString());
// Expected output: "Wed, 01 Jan 1975 02:15:30 GMT"
```

Syntax

```js
setUTCFullYear(yearValue)
setUTCFullYear(yearValue, monthValue)
setUTCFullYear(yearValue, monthValue, dateValue)
```

Parameters

`yearValue`

An integer representing the year. For example, 1995.

`monthValue` Optional

An integer representing the month: 0 for January, 1 for February, and so on.

`dateValue` Optional

An integer between 1 and 31 representing the day of the month. If you specify `dateValue`, you must also specify `monthValue`.

Return value

Changes the `Date` object in place, and returns its new timestamp. If a parameter is `NaN` (or other values that get coerced to `NaN`, such as `undefined`), the date is set to Invalid Date and `NaN` is returned.

Description

If you do not specify the `monthValue` and `dateValue` parameters, the values returned from the `getUTCMonth()` and `getUTCDate()` methods are used.

If a parameter you specify is outside of the expected range, `setUTCFullYear()` attempts to update the other parameters and the date information in the `Date` object accordingly. For example, if you specify 15 for `monthValue`, the year is incremented by 1 (`yearValue + 1`), and 3 is used for the month.

Examples

Using setUTCFullYear()

```js
const theBigDay = new Date();
theBigDay.setUTCFullYear(1997);
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-date.prototype.setutcfullyear

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`setUTCFullYear`	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Date.setUTCFullYear`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Date.prototype.getUTCFullYear()`
- `Date.prototype.setFullYear()`

JavaScript › Reference › Standard built-in objects › Date

Date.prototype.setUTCHours()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Date/setUTCHours · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **`setUTCHours()`** method of `Date` instances changes the hours, minutes, seconds, and/or milliseconds for this date according to universal time.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Date.prototype.setUTCHours()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const event = new Date("August 19, 1975 23:15:30 GMT-3:00");

console.log(event.toUTCString());
// Expected output: "Wed, 20 Aug 1975 02:15:30 GMT"

console.log(event.getUTCHours());
// Expected output: 2

event.setUTCHours(23);

console.log(event.toUTCString());
// Expected output: "Wed, 20 Aug 1975 23:15:30 GMT"
```

Syntax

js

```
setUTCHours(hoursValue)
setUTCHours(hoursValue, minutesValue)
setUTCHours(hoursValue, minutesValue, secondsValue)
setUTCHours(hoursValue, minutesValue, secondsValue, msValue)
```

Parameters

`hoursValue`

An integer between 0 and 23 representing the hours.

`minutesValue` Optional

An integer between 0 and 59 representing the minutes.

`secondsValue` Optional

An integer between 0 and 59 representing the seconds. If you specify `secondsValue`, you must also specify `minutesValue`.

`msValue` Optional

An integer between 0 and 999 representing the milliseconds. If you specify `msValue`, you must also specify `minutesValue` and `secondsValue`.

Return value

Changes the `Date` object in place, and returns its new timestamp. If a parameter is `NaN` (or other values that get coerced to `NaN`, such as `undefined`), the date is set to Invalid Date and `NaN` is returned.

Description

If you do not specify the `minutesValue`, `secondsValue`, and `msValue` parameters, the values returned from the `getUTCMinutes()`, `getUTCSeconds()`, and `getUTCMilliseconds()` methods are used.

If a parameter you specify is outside of the expected range, `setUTCHours()` attempts to update the date information in the `Date` object accordingly. For example, if you use 100 for `secondsValue`, the minutes will be incremented by 1 (`minutesValue + 1`), and 40 will be used for seconds.

Examples

Using setUTCHours()

```js
const theBigDay = new Date();
theBigDay.setUTCHours(8);
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-date.prototype.setutchours

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`setUTCHours`	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Date.setUTCHours`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Date.prototype.getUTCHours()`
- `Date.prototype.setHours()`

JavaScript › Reference › Standard built-in objects › Date

Date.prototype.setUTCMilliseconds()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Date/setUTCMilliseconds · MDN page last modified 2025-07-20

Baseline: Widely available — available across major browsers since July 2015.

The **`setUTCMilliseconds()`** method of `Date` instances changes the milliseconds for this date according to universal time.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Date.prototype.setUTCMilliseconds()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const date = new Date("2018-01-24T12:38:29.069Z");

console.log(date.getUTCMilliseconds());
// Expected output: 69

date.setUTCMilliseconds(420);

console.log(date.getUTCMilliseconds());
// Expected output: 420
```

Syntax

js

```
setUTCMilliseconds(millisecondsValue)
```

Parameters

`millisecondsValue`

An integer between 0 and 999 representing the milliseconds.

Return value

Changes the `Date` object in place, and returns its new timestamp. If `millisecondsValue` is `NaN` (or other values that get coerced to `NaN`, such as `undefined`), the date is set to Invalid Date and `NaN` is returned.

Description

If a parameter you specify is outside of the expected range, `setUTCMilliseconds()` attempts to update the date information in the `Date` object accordingly. For example, if you use 1100 for `millisecondsValue`, the seconds stored in the `Date` object will be incremented by 1, and 100 will be used for milliseconds.

Examples

Using setUTCMilliseconds()

```js
const theBigDay = new Date();
theBigDay.setUTCMilliseconds(500);
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-date.prototype.setutcmilliseconds

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
setUTCMilliseconds	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Date.setUTCMilliseconds`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Date.prototype.getUTCMilliseconds()`
- `Date.prototype.setMilliseconds()`

JavaScript › Reference › Standard built-in objects › Date

Date.prototype.setUTCMinutes()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Date/setUTCMinutes · MDN page last modified 2025-07-20

Baseline: Widely available — available across major browsers since July 2015.

The **`setUTCMinutes()`** method of `Date` instances changes the minutes for this date according to universal time.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Date.prototype.setUTCMinutes()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const date = new Date("December 31, 1975, 23:15:30 GMT+11:00");

console.log(date.getUTCMinutes());
// Expected output: 15

date.setUTCMinutes(25);

console.log(date.getUTCMinutes());
// Expected output: 25
```

Syntax

js

```
setUTCMinutes(minutesValue)
setUTCMinutes(minutesValue, secondsValue)
setUTCMinutes(minutesValue, secondsValue, msValue)
```

Parameters

`minutesValue`

An integer between 0 and 59 representing the minutes.

`secondsValue` Optional

An integer between 0 and 59 representing the seconds. If you specify `secondsValue`, you must also specify `minutesValue`.

`msValue` Optional

An integer between 0 and 999 representing the milliseconds. If you specify `msValue`, you must also specify `minutesValue` and `secondsValue`.

Return value

Changes the `Date` object in place, and returns its new timestamp. If a parameter is `NaN` (or other values that get coerced to `NaN`, such as `undefined`), the date is set to Invalid Date and `NaN` is returned.

Description

If you do not specify the `secondsValue` and `msValue` parameters, the values returned from `getUTCSeconds()` and `getUTCMilliseconds()` methods are used.

If a parameter you specify is outside of the expected range, `setUTCMinutes()` attempts to update the date information in the `Date` object accordingly. For example, if you use 100 for `secondsValue`, the minutes will be incremented by 1 (`minutesValue + 1`), and 40 will be used for seconds.

Examples

Using setUTCMinutes()

```js
const theBigDay = new Date();
theBigDay.setUTCMinutes(43);
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-date.prototype.setutcminutes

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
setUTCMinutes	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Date.setUTCMinutes`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Date.prototype.getUTCMinutes()`
- `Date.prototype.setMinutes()`

JavaScript › Reference › Standard built-in objects › Date

Date.prototype.setUTCMonth()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Date/setUTCMonth · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **`setUTCMonth()`** method of `Date` instances changes the month and/or day of the month for this date according to universal time.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Date.prototype.setUTCMonth()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const event = new Date("December 31, 1975 23:15:30 GMT-3:00");

console.log(event.toUTCString());
// Expected output: "Thu, 01 Jan 1976 02:15:30 GMT"

console.log(event.getUTCMonth());
// Expected output: 0

event.setUTCMonth(11);

console.log(event.toUTCString());
// Expected output: "Wed, 01 Dec 1976 02:15:30 GMT"
```

Syntax

js

```
setUTCMonth(monthValue)
setUTCMonth(monthValue, dateValue)
```

Parameters

`monthValue`

An integer representing the month: 0 for January, 1 for February, and so on.

`dateValue` Optional

An integer from 1 to 31 representing the day of the month.

Return value

Changes the `Date` object in place, and returns its new timestamp. If a parameter is `NaN` (or other values that get coerced to `NaN`, such as `undefined`), the date is set to Invalid Date and `NaN` is returned.

Description

If you do not specify the `dateValue` parameter, the value returned from the `getUTCDate()` method is used.

If a parameter you specify is outside of the expected range, `setUTCMonth()` attempts to update the date information in the `Date` object accordingly. For example, if you use 15 for `monthValue`, the year will be incremented by 1, and 3 will be used for month.

Examples

Using setUTCMonth()

js

```
const theBigDay = new Date();
theBigDay.setUTCMonth(11);
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-date.prototype.setutcmonth

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
setUTCMonth	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Date.setUTCMonth`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Date.prototype.getUTCMonth()`
- `Date.prototype.setMonth()`

JavaScript › Reference › Standard built-in objects › Date

Date.prototype.setUTCSeconds()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Date/setUTCSeconds · MDN page last modified 2025-07-20

Baseline: Widely available — available across major browsers since July 2015.

The **`setUTCSeconds()`** method of `Date` instances changes the seconds and/or milliseconds for this date according to universal time.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Date.prototype.setUTCSeconds()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const date = new Date("December 31, 1975, 23:15:30 GMT+11:00");

console.log(date.getUTCSeconds());
// Expected output: 30

date.setUTCSeconds(39);

console.log(date.getUTCSeconds());
// Expected output: 39
```

Syntax

```js
setUTCSeconds(secondsValue)
setUTCSeconds(secondsValue, msValue)
```

Parameters

`secondsValue`

An integer between 0 and 59 representing the seconds.

`msValue` Optional

An integer between 0 and 999 representing the milliseconds.

Return value

Changes the `Date` object in place, and returns its new timestamp. If a parameter is `NaN` (or other values that get coerced to `NaN`, such as `undefined`), the date is set to Invalid Date and `NaN` is returned.

Description

If you do not specify the `msValue` parameter, the value returned from the `getUTCMilliseconds()` method is used.

If a parameter you specify is outside of the expected range, `setUTCSeconds()` attempts to update the date information in the `Date` object accordingly. For example, if you use 100 for `secondsValue`, the minutes stored in the `Date` object will be incremented by 1, and 40 will be used for seconds.

Examples

Using setUTCSeconds()

```js
const theBigDay = new Date();
theBigDay.setUTCSeconds(20);
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-date.prototype.setutcseconds`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`setUTCSeconds`	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Date.setUTCSeconds`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Date.prototype.getUTCSeconds()
- Date.prototype.setSeconds()

Date.prototype.setYear()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Date/setYear · MDN page last modified 2026-10-26

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`setYear()`** method of Date instances sets the year for a specified date according to local time.

However, the way the legacy `setYear()` method sets year values is different from how the preferred `setFullYear()` method sets year values — and in some cases, also different from how `new Date()` and `Date.parse()` set year values. Specifically, given two-digit numbers, such as `22` and `61`:

- `setYear()` interprets any two-digit number as an offset to `1900`; so `date.setYear(22)` results in the year value being set to `1922`, and `date.setYear(61)` results in the year value being set to `1961`. (In contrast, while `new Date(61, 1)` also results in the year value being set to `1961`, `new Date("2/1/22")` results in the year value being set to `2022`; and similarly for `Date.parse()`).
- `setFullYear()` does no special interpretation but instead uses the literal two-digit value as-is to set the year; so `date.setFullYear(61)` results in the year value being set to `0061`, and `date.setFullYear(22)` results in the year value being set to `0022`.

Because of those differences in behavior, you should no longer use the legacy `setYear()` method, but should instead use the preferred `setFullYear()` method.

Syntax

```
setYear(yearValue)
```

Parameters

`yearValue`

An integer.

Return value

Changes the `Date` object in place, and returns its new timestamp. If `yearValue` is `NaN` (or other values that get coerced to `NaN`, such as `undefined`), the date is set to Invalid Date and `NaN` is returned.

Description

If `yearValue` is a number between 0 and 99 (inclusive), then the year for `dateObj` is set to `1900 + yearValue`. Otherwise, the year for `dateObj` is set to `yearValue`.

Examples

Using setYear()

The first two lines set the year to 1996. The third sets the year to 2000.

```js
const theBigDay = new Date();

theBigDay.setYear(96);
theBigDay.setYear(1996);
theBigDay.setYear(2000);
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/additional-ecmascript-features-for-web-browsers.html#sec-date.prototype.setyear

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
setYear Deprecated	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Date.setYear`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Polyfill of `Date.prototype.setYear` in `core-js`
- `Date.prototype.getFullYear()`
- `Date.prototype.getUTCFullYear()`
- `Date.prototype.setFullYear()`
- `Date.prototype.setUTCFullYear()`

JavaScript › Reference › Standard built-in objects › Date

Date.prototype.toDateString()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Date/toDateString · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **`toDateString()`** method of `Date` instances returns a string representing the date portion of this date interpreted in the local timezone.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Date.prototype.toDateString()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const event = new Date(1993, 6, 28, 14, 39, 7);

console.log(event.toString());
// Expected output: "Wed Jul 28 1993 14:39:07 GMT+0200 (CEST)"
// Note: your timezone may vary

console.log(event.toDateString());
// Expected output: "Wed Jul 28 1993"
```

Syntax

js

```
toDateString()
```

Parameters

None.

Return value

A string representing the date portion of the given date (see description for the format). Returns `"Invalid Date"` if the date is invalid.

Description

`Date` instances refer to a specific point in time. `toDateString()` interprets the date in the local timezone and formats the *date* part in English. It always uses the following format, separated by spaces:

1. First three letters of the week day name
2. First three letters of the month name
3. Two-digit day of the month, padded on the left a zero if necessary
4. Four-digit year (at least), padded on the left with zeros if necessary. May have a negative sign

For example: "Thu Jan 01 1970".

- If you only want to get the *time* part, use `toTimeString()`.
- If you want to get both the date and time, use `toString()`.
- If you want to make the date interpreted as UTC instead of local timezone, use `toUTCString()`.
- If you want to format the date in a more user-friendly format (e.g., localization), use `toLocaleDateString()`.

Examples

Using toDateString()

```js
const d = new Date(0);

console.log(d.toString()); // "Thu Jan 01 1970 00:00:00 GMT+0000 (Coordinated Universal Time)"
console.log(d.toDateString()); // "Thu Jan 01 1970"
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-date.prototype.todatestring

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
toDateString	1	12	1	5	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Date.toDateString`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Date.prototype.toLocaleDateString()`
- `Date.prototype.toTimeString()`
- `Date.prototype.toString()`

JavaScript › Reference › Standard built-in objects › Date

Date.prototype.toISOString()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Date/toISOString · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **`toISOString()`** method of `Date` instances returns a string representing this date in the date time string format, a *simplified* format based on ISO 8601, which is always 24 or 27 characters long (`YYYY-MM-DDTHH:mm:ss.sssZ` or `±YYYYYY-MM-DDTHH:mm:ss.sssZ`, respectively). The timezone is always UTC, as denoted by the suffix `Z`.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Date.prototype.toISOString()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const event = new Date("05 October 2011 14:48 UTC");
console.log(event.toString());
// Expected output: "Wed Oct 05 2011 16:48:00 GMT+0200 (CEST)"
// Note: your timezone may vary

console.log(event.toISOString());
// Expected output: "2011-10-05T14:48:00.000Z"
```

Syntax

js

```
toISOString()
```

Parameters

None.

Return value

A string representing the given date in the date time string format according to universal time. It's the same format as the one required to be recognized by `Date.parse()`.

Exceptions

`RangeError`

Thrown if the date is invalid or if it corresponds to a year that cannot be represented in the date string format.

Examples

Using toISOString()

js

```
const d = new Date(0);

console.log(d.toISOString()); // "1970-01-01T00:00:00.000Z"
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-date.prototype.toisostring

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`toISOString`	3	12	1	10.5	4	18	4	11	3.2	1.0	4.4	3.2	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Date.toISOString`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Date.prototype.toLocaleDateString()`
- `Date.prototype.toString()`
- `Date.prototype.toUTCString()`

JavaScript › Reference › Standard built-in objects › Date

Date.prototype.toJSON()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Date/toJSON · MDN page last modified 2026-10-30

Baseline: Widely available — available across major browsers since July 2015.

The **`toJSON()`** method of `Date` instances returns a string representing this date in the same ISO format as `toISOString()`.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Date.prototype.toJSON()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const event = new Date("August 19, 1975 23:15:30 UTC");

const jsonDate = event.toJSON();

console.log(jsonDate);
// Expected output: "1975-08-19T23:15:30.000Z"

console.log(new Date(jsonDate).toUTCString());
// Expected output: "Tue, 19 Aug 1975 23:15:30 GMT"
```

Syntax

js

```
toJSON()
```

Parameters

None.

Return value

A string representing the given date in the date time string format according to universal time, or `null` when the date is invalid. For valid dates, the return value is the same as that of `toISOString()`.

Description

The method is automatically called by `JSON.stringify()` when a `Date` object is encountered during JSON serialization. The serialized value can then be deserialized using the `Date()` constructor within the reviver of `JSON.parse()`.

The method first attempts to convert its `this` value to a primitive by calling its `[Symbol.toPrimitive]()` (with `"number"` as hint), `valueOf()`, and `toString()` methods, in that order. If the result is a non-finite number, `null` is returned. (This generally corresponds to an invalid date, whose `valueOf()` returns `NaN`.) Otherwise, if the converted primitive is not a number or is a finite number, the return value of `this.toISOString()` is returned.

Note that the method does not check whether the `this` value is a valid `Date` object. However, calling `Date.prototype.toJSON()` on non-`Date` objects fails unless the object's number primitive representation is `NaN`, or the object also has a `toISOString()` method.

Examples

Using toJSON()

```js
const jsonDate = new Date(0).toJSON(); // '1970-01-01T00:00:00.000Z'
const backToDate = new Date(jsonDate);

console.log(jsonDate); // 1970-01-01T00:00:00.000Z
```

Serialization round-tripping

When parsing JSON containing date strings, you can use the `Date()` constructor to revive them into the original date objects.

```js
const fileData = {
  author: "Maria",
  title: "Date.prototype.toJSON()",
  createdAt: new Date(2019, 3, 15),
  updatedAt: new Date(2020, 6, 26),
};
const response = JSON.stringify(fileData);

// Imagine transmission through network

const data = JSON.parse(response, (key, value) => {
  if (key === "createdAt" || key === "updatedAt") {
    return new Date(value);
  }
  return value;
});

console.log(data);
```

> **Note:** The reviver of `JSON.parse()` must be specific to the payload shape you expect, because the serialization is *irreversible*: it's not possible to distinguish between a string that represents a Date and a normal string.

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-date.prototype.tojson`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`toJSON`	3	12	1	10.5	4	18	4	11	3.2	1.0	4.4	3.2	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Date.toJSON`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Date.prototype.toLocaleDateString()`
- `Date.prototype.toTimeString()`
- `Date.prototype.toUTCString()`

JavaScript › Reference › Standard built-in objects › Date

Date.prototype.toLocaleDateString()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Date/toLocaleDateString · MDN page last modified 2025-10-30

Baseline: Widely available — available across major browsers since July 2015.

The **`toLocaleDateString()`** method of `Date` instances returns a string with a language-sensitive representation of the date portion of this date in the local timezone. In implementations with `Intl.DateTimeFormat API` support, this method delegates to `Intl.DateTimeFormat`.

Every time `toLocaleDateString` is called, it has to perform a search in a big database of localization strings, which is potentially inefficient. When the method is called many times with the same arguments, it is better to create an `Intl.DateTimeFormat` object and use its `format()` method, because a `DateTimeFormat` object remembers the arguments passed to it and may decide to cache a slice of the database, so future `format` calls can search for localization strings within a more constrained context.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Date.prototype.toLocaleDateString()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const event = new Date(Date.UTC(2012, 11, 20, 3, 0, 0));
const options = {
  weekday: "long",
  year: "numeric",
  month: "long",
  day: "numeric",
};

console.log(event.toLocaleDateString("de-DE", options));
// Expected output (varies according to local timezone): Donnerstag, 20. Dezember 2012

console.log(event.toLocaleDateString("ar-EG", options));
// Expected output (varies according to local timezone): الخميس، ٢٠ ديسمبر، ٢٠١٢

console.log(event.toLocaleDateString(undefined, options));
// Expected output (varies according to local timezone and default locale): Thursday,
December 20, 2012
```

Syntax

js

```
toLocaleDateString()
toLocaleDateString(locales)
toLocaleDateString(locales, options)
```

Parameters

The `locales` and `options` parameters customize the behavior of the function and let applications specify the language whose formatting conventions should be used.

In implementations that support the `Intl.DateTimeFormat` API, these parameters correspond exactly to the `Intl.DateTimeFormat()` constructor's parameters. Implementations without `Intl.DateTimeFormat` support are asked to ignore both parameters, making the locale used and the form of the string returned entirely implementation-dependent.

`locales` Optional

A string with a BCP 47 language tag, or an array of such strings. Corresponds to the `locales` parameter of the `Intl.DateTimeFormat()` constructor.

In implementations without `Intl.DateTimeFormat` support, this parameter is ignored and the host's locale is usually used.

`options` Optional

An object adjusting the output format. Corresponds to the `options` parameter of the `Intl.DateTimeFormat()` constructor. The `timeStyle` option must be undefined, or a `TypeError` would be thrown. If `weekday`, `year`, `month`, and `day` are all undefined, then `year`, `month`, and `day` will be set to `"numeric"`.

In implementations without `Intl.DateTimeFormat` support, this parameter is ignored.

See the `Intl.DateTimeFormat()` constructor for details on these parameters and how to use them.

Return value

A string representing the date portion of the given date according to language-specific conventions.

In implementations with `Intl.DateTimeFormat`, this is equivalent to `new Intl.DateTimeFormat(locales, options).format(date)`, where `options` has been normalized as described above.

> **Note:** Most of the time, the formatting returned by `toLocaleDateString()` is consistent. However, the output may vary between implementations, even within the same locale — output variations are by design and allowed by the specification. It may also not be what you expect. For example, the string may use non-breaking spaces or be surrounded by bidirectional control characters. You should not compare the results of `toLocaleDateString()` to hardcoded constants.

Examples

Using toLocaleDateString()

Basic use of this method without specifying a `locale` returns a formatted string in the default locale and with default options.

```js
const date = new Date(Date.UTC(2012, 11, 12, 3, 0, 0));

// toLocaleDateString() without arguments depends on the implementation,
// the default locale, and the default time zone
console.log(date.toLocaleDateString());
// "12/11/2012" if run in en-US locale with time zone America/Los_Angeles
```

Checking for support for locales and options parameters

The `locales` and `options` parameters may not be supported in all implementations, because support for the internationalization API is optional, and some systems may not have the necessary data. For implementations without internationalization support, `toLocaleDateString()` always uses the system's locale, which may not be what you want. Because any implementation that supports the `locales` and `options` parameters must support the `Intl` API, you can check the existence of the latter for support:

```js
function toLocaleDateStringSupportsLocales() {
  return (
    typeof Intl === "object" &&
    !!Intl &&
    typeof Intl.DateTimeFormat === "function"
  );
}
```

Using locales

This example shows some of the variations in localized date formats. In order to get the format of the language used in the user interface of your application, make sure to specify that language (and possibly some fallback languages) using the `locales` argument:

```js
const date = new Date(Date.UTC(2012, 11, 20, 3, 0, 0));

// formats below assume the local time zone of the locale;
// America/Los_Angeles for the US

// US English uses month-day-year order
console.log(date.toLocaleDateString("en-US"));
// "12/20/2012"

// British English uses day-month-year order
console.log(date.toLocaleDateString("en-GB"));
// "20/12/2012"

// Korean uses year-month-day order
console.log(date.toLocaleDateString("ko-KR"));
// "2012. 12. 20."

// Event for Persian, It's hard to manually convert date to Solar Hijri
console.log(date.toLocaleDateString("fa-IR"));
// "۱۳۹۱/۹/۳۰"

// Arabic in most Arabic speaking countries uses real Arabic digits
console.log(date.toLocaleDateString("ar-EG"));
// "٢٠١٢/١٢/٢٠"

// for Japanese, applications may want to use the Japanese calendar,
// where 2012 was the year 24 of the Heisei era
console.log(date.toLocaleDateString("ja-JP-u-ca-japanese"));
// "24/12/20"

// when requesting a language that may not be supported, such as
// Balinese, include a fallback language, in this case Indonesian
console.log(date.toLocaleDateString(["ban", "id"]));
// "20/12/2012"
```

Using options

The results provided by `toLocaleDateString()` can be customized using the `options` parameter:

```js
const date = new Date(Date.UTC(2012, 11, 20, 3, 0, 0));

// Request a weekday along with a long date
const options = {
  weekday: "long",
  year: "numeric",
  month: "long",
  day: "numeric",
};
console.log(date.toLocaleDateString("de-DE", options));
// "Donnerstag, 20. Dezember 2012"

// An application may want to use UTC and make that visible
options.timeZone = "UTC";
options.timeZoneName = "short";
console.log(date.toLocaleDateString("en-US", options));
// "Thursday, December 20, 2012, UTC"
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-date.prototype.tolocaledatestring`
ECMAScript® 2027 Internationalization API Specification `https://tc39.es/ecma402/#sup-date.prototype.tolocaledatestring`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`toLocaleDateString`	1	12	1	5	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
IANA time zone names in `timeZone` option	24	14	52	15	7	25	56	14	7	1.5	4.4	7	1.0.0	1.8	0.12.0
`locales` parameter	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8[1,2]	13.0.0[3,4]
`options` parameter	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.0	0.12.0

1. Also from version 1.0–1.8: partial support.
2. Also from version 1.0–1.8: Only the locale data for `en-US` is available.
3. Also from version 0.12.0–13.0.0: partial support.
4. Also from version 0.12.0–13.0.0: Before version 13.0.0, only the locale data for `en-US` is available by default. When other locales are specified, the function silently falls back to `en-US`. To make full ICU (locale) data available before version 13, see Node.js documentation on the `--with-intl` option and how to provide the data.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Date.toLocaleDateString`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Intl.DateTimeFormat`
- `Date.prototype.toLocaleString()`
- `Date.prototype.toLocaleTimeString()`
- `Date.prototype.toString()`

JavaScript › Reference › Standard built-in objects › Date

Date.prototype.toLocaleString()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Date/toLocaleString · MDN page last modified 2025-10-30

Baseline: Widely available — available across major browsers since July 2015.

The **`toLocaleString()`** method of `Date` instances returns a string with a language-sensitive representation of this date in the local timezone. In implementations with `Intl.DateTimeFormat` API support, this method delegates to `Intl.DateTimeFormat`.

Every time `toLocaleString` is called, it has to perform a search in a big database of localization strings, which is potentially inefficient. When the method is called many times with the same arguments, it is better to create an `Intl.DateTimeFormat` object and use its `format()` method, because a

DateTimeFormat object remembers the arguments passed to it and may decide to cache a slice of the database, so future format calls can search for localization strings within a more constrained context.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Date.prototype.toLocaleString()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const event = new Date(Date.UTC(2012, 11, 20, 3, 0, 0));

// British English uses day-month-year order and 24-hour time without AM/PM
console.log(event.toLocaleString("en-GB", { timeZone: "UTC" }));
// Expected output: "20/12/2012, 03:00:00"

// Korean uses year-month-day order and 12-hour time with AM/PM
console.log(event.toLocaleString("ko-KR", { timeZone: "UTC" }));
// Expected output: "2012. 12. 20. 오전 3:00:00"
```

Syntax

js

```
toLocaleString()
toLocaleString(locales)
toLocaleString(locales, options)
```

Parameters

The locales and options parameters customize the behavior of the function and let applications specify the language whose formatting conventions should be used.

In implementations that support the Intl.DateTimeFormat API, these parameters correspond exactly to the Intl.DateTimeFormat() constructor's parameters. Implementations without Intl.DateTimeFormat support are asked to ignore both parameters, making the locale used and the form of the string returned entirely implementation-dependent.

locales Optional

A string with a BCP 47 language tag, or an array of such strings. Corresponds to the locales parameter of the Intl.DateTimeFormat() constructor.

In implementations without Intl.DateTimeFormat support, this parameter is ignored and the host's locale is usually used.

options Optional

An object adjusting the output format. Corresponds to the options parameter of the Intl.DateTimeFormat() constructor. If weekday, year, month, day, dayPeriod, hour, minute, second, and fractionalSecondDigits are all undefined, then year, month, day, hour, minute, second will be set to "numeric".

In implementations without Intl.DateTimeFormat support, this parameter is ignored.

See the Intl.DateTimeFormat() constructor for details on these parameters and how to use them.

Return value

A string representing the given date according to language-specific conventions.

In implementations with `Intl.DateTimeFormat`, this is equivalent to `new Intl.DateTimeFormat(locales, options).format(date)`.

> **Note:** Most of the time, the formatting returned by `toLocaleString()` is consistent. However, the output may vary between implementations, even within the same locale — output variations are by design and allowed by the specification. It may also not be what you expect. For example, the string may use non-breaking spaces or be surrounded by bidirectional control characters. You should not compare the results of `toLocaleString()` to hardcoded constants.

Examples

Using toLocaleString()

Basic use of this method – without specifying `locale` or `options` – depends on the implementation and returns a string formatted based on the default locale and time zone, and with default options.

```js
const date = new Date(Date.UTC(2012, 11, 12, 3, 0, 0));

console.log(date.toLocaleString());
// "12/11/2012, 7:00:00 PM" if run in en-US locale with time zone America/Los_Angeles
```

Checking for support for locales and options parameters

The `locales` and `options` parameters may not be supported in all implementations, because support for the internationalization API is optional, and some systems may not have the necessary data. For implementations without internationalization support, `toLocaleString()` always uses the system's locale, which may not be what you want. Because any implementation that supports the `locales` and `options` parameters must support the `Intl` API, you can check the existence of the latter for support:

```js
function toLocaleStringSupportsLocales() {
  return (
    typeof Intl === "object" &&
    !!Intl &&
    typeof Intl.DateTimeFormat === "function"
  );
}
```

Using locales

This example shows some of the variations in localized date and time formats. In order to get the format of the language used in the user interface of your application, make sure to specify that language (and possibly some fallback languages) using the `locales` argument:

```js
const date = new Date(Date.UTC(2012, 1, 2, 3, 0, 0));

// Formats below assume the local time zone of the locale;
// America/Los_Angeles for the US
```

(code continues on the next page)

(code continued from the previous page)

```
// US English uses month-day-year order and 12-hour time with AM/PM
console.log(date.toLocaleString("en-US"));
// "2/1/2012, 7:00:00 PM" (UTC-8 is the previous day)

// British English uses day-month-year order and 24-hour time without AM/PM
console.log(date.toLocaleString("en-GB"));
// "02/02/2012, 03:00:00" (UTC+0 or UTC+1 depending on time of the year)

// Korean uses year-month-day order and 12-hour time with AM/PM
console.log(date.toLocaleString("ko-KR"));
// "2012. 2. 2. 오후 12:00:00"

// Arabic in most Arabic-speaking countries uses Eastern Arabic numerals
console.log(date.toLocaleString("ar-EG"));
// "٢/٢/٢٠١٢ ٥:٠٠:٠٠ ص"

// For Japanese, applications may want to use the Japanese calendar,
// where 2012 was the year 24 of the Heisei era
console.log(date.toLocaleString("ja-JP-u-ca-japanese"));
// "H24/2/2 12:00:00"

// When requesting a language that may not be supported, such as
// Balinese, include a fallback language (in this case, Indonesian)
console.log(date.toLocaleString(["ban", "id"]));
// "2/2/2012 11.00.00"
```

Using options

The results provided by `toLocaleString()` can be customized using the `options` parameter:

js

```
const date = new Date(Date.UTC(2012, 11, 20, 3, 0, 0));

// Request a weekday along with a long date
const options = {
  weekday: "long",
  year: "numeric",
  month: "long",
  day: "numeric",
};
console.log(date.toLocaleString("de-DE", options));
// Example output: "Donnerstag, 20. Dezember 2012"
// The exact date may shift depending on your local time zone.

// An application may want to use UTC and make that visible
options.timeZone = "UTC";
options.timeZoneName = "short";
console.log(date.toLocaleString("en-US", options));
// Example output: "Thursday, December 20, 2012 at UTC"

// Sometimes even the US needs 24-hour time
console.log(date.toLocaleString("en-US", { hour12: false }));
// Example output: "12/19/2012, 19:00:00"
// The exact date and time may shift depending on your local time zone.
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-date.prototype.tolocalestring`
ECMAScript® 2027 Internationalization API Specification `https://tc39.es/ecma402/#sup-date.prototype.tolocalestring`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`toLocaleString`	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
IANA time zone names in `timeZone` option	24	14	52	15	7	25	56	14	7	1.5	4.4	7	1.0.0	1.8	0.12.0
`locales` parameter	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8[1,2]	13.0.0[3,4]
`options` parameter	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.0	0.12.0

1. Also from version 1.0–1.8: partial support.
2. Also from version 1.0–1.8: Only the locale data for `en-US` is available.
3. Also from version 0.12.0–13.0.0: partial support.
4. Also from version 0.12.0–13.0.0: Before version 13.0.0, only the locale data for `en-US` is available by default. When other locales are specified, the function silently falls back to `en-US`. To make full ICU (locale) data available before version 13, see Node.js documentation on the `--with-intl` option and how to provide the data.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Date.toLocaleString`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Intl.DateTimeFormat`
- `Date.prototype.toLocaleDateString()`
- `Date.prototype.toLocaleTimeString()`
- `Date.prototype.toString()`

JavaScript › Reference › Standard built-in objects › Date

Date.prototype.toLocaleTimeString()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Date/toLocaleTimeString · MDN page last modified 2025-10-30

Baseline: Widely available — available across major browsers since July 2015.

The **`toLocaleTimeString()`** method of `Date` instances returns a string with a language-sensitive representation of the time portion of this date in the local timezone. In implementations with `Intl.DateTimeFormat` API support, this method delegates to `Intl.DateTimeFormat`.

Every time `toLocaleTimeString` is called, it has to perform a search in a big database of localization strings, which is potentially inefficient. When the method is called many times with the same arguments, it is better to create an `Intl.DateTimeFormat` object and use its `format()` method, because a

`DateTimeFormat` object remembers the arguments passed to it and may decide to cache a slice of the database, so future `format` calls can search for localization strings within a more constrained context.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Date.prototype.toLocaleTimeString()". It runs live in the browser there; its source code follows.

js · interactive example source

```
// Depending on timezone, your results will vary
const event = new Date("August 19, 1975 23:15:30 GMT+00:00");

console.log(event.toLocaleTimeString("en-US"));
// Expected output: "1:15:30 AM"

console.log(event.toLocaleTimeString("it-IT"));
// Expected output: "01:15:30"

console.log(event.toLocaleTimeString("ar-EG"));
// Expected output: "١٢:١٥:٣٠ ص"
```

Syntax

js

```
toLocaleTimeString()
toLocaleTimeString(locales)
toLocaleTimeString(locales, options)
```

Parameters

The `locales` and `options` parameters customize the behavior of the function and let applications specify the language whose formatting conventions should be used.

In implementations that support the `Intl.DateTimeFormat` API, these parameters correspond exactly to the `Intl.DateTimeFormat()` constructor's parameters. Implementations without `Intl.DateTimeFormat` support are asked to ignore both parameters, making the locale used and the form of the string returned entirely implementation-dependent.

`locales` Optional

A string with a BCP 47 language tag, or an array of such strings. Corresponds to the `locales` parameter of the `Intl.DateTimeFormat()` constructor.

In implementations without `Intl.DateTimeFormat` support, this parameter is ignored and the host's locale is usually used.

`options` Optional

An object adjusting the output format. Corresponds to the `options` parameter of the `Intl.DateTimeFormat()` constructor. If `dayPeriod`, `hour`, `minute`, `second`, and `fractionalSecondDigits` are all undefined, then `hour`, `minute`, `second` will be set to `"numeric"`.

In implementations without `Intl.DateTimeFormat` support, this parameter is ignored.

See the `Intl.DateTimeFormat()` constructor for details on these parameters and how to use them.

Return value

A string representing the time portion of the given date according to language-specific conventions.

In implementations with `Intl.DateTimeFormat`, this is equivalent to `new Intl.DateTimeFormat(locales, options).format(date)`, where `options` has been normalized as described above.

> **Note:** Most of the time, the formatting returned by `toLocaleTimeString()` is consistent. However, the output may vary between implementations, even within the same locale — output variations are by design and allowed by the specification. It may also not be what you expect. For example, the string may use non-breaking spaces or be surrounded by bidirectional control characters. You should not compare the results of `toLocaleTimeString()` to hardcoded constants.

Examples

Using toLocaleTimeString()

Basic use of this method without specifying a `locale` returns a formatted string in the default locale and with default options.

```js
const date = new Date(Date.UTC(2012, 11, 12, 3, 0, 0));

// toLocaleTimeString() without arguments depends on the implementation,
// the default locale, and the default time zone
console.log(date.toLocaleTimeString());
// "7:00:00 PM" if run in en-US locale with time zone America/Los_Angeles
```

Checking for support for locales and options parameters

The `locales` and `options` parameters may not be supported in all implementations, because support for the internationalization API is optional, and some systems may not have the necessary data. For implementations without internationalization support, `toLocaleTimeString()` always uses the system's locale, which may not be what you want. Because any implementation that supports the `locales` and `options` parameters must support the `Intl` API, you can check the existence of the latter for support:

```js
function toLocaleTimeStringSupportsLocales() {
  return (
    typeof Intl === "object" &&
    !!Intl &&
    typeof Intl.DateTimeFormat === "function"
  );
}
```

Using locales

This example shows some of the variations in localized time formats. In order to get the format of the language used in the user interface of your application, make sure to specify that language (and possibly some fallback languages) using the `locales` argument:

```js
const date = new Date(Date.UTC(2012, 11, 20, 3, 0, 0));

// formats below assume the local time zone of the locale;
// America/Los_Angeles for the US

// US English uses 12-hour time with AM/PM
console.log(date.toLocaleTimeString("en-US"));
// "7:00:00 PM"

// British English uses 24-hour time without AM/PM
console.log(date.toLocaleTimeString("en-GB"));
// "03:00:00"

// Korean uses 12-hour time with AM/PM
console.log(date.toLocaleTimeString("ko-KR"));
// "오후 12:00:00"

// Arabic in most Arabic speaking countries uses real Arabic digits
console.log(date.toLocaleTimeString("ar-EG"));
// "م ٧:٠٠:٠٠"

// when requesting a language that may not be supported, such as
// Balinese, include a fallback language, in this case Indonesian
console.log(date.toLocaleTimeString(["ban", "id"]));
// "11.00.00"
```

Using options

The results provided by `toLocaleTimeString()` can be customized using the `options` parameter:

```js
const date = new Date(Date.UTC(2012, 11, 20, 3, 0, 0));

// An application may want to use UTC and make that visible
const options = { timeZone: "UTC", timeZoneName: "short" };
console.log(date.toLocaleTimeString("en-US", options));
// "3:00:00 AM GMT"

// Sometimes even the US needs 24-hour time
console.log(date.toLocaleTimeString("en-US", { hour12: false }));
// "19:00:00"

// Show only hours and minutes, use options with the default locale - use an empty array
console.log(
  date.toLocaleTimeString([], { hour: "2-digit", minute: "2-digit" }),
);
// "20:01"
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-date.prototype.tolocaletimestring
ECMAScript® 2027 Internationalization API Specification https://tc39.es/ecma402/#sup-date.prototype.tolocaletimestring

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`toLocaleTimeString`	1	12	1	5	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
IANA time zone names in `timeZone` option	24	14	52	15	7	25	56	14	7	1.5	4.4	7	1.0.0	1.8	0.12.0
`locales` parameter	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8[1,2]	13.0.0[3,4]
`options` parameter	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.0	0.12.0

1. Also from version 1.0–1.8: partial support.
2. Also from version 1.0–1.8: Only the locale data for `en-US` is available.
3. Also from version 0.12.0–13.0.0: partial support.
4. Also from version 0.12.0–13.0.0: Before version 13.0.0, only the locale data for `en-US` is available by default. When other locales are specified, the function silently falls back to `en-US`. To make full ICU (locale) data available before version 13, see Node.js documentation on the `--with-intl` option and how to provide the data.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Date.toLocaleTimeString`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Intl.DateTimeFormat`
- `Date.prototype.toLocaleDateString()`
- `Date.prototype.toLocaleString()`
- `Date.prototype.toTimeString()`
- `Date.prototype.toString()`

JavaScript › Reference › Standard built-in objects › Date

Date.prototype.toString()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Date/toString · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **`toString()`** method of `Date` instances returns a string representing this date interpreted in the local timezone.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Date.prototype.toString()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const event = new Date("August 19, 1975 23:15:30");

console.log(event.toString());
// Expected output: "Tue Aug 19 1975 23:15:30 GMT+0200 (CEST)"
// Note: your timezone may vary
```

Syntax

```js
toString()
```

Parameters

None.

Return value

A string representing the given date (see description for the format). Returns `"Invalid Date"` if the date is invalid.

Description

The `toString()` method is part of the type coercion protocol. Because `Date` has a `[Symbol.toPrimitive]()` method, that method always takes priority over `toString()` when a `Date` object is implicitly coerced to a string. However, `Date.prototype[Symbol.toPrimitive]()` still calls `this.toString()` internally.

The `Date` object overrides the `toString()` method of `Object`. `Date.prototype.toString()` returns a string representation of the Date as interpreted in the local timezone, containing both the date and the time — it joins the string representation specified in `toDateString()` and `toTimeString()` together, adding a space in between. For example: "Thu Jan 01 1970 00:00:00 GMT+0000 (Coordinated Universal Time)".

`Date.prototype.toString()` must be called on `Date` instances. If the `this` value does not inherit from `Date.prototype`, a `TypeError` is thrown.

- If you only want to get the *date* part, use `toDateString()`.
- If you only want to get the *time* part, use `toTimeString()`.
- If you want to make the date interpreted as UTC instead of local timezone, use `toUTCString()`.
- If you want to format the date in a more user-friendly format (e.g., localization), use `toLocaleString()`.

Examples

Using toString()

```js
const d = new Date(0);
console.log(d.toString()); // "Thu Jan 01 1970 00:00:00 GMT+0000 (Coordinated Universal Time)"
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-date.prototype.tostring

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`toString`	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Date.toString`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Object.prototype.toString()`
- `Date.prototype.toDateString()`
- `Date.prototype.toLocaleString()`
- `Date.prototype.toTimeString()`

JavaScript › Reference › Standard built-in objects › Date

Date.prototype.toTemporalInstant()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Date/toTemporalInstant · MDN page last modified 2025-12-08

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`toTemporalInstant()`** method of `Date` instances returns a new `Temporal.Instant` object with the same `epochMilliseconds` value as this date's timestamp.

Use this method to convert legacy `Date` values to the `Temporal` API, then further convert it to other `Temporal` classes as necessary.

Syntax

```js
toTemporalInstant()
```

Parameters

None.

Return value

A new `Temporal.Instant` object with the same `epochMilliseconds` value as this date's timestamp. Its microsecond and nanosecond components are always `0`.

Exceptions

`RangeError`

Thrown if the date is invalid (it has a timestamp of `NaN`).

Examples

Using toTemporalInstant()

```js
const legacyDate = new Date("2021-07-01T12:34:56.789Z");
const instant = legacyDate.toTemporalInstant();

// Further convert it to other objects
const zdt = instant.toZonedDateTimeISO("UTC");
const date = zdt.toPlainDate();
console.log(date.toString()); // 2021-07-01
```

Specifications

Specification
Temporal https://tc39.es/proposal-temporal/#sec-date.prototype.totemporalinstant

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
toTemporalInstant	144	144	139	128	Prev.	144	139	95	**No**	**No**	144	**No**	1.4.0	2.7	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Date.toTemporalInstant`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Temporal.Instant`
- `Temporal.ZonedDateTime`
- `Temporal.Instant.fromEpochMilliseconds()`

JavaScript › Reference › Standard built-in objects › Date

Date.prototype.toTimeString()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Date/toTimeString · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **`toTimeString()`** method of `Date` instances returns a string representing the time portion of this date interpreted in the local timezone.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Date.prototype.toTimeString()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const event = new Date("August 19, 1975 23:15:30");

console.log(event.toTimeString());
// Expected output: "23:15:30 GMT+0200 (CEST)"
// Note: your timezone may vary
```

Syntax

js

```
toTimeString()
```

Parameters

None.

Return value

A string representing the time portion of the given date (see description for the format). Returns `"Invalid Date"` if the date is invalid.

Description

`Date` instances refer to a specific point in time. `toTimeString()` interprets the date in the local timezone and formats the *time* part in English. It always uses the format of `HH:mm:ss GMT±xxxx (TZ)`, where:

Format String	Description
`HH`	Hour, as two digits with leading zero if required
`mm`	Minute, as two digits with leading zero if required
`ss`	Seconds, as two digits with leading zero if required
`±xxxx`	The local timezone's offset — two digits for hours and two digits for minutes (e.g., `-0500`, `+0800`)
`TZ`	The timezone's name (e.g., `PDT`, `PST`)

For example: "04:42:04 GMT+0000 (Coordinated Universal Time)".

- If you only want to get the *date* part, use `toDateString()`.
- If you want to get both the date and time, use `toString()`.
- If you want to make the date interpreted as UTC instead of local timezone, use `toUTCString()`.
- If you want to format the date in a more user-friendly format (e.g., localization), use `toLocaleTimeString()`.

Examples

Using toTimeString()

```js
const d = new Date(0);

console.log(d.toString()); // "Thu Jan 01 1970 00:00:00 GMT+0000 (Coordinated Universal Time)"
console.log(d.toTimeString()); // "00:00:00 GMT+0000 (Coordinated Universal Time)"
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-date.prototype.totimestring

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
toTimeString	1	12	1	5	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Date.toTimeString`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Date.prototype.toLocaleTimeString()`
- `Date.prototype.toDateString()`
- `Date.prototype.toString()`

JavaScript › Reference › Standard built-in objects › Date

Date.prototype.toUTCString()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Date/toUTCString · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **`toUTCString()`** method of `Date` instances returns a string representing this date in the RFC 7231 format, with negative years allowed. The timezone is always UTC. `toGMTString()` is an alias of this method.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Date.prototype.toUTCString()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const event = new Date("14 Jun 2017 00:00:00 PDT");

console.log(event.toUTCString());
// Expected output: "Wed, 14 Jun 2017 07:00:00 GMT"
```

Syntax

js

```
toUTCString()
```

Parameters

None.

Return value

A string representing the given date using the UTC time zone (see description for the format). Returns `"Invalid Date"` if the date is invalid.

Description

The value returned by `toUTCString()` is a string in the form `Www, dd Mmm yyyy HH:mm:ss GMT`, where:

Format String	Description
`Www`	Day of week, as three letters (e.g., `Sun`, `Mon`)
`dd`	Day of month, as two digits with leading zero if required
`Mmm`	Month, as three letters (e.g., `Jan`, `Feb`)
`yyyy`	Year, as four or more digits with leading zeroes if required
`HH`	Hour, as two digits with leading zero if required
`mm`	Minute, as two digits with leading zero if required
`ss`	Seconds, as two digits with leading zero if required

Aliasing

JavaScript's `Date` API was inspired by Java's `java.util.Date` library (while the latter had become de facto legacy since Java 1.1 in 1997). In particular, the Java `Date` class had a method called `toGMTString` — which was poorly named, because the Greenwich Mean Time is not equivalent to the Coordinated Universal Time, while JavaScript dates always operate by UTC time. For web compatibility reasons, `toGMTString` remains as an alias to `toUTCString`, and they refer to the exact same function object. This means:

js

```
Date.prototype.toGMTString.name === "toUTCString";
```

Examples

Using toUTCString()

```js
const d = new Date(0);
console.log(d.toUTCString()); // 'Thu, 01 Jan 1970 00:00:00 GMT'
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-date.prototype.toutcstring

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
toUTCString	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, javascript.builtins.Date.toUTCString. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Date.prototype.toLocaleString()
- Date.prototype.toString()
- Date.prototype.toISOString()

JavaScript › Reference › Standard built-in objects › Date

Date.prototype.valueOf()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Date/valueOf · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **valueOf()** method of Date instances returns the number of milliseconds for this date since the epoch, which is defined as the midnight at the beginning of January 1, 1970, UTC.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Date.prototype.valueOf()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const date1 = new Date(Date.UTC(96, 1, 2, 3, 4, 5));

console.log(date1.valueOf());
// Expected output: 823230245000

const date2 = new Date("02 Feb 1996 03:04:05 GMT");

console.log(date2.valueOf());
// Expected output: 823230245000
```

Syntax

js

```
valueOf()
```

Parameters

None.

Return value

A number representing the timestamp, in milliseconds, of this date. Returns `NaN` if the date is invalid.

Description

The `valueOf()` method is part of the type coercion protocol. Because `Date` has a `[Symbol.toPrimitive]()` method, that method always takes priority over `valueOf()` when a `Date` object is implicitly coerced to a number. However, `Date.prototype[Symbol.toPrimitive]()` still calls `this.valueOf()` internally.

The `Date` object overrides the `valueOf()` method of `Object`. `Date.prototype.valueOf()` returns the timestamp of the date, which is functionally equivalent to the `Date.prototype.getTime()` method.

Examples

Using valueOf()

js

```
const d = new Date(0); // 1970-01-01T00:00:00.000Z
console.log(d.valueOf()); // 0
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-date.prototype.valueof

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`valueOf`	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Date.valueOf`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Object.prototype.valueOf()`
- `Date.prototype.getTime()`

Date.prototype[Symbol.toPrimitive]()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Date/Symbol.toPrimitive · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since April 2017.

The **`[Symbol.toPrimitive]()`** method of `Date` instances returns a primitive value representing this date. It may either be a string or a number, depending on the hint given.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Date.prototype[Symbol.toPrimitive]". It runs live in the browser there; its source code follows.

js · interactive example source

```
// Depending on timezone, your results will vary
const date = new Date("20 December 2019 14:48");

console.log(date[Symbol.toPrimitive]("string"));
// Expected output: "Fri Dec 20 2019 14:48:00 GMT+0530 (India Standard Time)"

console.log(date[Symbol.toPrimitive]("number"));
// Expected output: 1576833480000
```

Syntax

js

```
date[Symbol.toPrimitive](hint)
```

Parameters

`hint`

A string representing the type of the primitive value to return. The following values are valid:

- `"string"` or `"default"`: The method should return a string.

- `"number"`: The method should return a number.

Return value

If `hint` is `"string"` or `"default"`, this method returns a string by coercing the `this` value to a string (first trying `toString()` then trying `valueOf()`).

If `hint` is `"number"`, this method returns a number by coercing the `this` value to a number (first trying `valueOf()` then trying `toString()`).

Exceptions

`TypeError`

Thrown if the `hint` argument is not one of the three valid values.

Description

The `[Symbol.toPrimitive]()` method is part of the type coercion protocol. JavaScript always calls the `[Symbol.toPrimitive]()` method in priority to convert an object to a primitive value. You rarely need to invoke the `[Symbol.toPrimitive]()` method yourself; JavaScript automatically invokes it when encountering an object where a primitive value is expected.

The `[Symbol.toPrimitive]()` method of the `Date` object returns a primitive value by either invoking `this.valueOf()` and returning a number, or invoking `this.toString()` and returning a string. It exists to override the default primitive coercion process to return a string instead of a number, because primitive coercion, by default, calls `valueOf()` before `toString()`. With the custom `[Symbol.toPrimitive]()`, `new Date(0) + 1` returns `"Thu Jan 01 1970 00:00:00 GMT+0000 (Coordinated Universal Time)1"` (a string) instead of `1` (a number).

Examples

Using [Symbol.toPrimitive]()

```js
const d = new Date(0); // 1970-01-01T00:00:00.000Z

d[Symbol.toPrimitive]("string"); // "Thu Jan 01 1970 00:00:00 GMT+0000 (Coordinated
Universal Time)"
d[Symbol.toPrimitive]("number"); // 0
d[Symbol.toPrimitive]("default"); // "Thu Jan 01 1970 00:00:00 GMT+0000 (Coordinated
Universal Time)"
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/numbers-and-dates.html#sec-date.prototype-%symbol.toprimitive%

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
[Symbol.toPrimitive]	47	15	44	34	10	47	44	34	10	5.0	47	10	1.0.0	1.0	6.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Date.@@toPrimitive`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Symbol.toPrimitive`